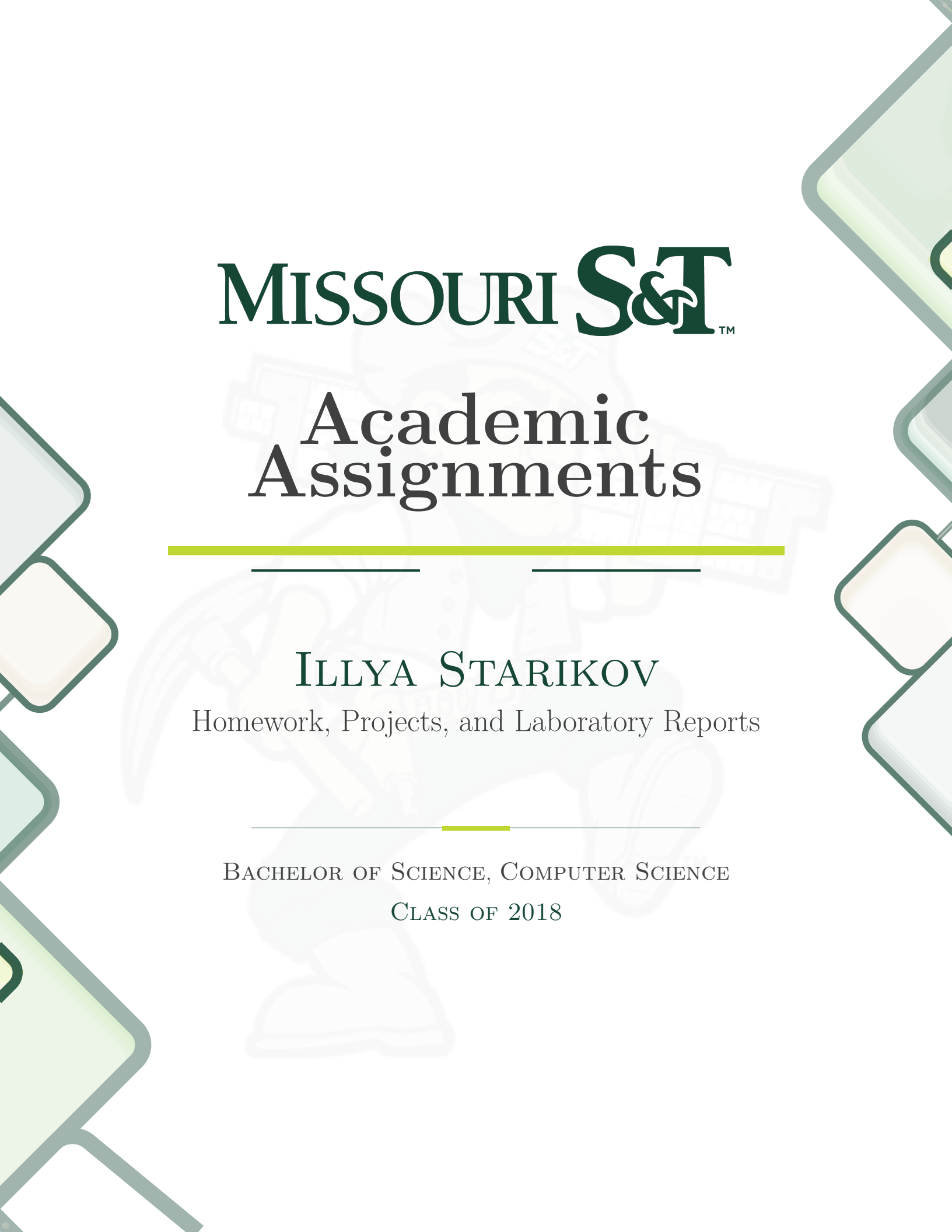




MISSOURI S&T™

Academic Assignments

ILLYA STARIKOV

Homework, Projects, and Laboratory Reports

BACHELOR OF SCIENCE, COMPUTER SCIENCE

CLASS OF 2018

Preface

This collection of academic assignments showcases the practical application of theoretical knowledge gained during my studies at Missouri University of Science and Technology. Through homework assignments, programming projects, and laboratory reports, these materials demonstrate my ability to translate complex concepts into working solutions across diverse areas of computer science and engineering.

Each assignment represents a stepping stone in my technical development, from early programming exercises in introductory courses to sophisticated implementations in advanced algorithms and software engineering. The progression visible in these works illustrates not only mastery of individual topics but also the integration of knowledge across disciplines—a hallmark of the comprehensive education provided at Missouri S&T.

Illya Starikov

Bachelor of Science, Computer Science
Missouri University of Science and Technology

Academic Advisors: Dr. Jennifer Leopold, Dr. A. Ricardo Morales,
Dr. Simone Silvestri, Professor Clayton Price

Class of 2018
starikov@mst.edu

“Miners Dig Deeper”



Table of Contents

Contents

Table of Contents	1
[CPE2210] Introduction To Computer Engineering	4
Lab 1	5
Lab 2	7
[CPE3150] Micro Embedded Design	9
Project 1 Report	10
Project 3 Report	17
Project 1 Specification	26
Homework 2	28
Homework 3	29
Homework 4	32
Homework 5	37
Homework 6	44
Homework 7	50
Homework 8	53
Arduino Research Paper	59
[CS1001] Data Structures Lab	64
Lab 1	65
Lab 5	67
Lab 8	68
Lab 9	69

Final Project	70
[CS1200] Discrete Math	74
Homework 4	75
Homework 5	76
[CS2200] Theory Of Computer Science	77
Quiz 10	78
Quiz 11	82
[CS2300] Databases	89
Homework 1	90
Homework 2	94
Homework 4	96
Final Report	98
[CS2500] Algorithms	123
Homework 1	124
Homework 2	128
Homework 3	133
Homework 4	135
Project 1: Report I	140
Project 1: Report II	147
Project 2: Report I	153
Project 2: Report II	168
[CS3001] Skills Development	186
Assignment Grade	187
Presentation 1	188
Presentation 2	195
Presentation 3	205
Presentation 4	216
Presentation 5	227
Presentation 7	240
Presentation 8	250
Presentation 9	263
Presentation 10	275
[CS3200] Numerical Methods	287
Homework 9	288
Homework 10	294
[CS3800] Operating Systems	295
Homework 2 Report	296
[CS5200] Analysis Of Algorithms	298
Homework 1	299
Homework 2	310
Homework 3	326

Homework 4	335
Homework 5	344
Homework 6	350
Homework 7	363
Homework 8	398
[CS5400] Artificial Intelligence	411
Exercise 1	412
Exercise 2	415
Exercise 3	421
Exercise 4	423
Exercise 5	424
Exercise 6	426
Exercise 7	428
Exercise 8	430
Bonus 1	431
[CS5402] Data Mining	432
Homework 1	433
Homework 2	440
Homework 3	444
Homework 4	447
Homework 5	450
Homework 6	454
[MATH2100] Foundations Of Mathematics	458
Homework 1	459
[PHIL3225] Business Ethics	460
Essay 1	461
Paper 1	464
Paper 2	466
Paper 3	468
Paper 4	471
Test 1	473
Test 2	477
[PSYC1101] Psychology 101	483
Sleep States	484
Steven Pinker Reflection	487
Quantum Cognition	493

CPE2210

Introduction To Computer Engineering



Lab #1: Laboratory Equipment

Illya Starikov

January 25, 2016

1 Objective

To familiarize self with lab equipment, namely the DC Power Supply, Function Generator, Frequency Counter/Timer, Digital Multimeter, and Mixed Signal Oscilloscope.

2 Equipment Used

- DC Power Supply
- Function Generator
- Frequency Counter/Timer
- Digital Multimeter
- Mixed Signal Oscilloscope

3 Procedures

1. Generate a square wave with an amplitude of 3 V and frequency of 900 Hz.
 - Offset DC by 2 V.
2. Generate 3 signals:
 - 1 kHz
 - 10 kHz
 - 50 kHz

3. Generate 7 V DC, measuring with the Digital Multimeter.
4. Follow the steps of category 5.

4 Conclusion and Results

4.1 Function Generator

4.2 Frequency Counter/Timer

Function Generator	Function Counter
1.017 kHz	1.1 kHz
10.02 kHz	10.1 kHz
49.99 kHz	50.1 kHz

4.3 Digital Multimeter (DMM)

Multimeter:	7.015 V
Power Supply:	7.1 V

4.4 Mixed Signal Oscilloscope

At approximately 2.430 MHz the square wave starts becoming indistinguishable, taking the shape of a sinusoidal wave.

5 Notes And Comments

- $v_{rms} = \frac{VP}{\sqrt{2}}$
- $p - p$ means peak to peak.

Lab # 2

Illya Starikov

February 01, 2016

1 Objective

To familiarize self with Quartus and ModelSim software.

2 Equipment Used

- Computer
 - Quartus Software
 - ModelSim Software

3 Procedures

1. Launch Quartus and go through initial project startup
2. Create a new Block Diagram / Schematic File
3. Add logic gates (as specified)
4. Compile the logic gates.
 - If any compiler errors, fix.
 - Rinse and repeat.
5. Create VHDL from schematic.
6. Launch ModelSim, go through initial setup.
7. Add VHDL to the project.
8. Simulate the schematic built and print the results.

4 Conclusion and Results

ModelSim and Quartus is quite useful software that makes the simulation of circuits a trivial task.

5 Notes And Comments

- Every project must have a file named the same as the project.
- Do **not** save into the C directory.

CPE3150

Micro Embedded Design



Project #2

Michael Schoen, Abdirahman Osman, Illya Starikov

Due Date: November 8th, 2016

For our project, we have decided to implement a memory matching game. The game will start off by enabling a single light. If the user correctly presses said light, the user will move onto the next level and enable a light sequence. For every successive level, an additional light have to be kept in mind (i.e. for the n^{th} level, there will be n lights that will have to be pressed).

This will continue for 15 games, where upon a successful finish, a special light sequence and a song will asynchronously play. From here, the user can choose to play another game if they so choose.

1 Explanation

Below we will take a more in-depth look into the implementation of our project.

1.1 Random Number Generation

Because we want our game to be different every round, we have to have some form of random number generation. Initially, we had tried to implement a *linear congruential generator*; unfortunately, we were unsuccessful. We instead went with a different implementation.

Both implementations require a seed value (and to have different results, we need a different seed value) — we “cheat” to get this. During our input, we not only check to see if the user initiated a game, but we also increment the register the seed is stored in. This way, there is only a $1/255$ chance the user will get the same game¹.

¹Although this is a significant value (0.39%), it serves well for demo purposes.

Failed Attempt

The basic summary is as follows: beginning with a seed value (named X_n), a new linear, psuedo-random number can be calculated via:

$$X_{n+1} = (aX_n + c) \mod m \quad (1)$$

where the follow conditions hold

1. $a, X_n, c, m \in \mathbb{Z}^+$
2. $0 \leq X_n, c < m$
3. $0 < a < m$
4. $m \neq 0$

The issue we face is it is *highly* recommended that m be quite large (most popular implementations range from 2^{31} to 2^{48}); unfortunately, we only have 2^8 available to us. Because of this, we either get a cycle roughly every 10 iterations or the same number produced. As one might imagine, this is a game-breaking bug; we decided to go with something different.

Our failed attempt for a linear congruential generator is as follows.

```
; Check writeup for how this works
; For now, the formula is

; X_n+1 = aX_n + c mod m

; For our purposes, a = 7, c = incrementor, m = 72
; for our purposes, incrementor can't be a multiple of 7
; X_n is stored in R7, incrementor in R6

; Result will be stored in A
RNG:
    MOV A, R7
    MOV B, #7D
    DIV AB
    MOV A, B
```

```

                                JNZ SKIP1
                                INC R6
SKIP1:                        MOV A,  #7D
                                MOV B, R7
                                MUL AB                                ; A = aX_n

                                ADD A, R6
                                MOV B, #72D
                                DIV AB

                                MOV R7, B

                                MOV A, B
                                MOV B, #10D
                                DIV AB

                                MOV A, B
                                RET

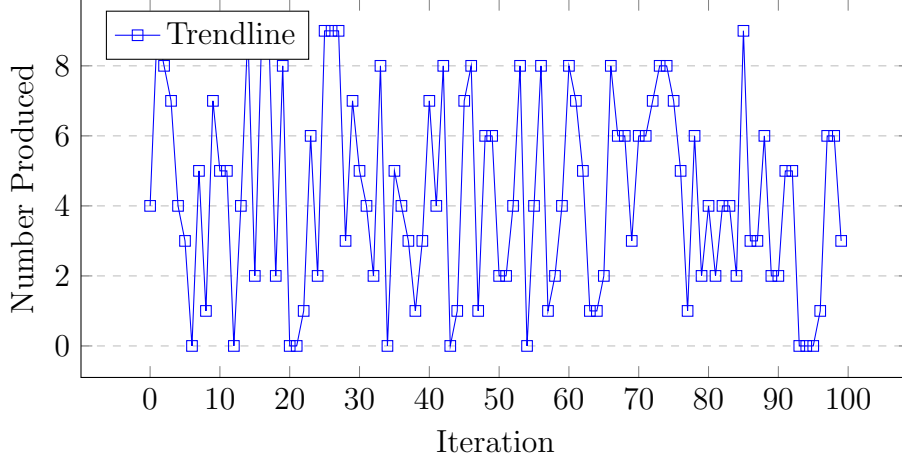
```

Successful Attempt

For our successful attempt, we ported a random number generating from an open source code base². It similar to the linear congruential generator in the sense that it linearly produces a psuedo-random number; however, it is a different formula (one that can't be eloquently described in an equation). Below is a graph that shows the distribution of numbers up to 100 iterations.

²<https://www.pjrc.com/tech/8051/>

Random Numbers Generated (Seed: 7)



1.2 Light Show

When programming the lights, we notice there to be 9 lights; which just so happens to be the exact size of register *A* and the carry bit *C* in the `sfr`. So we decided to map every bit in *A* and *C* to a particular light. For simplicity, we used a 1_2 to represent an ‘on’ state and a 0_2 for the ‘off’ state.

Our mapping is as follows: we start the MSB of *A*, which represents the top-left light. The proceeding two bits make up the top-most row of the lighting board. The fourth bit represents the middle row, *first* light. We continue with this pattern until we come to the last light. The last light is represented by the carry bit *C*. Fig. 1 summarizes this lighting schematic.

As an example, `111000111 1` would enable only the top and bottom lights, leaving the middle row blank. `10110110 1` enables the left and right columns. `10101010 1` makes an X.

As mentioned, to represent an ‘on’ state, we used a 1_2 ; however, the lights are active low. So, the first thing we do is `CPL A` and `CPL C` to ensure we don’t have opposite affect. From there, we rotate through *C* to enable individual light. Finally, we `CPL A` and *C* again to bring the data back to its original state.

The individual lighting effects were done through trial and error of what looked aesthetically pleasing. There are three light patterns, one for losing, advancing to the next level, and the winning light sequence. The winning light sequence has five parts, eloquently named:

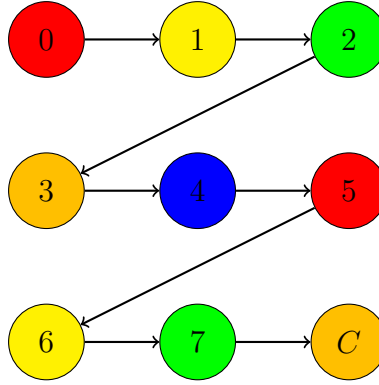


Figure 1 – The lighting schematic for “lights” subroutine.

1. Light Spiral Part I
2. Light Spiral Part II
3. Full Light Spiral
4. Os
5. Bring the Thundaaa

1.3 Music

We know the frequency of the Philips P89LPC932A1 to be 7.373 MHz, with 2 cycles per machine cycle. Therefore,

$$\frac{2 \text{ cycles}}{\text{machine cycle}} \cdot \frac{1 \text{ Period}}{7.373 \text{ MHz}} = 0.271 \text{ } 26 \text{ } \mu\text{s}/_{\text{mc}} \quad (2)$$

We use this calculation as the base of our music.

E5	$f = 659.255 \text{ Hz} \implies T = 1516 \text{ }\mu\text{s}$ $1516 \text{ }\mu\text{s} \div 0.27126 \text{ }\mu\text{s}/\text{mc} = 5589 \text{ mc}$ $5589 \text{ mc} \div 4 = 1398 \text{ mc}$ $1398 \text{ mc} \implies 699 \text{ iterations (with DJNZ)}$
F5	$f = 698.456 \text{ Hz} \implies T = 1431 \text{ }\mu\text{s}$ $1431 \text{ }\mu\text{s} \div 0.27126 \text{ }\mu\text{s}/\text{mc} = 5275 \text{ mc}$ $5589 \text{ mc} \div 4 = 1318 \text{ mc}$ $1318 \text{ mc} \implies 569 \text{ iterations (with DJNZ)}$
G5	$f = 783.991 \text{ Hz} \implies T = 1275.5 \text{ }\mu\text{s}$ $1275.5 \text{ }\mu\text{s} \div 0.27126 \text{ }\mu\text{s}/\text{mc} = 4702 \text{ mc}$ $4702 \text{ mc} \div 4 = 1176 \text{ mc}$ $1176 \text{ mc} \implies 588 \text{ iterations (with DJNZ)}$
D5	$f = 587.330 \text{ Hz} \implies T = 1702.6 \text{ }\mu\text{s}$ $1702.6 \text{ }\mu\text{s} \div 0.27126 \text{ }\mu\text{s}/\text{mc} = 6277 \text{ mc}$ $7045 \text{ mc} \div 4 = 1570 \text{ mc}$ $1570 \text{ mc} \implies 785 \text{ iterations (with DJNZ)}$
C5	$f = 523.251 \text{ Hz} \implies T = 1911.1 \text{ }\mu\text{s}$ $1911.1 \text{ }\mu\text{s} \div 0.27126 \text{ }\mu\text{s}/\text{mc} = 7045 \text{ mc}$ $7045 \text{ mc} \div 4 = 1760 \text{ mc}$ $1760 \text{ mc} \implies 880 \text{ iterations (with DJNZ)}$
Flat D5	$f = 554.365 \text{ Hz} \implies T = 1803.8 \text{ }\mu\text{s}$ $1803.8 \text{ }\mu\text{s} \div 0.27126 \text{ }\mu\text{s}/\text{mc} = 6650 \text{ mc}$ $6650 \text{ mc} \div 4 = 1662 \text{ mc}$ $1662 \text{ mc} \implies 831 \text{ iterations (with DJNZ)}$

2 Future Work

3 Work Effort

- Michael Schoen
 - Programmed binary counter.

- Programmed game logic.
- Osman Abdirahman
 - Programmed initial beep.
 - Programmed song implementation.
- Illya Starikov
 - Programmed random number renerator.
 - Programmed light sequence.

Project #3

Michael Schoen, Abdirahman Osman, Illya Starikov

Due Date: Tuesday, December 6th, 2016

For the second project, we were delighted to be able to use a higher level programming language; we¹ decided to apply this new-found excitement to implement a retro game from the 70s: Space Invaders.

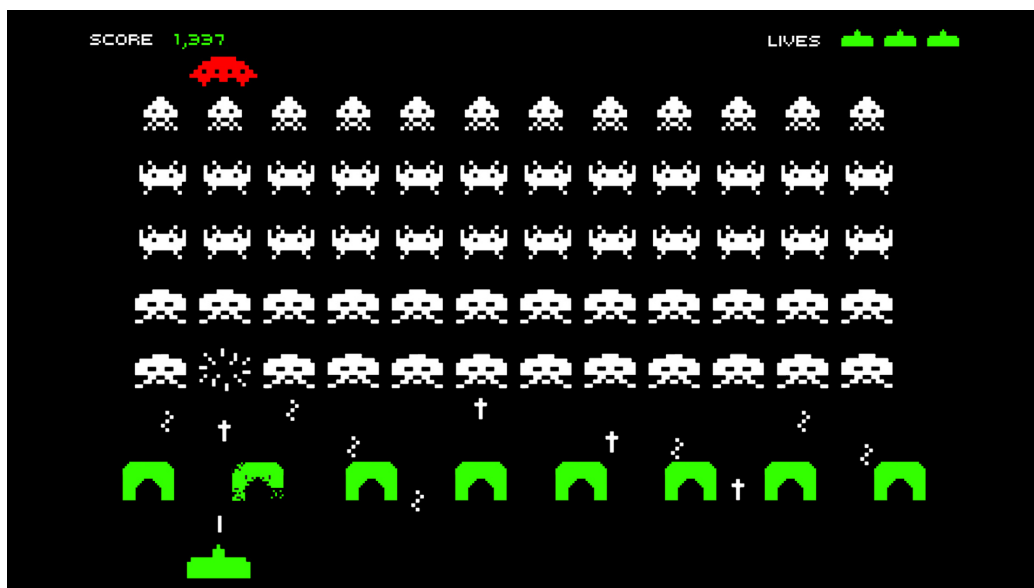


Figure 1 – The original space invaders.

When this idea came crashing and burning down to the ground, we decided to combat this with a additional music, an elegant user interface, and a keyboard (with some other miscellanies).

¹Illya.

1 Project Description

Below we will go into more detail about each individual parts of our project.

1.1 The Game

The game we initially decided to go with was space invaders. We had intention of doing the game logic on the microcontroller through serial communication; however, we learned early that this was likely not be possible². We describe in *Problems Encountered* how we absolved this. From here, we decided our game would exclusively be in the terminal.

Our game essentially creates a two dimensional array (in three segments — the header to show score and level, the aliens, and the shooter). Then we loop through depending on the input:

- Move the shooter left.
- Move the shooter left.
- Quits the game
- Shoots with the gunner.
- No input meant refresh game.

We achieved the drawing through `curses`³. Ultimately, we did not get our game fully done, but a good majority of it. See Figure. 2 for a look at the UI of the game.

1.2 Music

The formula for each note is

$$\frac{1/4 \text{ Oscillator Frequency}}{\text{Note Frequency}}$$

So for supposing we want to produce the note *C4*,

²Our hex file with very basic functionality was 25 kB.

³Can be read about here [https://en.wikipedia.org/wiki/Curses_\(programming_library\)](https://en.wikipedia.org/wiki/Curses_(programming_library))

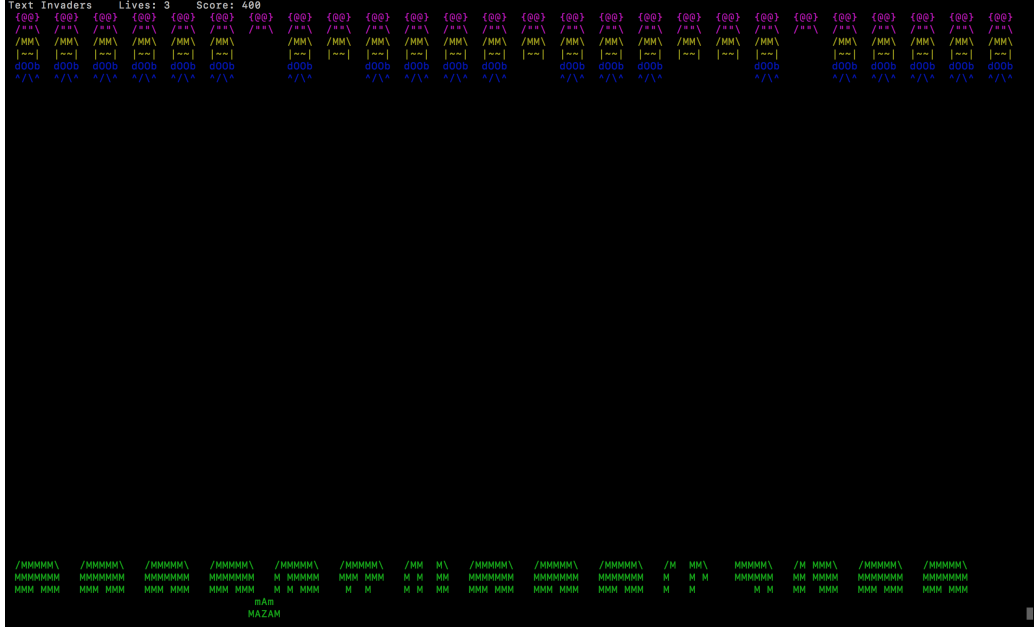


Figure 2 – Text Invaders, our version of space invaders..

$$\frac{1/4 \text{ Oscillator Frequency}}{\text{Note Frequency}} = \frac{1/4 \cdot 7.373 \text{ Hz}}{261.63 \text{ Hz}} = 7.045 \text{ ms} \quad (1)$$

To produce the music, we had a function that took in which note and the type of note (16th, 8th, 4th, half or whole note). The function multiplied the base length of a note by a constant depending on the type of note. A 16th note would be multiplied by 1, an 8th by 2, a quarter note by 4, a half-note by 8, and a whole note by 16. We used timers and interrupts for the period of the note and the length of the note.

We also used sheet music to get the notes. For speeding and slowing down the music, the type of note was just multiplied by 2 or divided by 2 respectively.

1.3 Interactive User Interface

The menu goes through three stages; printing the pen-rose triangle background, printing options, then allowing the selection of items. The following are printed

1. Counter

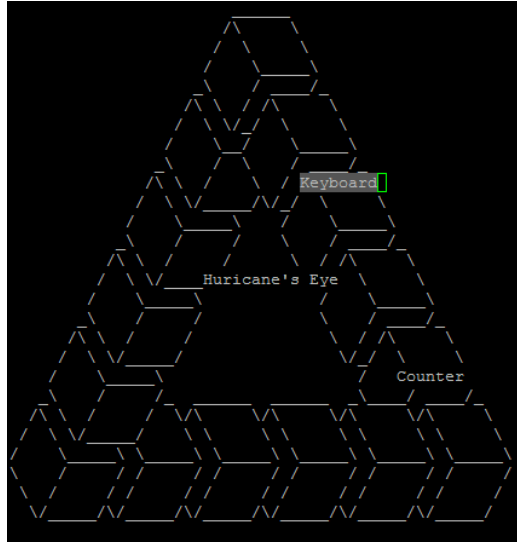


Figure 3 – The pen-rose style menu.

2. Hurricane's Eye
3. Keyboard

respectively. From there the user has the option to cycle upwards using P_2 (see Figure. 4), cycle down using P_8 , or select using P_5 . The cycle starts with 1, highlighting Counter if P_2 is pressed, then Hurricane's eye will be highlighted, and if printed once again Counter will be selected. If the menu user presses P_1 once at the 3rd option (Keyboard) then the menu will cycle back to option 1 and vice versa if P_8 were clicked at the 1st option. Selecting an menu option with P_5 while highlighted will start its respective function. Also each menu option has a corresponding number displayed on a seven segment display.

1.4 Seven Segment Display

For the seven segment display, we had to set P_2 to be push/pull so that the display would receive enough power. We then looked up the correct values to output to the pins to make each number show up on the display and used those values in the increment/decremented function as well as the simple function that lets you display any number. We used the data sheet for the

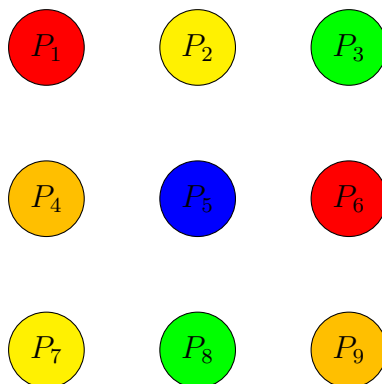


Figure 4 – The Main Menu.

seven segment display to see how to connect it on the breadboard. See Figure. 5 for additional information.

1.5 Keyboard

Our keyboard has a C scale set to switches $P_1 \dots P_8$ with P_9 escaping to the menu. Each switch uses the corresponding light associated with its position save P_9 which is the escape button.

1.6 Curses

In order to print to any part of the screen we needed to implement a structure to send ANSI control sequences and characters to the terminal using `uart_get()` as a base. To do this we implemented an 8051 port of curses that takes in (y, x) coordinates as arguments for printing `chars` and `strings` at any defined (x, y) location. This allows for the implementation of the menu since it requires printing of options in sometimes nonlinear fashion.

We found a whole host of others issues associated with printing characters to a terminal, printing to an arbitrary location, control sequences, fluctuation in crystal timings, and requesting information from a terminal. We accounted for these individually.

PIN NO.			
1	Cathode E	6	Cathode B
2	Cathode D	7	Cathode A
3	Common Anode DIG. 1	8	Common Anode DIG. 2
4	Cathode C	9	Cathode F
5	Cathode DP	10	Cathode G

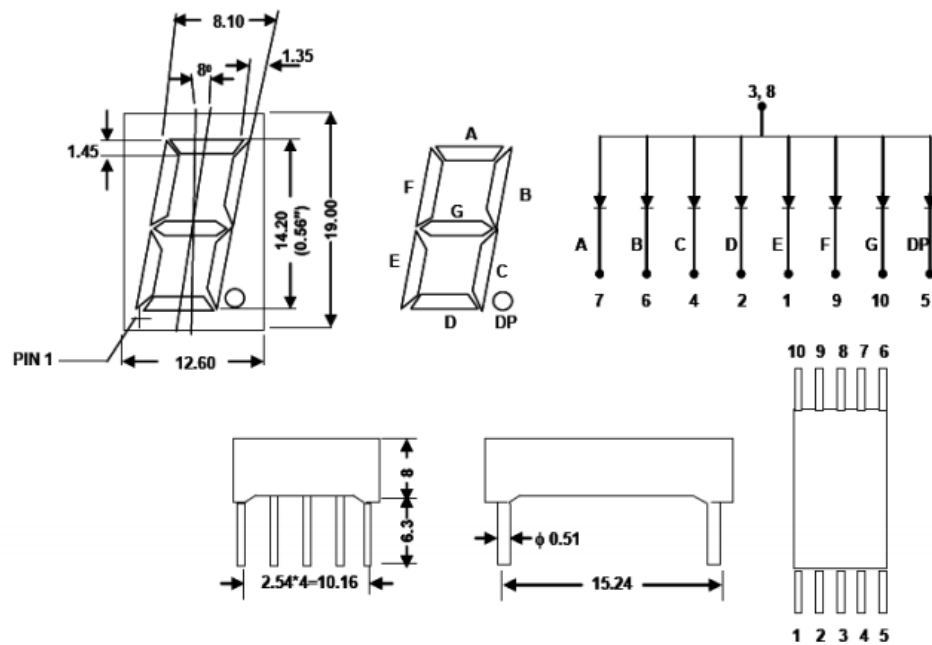


Figure 5 – The schematic for the seven segment display.

2 Problems Encountered

Below we will list some of the “several” problems we ran into.

2.1 Memory Constraints

By far, the biggest problem we encountered was the memory constraint. The 8051 has 8 kB of memory; our code base, with the exclusion of all the `malloc`s⁴, it is roughly 25 kB — a size a bit larger than the 8051 allows. Our workaround was to fight fire with fire.

Instead of “dumbing” down our game⁵ to get it to fit, we decided to have an external interface; specifically, a port sniffer. It would listen for input from the 8051, and if there is a signal on the serial port, use that as input. If not, default to the keyboard input.

Ultimately, we were unsuccessful with the port sniffer. Originally, we had tried to use a Linux port sniffer so we can just embed it into our program, `slsnif`⁶. Unfortunately this port sniffer does not support “legacy” ports, and the 8051 falls in this category. So we moved onto a Windows port sniffer, `Serial Input For Windows`⁷. This too did not work, because we could only have one interface use the serial port, so we would need a dedicated socket to intercept the `COM1` port’s input — something we were not familiar with.

Ultimately, we were simply unsuccessful and had to scrap this.

2.2 Printing Lines and Columns.

We had an issue with the timing crystals of the Simon board where the board heating up would cause the serial port to start printing unrecognizable characters. This lead us to think we were causing a segfault when we were printing string literals. This caused hours of wasted time since nothing we could change would cause the board to start printing what we wanted, but one afternoon after hours of exhaustive research and help at the lead sessions the conclusion was reached that heat changes were causing the board to lose its timing and print out nonsense. The fix for this was to let the board cool off while writing code. After this change garbage outputs dropped significantly.

⁴Since `unsigned char` = 1 B, the terminal window will roughly be 20 height × 80 width, we can roughly expect 1.6 kB to be allocated on the heap; a non-insignificant amount compared to 8 kB.

⁵According to back of the hand calculations, a space-optimized version of the game would still be 7 kB. This was likely to be impossible.

⁶Can be found at <https://sourceforge.net/projects/slsnif/>.

⁷Can be found at <http://www.randomnoun.com/wp/2013/02/03/serial-input-for-windows/>.

2.3 Seven Segment Display Buttons

One problem we ran into was some of the buttons use the same pins as we used for the seven segment display. The display used push/pull which makes the buttons not work, so we had to carefully chose buttons that didn't use the same pins.

3 8051 Architecture

For this project, we made sure to utilize the additional functionality we learned in the later parts of the semester — specifically, timers, serialization, and interrupts.

We spent a great deal of time with the serial communication aspect of the board; we took advantage of this the most (with respect to the 8051 architecture). We have already discussed the things that made our board unique; however, we will show an example of how our implimination of `curses`.

```
1 void move(unsigned char y, unsigned char x) {
2     unsigned char i; // counter
3     zero_cursor(); // moves the code to (0, 0)
4
5     for(i = 0; i < x; i++) {
6         cursor_jump(1, 'R'); // move cursor to the x position
7     }
8     for(i=0; i < y; i++) {
9         cursor_jump(1, 'D'); // now to the y
10    }
11 }
```

Now combining that with `addch(char)`,

```
1 void addch(unsigned char value) {
2     uart_isr();
3     uart_transmit(value);
4     while(TI==0);
5 }
```

we now have a fundamental idea to `curses`, printing anywhere!

Next, we implemented all delays with timer interrupts. For example,

```
1
2 void delay1ms() {
3     TH0=-0x0E;
```

```

4     TLO=-0x66;
5     TR0 = 1; // start
6     while(TF0 == 0); // poll to finish
7     TR0 = 0; // stop
8     TF0 = 0; // clear the finish
9 }

```

delays for 1 ms, because $(FFFF_{16} - 0E66_{16} + 1) \times 1.085 \mu s \approx 1 \text{ ms}$. As it turns out, we mostly needed millisecond precision, so we used this as the foundation for the rest of the program.

4 Individual Features

- Michael Schoen — 33% Contribution
 - Song
 - Seven Segment Display
- Abdirahman Osman — 33% Contribution
 - Port Serialization
 - Text User Interface
 - Menu
- Illya Starikov — 33% Contribution
 - Space Invaders Game
 - Keyboard

Happy Holidays
From Michael, Abdirahman, and Illya!

Project I

Illya Starikov

Due Date: October 4th, 2016

When coding, I started by breaking up the assignments into subproblems; problems small enough that could be handled in a subroutine of their own.

To begin, the first specified task was to read an input from P1; which was nothing more than a couple of `MOV`s and a loop (`JZ`, to be precise). Then I made sure to clear the upper four-bits (because they were to be set as output ports) to prevent an extra 240¹ iterations from occurring. Then one of two things could occur based on the input: read from `ROM` or calculate the Fibonacci sequence on the fly.

Because `ROM` has to be read only, and we can write only at compile time, Fibonacci (up to the first 15 digits) had to be defined before the start of the program via a `DB` at `200H`. However, calculation on the fly was a bit more complicated. We know Fibonacci to be

$$0, 1, 1, 2, 3, 5, \dots, a_{n+1} = a_n + a_{n-1} = \sum_{k=n-2}^{n-1} a_k \quad (1)$$

with the summation's initial conditions being $a_0 = 0$ and $a_1 = 1$. Using this more compact summation, we can write this programmatically. Store the initial conditions in two registers, add together to get a result, rotate the result into the register (and removing a_{n-1}) can produce an effective Fibonacci sequence generator.

Next is to output the value; this depends on the original input (because that determine how Fibonacci would be generated). There are, again, two methods based on the input.

One would be to read from the value, a simple enough tasks. `MOV` the `DPTR` to the location of the value, and read it in. However, the fifteenth value

¹11110000₂ = 240₁₀

is odd — we had stored 121_{10} in ROM, which is 79_{16} in base-16 encoding. We account for this by appending the **CY** flag. Now, $\text{Fibonacci}(15) = \text{CY} + 79_{16} = 179_{16} = 377_{10}$. But this doesn't affect lower numbers; $\text{Fibonacci}(14) = \text{CY} + 90_{16} = 090_{16} = 144_{10}$.

Outputting the result of the generated Fibonacci number is even easier; our value generated for the Fibonacci is still in the accumulator. After making P3 an output port, **MOV** from the accumulator to P3. The carry flag was already set during the arithmetic if the number was 15^2 .

After the output, there is to be a delay. This delay has to be $27,250\mu s$, which is a relatively large number since the loops can only have repeat 255 times. So to do this, I knew I needed to undershoot by a small margin (effectively $< 553.35\mu s$). So arbitrarily choosing 255 as one of the loops, a reasonable assumption would be there has to an inner loop of 47 iteration to give $26,007.45\mu s$ for the main loop. The rest of the subroutines was just asymptotically approaching $27,250\mu s$. This is summarized in Table 1, where the final value of the delay $27,250.86\mu s$.

Code	MC	Iter	Time
MOV R3, _num_255	2	1	$2.17\mu s$
DELAY1: MOV R4, _num_47	2	255	$553.35\mu s$
DELAY2: DJNZ R4, DELAY2	2	255×47	$26007.45\mu s$
DJNZ R3, DELAY1	2	255	$553.35\mu s$
MOV R3, _num_109	2	1	$2.17\mu s$
DELAY3: DJNZ R3, DELAY3	2	61	$132.37\mu s$

Table 1: Calculation of the delay

² $\text{Fibonacci}(15) > 255_{10} \implies \text{carry} \implies \text{CY} = 1$

Homework #2

Illya Starikov

Due Date: September 15, 2016

1. Printer, Disk, Keyboard, Monitor.
2. Because Windows and Android both support the x86 architecture, there is a monopolization of the x86 architecture support — meaning millions of computers could run x86 compatible software. Other benefits could include more functionality (i.e. BIOS Bootloading) and computing power.
3. The ROM is where the program is stores; too large of a program could mean not being able to run the executable.
4. Power consumption.
5. Addressing modes are as follows:
 - (a) Directed, Indexed
 - (b) Indexed, Immediate
 - (c) Absolute
 - (d) Direct
 - (e) Relative
 - (f) Register, Relative
 - (g) Register
 - (h) Long
6. Addressing maps is as follows:

Address	Contents
0000	79
0001	A3
0002	74
0003	2D
0004	27
0005	04
0006	C4
0007	82

Homework #3

Illya Starikov

Due Date: September 22, 2016

Problem #1

- (a) Illegal (*ADD only applies to A.*)
- (b) Illegal (*255H is too large*)
- (c) Legal
- (d) Illegal (*Same reasoning as (a)*)
- (e) Legal
- (f) Illegal (*F5H will be treated as a label instead of a number.*)

Problem #2

```
MOV R4, #25H      ; Moves 37 into register 4.
MOV A, #1FH       ; Moves 31 into temporary register A
ADD A, R4         ; Adds register 4's value to register A's value
```

It's a simple add operation, resulting in the value of $37_{10} + 31_{10} = 68_{10} = 44_{16}$; it's kept in ROM.

Problem #3

```

ORG 0;
MOV R4, #0FH      ; Move 0F to register 4
MOV A, #0A2H      ; Move A2 to register A
ADD A, R4         ; Add R4 To A
MOV R7, A         ; Move the result to R7
END;

```

Address	Contents
0000	0F
0001	A2
0002	B1
0003	B1

Problem #4

```

ORG 0

MOV R0, #30H      ; Move the value 30H to R0
LOOP: MOV @R0, #0FFH ; Move FFH at R0
      INC R0       ; Increment R0
      CJNE R0, #40H, LOOP
END

```

Problem #5

```

ORG 0;

SETB PSW.3      ; Switch to register bank 3
MOV R0, #0A5H   ; Load A5 into register 0
PUSH 0          ; Push R0 onto stack

SETB PSW.0      ; Switch to register bank 4
POP 5           ; Pop onto register 5

END;

```

Problem #6

Assuming A is the register to store results,

```

ORG 0;

```

```

MOV R6, #5FH          ; Load 5F into R6
MOV R5, #01001001B    ; Load 01001001 into R5

/* Swap */
MOV A, R6;             ; temp (A) = R6
MOV 6, 5;              ; R6 = R5
MOV R5, A              ; R5 = Temp (A)

/* Add */
ADD A, R6;             ; A already had R6's values, just add original R5
                     ; Which got switched.. so actually add R6
ADD A, #20;

END;

```

Homework #4

Illya Starikov

Due Date: October 20th, 2016

Problem #1

```
ORG 0

GREEN BIT P2.0
YELLOW BIT P2.1
RED BIT P2.3

MOV P1, #0FFH ; Make an import port

INCT5:
    CPL RED
    ACALL DELAY5S
    CPL RED
    ACALL DELAY5S
    MOV R0, #5H ; Attempt counter
    MOV R2, #30H ; First correct numbers
    MOV R4, 3 ; For yellow light

    SETB C

INCT:
    MOV R1, #3H ; There are three numbers to get right
    JC SKIP ; Basically, if first time, skip the delay

    MOV A, R4
    JZ SKIP1 ; If 3 attempts have been used, yellow
    MOV R4, #0FFH ; To make sure you don't hit this twice
    CPL YELLOW
    ACALL DELAY5S
    CPL YELLOW
```

```

        SJMP SKIP2            ; Because we don't want two delays

SKIP1:  ACALL DELAY5S
SKIP2:  DEC R4

        MOV A, R0
        JZ INCT5              ; A = R0 = 0, then 5 attempts have been made

SKIP:   CLR C
        CRCT:
        MOV 3, @R1            ; R3 = @R1
        MOV A, P1            ; A = P1
        CJNE A, 1, INCT

        INC R2
        DJNZ R1, CRCT;
        SETB P0.0            ; If this far, it's correct

        CPL GREEN
        ACALL DELAY5S
        CPL GREEN
        CLR P0.1            ; Re-lock after five seconds

END

```

Problem #2

```

ORG 0

        SETB PSW.4
        SETB PSW.3            ; set to register bank 3

        MOV A, 7              ; A = B3,R7

        CJNE A, #64, NEXT    ; This is literally to set the flag, nothing else
NEXT:

        CLR PSW.4
        CLR PSW.3            ; set register bank to bank 0

```

```

/*
    R6 IS UPPER BYTE
    R5 IS LOWER BYTE
*/

```

```

MOV B, #100D
DIV AB

```

```

MOV R6, A
MOV R5, B

```

```

END

```

Problem #3

```

ORG 0
MOV R7, #0           ; 1's occurrence counter
MOV R0, #8           ; Loop counter
MOV R1, #30H         ; For writing to ROM
MOV R2, #0           ; Port counter, i.e. will be 0 when P0, 1 when P1

CLR C                ; Just in case

MOV P1, #0FFH        ; Make P1 an input port
MOV A, P1

/* Determine the location of 1s */
ILOOP: RRC A          ; Start scanning at LSB
      INC R2
      JNC SKIP        ; If it is 1, next immediate line will execute
      INC R7          ; Another occurrence of 1
      MOV @R1, 2       ; Rom[30H + i] = Port count
      INC R2
SKIP:  DJNZ R0, ILOOP

/*Output to P2 */
MOV P2, #0           ; Make P2 an output port
REPEAT:              ; For repeating output indefinetly
MOV R1, #30H         ; Again, to be used for output
MOV 3, 7             ; R3 = R7 (The number of occurrences)

```



```

        OLOOP:                ; For outputting a single time
        MOV P2, @R1;          ; P2 = @R1, the value of where the 1 occurred
        ACALL DELAY;
        INC R1;

        DJNZ R3, OLOOP;
        SJMP REPEAT;

DELAY:  MOV R7, #118           ; DJNZ = 2MC. (256/1.085069444) / 2 MC = 117.9648 MC
DLOOP:  DJNZ R7, DLOOP
        RET

        END

```

Problem #4

```

        ORG 0

        RESTART:
        SETB P1.1
        SETB P1.2
        SETB P1.3

        ACALL DELAY           ; 12.5%
        CLR P1.1

        ACALL DELAY           ; 25 %
        CLR P1.2

        ACALL DELAY           ; 37.5%
        ACALL DELAY           ; 50%
        CLR P1.3

        ACALL DELAY           ; 62.5
        ACALL DELAY           ; 75
        ACALL DELAY           ; 87.5
        ACALL DELAY           ; 100

        AJMP RESTART;

DELAY:  MOV R0, 5              ; 1khz = 100 us. So 12.5% = 12.5 us.

```

```
DELL:  DJNZ R0, DELL      ; So ~5 iterations
      RET

      END
```

Homework #5

Illya Starikov

Dude Date: November 1st, 2016

Problem #1

Letter	Ascii Hex
R	52_{16}
o	$6F_{16}$
l	$6C_{16}$
l	$6C_{16}$
a	61_{16}
Σ	$1FA_{16}$

Because we ignore carries, we consider FA_{16} .

$$\begin{aligned}FA_{16} &= 1111\ 1010_2 \\1's\ comp &= 0000\ 0101_2 \\2's\ comp &= 0000\ 0110_2 \\&= 06_{16}\end{aligned}$$

The checksum value is 06_{16} . Performing the checksum, we get $FA_{16} + 06_{16} = 100_{16}$. Because we ignore carries, the check is correct.

Problem #2

```
ORG 200H
STRING: DB 'Missouri S&T'
ORG 0H

REPEAT: MOV R0, #12D           ; Size of 'Missouri S&T'
        MOV DPTR, #STRING
        ACALL BLINK
```

```

LOOP:  MOV P1, #0H                ; P1 = output port
        CLR A
        MOVC A, @A+DPTR
        MOV P1, A                ; P1 = string
        INC DPTR                 ; DPTR++
        ACALL DELAY

        DJNZ R0, LOOP

        ACALL BLINK
        ACALL BLINK
        ACALL DELAY
        ACALL DELAY
        SJMP REPEAT

DELAY:  MOV R7, #118D             ; 256 / 1.085 = 235.9269 * 1/2 (djnz) = 118
HERE:   DJNZ R7, HERE
        RET

BLINK:  MOV P0, #0H              ; Make output port
        CLR P0.1                ; Low
        SETB P0.1               ; To High
        CLR P0.1                ; To low
        RET

        END

```

Problem #3

```

ORG 0H

/*
PORTS
R0: Loop counter
R1: Holds hundreds, tens, ones
R2: Hold the additive value
*/

MOV P1, #0FFH                ; P1 = Input Port
MOV P2, #0H                 ; P2 = Output Port
MOV P1, #32H

```

```

REPEAT: MOV R0, #3D          ; Number to read in
        MOV R1, #100D
        MOV R2, #0D

        CLR A
        CLR C

LOOP:   MOV A, P1
        ANL A, #0FH          ; By and-ing with 0F, upper bits are cleared.
                                ; This way, goes from ascii to a hex number

        MOV B, R1
        MUL AB                ; Multiply by hundreds, tens, one+

        ADD A, R2             ; Add to the values we have received so far
        MOV R2, A             ; R2 = Sum value

        ACALL SWAPAB
        JC BIG                ; if this additive value sum is > 255, Carry
        JNZ BIG               ; if there was something in b, Carry

        MOV A, R1             ; A = Tens, hundred, ones
        MOV B, #10D
        DIV AB                ; So now, we have values for tens, ones
        MOV R1, A

        ACALL DELAY1
        DJNZ R0, LOOP

        MOV P2, R2

        SJMP SKIP
BIG:    ACALL DELAY2
SKIP:   LJMP REPEAT

        ACALL REPEAT

DELAY1: MOV R7, #118D         ; 256 / 1.085 = 235.9269 * 1/2 (djnz) = 118
HERE1:  DJNZ R7, HERE1
        RET

```

```

DELAY2: MOV R6, #4D
HERE2:  ACALL DELAY1          ; 1.024 ms = 1024 us = 4*Delay1
        DJNZ R6, HERE2
        RET

```

```

SWAPAB: MOV R5, A
        MOV A, B
        MOV B, R5
        RET

```

```

END

```

Problem #4

```

        ORG 250H
MYDATA: DB 3, 9, 6, 9, 7, 6, 4, 2, 8
        ORG 0H

        ACALL R2R
        ACALL ADDV
        ACALL AVG

        ACALL DONE

R2R:    MOV R0, #30H
        MOV DPTR, #MYDATA
        MOV R1, #9D          ; Size of 'MYDATA'

LOOP1:  CLR A
        MOVC A, @A+DPTR
        MOV @R0, A

        INC R0
        INC DPTR
        DJNZ R1, LOOP1

        RET

```

```

ADDV:  MOV R0, #30H           ; Where values are stored
      MOV R1, #9D            ; Size of 'MYDATA'
      CLR A
LOOP2:  ADD A, @R0
      INC R0

      DJNZ R1, LOOP2

      MOV 70H, A
      RET

AVG:    MOV R0, #30H           ; Where values are stored
      MOV R1, #0D            ; Counter
      MOV R2, #0D            ; Sum

      CLR A
      /*
      So.. not entirely sure how to do this part, so here's my guess..

      I don't write to ROM anywhere past 30H, so it should theoretically be 0
      So keep reading and adding until I hit an 0 in RAM
      */
LOOP3:  MOV A, @R0
      JZ LDONE

      ADD A, R2
      MOV R2, A

      INC R0
      INC R1
      SJMP LOOP3

LDONE:  MOV B, R1
      MOV A, R2
      DIV AB

      MOV R7, A

      RET

DONE:   NOP

```

END

Problem #5

```
ORG 0

/*
R0 = x

R1 = 2x^2
R2 = 3x
R3 = 1
*/

MOV P1, #0FFH      ; P1 = input port
MOV P2, #0H        ; P2 = output port

MOV R0, P2          ; R0 = P2

CLR C              ; CY = 0
CJNE R0, #4D, NEXT
NEXT: JC SKIP       ; if CY = 0, outside of range (by CJNE)
MOV P2, #0H        ; R2 = 0, if outside of 0 <= x <= 3
ACALL DONE         ; Finish

SKIP: /* 2x^2 */

MOV A, R0           ; A = x
MOV B, R0           ; B = x
MUL AB             ; A = x^2, but the lower value. However,
                  ; x will always be in the lower bit, as in A

MOV B, #2D
MUL AB             ; A = 2x^2
MOV R1, A           ; R1 = 2x^2

/* 3x */
MOV A, R0           ; A = x
MOV B, #3D          ; B = 3
MUL AB             ; A = 3x

MOV R2, A           ; R2 = 3x
```



```

/* 1
  I know this is kinda wasting MC, but
  it's easier to manipulate the equation this way
  */
MOV R3, #1D      ; R3 = 1

MOV A, R1
ADD A, R2
ADD A, R3          ; A = 2x^2 + 3x + 1

MOV P2, A          ; P2 = 2x^2 + 3x + 1

DONE:  NOP
      END

```

Homework #6

Illya Starikov

Due Date: November 3rd, 2016

Problem #1

```
#include <stdio.h>
#include <reg51.h>

unsigned char input();
void delay(int time) {} // To prevent compiler errors
int power(int base, int exponent) reentrant;

typedef enum { false, true } bool;

int main() {
    unsigned char input, i;
    unsigned char sum, asciiInput;

    P2 = 0x00; // output port
    P1 = 0xFF; // Input Port

    do {
        sum = 0;
        for (i = 3; i > 0 && sum < 255; i--) {
            asciiInput = input();
            input = asciiInput & 0x0F; // convert to HEX

            sum += input * power(10, i - 1); // during execution: 100*input ->
                                            // 10*input -> 1*input

            delay(256);
        }

        if (sum <= 255) {
```

```

        P2 = sum;
    } else {
        CY = true;
        delay(1024);
    }
} while (true);

return 0;
}

int power(int base, int exponent) reentrant {
    if (exponent == 0) {
        return 1;
    }

    return base * power(base, exponent - 1);
}

unsigned char input() {
    return P1;
}

```

Problem #2

```

#include <stdio.h>
#include <reg51.h>

typedef enum { false, true } bool;
sbit readFromROM = P2^1;

unsigned char fibonacci(const unsigned char number) reentrant;
unsigned char get_input();

void delay(int microsecond) {} // To prevent compiler errors

int main() {
    code unsigned char sequence[] = { 0, 1, 1, 2, 3, 5, 8, 13, 21,
                                      34, 55, 89, 144, 233, 377 };

    unsigned char input, i;

    P3 = 0x00; // Output port

```

```

P1 = 0x0FF; // Input port

input = get_input();

for (i = 0; i < input; i++) {
    if (readFromROM) {
        P3 = sequence[i];
    } else {
        P3 = fibonacci(i);
    }

    delay(27250);
}

if (i > 14) {
    CY = true;
}

return 0;
}

unsigned char fibonacci(const unsigned char number) reentrant {
    if (number == 0) {
        return 0;
    } else if (number == 1) {
        return 1;
    }

    return fibonacci(number - 1) + fibonacci(number - 2);
}

unsigned char get_input() {
    unsigned char input;

    do {
        input = P1 & 0x0F; // to clear upper four bits
    } while (input == 0);

    return input;
}

```

Problem #3

```
char swap_bits(const char byte) {
    char low = byte << 4,
    high = byte >> 4;

    return low | high;
}
```

Problem #4

```
#include <stdio.h>
#include <reg51.h>

unsigned char getInput();
void output(const char low, const char high);

int main() {
    unsigned char input = getInput();
    unsigned char low, high;

    P0 = 0xFF; // input port

    if (input <= 99) {
        high = (input & 0xF0) >> 4; // The clears the low bits,
        // but it's still the tens place. shift bits to make make it a ones place
        low = input & 0x0F;
        output(low, high);
    } else {
        P0 = 0xFF;
        P1 = 0xFF;
    }

    return 0;
}

unsigned char getInput() {
    return P0;
}

void output(const char low, const char high) {
    unsigned char asciiLow = 0x30 | low,
```

```

    asciiHigh = 0x30 | high;

    P1 = asciiLow;
    P2 = asciiHigh;
}

```

Problem #5

```

#include <stdio.h>
#include <reg51.h>

int power(int base, int exponent) reentrant;
unsigned char input();
unsigned char function(const unsigned char x);

int main() {
    unsigned char x;

    P1 = 0xFF; // input port
    x = input();

    if (x >= 0 && x <= 3) {
        P2 = function(x);
    } else {
        P2 = 0x00;
    }

    return 0;
}

int power(int base, int exponent) reentrant {
    if (exponent == 0) {
        return 1;
    }

    return base * power(base, exponent - 1);
}

unsigned char input() {
    return P1;
}

```

```
unsigned char function(const unsigned char x) {  
    return 2*power(x, 2) + 3*x + 1;  
}
```

Homework #7

Illya Starikov

Due Date: November 10th, 2016

Problem #1

$$(65536 - x) \times 1.085 \mu\text{s} = 5 \text{ ms} = 5000 \mu\text{s} \quad (1)$$

$$= 60928_{10} \quad (2)$$

$$= EE00_{16} \quad (3)$$

Therefore, TH1 = EE_{16} and TL1 = 00_{16}

Problem #2

Assuming Problem #1 is correct,

```
ORG 0H

MOV R0, #100D
LOOP:  MOV TMOD, #01D
        MOV TH0, #EEH
        MOV TLO, #00H    ; (FFFF - EE00 + 1)*1.085 = 5ms
        ACALL DELAY
        DJNZ R0, LOOP    ; 100*5ms = 500 ms

        AJMP DONE

DELAY:  SETB TR0
LOOP2:  JNB TF0, LOOP2
        CLR TR0
        CLR TF0
        RET

DONE:   NOP
        END
```


Problem #3

5 kHz = 500 μ s. So,

$$TH1 = EE_{16} \quad \text{and} \quad TL1 = 00_{16} \quad (4)$$

The code is as follows,

```
ORG 0H

MOV R0, #100D
CLR P0.0

LOOP:  MOV TMOD, #01D
        MOV TH0, #EEH
        MOV TLO, #00H    ; (FFFF - EE00 + 1)*1.085 = 5ms
        ACALL DELAY

        CJNE R0, #75D, SKIP
        SETB P0.0

SKIP:   DJNZ R0, LOOP    ; 100*5ms = 500 ms

        AJMP DONE

DELAY:  SETB TR0
LOOP2:  JNB TF0, LOOP2
        CLR TR0
        CLR TF0
        RET

DONE:   NOP
        END
```

Problem #4

Because we know frequency $(f) = \frac{1}{\text{period}(T)}$, $\lfloor f \rfloor \implies \lceil T \rceil, \{\forall f, T \in \mathbb{R}^+ \mid f \neq 0, T \neq 0\}$. Using this, we find the smallest period to be,

$$(FFFF_{16} - FFFF_{16} + 1) \cdot 1.085 \mu\text{s} = 1.085 \mu\text{s} \quad (5)$$

From this, we use $f = \frac{1}{T} = \frac{1}{1.085} = 0.92 \mu\text{Hz}$. We do a similar process for lowest frequency.

```

        ORG 0H
        CLR P1.3

LOOP:    MOV TMOD, #01D
        MOV TH0, #0D    ; lowest frequency => highest period
        MOV TL0, #0D    ; highest period => Smallest TH, TL
        SETB P1.3
        ACALL DELAY

        CLR P1.3
        AJMP DONE

DELAY:   SETB TR0
LOOP2:   JNB TF0, LOOP2
        CLR TR0
        CLR TF0
        RET

DONE:    NOP
        END

```

Problem #5

Part A

$$\begin{aligned}
 2C_{16} + 11_{16} &\implies 2C11_{16} & (6) \\
 FFFF_{16} - 2C11_{16} + 1_{16} &= D3EF_{16} & (7) \\
 &= 54254_{10} & (8) \\
 &\implies 58\,865.59\,\mu\text{s} & (9) \\
 &= 58.87\,\text{ms} & (10)
 \end{aligned}$$

Part B

$$\begin{aligned}
 10010011_2 &\implies 93_{16} \implies 147_{10} & (11) \\
 (65536 - 147) \times 1.085 &= 70\,947.07\,\mu\text{s} & (12) \\
 &= 70.95\,\text{ms} & (13)
 \end{aligned}$$

Homework 8

Illya Starikov

Due Date: December 1st, 2016

This homework will be **extra credit**.

Problem 1

```
1  #include <reg51.h>
2
3  typedef enum { false, true } bool;
4  unsigned char R0;
5
6  void main() {
7      TMOD = 0x06; // Counter 2, Mode 2
8      TH0 = -0x60; // Initial value given
9
10     R0 = 0; // Just in case
11     TR0 = true; // Start the count
12
13     do {
14         if (TF0) {
15             R0++;
16             TF0 = false; // continue the count
17         }
18     } while (R0 != 60);
19
20     TR0 = false; // Stop the count
21 }
```

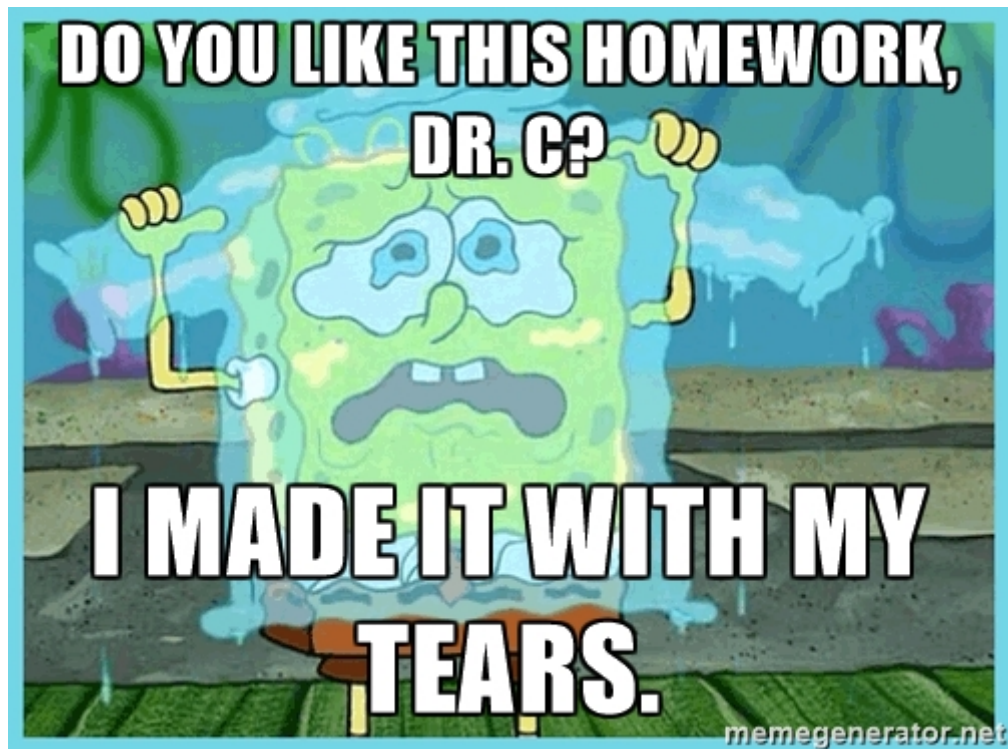
Problem 2

```
1      ORG 0H
2
3      LJMP MAIN
4
5      /* Timer 1 Interrupt */
6      ORG 001BH
```

```

7      CPL P1.1
8      RETI
9
10     /* Main */
11     ORG 0030H
12 MAIN: MOV P2, _num_00H      ; P2 = output port
13     MOV TMOD, _num_20H      ; Timer 1, mode 2
14     MOV TH1, #-37D          ; 5 kHz => 25000 Hz => 4^-5 s
15                               ; => 40 us => 40/1.085 ticks => 37 ticks
16     MOV IE, _num_88H
17     SETB TR1
18
19 LOOP: INC R0
20     MOV P2, R0              ; P2 = R0
21     ACALL DELAY
22     SJMP LOOP
23
24 ; so we need to generate a 10.85us delay
25 ; 10.85/1.085 = 10MC. Easier to just do a brute force
26 DELAY: MOV R1, _num_4D      ; 2 MC
27 HERE: DJNZ R1, HERE         ; 4 * 2 = 8 MC
28     RET                     ; 8 + 2 = 10 MC
29
30
31     END

```

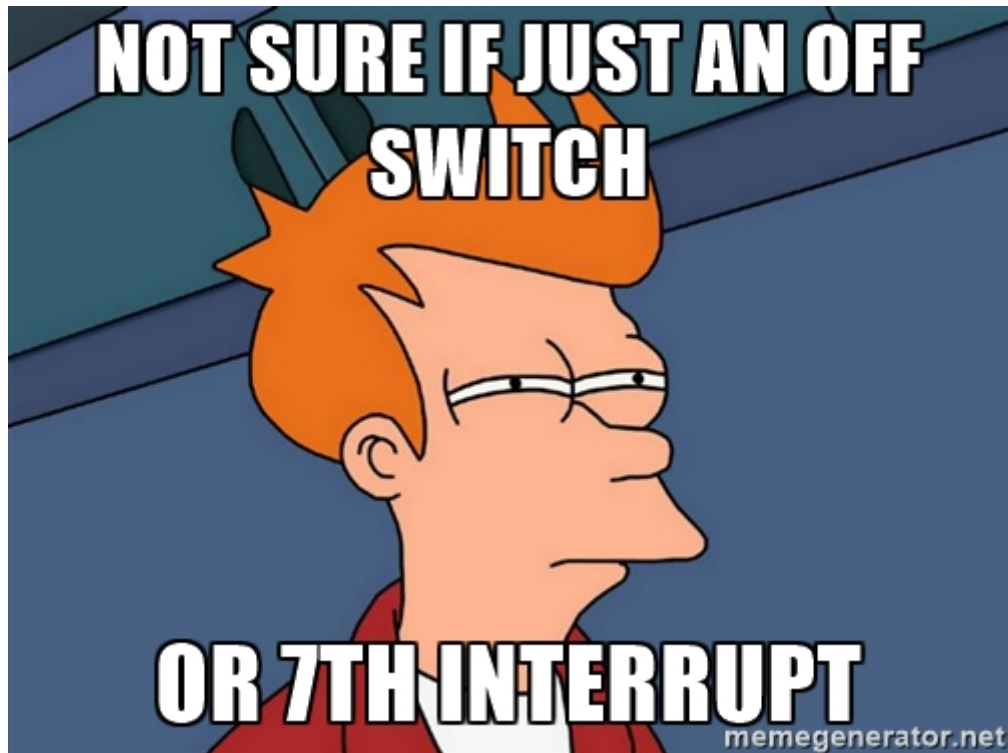


Problem 3

```
1      ORG 0
2      LJMP MAIN
3
4      /* Timer 0 Interrupt */
5      ORG 000B
6      CPL P1.1
7      RETI
8
9      /* Timer 1 Interrupt */
10     ORG 001B
11     CPL P1.2
12     RETI
13
14     /* Main */
15     ORG 30H
16 MAIN: MOV TMOD, _num_12H      ; T0M2 & T1M1
17     MOV P1, _num_0           ; P1 = output port
18     ACALL WAVE0
19     ACALL WAVE1
20
21     MOV IE, _num_8AH
22 REPEAT: SJMP REPEAT
23
24 ; 5kHz => 1/5000 s = .0002 s => 200 microseconds
25 ; 200 us => 184 ticks PER PERIOD. Assuming a 50%
26 ; duty cycle. So actual value is 184/2 = 92 ticks
27 WAVE0: MOV TH0, #-92
28     SETB TR0
29     RET
30
31 ; 500 Hz => .002 s => 2000 us
32 ; (FFFF - x + 1)(1.085) = 2000
33 ; x = FFFF - 1842 => E7 BD
34 WAVE1: MOV TH1, _num_0E7H
35     MOV TL1, _num_0BDH
36     SETB TR1
37     RET
38
39
40     END
```

Problem 4

```
1      ORG 0H
2      LJMP MAIN
3
4      /* Interrupts */
```

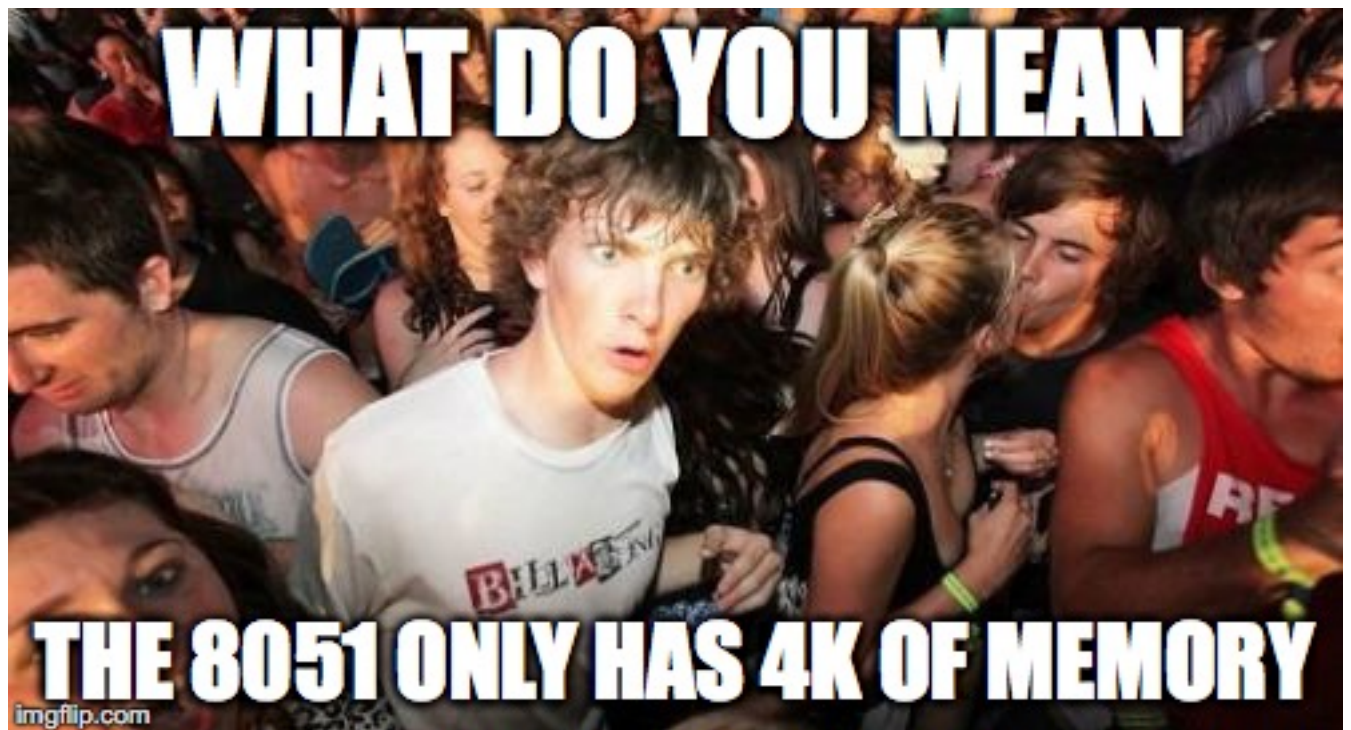


```
5      ORG 001BH
6      LJMP SWAVE
7      RETI
8
9      /* Main */
10     ORG 30H
11 MAIN:  MOV R7, _num_1D
12         MOV P1, _num_0           ; Make P1 an output port
13         MOV IE, _num_88H        ; Enable interrupt on T1
14
15 LOOP:  MOV P2, R7                ; R7 holds the value of the div series
16         ACALL INFSER
17         SJMP LOOP
18
19 ; code for the square wave interrupt
20 ; determines via R6 what the last wave was
21 SWAVE:  MOV TMOD, _num_20H      ; Timer 2, mode 2
22
23         CJNE R6, _num_70, SKIP2LOW
24         ACALL HWAVE
25         SJMP NEXT
26
27 SKIP2LOW:
28         ACALL LWAVE
29
30 NEXT:  SETB TR1
```

```

31         RET
32
33 ; the wave has a duty cycle of 70%
34 ; so the high wave needs a high wave of 7kHz
35 ; & kHz => 7000 Hz => 1.42^-4 s => 142 us => 130 ticks
36 ; 130.8 *.7 = 92 ticks for 70% duty cycle of a 7kHz wave
37 HWAVE:
38     SETB P1.1
39     MOV TH1, #-92
40     MOV R6, _num_70                ; Just to keep track of what our last
        value was
41     RET
42
43 ; so, exact same reasoning as before, we need 130 ticks for
44 ; the period of the wave, but now we just subtract 92 from 130
45 ; so, 130 - 92 = 38
46 LWAVE:
47     CLR P1.1
48     MOV TH1, #-38
49     MOV R6, _num_30
50     RET
51
52 ; computes infinite divergent series incrementally
53 ; uses R7
54 INFSER:
55     MOV A, R7
56     ACALL TWOCOMP
57     ACALL ISNEGATIVE
58
59     JC RETURN
60     INC A
61
62 RETURN: MOV R7, A
63     RET
64
65
66 ; returns two's compliment of A
67 TWOCOMP:
68     CPL A
69     ADD A, _num_1D
70     RET
71
72
73 ; returns 1 in C if B is negative
74 ISNEGATIVE:
75     CJNE A, _num_10000000B, NEXT2
76 NEXT2:  CPL C
77     RET
78
79
80     END

```



Paradigms and Applications of the Arduino Uno

Computer Engineering 3150 Research Paper

Illya Starikov

1 Introduction

The Arduino is an open-sourced family of micro-controllers aimed at not just the specialist, but also the hobbyist or enthusiast. With a multi-platform¹ IDE (Integrated Development Environment) provided and maintained by Arduino, code can be written and deployed within seconds, making it easy for anyone to get started. A “hello world”-esque program (cause an led light to flash on and off) can be implemented in only 12 lines of code² [1].

```
int ledPin = 10;

void setup() {
    pinMode(ledPin, OUTPUT);
}

void loop() {
    digitalWrite(ledPin, HIGH);
    delay(1000);
    digitalWrite(ledPin, LOW);
    delay(1000);
}
```

Anyone familiar with C or C++ can get started by briefly skimming documentation. However, the flexibility of and power of Arduino’s microcontrollers is why professionals use it.

What makes Arduino so powerful is its ease of expandability, a process known as shielding. Such shields can be provide additional functionality, such as: [2]

- Bluetooth
- Internet connectivity via WiFi or Ethernet
- Sound and speech synthesizer
- Allowing a stack of Arduino boards to communicate

This readily available power of any other board makes Arduino ideal for any project. And if the functionality is not there, producing one is fairly simple — design a PCB (print circuit board) and mount it.

As mentioned prior, Arduino open sources everything; by everything, this covers to code, schematics, and designs. Because of the open source license (Creative Commons Attribution Share-Alike), it is possible to entirely replicate the board³. This is actually a common occurrence, giving rise to Arduino “clones”, such as the Aliexpress and Teensy — of course, the Arduino Uno (the topic of this paper) is also a “clone”⁴ of ATmega328.

1.1 The Arduino Uno

The Arduino Uno is an 8-bit microcontroller with 32 kB of flash storage and 2 kB of RAM (Random Access Memory). Because it operates at 5 V, it can be charged through the provided power jack or a typical 5 V charging block (e.g. a typical phone charger), or a 7 V - 12 V input pin.⁵ As for as analog controls and peripherals, the Uno has “14 digital input/output pins (of which 6 can be used as PWM outputs), 6 analog inputs, a 16 MHz ceramic resonator, a USB connection, a power jack, an ICSP header, and a reset button” [3]. Another great feature is the price: \$29.95.

Because of its open-sourcing and given the 6 years to mature [5], the Uno has gather quite a community on the internet. The forums, hosted on the Arduino website, are active with some threads reaching up to 100 000 posts. Topics include [6]:

- Robotics (“Platforms, motors, algorithms and sensors to conform robotics projects”)

³Excluding the name Arduino or Uno

⁴Based on the technology of the ATmega328.

⁵However, the pin “bypasses the regulator, and can damage your board.” It is not advised charging anywhere out of the recommended range

¹Supported on MAC OS X, Linux, and Windows.

²Excluding `int main()` and preprocessor directives.



Figure 1: The Arduino Uno Microcontroller [4].

- Home Automation and Networked Objects (Internet of Things)
- E-Textile and Craft (“flexible conductive materials, electroluminescent materials”)
- Device Hacking (“reappropriation of devices, tricks of the trade”)

2 Applications of The Arduino Uno

The Arduino Uno’s flexibility makes it pragmatic to be used in a wide array of industries; three of these industries are Education, Healthcare, and Home Automation.

2.1 Education

In the education market, there are many factors that are needed to be accounted for in adopting new technology, and the Arduino Uno usually accommodates for these factors, typically making it an ideal candidate. The two most important factors are affordability and ease of use.

When considering purchasing equipment, price is the most important element. Students need to have the ability to experiment with technology without fear of breaking it; not only that, but there is maintenance, replacement and installation costs associated with it. Fortunately, the Uno fits this niche. As aforementioned, the Uno is competitively at \$29.95. The Arduino Starter Kit, which not only

contains the Uno, but many sample projects that gives a hand-on experience with it, costs \$99.99. Lastly, the multiple shields that can be purchased range from \$1.00 to little over \$100, but most usually lower than \$40.

For such a competitive price, the Uno still manages to have a lot of thought and consideration with regards to the user. The cross-platform IDE ensures frictionless setup; programming the microcontroller can begin immediately after installation. The online documentation and forums are house enough information to get started with any kind of project. This makes the Arduino Uno the ideal microcontroller to get started with.

This was the mentality behind a team of researchers from Federal University of Rio Grande do Sul Porto Alegre-Rs and Criciuma-Sc from Brazil. In a recent attempt to further engage electrical engineering students, the group built a “remote educational experiment of a current motor (DC motor) control speed, which allow the students to interact with the control systems P, PI, PID applied to the rotation control of the remote mode engine” [7].

Instead of being constrained to the labs, where traditional equipment is kept, the researchers moved the lab online where it is accessible from anywhere at anytime. The user can, through an online Java application, access the hardware in the lab. The user interaction is still natural, because the application is tethered to a camera that sends a continuous live video that will be streamed to the user’s browser. The user can send live commands to the device, and get instantaneous feedback. As the user is interacting with the application, the commands are being sent to the Arduino microcontroller, which is not only controlling the engine, but is acting as a PID control machine.

Figure 2: The Java applet, with the livestream on the left and PID controls on the right [7].

A PID controller (proportional-integral-derivative) is a self-correcting feedback mechanism that asymptotically approaches a stable point, also referred to as a setpoint. The error correcting equation, as a function of time, is an integro-differential equation as such:

$$u(t) = k_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{d e(t)}{dt}. \quad (1)$$

It reads “as the error tolerance at a set time $u(t)$ is calculated by the current error ($k_p e(t)$) with the potential error that needs to be considered ($K_i \int e(\tau) d\tau$), while taking away the excess error ($K_d \frac{d e(t)}{dt}$)”. As this function changes with respect to time, the system (in this case, the DC motor) calculates the error and asymptotically reaches a setpoint. In this case, the PID controller tries to regulate the voltage and bring the engine to a steady state. And the Uno does all of the calculations in real time and lets the user see at a glance what the each state (P, I, or D) is at, and what the setpoint is.

And at the end of the session, a majority of the participants enjoyed the project. The original goal of further engaging students outside of the classroom was achieved; the project was an overwhelming success.

Experiment contributed to better understanding the concepts developed in the discipline of control systems?			
Excellent 43%	Good 36%	Regular 11%	Bad 10%
What is your impression about developing experiments remote related to control systems?			
Excellent 40%	Good 38%	Regular 14%	Bad 8%
In your opinion greater learning control systems in the development of activities linked to experiments remote or simulations?			
Simulation 5%	Real experiments 27%	Both 44%	Combination 24%

Table 1: Satisfaction of the PID controller project [7].

However, this is only one of the many interesting projects — all of which don’t have to be a strict application to Electrical Engineering. A group of researches from IBM decided to target a younger age-group: 9th–12th grade students. Seeing as this was the age group that had a rapid loss of interest in STEM (Science, Technology, Engineering, and

Mathematics) based fields, the IBM group realized this was a delicate experiment. The experiments could not be too technical, because of the lack of knowledge and experience in high schools, but could not be mundane; all of this had to be considered while trying to teach students the basics of electronics and programming.

The students received hands-on experience by doing a simplified experiment: programming a blinking LED light; and then further expanding this concept by placing the LED lights to a bag and making text light up. The researches said the students “were very engaged to and excited about open source, which is affordable and accessible. The students conducted the experiments with no major issues, except the lack of time” [8].

After the experiment, there were presentations of the more advanced things that could be done. Some of these were: a Twitter client that projects negative or positive feelings in regards to a debated topic, a toy car that could be controlled with a PlayStation controller, and even a quadcopter. All of these demonstrations were powered with the Arduino Uno and a couple of shields.

The IBM group is continuing to expand this project in the coming years and hoping to educate more high school students about the power of programming, electronics, and open source.

2.2 Healthcare

As previously mentioned, the Arduino Uno is based on the ATmega328P, the high performance microcontroller from Atmel. However, performance is not exclusively what the team at Arduino focused on; they weighed performance with reliability. This blend of high performance and reliability makes the Arduino Uno a suitable microcontroller to be used in healthcare. Two graduate students at the Institute of Biophysics and Biomedical Engineering in Portugal have found an interesting use for it: hand rehabilitation therapy.

Because modern, technological approaches to hand rehabilitation therapy have been proven to be quite expensive and needed heavy technical experience and support, the team decided to make an affordable and simple system that can be used at the comfort of your own home. It’s called a “hand

therapist”.

The team started with Unity3D, a popular game engine, to simulate an environment on a personal computer. The environment is essentially a canvas with a working hand that be controlled with a series of commands in the *xyz*-plane, making it easy to wire up a working hand to a microcontroller to interpret said commands; and that’s exactly what the team did.

The Uno interprets the movement and synchronously plots the movement on the computer. Then the user can “grab, hold, transport and drop a cube in several increasingly difficult puzzle levels”. Not only does this make hand therapy much easier, but more affordable (less than \$600) [9].

Figure 3: The rehabilitation system [9].

But complicated hand therapy is not the only application for the Uno in healthcare. Another research team in India have managed to hook up a microcontroller to monitor patient heartrate through the course of the day with an Uno and a bright LED light and transmit the information to the doctor at a fixed schedule, making it easy to regularly check up on patients [10].

Thanks to the Uno, not only is healthcare becoming more reliable, but less expensive too.

2.3 Home Automation

Today’s technology is significantly more advanced then ever; devices are getting smarter, better at communicating, and even more efficient. Something that came out of this “technological revolution” is the idea of a smarthome — powered by the IoT (Internet of Things). A smarthome is nothing more than a home “that constitutes advanced automation systems to provide the occupants with sophisticated monitoring and controlling over the different functions of the building” [11].

Home automation is a very broad topic, ranging from things such as temperature monitoring to motion tracking to light automation; fortunately, the Arduino Uno can do all of the above. Researchers from MIT managed to mount input/output blocks consisting of “PIR (Passive Infra-Red) motion sensor, LDR (Light Dependent Resistor) and an LM35

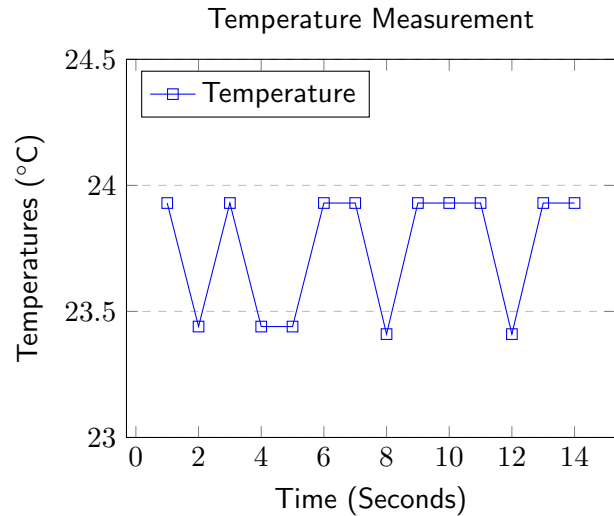


Figure 4: The system usually keeps the room within a 1°C.

temperature sensor as inputs and some lamps” and get the blocks communicating.

The PIR motion sense was to detect motion; not only could it turn lights on/off (via the LDR) if anyone was present, but it could act as a security system as well. If there was any suspicious movement during irregular hours, an alarm is buzzed. This buzzing acts as a warning system to any intruders or potential wild animals. The LM35 is not only to monitor temperature, but to enable and Air Conditioning or fans when a certain temperature threshold has been reached.

What makes these sensor different than switches on the wall or a regular AC controller is these can be monitored, synchronously, virtually. The current front-end is a MATLAB GUI (Graphical User Interface). Instead of having to traverse the typical household, the entire system is a few clicks away. This makes the entire household not only systematic and tidy, but more cost and energy efficient; and because it runs on a 5 volt Arduino Uno, it is using significantly less power than the AC unit, lighting, and security systems⁶.

⁶However, the energy efficiency should be traded for one’s safety.

3 Conclusion

As laid out previously, the Arduino Uno has incredible uses in three major fields: Education, Healthcare, and Home Automation. The Uno's ease of use makes it an ideal electronic to get interest in Engineering-based fields, while its price makes a candidate to be used in the classroom without fear of breaking hardware. The reliability and processing power makes it a great aid in healthcare, such as "hand therapy" or monitoring a patient status. Also, the Uno is great at home automation, where it can simultaneously monitor movement, temperature, and lighting based on room occupancy. Looking forward, one can surely expect many more great microcontrollers from Arduino that will get be in many, interesting ways.

References

- [1] M. McRoberts, *Beginning Arduino*.
- [2] "Arduino - ArduinoShields", *Arduino.cc*, 2016. [Online]. Available: <https://www.arduino.cc/en/Main/arduinoShields>. [Accessed: 11- Sep- 2016].
- [3] "Arduino - ArduinoBoardUno", *Arduino.cc*, 2016. [Online]. Available: <https://www.arduino.cc/en/Main/ArduinoBoardUno>. [Accessed: 11- Sep- 2016].
- [4] "Arduino Uno", *Wikimedia*, 2016. [Online]. Available: https://upload.wikimedia.org/wikipedia/commons/9/90/Barbone_Arduino_Uno.jpg. [Accessed: 11- Sep- 2016].
- [5] M. Banz, "Arduino Blog – Dinner is Ready", *Blog.arduino.cc*, 2010. [Online]. Available: <https://blog.arduino.cc/2010/09/24/dinner-is-ready/>. [Accessed: 11- Sep- 2016].
- [6] "Arduino Forum - Index", *Forum.arduino.cc*, 2016. [Online]. Available: <http://forum.arduino.cc>. [Accessed: 11- Sep- 2016].
- [7] J. M. Neto, S. Paladini, C. E. Pereira and R. Marcelino, "Remote educational experiment applied to electrical engineering," *Remote Engineering and Virtual Instrumentation (REV)*, 2012 9th International Conference on, Bilbao, 2012, pp. 1-5.
- [8] L. M. Herger and M. Bodarky, "Engaging students with open source technologies and Arduino," *Integrated STEM Education Conference (ISEC)*, 2015 IEEE, Princeton, NJ, 2015, pp. 27-32.
- [9] R. Lipovský and H. A. Ferreira, "Hand therapist: A rehabilitation approach based on wearable technology and video gaming," *Bioengineering (ENBENG)*, 2015 IEEE 4th Portuguese Meeting on, Porto, 2015, pp. 1-2.
- [10] P. A. Pawar, "Heart rate monitoring system using IR base sensor & Arduino Uno," *IT in Business, Industry and Government (CSIBIG)*, 2014 Conference on, Indore, 2014, pp. 1-3.
- [11] A. R. Behera, J. Devi and D. S. Mishra, "A comparative study and implementation of real time home automation system," *2015 International Conference on Energy Systems and Applications*, Pune, 2015, pp. 28-33.

CS1001

Data Structures Lab



Lab #1

Illya Starikov

July 27, 2025

1 Output Redirection

1. foo
2. bar. I conclude that it is echoing the variable (foo and bar) into the file.txt.
3. `bar`
`baz`
4. `»`is prepending a newline.

2 ls and Friends

1. Outputs all directories (hidden and public) as with the subdirectories. `*` matches all directories and subdirectories.
2. It outputs all super directories, `.*` matches anything with that in the name.
3. It will annihilate all super directories, including your home directory.

3 Intermediate man usage

1. Shows the manual for the `passwd` file.
2. `man 5 passwd`, a lot of tinkering.
3. `$Home`, `$SHELL`, `$PATH`, `$LOGNAME`, `$Mail`

4 A series of tubes

1. Done.
2. `cat My Cow Story | foo` (The Build File)
3. Bash error.

5 Submitting your work

1. Done.
2. `zip -r Lab\ #2 Lab\ #2/`
3. Done.

1 That filter problem, AGAIN

1. `grep \# -v story-plain.txt`
2. `grep -v "[\#]" story-space.txt`

2 Lk m, n vwls!

1. `sed s/[aeuio]//gi story-plain.txt`

3 Counting Lines

1. `grep -P "\d{4}-01" numbers.log | wc -l`

4 Phone numbers

1. `grep -Po "([?d{3}[]\s-]?d{3}[\s-]?d{4}" phonebook.txt`

Debuggers

Illya Starikov

March 14, 2016

1 Segfaults

1. `List::append`
2. Passed a `nullptr`. It's fixed!

2 Loopy

1. `Print`
2. They repeat. It's a looped List!
3. Fixed.

3 Math is hard

1. Fixed!

Lab #9

Illya Starikov

March 21, 2016

1 Drip, Drip, Drip, Drip

1. Memory leak, indirectly lost: 80 bytes in 5 blocks and definitely lost: 16 bytes in 1 blocks
2. Because a pointer to a list is being created, but not freed.
3. Done!

2 Imagine the possibilities!

1. possibly lost, still reachable, suppressed.
2. I was.. I knew it.
3. There is a destructor in `dtor` and `plain` only has the default (which does not take care of manual memory allocation).

3 Use wisely

1. Memory Leak.
2. You have a memory address assigned to an memory address.
3. Done.

Project #1

Illya Starikov

Due Date: March 25, 2016

0 Introduction

- Install 2 editor plugins
- Document 5 editor features (with vim, this'll be a cinch)
- Do the job control section of project B (I use this all the time)
- What does the -exec option on find do?
- What does xargs do?
- Look up and use tmux.

1 Two Editor Plugins

For two text editors plugins, I chose (facetiously) pathogen and powerline.

1.1 pathogen.vim

Naturally when I knew I was going to install packages, my first thought was to use a package manager. Easily manipulating and deleting new packages was going to be critical, so Pathogen looked to be the proper choice.

1.1.1 What It Does

In its simplest terms, pathogen makes package management as easy as `rm -r`. My main use has been mostly to easily locate and delete new plugins that I do not use in my `bundle` directory.

1.1.2 How To Use It

After **initial setup**, pathogen actually does the rest.

1.2 Powerline

While on my quest to find text editor plugins, I stumbled upon and installed powerline. While trying to think of my second plugin to write in this report, I couldn't help but notice how much I was using Powerline.

1.2.1 What It Does

Powerline is an alternative to the standard Vim statusline.

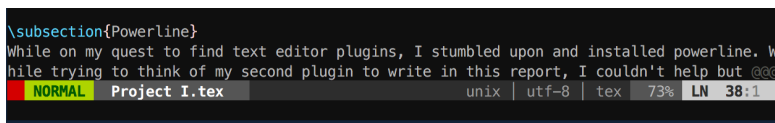


Figure 1: The Powerline In Action

It changes the configuration based on the mode that you are.

1.2.2 How To Use It

Use of powerline is automatic.

2 Five Editor Features

For my five editor features, I decided to do:

External commands (`(r ! Command)`) To run an external command outside of vim, and pipe the input back into the file. Makes inserting the date into LaTeX documents much, much faster.

Searching (`(/ ?)`) `Command F` for Vim.

Go to line (`:number`) Jumping to a specified line number.

Autocomplete (`(control + n)`) Attempts to finish the work being typed (whether it be a command or variable)

Text folding (`(z + f)a`)

3 Job Control

1. It terminates.
2. It terminates the command from the background.
3. Waits until all previous processes have finished until continueing.

4 Find `exec` ution

According to `Man`,

```
-exec utility [argument ...] ;
    True if the program named utility returns a zero value as its exit
    status. Optional arguments may be passed to the utility. The
    expression must be terminated by a semicolon (';'). If you
    invoke find from a shell you may need to quote the semicolon if the
    shell would otherwise treat it as a control operator. If the
    string ``{}`` appears anywhere in the utility name or the arguments
    it is replaced by the pathname of the current file. Utility will
    be executed from the directory from which find was executed.
    Utility and arguments are not subject to the further expansion of
    shell patterns and constructs.
```

Essentially `exec` allows the `exec` ution of certain programs during a command. For example, to make the use relevant to `find`, searching for an instance of a keyword in open files would `find . -exec grep KEYWORD +.`

5 *Terminal Multiplexor*

Finished, used for the job controls section (SSH to tmux to logout).

6 `Xargs`

Back to `Man`,

The `xargs` utility reads space, tab, newline and end-of-file delimited strings from the standard input and executes utility with the strings as arguments.

Any arguments specified on the command line are given to utility upon each invocation, followed by some number of the arguments read from the standard input of `xargs`. The utility is repeatedly executed until standard input is exhausted.

Spaces, tabs and newlines may be embedded in arguments using single ('` ' ') or double ('`"`) quotes or backslashes ('`\\`'). Single quotes escape all non-single quote characters, excluding newlines, up to the matching single quote. Double quotes escape all non-double quote characters, excluding newlines, up to the matching double quote. Any single character, including newlines, may be escaped by a backslash.

`xargs` allow for the manipulation of standard input, becoming quite useful with commands such as `find` or `ls`.

7 Conclusion

In conclusion, I have gained more familiarity with the command line. To take the assignment another step up, the entirety of this assignment has been created using the command line — Vim, `pdflatex`, `rm` for removing of $\text{\LaTeX}$ files, etc.

CS1200

Discrete Math



COMP SCI 1200 Section B FS2015 Assignment 4

Illya Starikov

November 8, 2015

To compile: Run a standard makefile:

```
fg++ *.cpp -o foo
```

Notes:

- Any two int are acceptable (within the scope of the primitive int).
- Negative integers are also accepted, but they will be converted to positive.

COMP SCI 1200 Section B FS2015 Assignment 5

Illya Starikov

November 17, 2015

To compile: Run a standard makefile:

```
fg++ *.cpp -o foo
```

Notes:

- Any two non-negative int are acceptable (within the size of the primitive int).

CS2200

Theory Of Computer Science



Quiz 10

Illya Starikov

August 7, 2025

Problem #1

	S	
$\Rightarrow AB$	via $S \rightarrow AB$	
$\Rightarrow aXB$	via $A \rightarrow aX$	
$\Rightarrow aXbYd$	via $B \rightarrow bYd$	
$\Rightarrow aXbYd$	via $B \rightarrow bYd$	
$\Rightarrow abXYd$	via $Xb \rightarrow bX$	
$\Rightarrow abYcd$	via $XY \rightarrow Yc$	
$\Rightarrow abcd$	via $Y \rightarrow \lambda$	

Problem #2

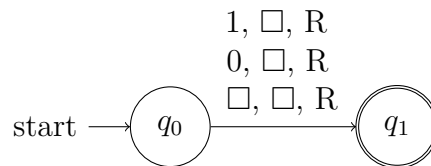
	S	
$\Rightarrow$	TbC	via $S \rightarrow TbC$
$\Rightarrow$	gC	via $Tb \rightarrow g$
$\Rightarrow$	Sg	via $gC \rightarrow Sg$
$\Rightarrow$	$TbCg$	via $S \rightarrow TbC$
$\Rightarrow$	gCg	via $Tb \rightarrow g$
$\Rightarrow$	Sgg	via $gC \rightarrow Sg$
$\Rightarrow$	$TbCgg$	via $S \rightarrow TbC$
$\Rightarrow$	$gCgg$	via $Tb \rightarrow g$
$\Rightarrow$	egg	via $gC \rightarrow e$

Problem #3

The language is derived by the grammar is $L = \{aa^n b^k \mid 0 \leq n \leq k, n + k = 2c\}$, for some arbitrary constant c (i.e. $n + k$ is even).

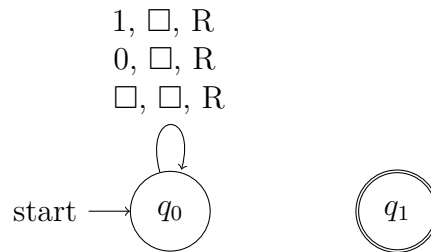
Problem #4

The following always halts, regardless of the input from $\Sigma = \{0, 1\}$.



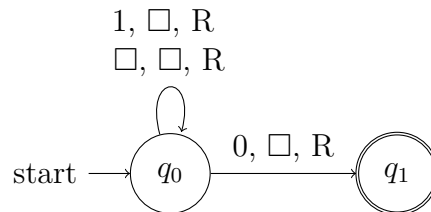
Problem #5

The following never halts, regardless of the input from $\Sigma = \{0, 1\}$. Note it also never halts even if the tape is blank (λ).



Problem #6

The following halts for some, but not all, input from $\Sigma = \{0, 1\}$. In this case, the only input that would cause the machine to halt would be 0.



Problem #7

No, we will prove so as follows.

Theorem 1. *Suppose A to be an arbitrary program — that is, $A \in$ all programs. Then, $\forall A \in$ all program, there **does not** exists a program P such that P can determine if A will halt **regardless of input**.*

Proof. Suppose not. That is, suppose $\exists P \in$ all programs such that P can determine if $\forall A \in$ all programs, A will halt. Suppose the following to be such program P , with an additional source code at the end:

```

bool willHalt(program P) { ... }

int recursiveNightmare() {
    if (willHalt(P)) {
        return recursiveNightmare();
    }

    return 42;
}
  
```

If we were to run this program through P , that is $P(P)$, we have the following cases:

Case 1: The program P loops forever. Assuming the program to be correct, this will return false; hence program P halts on input P , which is a contradiction (by our **recursive nightmare**).

Case 2: The program halts on input P . Assuming the program to be correct, this will return true; hence program P runs indefinitely on input P ; this is a contradiction.

We see this to be a paradox; P only halts when `willHalt` will not halt, and runs indefinitely when it does halt. Either way, we see this to be a contradiction. Therefore, there **does not** exist a program P such that P can determine if A will halt **regardless of input**.

□

Quiz #11

Ilya Starikov

July 27, 2025

1 Source Code

```
1      /*      example.l
2
3          Illya Starikov
4      */
5
6      %{
7      /*
8      Definitions of constants, variables, & function prototypes
9      go here
10     */
11
12     #define T_IDENT      1
13     #define T_INTCONST   2
14     #define T_UNKNOWN    3
15     #define T_FOO        4
16     #define T_KEYWORD    5
17     #define T_OPERATOR   6
18     #define T_NUMBER     7
19     #define T_ALPHA      8
20
21     int numLines = 1;
22
23     void printTokenInfo(const char* tokenType, const char*
24     lexeme);
25
26     /* Defintions of regular expressions go here */
27
```



```

28 WSPACE      [ \t\v\r]+
29 NEWLINE     \n
30
31 KEYWORD      (var|if|then|else|while|true|false)
32 OPERATOR     (;|\{|\}|\(|\)|\[[\]]|=|\(|\)|\+|\>|\<|!=|>=|<=|==)
33 NUMBER       -?{INTCONST}
34 ALPHA        {LETTER}
35
36 DIGIT        [0-9]
37 LETTER       [a-zA-Z]
38
39 IDENT        {LETTER}({LETTER}|{DIGIT})*
40 INTCONST     {DIGIT}+
41
42 %%
43
44 "foo"        {
45               printTokenInfo("FOO", yytext);
46               return T_FOO;
47             }
48 {KEYWORD}    {
49               printTokenInfo("KEYWORD", yytext);
50               return T_KEYWORD;
51             }
52 {OPERATOR}   {
53               printTokenInfo("OPERATOR", yytext);
54               return T_OPERATOR;
55             }
56 {ALPHA} {
57               printTokenInfo("ALPHA", yytext);
58               return T_ALPHA;
59             }
60 {INTCONST}   {
61               printTokenInfo("INTCONST", yytext);
62               return T_INTCONST;
63             }
64 {INTCONST}   {
65               printTokenInfo("INTCONST", yytext);
66               return T_INTCONST;
67             }
68 {IDENT}      {
69               printTokenInfo("IDENT", yytext);
70               return T_IDENT;
71             }
72 {NEWLINE}    {

```

```

73             numLines++;
74         }
75     {WSPACE} { }
76     . {
77         printTokenInfo("UNKNOWN", yytext);
78         return T_UNKNOWN;
79     }
80     %%
81
82     // User-written code goes here
83
84     void printTokenInfo(const char* tokenType, const char*
lexeme)
85     {
86         printf("TOKEN: %s LEXEME: %s\n", tokenType, lexeme);
87     }
88
89     // You should supply a yywrap function.
90     // Having it return 1 means only 1 input file will be
scanned.
91     int yywrap() { return(1); }
92
93     int main()
94     {
95         while ( yylex() ) ; // Process tokens until 0 returned
96
97         printf("Processed %d lines\n", numLines);
98         return(0);
99     }

```

2 Input

```
var
x; y; z;
A[4];
B[2] [3];
\$\% \#
{
x = 12;
B[1] [2] = x;
A[x] = 5;
while (x > 100) x = x + 1;
if (A[2] <= A[B[x] [y]]) then x = y + 1 else z = 10;
\}
```

3 Output

```
TOKEN: KEYWORD  LEXEME: var
TOKEN: ALPHA    LEXEME: x
TOKEN: OPERATOR LEXEME: ;
TOKEN: ALPHA    LEXEME: y
TOKEN: OPERATOR LEXEME: ;
TOKEN: ALPHA    LEXEME: z
TOKEN: OPERATOR LEXEME: ;
TOKEN: ALPHA    LEXEME: A
TOKEN: OPERATOR LEXEME: [
TOKEN: INTCONST LEXEME: 4
TOKEN: OPERATOR LEXEME: ]
TOKEN: OPERATOR LEXEME: ;
TOKEN: ALPHA    LEXEME: B
TOKEN: OPERATOR LEXEME: [
TOKEN: INTCONST LEXEME: 2
TOKEN: OPERATOR LEXEME: ]
TOKEN: OPERATOR LEXEME: [
TOKEN: INTCONST LEXEME: 3
TOKEN: OPERATOR LEXEME: ]
TOKEN: OPERATOR LEXEME: ;
TOKEN: UNKNOWN  LEXEME: \$
TOKEN: UNKNOWN  LEXEME: \%
TOKEN: UNKNOWN  LEXEME: \#
TOKEN: OPERATOR LEXEME: {
TOKEN: ALPHA    LEXEME: x
TOKEN: OPERATOR LEXEME: =
TOKEN: INTCONST LEXEME: 12
TOKEN: OPERATOR LEXEME: ;
TOKEN: ALPHA    LEXEME: B
```

```
TOKEN: OPERATOR  LEXEME: [  
TOKEN: INTCONST  LEXEME: 1  
TOKEN: OPERATOR  LEXEME: ]  
TOKEN: OPERATOR  LEXEME: [  
TOKEN: INTCONST  LEXEME: 2  
TOKEN: OPERATOR  LEXEME: ]  
TOKEN: OPERATOR  LEXEME: =  
TOKEN: ALPHA     LEXEME: x  
TOKEN: OPERATOR  LEXEME: ;  
TOKEN: ALPHA     LEXEME: A  
TOKEN: OPERATOR  LEXEME: [  
TOKEN: ALPHA     LEXEME: x  
TOKEN: OPERATOR  LEXEME: ]  
TOKEN: OPERATOR  LEXEME: =  
TOKEN: INTCONST  LEXEME: 5  
TOKEN: OPERATOR  LEXEME: ;  
TOKEN: KEYWORD   LEXEME: while  
TOKEN: OPERATOR  LEXEME: (  
TOKEN: ALPHA     LEXEME: x  
TOKEN: OPERATOR  LEXEME: >  
TOKEN: INTCONST  LEXEME: 100  
TOKEN: OPERATOR  LEXEME: )  
TOKEN: ALPHA     LEXEME: x  
TOKEN: OPERATOR  LEXEME: =  
TOKEN: ALPHA     LEXEME: x  
TOKEN: OPERATOR  LEXEME: +  
TOKEN: INTCONST  LEXEME: 1  
TOKEN: OPERATOR  LEXEME: ;
```

```
TOKEN: KEYWORD  LEXEME: if
TOKEN: OPERATOR  LEXEME: (
TOKEN: ALPHA    LEXEME: A
TOKEN: OPERATOR  LEXEME: [
TOKEN: INTCONST  LEXEME: 2
TOKEN: OPERATOR  LEXEME: ]
TOKEN: OPERATOR  LEXEME: <=
TOKEN: ALPHA    LEXEME: A
TOKEN: OPERATOR  LEXEME: [
TOKEN: ALPHA    LEXEME: B
TOKEN: OPERATOR  LEXEME: [
TOKEN: ALPHA    LEXEME: x
TOKEN: OPERATOR  LEXEME: ]
TOKEN: OPERATOR  LEXEME: [
TOKEN: ALPHA    LEXEME: y
TOKEN: OPERATOR  LEXEME: ]
TOKEN: OPERATOR  LEXEME: ]
TOKEN: OPERATOR  LEXEME: )
TOKEN: KEYWORD  LEXEME: then
TOKEN: ALPHA    LEXEME: x
TOKEN: OPERATOR  LEXEME: =
TOKEN: ALPHA    LEXEME: y
TOKEN: OPERATOR  LEXEME: +
TOKEN: INTCONST  LEXEME: 1
TOKEN: KEYWORD  LEXEME: else
TOKEN: ALPHA    LEXEME: z
TOKEN: OPERATOR  LEXEME: =
TOKEN: INTCONST  LEXEME: 10
TOKEN: OPERATOR  LEXEME: ;
TOKEN: OPERATOR  LEXEME: }
```

CS2300

Databases



Homework #1

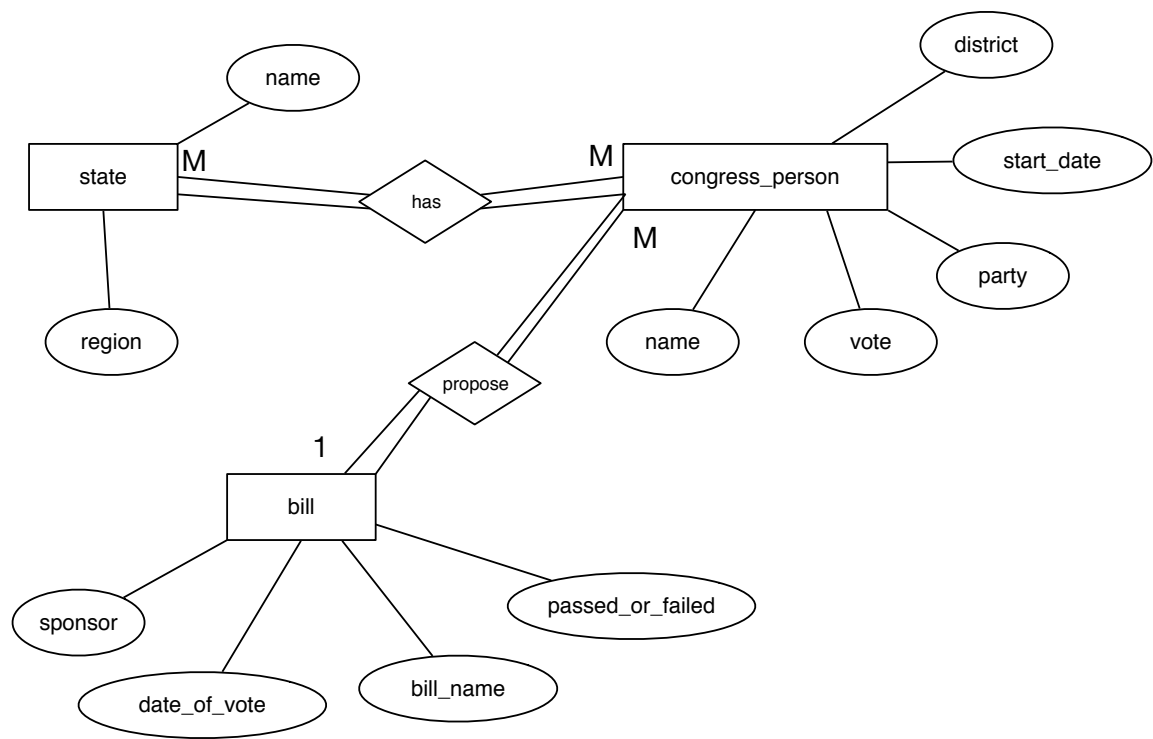
Claire Trebing, Jason Young, Illya Starikov

Due Date: February 04, 2016

Question 1

- a Section's CRoom (Building and Room Number).
- b Section's DayTime and Year.
- c Section's SecID for every Year.

Question 2



Question 3

a Strong Entity Types

- Bank
- Accounts
- Loans
- Customer

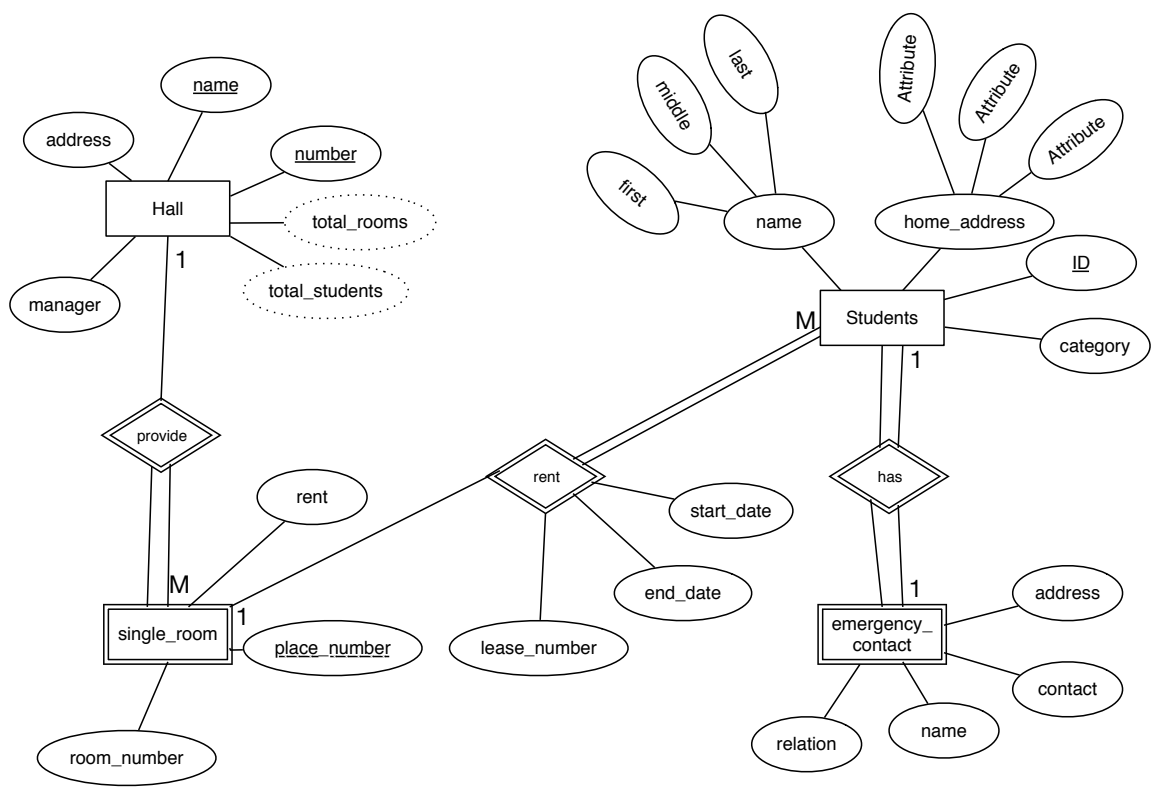
b Weak Entity Type

- Yes, the **Bank Branch** is a weak entity type. The partial key is the **Branch Number** and the identifying relationship is the **Branches** relationship.

c Constraints

- This information specifies that the branch number of a bank branch is only unique within a bank. For example, a bank branch in Bank A may have the same branch number as a bank branch in Bank B, but a bank branch in Bank A would not have the same branch number as any other bank branch in Bank A.

Question 4



Homework #2

Claire Trebing, Jason Young, Illya Starikov

February 25, 2016

Question #1

- Valid Operation
- Valid Operation
- Key Constraint (Key #4 is already taken)
- Key Constraint, Not Null Constraint (Key cannot be null)
- Valid Operation
- Valid Operation
- Integrity Constraint (Did not update WORKS_ON or DEPENDANT)
- Integrity Constraint (Did not update WORK_ON)
- Valid Operation
- Integrity Constraint (Did not update WORKS_ON)
- Valid Operation

Question #2

Grader note: the rho (ρ) is too small in L^AT_EX to be distinguished from a subscript, so instead P is used in it's place.

- $\Pi_{\text{Fname, Minit, Lname}}(\sigma_{\text{Dno} = '5' \wedge \text{hours} = > 10}(\text{project} \bowtie_{\text{Dname} = 'rojectX'} \text{on}))$
- $\Pi_{\text{Fname} = \text{Dependent_Name}}(\text{employee} \bowtie_{\text{Dependent}})$
- $\Pi_{\text{Fname, LName}}(\text{employee} \div \Pi_{\text{Super_SSN}})(\sigma_{\text{Fname} = 'Franklin'})$
- $\Pi_{\text{Lname, Sum_Hours}}(s_{\text{name}} \mathcal{F}_{\text{sum hours}}(\text{PROJECT} \bowtie_{\text{ESSN} = \text{SSN}}))$
- $\text{employee1} \rightarrow \text{employee} \bowtie_{\text{ESSN} = \text{SSN} \vee \text{ork_On}}$

- [illegible]

Question #3

- `ust# \end{verb} (\begin{verb} ORDER \end{verb})`
- `rder \end{verb} (\begin{verb} RDERiTEM, HIPMENT \end{verb})`
- `tem# \end{verb} (\begin{verb} ORDER \end{verb}, \begin{verb} TEM
)`
- `arehouse# \end{verb} (\begin{verb} SHIPMENT \end{verb})`

Assumptions

- Table headers with the same name are not unique.

Homework #4

Databases and File Structures

Illya Starikov, Jason Young, Claire Trebing

Due Date: April 21, 2016

1 Closures, Candidate Keys

- $A^+ = \{A\}$
- $AB^+ = \{A, B, C, D, E\}$ **Candidate Key**
- $AC^+ = \{A, B, D, C, E\}$ **Candidate Key**
- $ACD^+ = \{A, B, C, D, E\}$ **Candidate Key**
- $B^+ = \{B, D\}$
- $BC^+ = \{A, B, C, D, E\}$ **Candidate Key**
- $C^+ = \{C\}$
- $CD^+ = \{A, B, C, D, E\}$ **Candidate Key**
- $E^+ = \{A, B, C, D, E\}$ **Candidate Key**
- $DE^+ = \{A, B, C, D, E\}$ **Candidate Key**

2 Functional Dependence Equivalence

No, because there is no occurrence of $C \rightarrow D$ in G , but there is one in F .

3 Minimal Cover

Step 1

- $A \rightarrow B, A \rightarrow C, A \rightarrow D$
- $E \rightarrow G, E \rightarrow H$
- $C \rightarrow D$
- $AE \rightarrow J$

Step 2

- $A \rightarrow B, A \rightarrow C, A \rightarrow D$
- $E \rightarrow G, E \rightarrow H$
- $C \rightarrow D$
- $AE \rightarrow J$

Step 3

- $A \rightarrow B, A \rightarrow C$
- $E \rightarrow G, E \rightarrow H$
- $C \rightarrow D$
- $AE \rightarrow J$

4 3NF and BCNF

1. $F = \{AB \rightarrow C, C \rightarrow D, C \rightarrow A\}$, *Candidate Keys*: $(AB), (BC)$
 - No, because the in the functional dependency $C \rightarrow D$, D is not a part of any candidate key and A is not a super key.
 - No, because $\{C \rightarrow D\}$ is neither trivial nor a superkey.
2. $F = ACE \rightarrow BD, B \rightarrow C$, *Candidate Key*: (ACE)
 - Yes, because ACE is a super key and C is part of the candidate key.
 - No, because $\{B \rightarrow C\}$ is neither trivial nor a superkey.

ForFit

Never Quit Again

Illya Starikov, Jason Young, Claire Trebing

July 27, 2025

Contents

I. The Database Design Manual	3
1. Problem Statement	4
2. Conceptual Database Design	5
3. Logical Database Design	7
3.1. Relational Set	7
3.2. Summary Table	8
4. Application Program Design	10
4.1. Create a New User	10
4.2. Delete A Group	10
4.3. Modifying User Statistics	11
4.4. Query Other Users	12
4.5. User Leaderboards	13
II. User Manual	14
5. A Brief Introduction To ForFit	15
6. How It Works	16
7. Test Procedure	17
8. Functions	18
8.1. Delete Group	18
8.2. Modify User	18
8.3. New User	19
8.4. Query User	19
8.5. Total Users	20
8.6. User Leaderboards	20
9. Shell	22
9.1. Create Database	22
9.2. Random Input	22
9.3. Drop Database	23

Part I.

The Database Design Manual

1. Problem Statement

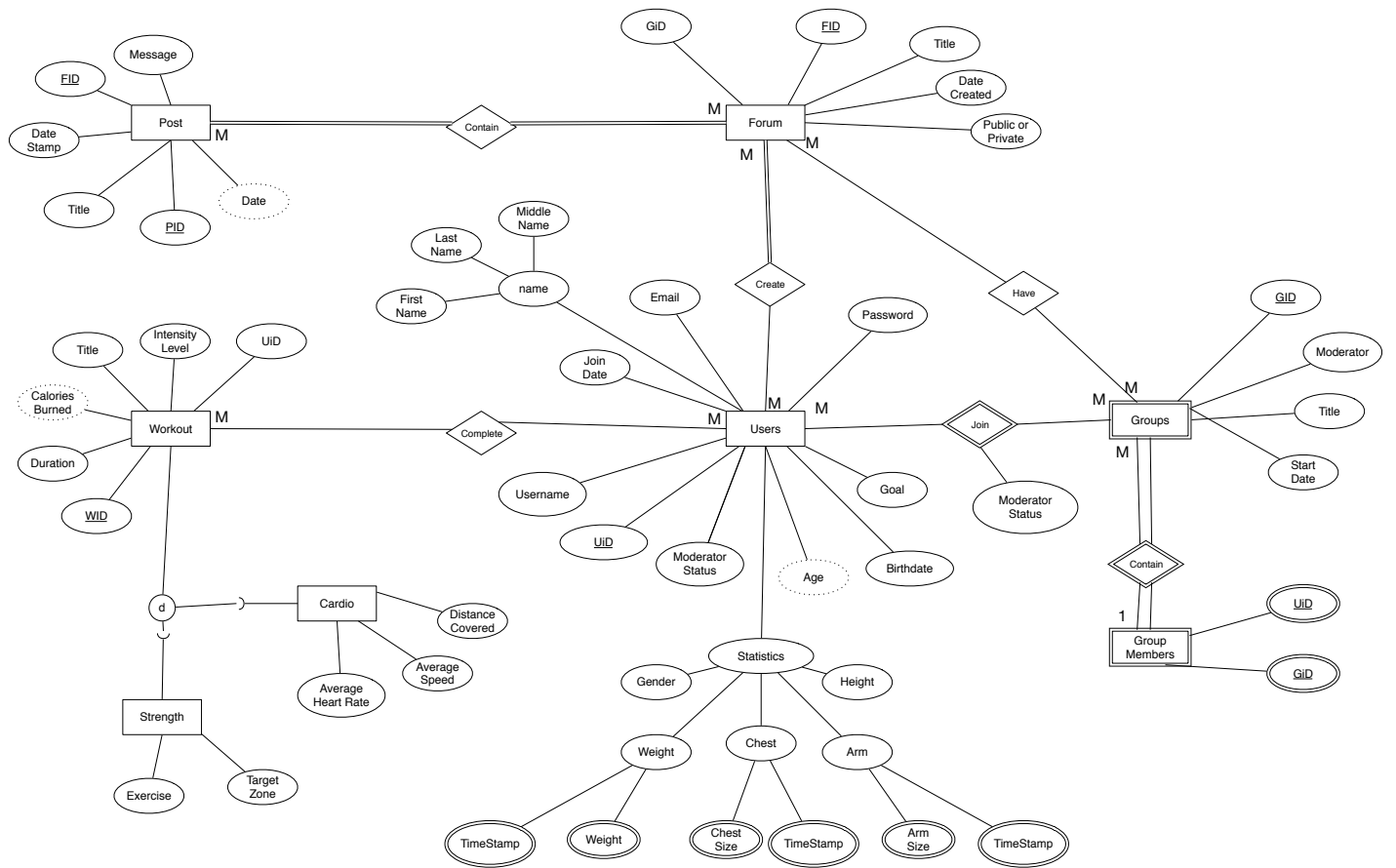
2015 marked a record year for Americans regularly exercising, hitting just over 55%. Keeping that momentum is difficult. As blue-collar jobs continue to decline, it has been more important than ever to keep a weekly regimen of healthy eating and exercising.

Living in the most interconnected generation poses quite a viable idea for social networking: healthy living. We can build a social network that connects users to friends, peers and family to make an online community of healthy living.

Our database will be essential because it will unite something so mundane and uninteresting with familiar faces. This will make exercise more enjoyable and offer a group to hold you accountable. We can shape an entire generation by having motivation a click away.

This database will consist of premade workouts, groups, and reminder emails to help you stay on top of your fitness plan.

2. Conceptual Database Design



3. Logical Database Design

3.1. Relational Set

Please note primary keys are signified by **PK** and foreign keys are signified by *FK*.

Post

<u>PiD</u> PK	<u>FiD</u> <i>FK</i>	Message	DateStamp	Title	Date
----------------------	----------------------	---------	-----------	-------	------

Forum

<u>FiD</u> PK	Title	DateCreated	PublicOrPrivate	GiD <i>FK</i>
----------------------	-------	-------------	-----------------	---------------

Groups

<u>GiD</u> PK	Moderator <i>FK</i>	Title	StartDate
----------------------	---------------------	-------	-----------

Workouts

<u>WiD</u> PK	Duration	Title	IntensityLevel	CaloriesBurned	UiD <i>FK</i>
----------------------	----------	-------	----------------	----------------	---------------

Strength

<u>WiD</u> PK	Duration	Title	IntensityLevel	CaloriesBurned	UiD <i>FK</i>	Target Zone
----------------------	----------	-------	----------------	----------------	---------------	-------------

Cardio

<u>WiD</u> PK	Duration	Title	IntensityLevel	CaloriesBurned	UiD <i>FK</i>	AverageHeartRate
AverageSpeed	DistanceCovered					

Users

<u>UiD</u> PK	Username	Height	Birthdate	Goal	Password	JoinDate	Gender
FirstName	MiddleName	LastName					

Weight

<u>UiD</u> <i>PK, FK</i>	<u>TimeStamp</u> PK	Weight
--------------------------	----------------------------	--------

Arm Size

<u>UiD</u> <i>PK, FK</i>	<u>TimeStamp</u> PK	Arm Size
--------------------------	----------------------------	----------

Chest Size

<u>UiD</u> <i>PK, FK</i>	<u>TimeStamp</u> PK	ChestSize
--------------------------	----------------------------	-----------

3. Logical Database Design

Group Members

<u>UiD</u> ^{PK}	<u>GiD</u> ^{FK}	ModeratorStatus
--------------------------	--------------------------	-----------------

3.2. Summary Table

3. Logical Database Design

Attribute	Data Type	Constraints	Meaning
Message	varchar	500 characters	Contents of a post.
FID	int	unique	Forum ID.
DateStamp	timestamp	timestamp of creation	Timestamp of post was creation.
Title	char	35 characters	Title of a post.
PID	int	unique	Post ID.
Date	timestamp	date of creation	Timestamp of post creation.
Title	char	35 characters	Title of a forum.
DateCreated	timestamp	timestamp of creation	Timestamp of forum creation.
PublicOrPrivate	boolean	-	Is forum public.
GID	int	unique	Group ID.
Moderator	char	must match a username	Group owner/moderator.
Title	char	35 characters	Title of the group.
StartDate	timestamp	timestamp of creation	Timestamp of group creation.
IntensityLevel	int	-	Enumerated value, code for different options.
Title	char	35 characters	Title of the workout.
CaloriesBurned	int	num > 0	How many calories burned during exercise.
Duration	timestamp	num > 0	Length of exercise.
WID	int	unique	Workout ID.
Exercise	char	35 characters	Exercise name.
TargetZone	char	35 characters	Area exercise is intended to workout.
AverageHeartRate	int	num > 0	Average heart rate recorded during exercise.
AverageSpeed	int	num > 0	Average speed of cardio exercise.
DistanceCovered	int	num > 0	Distance covered during cardio exercise.
Password	char	-	Password of user.
JoinDate	timestamp	timestamp of creation	Timestamp of users sign up.
Username	char	unique	Users username, used for sign in.
ModeratorStatus	boolean	-	Is a moderator.
Birthdate	date	-	Date of users birth.
Goal	varchar	500 characters	Users intended workout goals.
Gender	char	1 character, M or F	Users gender, male(M) or female(F).
Weight	int	num > 0	Users weight at a certain date.
ChestSize	int	num > 0	Users chest size at a certain date.
ArmSize	int	num > 0	Users arm size at a certain date.
Height	int	num > 0	Users height at a certain date.
UID	int	unique	Users ID.
TimeStamp	date	unique	Date of users weight mesurement.
TimeStamp	date	unique	Date of users chest measurement.
TimeStamp	date	unique	Date of users arm measurement

4. Application Program Design

4.1. Create a New User

This function creates a new user, accessing only the `user` table.

INPUT	Username, Height, Birthdate, Goal, Password, Gender.
STEPS	1. Check to see if the <code>username</code> is available. If available, proceed. If not, display appropriate message to notify the user. 2. Insert appropriate information to the User table. (Username to <code>username</code>, height to <code>height</code>) 3. Generate a user ID, assign it to the <code>UiD</code> attribute. 4. Take a time stamp, assign it to the <code>JoinDate</code> attribute. 5. Calculate the age based on the <code>BirthDate</code> attribute.
OUTPUT	A new <code>User</code> entity will be inserted into the table, with appropriate data into proper columns (along with computed properties and derived properties).
ASSUMPTIONS	<code>Username</code> , <code>Height</code> , <code>Birthdate</code> , <code>Goal</code> , <code>Password</code> , <code>Gender</code> are all correct (this will be validated in the sign up form).

4.2. Delete A Group

This function deletes a group by updating information in `Forum`, and removing the `Group` and `Group Members` tables.

4. Application Program Design

INPUT	The Group ID (GiD) that is to be deleted.
STEPS	1. Check to see if the request is made by the moderator via the Moderator column in the Groups table. If true, approve the request. If not, cancel the request and notify the user. 2. Remove the row that has a matching GiD that was provided for deletion in the Groups table. 3. Query the Group Members table, removing any row that match the GiD provided for deletion. 4. Query the Forum table for any matching Group Ids (GiD), setting the GiD to null if matching.
OUTPUT	The groups are deleted, the group members within that group are deleted, and any reference to the group is deallocated.
ASSUMPTIONS	None.

4.3. Modifying User Statistics

Our user statistics have the ability to fluctuate. We would like to accommodate for this fluctuation by allowing users to update their respective statics; specifically, we would like to let users update **Username**, **Height**, **Goal**, **Password**, **Gender** in the **Users** table.

4. Application Program Design

INPUT	The specific attribute(s) of the set Username , Height , Goal , Password , Gender that would like to be updated with the new value.
STEPS	1. Ensure the data is valid (e.g. is not null when applicable, in the proper domain). If it is valid, continue. If not, prompt the user with an error message and try again. 2. Modify the attribute to reflect the new value. 3. Repeat for any additional attributes provided.
OUTPUT	The attribute(s) should now reflect the new value provided.
ASSUMPTIONS	The data is within a proper range (will not overflow).

4.4. Query Other Users

This function allows for users to query other users; this can be done via **Username** or **FirstName**, **MiddleName**, and **LastName** from the **Users** table.

INPUT	Either a Username xor any subset of FirstName , MiddleName , or LastName .
STEPS	1. Check to see if input is valid. If is, proceed. If not, display error message to the user. 2. Query the Users table to see if the user exists. If the user exists, proceed. If not, display appropriate message to the user. 3. Project the profile.
OUTPUT	Either the search user will be projected or an error message is the user does not exist.
ASSUMPTIONS	The first, middle and last name are all provided. The names are unique (solely for the testing purposes).

4.5. User Leaderboards

Generate the leaderboard based on the workouts accomplished; specifically aggregating data from the **Strength** and **Cardio** table. *Note this is the function that requires multiple tables.*

INPUT	None.
STEPS	1. Merge the User and Workouts table, call the new table Merged. 2. Add up the total duration (call the new property TotalDuration) in the Merged table based on the UiD attribute, making a new table named Sums. 3. Sort the Sums by the TotalDuration attribute. 4. Display the top 10 on the sorted Sums table to the user. 5. Display the user their current rank.
OUTPUT	An eleven-row table displaying the top 10 leaderboards and the user's current rank.
ASSUMPTIONS	There is a bare minimum of eleven users.

Part II.

User Manual

5. A Brief Introduction To ForFit

ForFit is a new way to workout. Over the past year, just about 55% of Americans exercised regularly. ForFit is a way to connect you to your friend, family, and other exercise enthusiasts in your area. These exercises can range from traditional bike riding and swimming to newer forms of exercise such as parkour and yoga. ForFit encourages group activity and connection so that workouts are never dull and so that users will be continually motivated and pushed toward their workout goals.

6. How It Works

ForFit has two main components. The first part is the users themselves. Users sign up for the service and then begin recording their workouts. Workouts can be logged as many times as liked, from once a month to several times a day. The more workouts logged, the better data and the more progress toward your exercise goals.

Users can view other users and the workouts they've done. This leads into the second component of ForFit, which is the encouragement of community. Users can create groups and join existing groups. These groups can help users focus on specific areas. For example, a user wanting to run a marathon might join a group of other runners training for a marathon. This way the user can keep in touch and stay motivated through the group and also have a resource to ask questions or get other workouts.

Groups allow users to make forums, for in-depth discussion, and posts, for quick updates. These groups are the main area of community creation that ForFit focuses on in the design. Users can find other users and join groups with users they have been exercising with before. This creates a network of community interaction.

7. Test Procedure

This database was well tested before it reached you, the user. This section will explain how the test database was created and what tests the database went through before completion. The mock data was created using C++ code to randomly generate data. Data from online was used to create First Names, Last Names, Usernames, and Group Names as well as to populate the workout subclasses in strength and cardio. Using C++ code to randomly select and combine information, creating unique data. This mock data was then put through a series of tests to make sure the functions would work properly. Each function was tested over 500 times to error or improper analysis of the data. After confirming that these functions were working properly, they were implemented into the database.

8. Functions

8.1. Delete Group

Delete Group allows you to remove a group from the database. Once a group has been deleted, you can no longer make forums or posts in this group.

Parameters

groupId The GiD of the group that is to be deleted.

Results

The group who's ID matches the given **groupId** will be deleted. If the group does not exist, nothing is displayed.

```
[rMBP:~ postgres$ '/Applications/Postgres.app/Contents/Versions/9.5/bin'/psql -p5432
psql (9.5.2)
Type "help" for help.

postgres=# select delete_group(42);
 delete_group 
-----
(0 rows)

postgres=#
```

8.2. Modify User

Modify User allows users to change certain values in the user's data by providing the user name. Users can change their username, height, goal, password, and gender.

Parameters

userID The UiD of the user to who's statistics that will be modified. If this is not provided, there is no way of changing the user.

newUsername The new username property of the user. If this statistic will not change, pass a '0' to the function. Otherwise, proceed with changing.

newHeightFeet See above.

newHeightInches See above.

8. Functions

newGoal See above.

newPassword See above.

newGender See above.

Results

For all attributes that do not have the value of '0', the value of said attribute will be updated (provided it upholds all constraint).

```
(rMBP:~ postgres$ ~/Applications/Postgres.app/Contents/Versions/9.5/bin/psql -p5432
psql (9.5.2)
Type "help" for help.

postgres=# select modify_user(1, 'IllyaStarikov', 0, 0, 42, '0', 'F');
 modify_user
(0 rows)

postgres=# select * from USERS where Uid = 1;
 uid | username | email | heightfeet | heightinches | birthdate | goal | password | joindate | gender | fname | minit | lname
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
  1  | IllyaStarikov | CasieScarlett1@gmail.com | 6 | 2 | 1967-03-31 | 42 | EtZEdInfnIBAI | 1999-12-31 | F | Casie | M | Scarlett
(1 row)
```

8.3. New User

New User creates new users in the database. The user must input a username, email, height, birthdate, goal, and password. If any of those are not provided, an error message will be generated.

Parameters

id, username, email, heightFeet, heightInches, birthdate, goal, password All the attributes that can be inserted into user table.

joinDate, gender, firstName, mInitial, lastName See above.

Results

Provided all the attributes meet database constraints, a new user will be inserted into the database. If a constraint is not meant, there will be a message outputting the error and the insert will be rejected.

```
postgres=# select new_user(404, 'IllyaStarikov', 'IllyaStarikov@iCloud.com', 5, 10, '07-20-1994', 165, '42', '07-21-1994', 'M', 'Illya', 'A', 'Starikov');
 new_user
(0 rows)

postgres=# select * from USERS where Uid = 404;
 uid | username | email | heightfeet | heightinches | birthdate | goal | password | joindate | gender | fname | minit | lname
-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----
 404 | IllyaStarikov | IllyaStarikov@iCloud.com | 5 | 10 | 1994-07-20 | 165 | 42 | 1994-07-21 | M | Illya | A | Starikov
(1 row)
```

8.4. Query User

Query User allows users to find others by inputting a first and last name. If a user does not exist than the results are empty.

8. Functions

Parameters

firstName, lastName The first and last name of the user to be queried.

Results

If the user exists, the column of the user will be returned. If the user does not exist, not such column will be returned.

```
rMBP:~ postgres$ '/Applications/Postgres.app/Contents/Versions/9.5/bin'/psql -p5432
psql (9.5.2)
Type "help" for help.

postgres=# select query_user('Illya', 'Starikov');
               query_user
-----
(117,IllyaStarikov,IllyaStarikov@iCloud.com,5,10,1994-07-20,165,42,1994-07-21,M,Illya,A,Starikov)
(1 row)
```

8.5. Total Users

Total Users returns how many users are in the whole database. In the future, this function will give a user count for an individual group.

Parameters

None.

Results

Returns the total count of users, a maintenance function used to compare to the total count in the database.

```
rMBP:~ postgres$ '/Applications/Postgres.app/Contents/Versions/9.5/bin'/psql -p5432
psql (9.5.2)
Type "help" for help.

postgres=# select total_users();
 total_users
-----
          52
(1 row)
```

8.6. User Leaderboards

User Leaderboards gives you a top 10 user count. This is calculated by a count of workouts done by a user. This count includes all the users in a database and is not specific to a group.

8. Functions

Parameters

None.

Results

Returns the top 10 users (calculated via count of workouts done) in the database.

```
lrmBP:~ postgres$ '/Applications/Postgres.app/Contents/Versions/9.5/bin'/psql -p5432
psql (9.5.2)
Type "help" for help.

postgres=# select user_leaderboards();
 user_leaderboards
-----
 (Gregory,Patty,6)
 (Agueda,Heather,4)
 (Bertram,Gerardoy,4)
 (Gay,Arthur,4)
 (Garry,Shanony,4)
 (Brad,Lana,4)
 (Francisco,Bruce,4)
 (Edgar,Dennisy,4)
 (Alvina,Carter,4)
 (Darlene,Salvatore,3)
(10 rows)
```

9. Shell

We know doing tasks such as creating the database, generating a new set of random inputs, or even running all the function files to make sure all of the functions are created can be tedious. Fortunately, we have a solution — that solution is [Shell scripting](#).

We quickly realized that repetitively running the same commands¹ could get tedious, so we have provided multiple shell scripts that automate most of the monotonous tasks.

To run a shell script, first you must give it privileges to be able to run **Bash** commands. This is accomplished by `chmod -x /path/to/file.sh`. This gives read/write access to the shell script, which then allows the shell script to be run via `./path/to/file.sh`. This then allows the user to do tasks like setting up the entirety of the database (which spans multiple files!) one file path away.

Below are the shell scripts we provide.

9.1. Create Database

A lot of testing requires the use of setting up the database from scratch. Unfortunately, this usually means creating the database scheme, followed by inserting the data, followed by inserting the functions, and so forth. Although an easy fix would be to have all the code in a single file, this would combine concerns. We would rather have a lot of function specific files apposed to one giant file. Our solution: `./create_database.sh`

The create database script works like so:

1. Create the database schema by `create`ing all the tables and `modify`ing all for foreign key constraints.
2. Insert data, which is artificially generated (as will be mentioned below).
3. Go through the *entirety* of the functions directory, and running each function `sql` script to ensure all functions are inserted into the database.

This allows for easily bringing a database back up after it has been reset.

9.2. Random Input

Let face it, every social network occasionally needs some help. That is why we have a shell script that executes our C++ codebase to auto-populate the table. The shell script runs like so:

¹Like running ``/Applications/Postgres.app/Contents/Versions/9.5/bin'/psql -p5432 -f` to create every file!

9. *Shell*

1. Change to the directory of the C++ codebase.
2. Run the makefile
3. Run the executable, pipe that to an output file.
4. Move the shell script to the directory of all the other `sql` files.

This makes for easily produced, new data.

9.3. Drop Database

Drop database simply runs the `sql` script to drop all tables in a cascade fashion. It's a simple script but is much quicker than manually running the command to drop the database.

10. Website

To setup the website you only need to place the website files inside of an apache server and then rewrite the config.php file.

For the config.php file you need to modify the variables;

```
"  
    date_default_timezone_set('America/Chicago');  
    define('DB_SERVER', 'localhost:3306');  
    define('DB_USERNAME', 'JasonY');  
    define('DB_PASSWORD', 'Servant83');  
    define('DB_DATABASE', 'forfit');  
"
```

to fit the website's database.

The website is a blend of PHP and HTML. It allows you to sign up, logout, view groups and forums.

The main page contains the link to signup. After signing up or logging in you can open the menu and view the groups which contain forums. You may also enter your personal information in the profile which tracks your exercises and gains.

CS2500

Algorithms



Homework #1

Illya Starikov

Due Date: February 3, 2016

1 Insertion Sort

1.1 Operations of Insertion Sort

3 <u>41</u> 52 26 38 57 9 49	1 Comparison(s).
3 41 <u>52</u> 26 38 57 9 49	1 Comparison(s).
3 41 52 <u>26</u> 38 57 9 49	3 Comparison(s).
3 26 41 52 <u>38</u> 57 9 49	3 Comparison(s).
3 26 38 41 52 <u>57</u> 9 49	1 Comparison(s).
3 26 38 41 52 57 <u>9</u> 49	6 Comparison(s).
3 9 26 38 41 52 57 <u>49</u>	3 Comparison(s).
3 9 26 38 41 49 52 57	0 Comparison(s).

1.2 Number of Comparisons

18 total operations.

2 Question 2

B	A	O	o	Ω	ω	Θ
2^n	$2^{\frac{n}{2}}$	no	no	yes	yes	no
$n^{\lg c}$	$c^{\lg n}$	yes	no	yes	no	yes

2.1 Justification

To determine larger asymptotic growth, take the limit of one function over the other. Arbitrarily choosing 2^n for the numerator, we see that:

$$\lim_{n \rightarrow \infty} \frac{2^n}{2^{\frac{n}{2}}} = \lim_{n \rightarrow \infty} 2^{\frac{n}{2}} = \infty \quad (1)$$

Alternatively, if we chose 2^n as the denominator, we notice that $\lim_{n \rightarrow \infty} \frac{2^{\frac{n}{2}}}{2^n} = 0$, so we know that the condition $2^n > 2^{\frac{n}{2}}$ holds, and for certain conditions the functions are equal (take $n = 0$, $2^0 = 2^{\frac{0}{2}}$).
 $\therefore 2^n$ is $\omega(2^{\frac{n}{2}})$ and $\Omega(2^{\frac{n}{2}})$. QED.

2.2 Justification II

By the properties of logarithms, we will show that $n^{\lg c} = c^{\lg n}$.

$$n^{\lg c} = c^{\lg n} \quad (2)$$

$$= c^{\log_2 n} \quad (3)$$

$$= c^{\frac{\ln n}{\ln 2}} \quad (4)$$

$$= e^{\ln c \times \frac{\ln n}{\ln 2}} \quad (5)$$

$$= e^{\frac{\ln c}{\ln 2} \times \ln n} \quad (6)$$

$$= e^{\lg c \ln n} \quad (7)$$

$$= n^{\lg c} \quad (8)$$

$\therefore n^{\lg c}$ is $O(c^{\lg n})$, $\Omega(c^{\lg n})$ and $\Theta(c^{\lg n})$. QED.

3 Big-O Implies Big-Ω

Theorem: Let $f(n)$ and $g(n)$ be asymptotically positive functions, $O(g(n))$ be the set $\{f(n) : \exists c, n_0 \in \mathbb{R}^+, \forall n, n_0 \in \mathbb{R}, 0 \leq f(n) \leq c g(n) \wedge n > n_0\}$, and $\Omega(g(n))$ be the set $\{f(n) : \exists c, n_0 \in \mathbb{R}^+, \forall n, n_0 \in \mathbb{R}^+, 0 \leq c g(n) \leq f(n) \wedge n > n_0\}$. Then,

$$f(n) = O(g(n)) \Rightarrow g(n) = \Omega(f(n)) \quad (9)$$

[We will prove so by contradiction.]

Proof: Suppose not. That is, suppose

$$\sim [f(n) = O(g(n)) \Rightarrow g(n) = \Omega(f(n))] \quad (10)$$

$$f(n) = O(g(n)) \wedge \sim [g(n) = \Omega(f(n))] \quad (11)$$

Using the formal definition of $\Omega(g(n))$, we can see the negation is as follows:

$$\sim \Omega(g(n)) = \{f(n) : \forall c, n_0 \in \mathbb{R}^+, \exists n, n_0 \in \mathbb{R}^+, 0 > c g(n) > f(n) \wedge n \leq n_0\} \quad (12)$$

This is a contradiction, for there are no c, n_0 such that $0 \leq c g(n) \leq f(n) \wedge 0 > c f(n) > g(n)$ because c, n_0 are defined as *positive* constants, and $f(n), g(n)$ are defined as *positive* functions. Because no such positive function and positive constants exists to satisfy

$$0 > cf(n) > g(n) \quad (13)$$

This has led us to a contradiction. QED.

4 Asymptotic Proofs

4.1 Proof I

Assuming $n > 1$,

$$n^2 \leq 20n^2 + 2n + 5 \quad (14)$$

$$\leq 20n^2 + 2n^2 + 5 \quad (15)$$

$$\leq 27n^2 \quad (16)$$

$\therefore$ For $C = 27, n_0 = 1, 20n^2 + 2n + 5 = O(n^2)$. QED.

4.2 Proof II

Assume $C = 1, n_0 = 1$. $\therefore$ This satisfies the condition that c and n_0 are positive constants such that $0 \leq Cn^2 \leq 5n^2 - 15n + 100 \forall n \geq n_0$. QED.

4.3 Proof III

4.3.1 Lower Bound

Assume $C = 1, n_0 = 1$. $\therefore$ This satisfies the condition that c and n_0 are positive constants such that $0 \leq Cn^2 \leq 5n^2 + 2n \forall n \geq n_0$. QED.

4.3.2 Upper Bound

Assuming $n > 1$,

$$n^2 \leq 5n^2 + 2 \quad (17)$$

$$\leq 5n^2 + 2n^2 \quad (18)$$

$$\leq 7n^2 \quad (19)$$

$\therefore C = 7, n_0 = 1, 5n^2 + 2n = \Theta(n^2)$. QED.

4.4 Proof IV

To prove that $5n + 7 = o(n^2)$ we must show that $\exists c, n_0 \in \mathbb{R}^+, 0 \leq f(n) < Cg(n) \wedge n > n_0$. We do so by showing that $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$. [*with the help of L'Hôpital's Rule*]

$$\lim_{n \rightarrow \infty} \frac{5n + 1}{n^2} \tag{20}$$

$$= \lim_{n \rightarrow \infty} \frac{5}{2n} \tag{21}$$

$$= 0 \tag{22}$$

QED.

Homework #2

Illya Starikov

Date Due: February 15, 2016

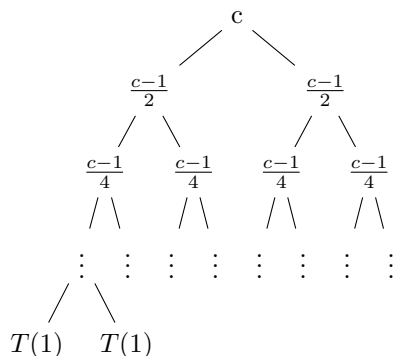
1 Programming Assignment

Code is attached. Binary search implementation is as follows.

```
1  extension Array where Element: Comparable {
2      func binarySearch(key: Element) -> Int? {
3          return binarySearch(key, low: 0, high: self.count - 1)
4      }
5
6      private func binarySearch(key: Element, low: Int, high: Int) -> Int? {
7          if low > high { return nil }
8
9          let middle = Int((low + high) / 2)
10
11         if key == self[middle] {
12             return middle
13         } else if key > self[middle] {
14             return binarySearch(key, low: middle + 1, high: high)
15         } else {
16             return binarySearch(key, low: low, high: middle - 1)
17         }
18     }
19 }
```

2 Recurrence Equation

2.1 Tower of Hanoi



The complexity of this tree is $O(\lg n)$, or alternatively, $O(2^n)$

2.2 Merge Sort

$$T(n) = 2 T\left(\frac{n}{2}\right) + n \quad (1)$$

From the equation, we can see that

$$a = 2, b = 2, c = 1, f(n) = n \quad (2)$$

We observe that

$$f(n) \in \Theta(n^{\log_b a}) = \Theta(n^{\log_2 2}) = \Theta(n) \quad (3)$$

It follows from Case #2 of the Master Theorem

$$T(n) \in \Theta(n^{\log_b a} \lg n) \quad (4)$$

$$= \Theta(n^{\log_2 2} \lg n) \quad (5)$$

$$= \Theta(n^1 \lg n) \quad (6)$$

$$= \Theta(n \lg n) \quad (7)$$

Thus the recurrence relation $T(n)$ is in $\Theta(n \log n)$. QED.

3 Loop Invariant

```

1 Merge(A, p, q, r)
2   leftIndex = q - p + 1
3   rightIndex = r - q
4   let L[1..left + 1] and R[1..right + 1] be new arrays
5   for i = 1 to leftIndex

```

```

6         L[i] = A[p + i - 1]
7     for j = 1 to rightIndex
8         R[j] = A[q + j]
9     L[leftIndex + 1] = sentinel
10    R[rightIndex + 1] = sentinel
11    i = 1
12    j = 1
13    for k = p to r
14        if L[i] <= R[j]
15            A[k] = L[i]
16            i = i + 1
17        else A[k] = R[j]
18            j = j + 1

```

At the end of the for loops on 4-5, 6-7, new arrays will be created, holding the left and right half of the data in the passed array.

Initialization Initially we have two arrays, the left and right side of the original passed array. This holds trivially.

Maintenance During each iteration, we copy over the array's data.

Termination Upon termination, the Left array has the data from $[p + i - 1]$ and Right array has the data of $[q + j]$.

During lines 12 - 17, we merge the arrays to get a properly sorted array.

Initialization Initially we have the left and right arrays, unsorted, and the original array. This holds trivially.

Maintenance During the iterations of the array, we replace the data of the originally passed array with the smaller of the Left and Right array.

Termination Upon termination, we have a fully sorted array from $q \dots r$.

4 Quicksort

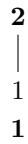
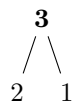
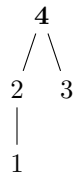
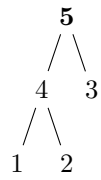
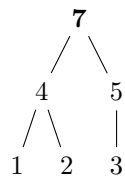
Let $_$ signify the pivot, $\|$ signify the wall.

1. $\|$ 4 2 7 6 3 5 1
2. 1 $\|$ $\|$ 4 2 7 6 3 5
3. 1 $\|$ 4 $\|$ 2 7 6 3 5
4. 1 $\|$ 4 2 $\|$ 7 6 3 5
5. 1 $\|$ 4 2 3 $\|$ 7 6 5
6. 1 $\|$ 4 2 3 $\|$ 5 $\|$ 7 6
7. 1 $\|$ $\|$ 4 3 $\|$ 5 $\|$ 7 6

8. 1 || 2 || 4 3 || 5 || 7 6
9. 1 || 2 || 3 || 4 || 5 || 7 6
10. 1 || 2 || 3 || 4 || 5 || || 7 6
11. 1 || 2 || 3 || 4 || 5 || 6 || 7
12. 1 || 2 || 3 || 4 || 5 || 6 || 7

5 Heapsort

5.1 Heap Representation



5.2 Enumerated Steps

1. 7
2. 7 5
3. 7 5 4

4. 7 5 4 3

5. 7 5 4 3 2

6. 7 5 4 3 2 1

Homework #3

Illya Starikov

Due Date: February 28, 2016

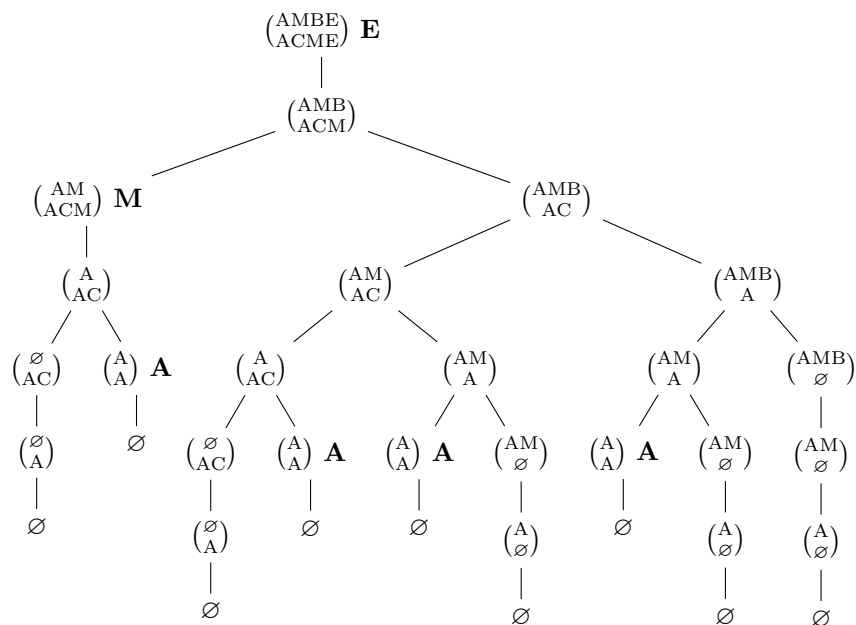
1 Matrix Multiplication

	1	2	3	4
1	0	$20^{[1]}$	$35^{[2]}$	$65^{[3]}$
2	X	0	$12^{[2]}$	$28^{[3]}$
3	X	X	0	$6^{[3]}$
4	X	X	X	0

Therefore, the minimum scalar operations can be achieved via $((A_1 \times A_2) \times A_3) \times A_4$

2 Longest Common Subsequence

2.1 Recursion Tree



Therefore, the longest common substring is AME.

2.1.1 Trace Table

	$\emptyset$	A	M	B	E
$\emptyset$	0	0	0	0	0
A	0	1	1	1	1
C	0	1	2	1	1
M	0	1	2	2	2
E	0	1	2	2	3

Therefore, the longest common substring is AME.

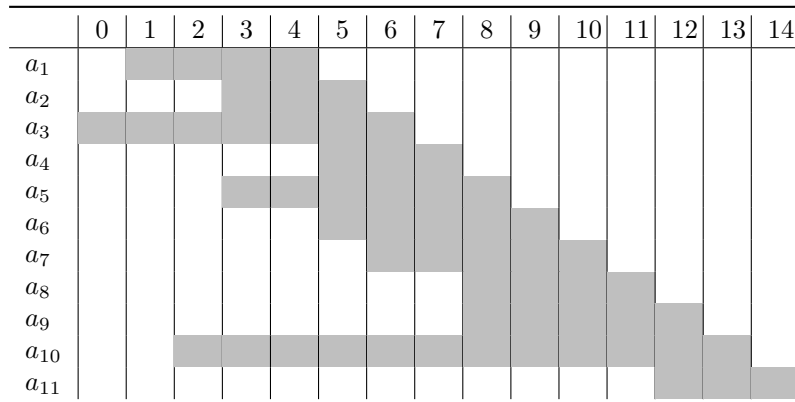
Homework #4

Illya Starikov

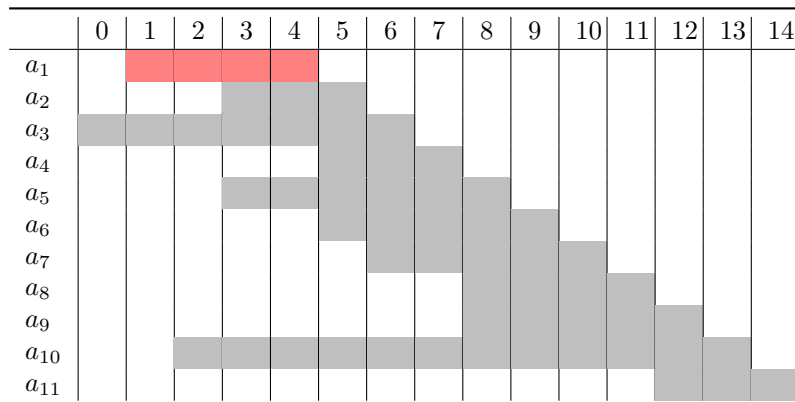
Due Date: March 23, 2016

1 Activity Selection Problem

Plotting this out on a graph,



We can plainly see the optimal solution to be $\{a_1, a_4, a_8, a_{11}\}$. We get this solution by using the ending first greedy algorithm. Here are the steps to obtain the solution.

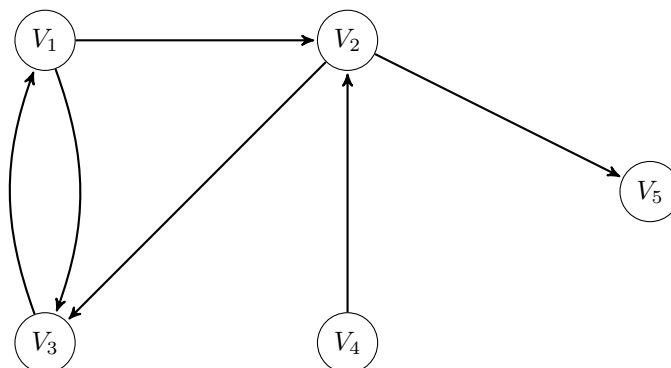


	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a_1															
a_2															
a_3															
a_4															
a_5															
a_6															
a_7															
a_8															
a_9															
a_{10}															
a_{11}															

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a_1															
a_2															
a_3															
a_4															
a_5															
a_6															
a_7															
a_8															
a_9															
a_{10}															
a_{11}															

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
a_1															
a_2															
a_3															
a_4															
a_5															
a_6															
a_7															
a_8															
a_9															
a_{10}															
a_{11}															

2 Graph And Depth First Search



2.1 Depth First Search

Ouput: V_1 V_2 V_3 V_5 V_4
 Distance: 0 1 1 2 ∞

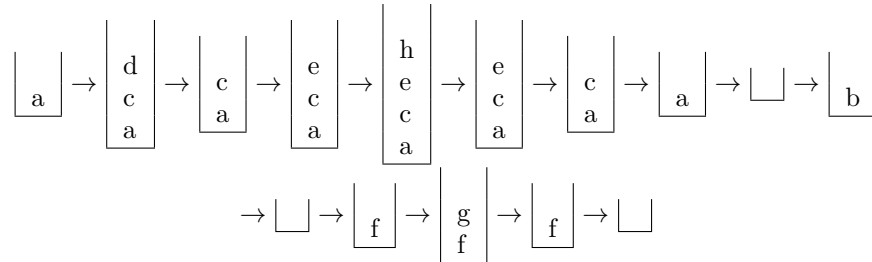
2.1.1 Queue Status

V_1
V_2 V_3
V_3 V_5
V_5
V_4

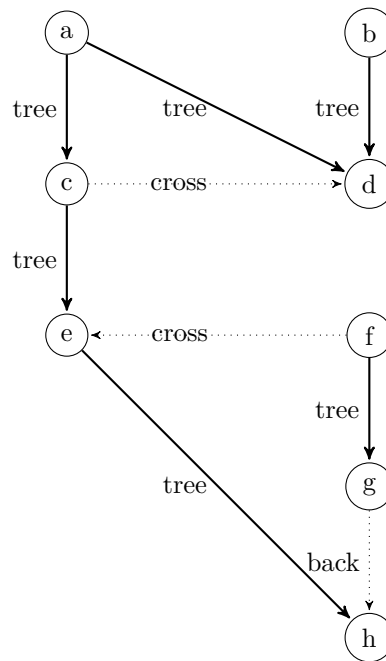
3 Depth First Search and Topological Sorting

Ouput	Discovery	Finish
a	1	10
c	2	9
d	3	4
e	5	8
h	6	7
b	11	12
f	13	16
g	14	15

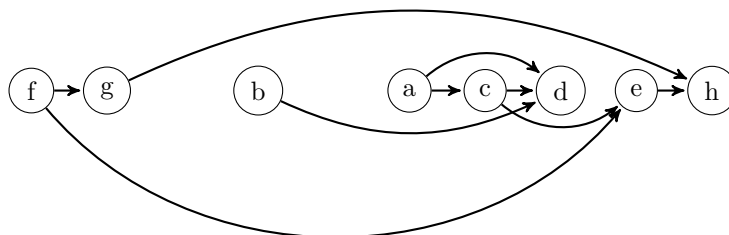
3.0.1 Queue Status



3.0.2 Edge Classification



3.0.3 Topological Sorting



4 Kruskal's and Prim's Algorithm

Please note that my tables are different, however the algorithm is the same is the same.

4.1 Kruskal

Kruskal's works the same way, except my column headers are more verbose and there an Iteration Number column, making it easy to keep track of large algorithms.

Iteration Number	Edge	Weight	Action Taken
1	$\{u, y\}$	1	Added
1	$\{u, x\}$	1	Added
2	$\{s, t\}$	2	Added
3	$\{t, y\}$	2	Added
4	$\{s, u\}$	3	Not Added
5	$\{s, x\}$	3	Not Added
6	$\{t, x\}$	3	Not Added

4.2 Prim

Prim's is also the same, with the exception that my headers are also verbose and the table is sorted linearly by the vertexes and edges that are added. The process is still the same of starting a vertex and branching out based on the best possible choice at the moment.

Iteration Number	Vertex Added	Edge Added	Weight
0	s		
1	t	$\{s, t\}$	2
2	x	$\{t, y\}$	2
3	u	$\{y, u\}$	1
4	y	$\{u, x\}$	1

Both algorithms produce the same result.

Programming Project I, First Report

Illya Starikov, Claire Trebing, & Timothy Ott

Due Date: March 07, 2016

1 Abstract

Smartphone users launch many apps everyday, however one of the most fundamental things a smartphone does is abstracted away: memory management.

Although smartphones have advanced significantly in many ways compared to their first predecessors (in terms of RAM, architecture, processors), deactivation, (the process of “the operating system needing to choose and remove some apps from the memory”, a subproblem of **memory management**) is a solution that is often less-than-perfect. Although Java’s **Garbage Collection** and Swift’s **Automatic Reference Counting** (ARC) have sufficed, there are other methods.

In this project I propose to solve this problem using three techniques:

- Brute Force
- Dynamic Programming
- Greedy Solution

2 Introduction and Motivation

As stated previously, memory management is solved in a less-than-perfect manner. Although current technology suffices, we would like to compare algorithms to show the significant gains via three different approaches (Brute Force, Dynamics Programming, and Greedy).

3 Proposed Solution

For our project we decided to take a more **skeuomorphic** and object oriented approach, modeling objects after their real world counterparts, such as Application or Smartphone. As for the approaches, we have the following solutions:

3.1 Brute Force

For the brute force method, we knew that we had to check every possible subset (and for a set of size n , we know there to be 2^n subsets). We used this to our advantage, creating a lookup table modeled after a truth table. As an example, suppose we have a three items in our knapsack:

Items A	Item B	Item C	Item Number
0	0	0	0
0	0	1	1
0	1	0	2
0	1	1	3
1	0	0	4
1	0	1	5
1	1	0	6
1	1	1	7

We notice that the knapsack combination can be directly summarized by the n -bit binary representation. Using the previous example of three item knapsack:

$$\text{Subset \#0 : } 0_{10} = 000_2 = \sim (ABC) = \text{No Items} \quad (1)$$

$$\text{Subset \#5 : } 5_{10} = 101_2 = A \sim (B)C = \text{Items A, B} \quad (2)$$

$$\text{Subset \#7 : } 7_{10} = 111_2 = ABC = \text{Items A, B, C} \quad (3)$$

Now that we have a representation of every subset, we can multiply the columns value by the knapsack value to get the item's value into the lookup table. Extending the example, suppose we have the following table:

App	Memory	Cost
A	2	4
B	3	6
C	7	9

Multiplying the columns by the values produces the following results:

App A	App B	App C		App A	App B	App C
0×4	0×6	0×9		0	0	0
0×4	0×6	1×9		0	0	9
0×4	1×6	0×9		0	6	0
0×4	1×6	1×9	=	0	6	9
1×4	0×6	0×9		4	0	0
1×4	0×6	1×9		4	0	9
1×4	1×6	0×9		4	6	0
1×4	1×6	1×9		4	6	9

Adding the columns across we produce the following, and factoring the weight:

Items A	Item B	Item C	Total Cost	Total Memory
0	0	0	0	0
0	0	9	9	7
0	6	0	6	3
0	6	9	15	10
4	0	0	4	2
4	0	9	12	9
4	6	0	10	5
4	6	9	19	12

Now finding the minimal value is straightforward, find the minimal value is traversing down the additive value while looking at the size it frees up. Suppose we have to free up 10 blocks of memory, items A, B would be the most efficient choice.

The pseudocode is as follows, with a few notes:

- Comparison is done while “generating” a table.
- A table is not generate, but the binary representation is used instead.

3.1.1 Pseudocode

```

knapsackBrute(items, knapsackSize)
    mininum = 0
    for i = 0 to  $2^n - 1$ 
        subset = binaryToInteger(i)
        sum = 0

        for i to subset.length
            sum = sum + item[i].benefit * subset[i]
            size = size + item[i].weight * subset[i]

        if size <= knapsackSize && sum < mininum
            mininum = sum
            greatestSubset = i

    subset = binaryToInteger(greatestSubset)
    for i = 0 to subset.size
        if subset[i] == 1
            optimalSolution.append(item[i])

    return optimalSolution

```

3.1.2 Time Complexity

Analyzing this algorithm, we can see the complexity:

$$\sum_{k=1}^n 2^k \times \sum_{k=1}^n c = \mathcal{O}(n2^n) \quad (4)$$

We run in $\mathcal{O}(n2^n)$ time.

3.2 Dynamic Programming

The approach to dynamic programming came from creating a table of the possible memory and cost combinations. The table would contain rows for the values from 0 to M, the largest possible memory we would have to free up. The program starts by finding the lowest amount of memory an app can take up and the lowest cost. This will act as a base case. For example, if the lowest memory amount is 3 and the cost is 1, then if the user needed to clear out 1, 2, or 3 memory spaces then the phone would close that app. Amounts greater than 4 would be calculated when the program started.

3.2.1 Pseudocode

Create_Delete_Table(numberOfApps, numberToDelete, Memory[], Cost[]){

Let d[1 ... M, 3] be new table as shown below

Memory To Delete | Nr. of Apps | Cost | List of Apps Used

0

1

2

3

...

M

int low = findMin(d[], Memory[], Cost[]);

//This function will find a base case for an app with
the lowest memory and lowest cost

for (j = 1 to low)

{

d[j, 0] = d[low, 0];

d[j, 1] = d[low, 1];

d[j, 2] = d[low, 2];

d[j, 3] = d[low, 3];

}

for(j = low to M)

{

for(i = 1 to N){

if(Mi == j && Ci < d[j-low,2] + d[low,2]){

d[j, 1] = 1;

d[j, 2] = Ci;

```

        d[j, 3] = i;
    }

    else{
        d[j,1] = d[j-low,1]++;
        d[j,2] = d[j-low,2] + d[low,2];
        //add app to the list
    }
}
}
return;
}

```

3.2.2 Time Complexity

We can see that this algorithm is $\mathcal{O}(n^2)$.

3.3 Greedy Solution

After considering several greedy algorithm solutions we settled on selecting based on a ratio of cost and memory freed by deactivating a particular app. Once each of the apps' ratio has been evaluated (and the ratio has been stored as a member variable) we sort the array of apps in an ascending order according to this new value. Because the apps are now in order in ascending cost per unit of memory we can now simply add apps to the solution set in the order they appear in the array until our memory requirements are met. For example, if we take our example set of three items:

App	Memory	Cost
A	2	4
B	3	6
C	7	9

We first take the ratio of each item's cost over the capacity of memory it takes up and sort the elements accordingly. Doing this gives us the following updated table:

App	Memory	Cost	Ratio
C	7	9	1.29
A	2	4	2
B	3	6	2

And now we simply add apps until our memory requirement has been met or exceeded. Suppose we need to free up 9 units of memory, according to this approach the optimal solution would be items C and A.

It is worth noting, however, that this solution doesn't always produce the absolute optimal solution. If we use the previous example of freeing up 10 units

of memory this algorithm produces a solution of all the apps, A, B and C as opposed to the actual optimal solution of app C and B. This inaccuracy is a sacrifice in order to benefit from a much lower runtime than that of the other solutions.

We could perhaps introduce a second greedy choice or to further refine our algorithm to improve the accuracy but we would do so at the cost of performance and the solutions chosen by our initial solution are accurate enough for our purposes.

3.3.1 Pseudocode

```
knapsackGreedy()
  A[] = Array of Apps
  S[] = Array of solution set
  M = Desired amount of memory to be freed
  sm = Amount of memory freed by solution set

  n = A.length

  for 0 to n
    A[i].ratio = A[i].cost/A[i].memory

  mergesort(A)

  GreedKnap(A, S)
    S[0] = A[0]
    sm = A[0].memory
    i = 1
    while (sm < M AND i < n)
      S[i] = A[i]
      sm += A[i].memory
```

3.3.2 Time Complexity

Analyzing this algorithm, including both going over the array to produce the ratio that will guide our greedy choices and choosing the actual solution set, we come up with the complexity of:

$$\sum_{k=1}^n k + \sum_{k=1}^n k = \mathcal{O}(2n) \quad (5)$$

This algorithm therefore runs in $\mathcal{O}(2n)$ time.

4 Plan of Experiments

The main objective of our experiments is to extrapolate time and efficiency from the algorithms for sufficiently large input, so our plan of experiments is thus:

1. Generate a random array of apps, and a smartphone with n memory.
2. Take a date stamp.
3. Run the Brute Force algorithm for the random array of apps and the smartphone.
4. Take another date stamp, record the difference.
5. Return to step 2 and run the Dynamic Programming algorithm.
6. Likewise, return to step 2 and run the Greedy algorithm.
7. Record the results from the experiment.
8. Return to step 1, repeat 9 additional times, take the average.
9. Run the algorithm again for a scale factor of n (the input size).

5 Team Roles

Illya Starikov Project Management, Development (Brute Force)

Timothy Ott Development (Greedy), Architecture

Claire Trebing Development (Dynamic), Quality Assurance, Documentation

Programming Project I, Second Report

Illya Starikov, Claire Trebing, & Timothy Ott

Due Date: March 17, 2016

1 Abstract

As proposed in the first report, we would like to compare the efficiency of different memory management methods via a **0-1 Knapsack**. We discussed the current memory management methods, the different approaches we would like to use (Brute Force, Dynamic Programming, Greedy Solution) and our plan of implementations.

After implementing the pseudocode provided in Report #1, rigorous testing and extensive running of sample of data, we have completed conclusive evidence to our proposed problem.

In this report we would like to discuss our findings.

2 Implementation

As we discussed in our earlier report we decided to go with a more object-oriented approach to this problem. We created a class called App to represent each application running on our platform. Each instance of the App class contains (as properties¹) an identifying number, its memory usage, and its cost to recover from storage. These properties are largely randomized in the construction of each instance. For example, every app has a random memory usage in the range of 32 to 1028; the cost is a derived value, taking anywhere from 20 to 50 percent of the memory usage of the App in question; we also have a property called ratio which consists of the cost property divided by the memory property and is calculated at creation of the instance by the constructor, this will be used in our Greedy Algorithm; and lastly each App is given an identifying number based on the index of the cell in the array in which they are initially placed. This ensures that each App will have a unique identifier to help differentiate them whilst checking our solutions against one another.

Initially we considered creating a separate class for our “Smartphone” or the platform on which our Apps would be running but later decided against this as this functionality could easily be represented simply by a list (or array) of all the Apps, cutting down on the amount of code we would need.

¹Or member variables, depending on your preferred notation.

We implemented our three solutions as one overarching test program that produces arrays of apps in ever increasing input sizes and feeding those arrays to our algorithms one at a time, testing the average runtime over 10 iterations. The general flow of our program is as follows:

First we specify a list of input sizes that our program will act on, we will discuss this in more detail further along but for now it should be noted that our first input size is 5. Our program then creates an array of newly created Apps of the specified input size². These newly created apps have each of their properties randomly assigned by its constructor. Next, because our Greedy Solution requires the inputs be sorted based on its ratio variable and the other solutions do not, we sort the list using Python’s built in sort function.

Now we simply feed the newly sorted list to each of the solution functions in turn, timing the run time of each by taking a timestamp before running the algorithm, again after, then subtracting the two times. We run each program ten times per input and take the average of these times for greater accuracy. Once each solution algorithm has run ten times the program outputs the data (which includes the solution set, the amount of memory freed by the solution, the combined cost of the solution and the average time it took) to two text files in the directory of the program.

It is at this point that our program returns to the top of the loop for the next input. To ensure a adequate spread of input sizes we defined a list of “base inputs” which we initialized as 5, 10, 15, and 25. Once the program finishes iterating over this list, testing each solution as mentioned above, the program returns to the beginning of the input list and multiplies each by 10 to the power of n , where n is incremented with each new round of the loop back to the beginning of the base inputs. This entire process is placed in a while loop that is set to infinitely repeat so that we are able to get as many data sets as the program can output in the time we have allotted for testing.

3 Experiment

As described above, the tests were run for a twelve hour period getting our input size to 1,000. Noting that the for input size of 1,000 it took the Dynamic Programming approach 1334.81 seconds, the total time to get accurate results was *3 hours, 42 minutes, and 28 seconds*³. We thought this was a good cut off point.

To ensure accuracy, we decided to run the tests on two independent machines. These tests can be seen on the graphs, but not the data sets. The tests were fairly comparable, so we decided that displaying data from one data set was sufficient.

We noticed that the Brute Force solution was taking a drastically longer time to run (the complexity being $n \times 2^n$). After a sample run, the Brute Force algorithm returned a Memory Error for an input size of 50. Seeing as we were

²Again, in our first case this is 5.

³1334.81 seconds $\times$ 10 iterations = 13348.1 seconds = 222.47 minutes.

not likely to be able to have sufficient enough time, we decided to limit the Brute Force to a maximum of 25 inputs. Below are the results.

3.1 Results Data Set

Below you can find the results from *one of our test*. As stated above, the Brute Force only goes to 25 inputs. Note that the input column is the Input size of applications, the Algorithm is the type of algorithm we used, the Memory was the goal memory we wanted to produce, the Cost was what we wanted to minimize, the Freed column is the memory that was actually freed (because we could go over the goal if it was an optimal solution) and finally the Time is the time taken (strictly of the algorithm in question) measured in seconds.

Input	Algorithm	Memory	Cost	Freed	Time
5	BRUTE	609	206.960013686	769	0.0000652
	DYNAMIC	609	413.920027372	1538	0.008588
	GREEDY	609	206.960013686	769	0.00572586666667
10	BRUTE	1254.0	306.007083914	1307	0.0023169
	DYNAMIC	1254.0	913.397426561	3126	0.033041
	GREEDY	1254.0	418.629747	1961	0.0220281
15	BRUTE	1896.2	462.268333337	1980	0.0933534
	DYNAMIC	1896.2	706.822856172	3129	0.10722175
	GREEDY	1896.2	506.866912451	2383	0.0714820333333
25	BRUTE	2469.0	619.523226819	2479	113.6533167
	DYNAMIC	2469.0	1204.68191427	4035	56.92951935
	GREEDY	2469.0	620.454984919	2562	37.9530137333
50	DYNAMIC	5421.2	1664.89861527	7120	0.870269
	GREEDY	5421.2	1177.03320938	5519	0.43513695
100	DYNAMIC	9783.2	2809.24740865	11268	3.8071061
	GREEDY	9783.2	2392.11409139	9849	1.90355725
150	DYNAMIC	14995.8	4027.50565053	16594	9.4551353
	GREEDY	14995.8	3696.37456362	15462	4.7275798
250	DYNAMIC	26793.4	6591.81933106	28200	32.8375575
	GREEDY	26793.4	6272.26215744	27018	16.41878765
500	DYNAMIC	52354.6	12547.9234913	53429	192.5988691
	GREEDY	52354.6	12365.0250774	52842	96.2994775
1000	DYNAMIC	107148.2	25671.8344823	109000	1334.8144697
	GREEDY	107148.2	25256.6939401	107895	667.4074229

3.2 Results

While looking at the data we got back, we notice a very noticeable trend immediately: the exponential nature of the quasi-exponential nature of the algorithms. When plotting on an input size vs. time graph, we could see even the algorithm with $\mathcal{O}(n)$ (the Greedy Solution) complexity showed some exponential growth. Taking into consideration the [RAM Model of Computing](#) is not completely applicable in this situation, the results are more than staggering.

As can be seen in the above figure, although the input follows the pattern of $5n, 10n, 15n, 25n | n \in \mathbb{Z}^+$, the results typically follow a pattern of $2\times$ their predecessor.

When comparing the Time vs. Costs, the trend is also quite exponential. For, as our algorithms get a larger cost, the time is unquestionably larger. When actually looking at the Time vs. Accuracy, the clear cut winner is Greedy. The time saving are immense compared to Dynamic and Brute Force. On certain cases, Greedy can finish before Dynamic and Brute force combined.

We also see that Greedy Solution outperformed the Dynamic in not only time but accuracy. We are unsure of the reasoning behind this, for our algorithm certainly seemed correct on paper.

Tying this back into the real world example, none of these solution would be able to work. Assuming a smartphone of 2 GB RAM⁴, figuring an optimal solution would take a bare minimum of 38 seconds (not to mention 2 minutes if we are using the Brute Force.)

While this may not be applicable to the real world, it was an interesting thought experiment.

4 Roles

- Illya Starikov
 - Project Manager
 - Brute Force Development
 - Abstract, Experiment Writement
- Timothy Ott
 - Greedy Algorithm Development
 - Development, Conclusion, Interesting Results Writeup
 - Testing Documents

⁴We are assuming our memory is equivalent to 1 MB. Therefore, 2 GB = 2000 MB

- Claire Trebing
 - Dynamic Programming Development
 - Experiment Writing Up
 - Quality Assurance

5 Interesting Results

In the process of going over the data generated in our experiments we noted several interesting results that we would like to highlight here.

- We noticed that for input sizes under roughly 15 the Brute Force method was both more accurate, which we expected, but also significantly faster than the other two methods. When we take into consideration that the Brute Force's complexity is $\mathcal{O}(n2^n)$ and the Greedy Solution is $\mathcal{O}(2n)$, this is an excellent highlight to the fact that the growth of the function is far more important than its initial run speeds.
- There is an interesting anomaly in the data set for the input size of 25. At this point the run times for each algorithm spike noticeably only to then immediately dip once again. What's more interesting is that as we mentioned earlier we ran these experiments twice, independently of one another on two different machines and this anomaly occurred in both rounds of experimentation. So, what is so special about 25 inputs? The short answer is we are not sure. Our best guess is that the increase and subsequent drop are somehow related to the Brute Force algorithm since that is where we stopped testing that particular algorithm. Particularly we think this may have been the fault of the language's garbage collection functionality but without further testing we cannot be sure.
- Also of note is that while initially our dynamic approach was a good deal less accurate than either the Brute Force or even the Greedy approach, as the input sizes increased the accuracy of the dynamic algorithm also increased.

6 Conclusion

Memory management is a complex issue with many interesting solutions of varying effectiveness. With this project we explored three approaches to solve this issue using a smartphone's memory management system as our testing ground. Our solutions took the forms of:

- A brute force approach, comparing every possible subset of solutions to find an optimal choice at the expense of a high complexity and eventual run-time;

- A dynamic programming solution which built a table of solutions to effectively try every solution but with a lower complexity due to being able to look up values that were already computed; and
- A greedy algorithm approach which took a ratio of the cost and memory and produced a solution that was simply taking the lowest ratio apps until the memory requirements were met, in effect trading the accuracy of the other two approaches for a significant increase in speed.

Through our data gathered during experimentation we see that these early predictions hold fairly consistently, despite some unaccounted for inaccuracy on the part of our dynamic solution. While it is tempting to label one solution as the “best” or “optimal” solution, this would be misleading as each approach shines under different circumstances. For instance, for small input sizes the brute force algorithm seemed to exceed both other solutions in terms of both accuracy and speed. For higher input sets the greedy algorithm was the clear winner in terms of speed but theoretically would lose out on accuracy to the dynamic solution every time. Thus, we see that is important to tailor your solutions to the problem and not simply favor one approach over another arbitrarily.

Programming Project II, First Report

Illya Starikov, Claire Trebing, Timothy Ott

Due Date: April 22, 2016

1 Abstract

Social Networks have revolutionized the way we communicate, meet others, consume information, and essentially influence our day-to-day lives. To show Social Network's prominence, [here are the percentages of online adults who use social media](#):

Facebook : 71% Adults

Twitter : 23% Adults

Instagram : 26% Adults

Pinterest : 28% Adults

LinkedIn : 28% Adults

This is unprecedented. Seeing as an overwhelming majority of adults have some sort of social media account, this can be used to model real world relationships — through social graphs.

In this experiment we would like to examine the social graph through the follower-followee relationship.

2 Introduction and Motivation

As stated above, social networks play a dominant role in our lives. Through social graphs, we can examine real world relationships.

There are many reasons for this to be valuable, such as common interest, community detection, influence amongst followers, etc. For this experiment, we would just like to test on the following measures:

Degree Distribution Test the direct amount of followers compared to followees a person has.

Shortest Path Distribution Test the degree of separation between a follower-followee.

Graph Diameter Test the breadth of a community.

Closeness Centrality Test the dependence of a degree of separation on a singular person.

Betweenness Centrality Distribution Same as previous.

Community Detection Based on Closeness Centrality, find the diameter of a community.

This will give us a reasonable dataset for the relationship of a community in a social graph.

3 Proposed Solutions

3.1 Unweighted In Degree Distribution

Unweighted In Degree Distribution goes through each entry to determine what the origin of each edge is. An array with cells for each possible vertex (total) is created and once the origin is determined the respective cell in total is increased by one. After all of the entries have been checked, the function goes through the totals.

The functions begins by assuming `total[0]` is the maximum number of In Degree edges and 0 is added to `arrayMax`. `arrayMax` is the array which holds all of the vertexes with the maximum number of in degree edges. Each entry in total is compared to max. If the total entry is larger than the current max, then the current max is replaced with current total. The `arrayMax` list is cleared and the current total origin is added to the `arrayMax`. If the current total is equal to the max, then the origin of current total is added to `arrayMax`.

```
void Unweighted_InDegree_Distribution(&int arrayMax[])
{
    N = the number of vertices
    int * total;
    total = new int [maxVertice];

    for(i = 0 to N){
        total[origin of i]++;
    }
    int degreeMax = total[0];
    add 0 to arrayMax

    for(i = 1 to maxVertice){
        if(total[i] > degreeMax){
            degreeMax = total[i];
            clear arrayMax
            add i to arrayMax
        }
    }
}
```



```

    }
    if(total[i] == degreeMax){
        add i to arrayMax
    }
}
delete[] total;
return;
}

```

The time complexity of this function is $2n = \mathcal{O}(n)$. The function must traverse through the array twice before finishing.

3.2 Unweighted Out Degree Distribution

Unweighted Out Degree Distribution goes through each entry to determine what the destination of each edge is. An array with cells for each possible vertex (`total`) is created and once the destination is determined the respective cell in `total` is increased by one. After all of the entries have been checked, the function goes through the totals. The function begins by assuming `total[0]` is the maximum number of Out Degree edges and 0 is added to `arrayMax`. `arrayMax` is the array which holds all of the vertexes with the maximum number of out degree edges. Each entry in `total` is compared to `max`. If the `total` entry is larger than the current `max`, then the current `max` is replaced with current `total`. The `arrayMax` list is cleared and the current `total` destination is added to the `arrayMax`. If the current `total` is equal to the `max`, then the destination of current `total` is added to the `arrayMax`.

```

void Unweighted_OutDegree_Distribution(&int arrayMax[])
{
    N = the number of vertices
    int * total;
    total = new int [maxDestination];

    for(i = 0 to N){
        total[Destination of i]++;
    }
    int degreeMax = total[0];
    add 0 to arrayMax

    for(i = 1 to maxOrigin){
        if(total[i] > degreeMax){
            degreeMax = total[i];
            clear arrayMax
            add i to arrayMax
        }
        if(total[i] == degreeMax){

```

```

        add i to arrayMax
    }
}
delete[] total;
return;
}

```

3.2.1 Complexity Analysis

The time complexity of this function is $2n = \mathcal{O}(n)$. The function must traverse through the array twice before finishing.

3.3 Weighted In Degree Distribution

Weighted In Degree Distribution goes through each entry to determine what the origin of each edge is and the sum of the weighted edges. An array with cells for each possible vertex (*total*) is created and once the origin is determined the respective cell in *total* is increased by weight of that edge. After all of the entries have been checked, the function goes through the totals.

The functions begins by assuming *total*[0] is the maximum number of Weighted In Degree edges and 0 is added to *arrayMax*. *arrayMax* is the array which holds all of the vertexes with the maximum number of weighted in degree edges. Each entry in *total* is compared to *max*. If the *total* entry is larger than the current *max*, then the current *max* is replaced with current *total*. The *arrayMax* list is cleared and the current *total* origin is added to the *arrayMax*. If the current *total* is equal to the *max*, then the origin of current *total* is added to the *arrayMax*.

```

void Weighted_InDegree_Diribution(&int arrayMax[])
{
    N = the number of vertices
    int * total;
    total = new int [maxOrigin];

    for(i = 0 to N){
        total[Origin of i] += (Weight of i);
    }
    int degreeMax = total[0];
    add 0 to arrayMax

    for(i = 1 to maxOrigin){
        if(total[i] > degreeMax){
            degreeMax = total[i];
            clear arrayMax
            add i to arrayMax
        }
    }
}

```

```

        if(total[i] == degreeMax){
            add i to arrayMax
        }
    }
    delete[] total;
    return;
}

```

3.3.1 Complexity Analysis

The time complexity of this function is $2n = \mathcal{O}(n)$. The function must traverse through the array twice before finishing.

3.4 Weighted Out Degree Distribution

Weighted Out Degree Distribution goes through each entry to determine what the destination of each edge is and the weight of those edges. An array with cells for each possible vertex (`total`) is created and once the destination is determined the respective cell in `total` is increased by the value of that edge. After all of the entries have been checked, the function goes through the totals. The functions begins by assuming `total[0]` is the maximum number of Weighted Out Degree edges and 0 is added to `arrayMax`. `arrayMax` is the array which holds all of the vertexes with the maximum number of weighted out degree edges. Each entry in `total` is compared to `max`. If the `total` entry is larger than the current `max`, then the current `max` is replaced with current `total`. The `arrayMax` list is cleared and the current `total` destination is added to the `arrayMax`. If the current `total` is equal to the `max`, then the destination of current `total` is added to the `arrayMax`.

```

void Weighted_OutDegree_Diribution(&int arrayMax[])
{
    N = the number of vertices
    int * total;
    total = new int [maxDestiation];

    for(i = 0 to N){
        total[Destination of i] += (Weight of i);
    }
    int degreeMax = total[0];
    add 0 to arrayMax

    for(i = 1 to maxDestiation){
        if(total[i] > degreeMax){
            degreeMax = total[i];
            clear arrayMax
            add i to arrayMax
        }
    }
}

```

```

    }
    if(total[i] == degreeMax){
        add i to arrayMax
    }
}
delete[] total;
return;
}

```

The time complexity of this function is $2n = O(n)$. The function must traverse through the array twice before finishing.

3.5 Shortest Path

For our Shortest Path algorithm we decided to implement the Floyd-Warshall algorithm. We chose this algorithm because of the need to examine the shortest path for all pairs of vertices as well as the lower complexity of the dynamic programming solution. The Floyd-Warshall algorithm creates an array of $V \times V$ size keeping a running tally of the shortest path while adding vertices to the list of possible intermediate points on those paths.

```

V = the number of vertices
distance = new array[V][V]
Initialize distance to -1 //since there are no negative weights this will
                        //represent infinity

```

```

Floyd(V):
    for each vertex v:
        distance[v][v] = 0
    for each edge (u,v)
        distance[u][v] = w(u,v) //the weight of the edge (u,v)
    for k from 1 to V
        for i from 1 to V
            for j from 1 to V
                if distance[i][j] > distance[i][k] + distance[k][j]
                    distance[i][j] = distance[i][k] + distance[k][j]

```

```

shortest = new array[100]
initialize shortest to 0
for i from 1 to V
    for j from 1 to V
        if distance[i][j] > 0
            if distance[i][j] > shortest.length
                temp = new array[distance[i][j]+10]
                for i from 0 to shortest.length
                    temp[i] = distance[i]
                delete distance

```

```

        distance = temp
        shortest[distance[i][j]]++

for i from 1 to shortest.length
    if shortest[i] > 0
        output shortest[i]

```

3.5.1 Complexity Analysis

As we know, the Floyd-Warshall algorithm operates on n^3 time. Combining this with the algorithm to iterate over the resulting $V \times V$ matrix results in a $n^2 + n^3$ complexity — or, $\mathcal{O}(n^3)$ time.

3.6 Unweighted Graph Diameter

Our solution first creates an $N \times N$ array to hold all of the shortest paths from each vertex to every other vertex. For an unweighted graph the shortest path from $i \rightarrow j$ is equal to the shortest path from $j \rightarrow i$. After creating and filling the matrix, the function assumes that the value at `distance[0][1]`. The function then tests every value in the matrix and compares it to the max. If the value is greater than the max then the max is replaced with `distance[i][j]`. Clear the `GraphDiameterList` and then add (i, j) to the graph diameter list. If `distance[i][j]` is equal to max then add (i, j) to the `GraphDiameterList`.

The pseudocode is as follows.

```

Unweighted Graph Diameter {
    N = the number of vertices
    distance = new array[N][N]
    all distances start at -1

    for(i = 0 to N){
        for(j = i+1 to N){
            distance[i][j] = shortest path(i, j)
        }
    }

    max = Diameter[0][1]
    Add (0,1) to GraphDiameterList

    for(i = 0 to N){
        for(j = 1 to N){
            if(distance[i][j] > max){
                clear GraphDiameterList
                max = distance[i][j]
                add (i,j) to GraphDiameterList
            }
        }
    }
}

```

```

        if(distance[i][j] = max){
            add (i,j) to GraphDiameterList
        }
    }
    return GraphDiameterList
}

```

3.6.1 Complexity Analysis

The time complexity of this code is $\mathcal{O}(n^2)$. You must go through the matrix twice to complete this function.

3.7 Weighted Graph Diameter

This functions first created an $N \times N$ array to hold all of the shortest paths from each vertex to every other vertex. After creating and filling the matrix, the solution assumes that the value at `distance[0][1]`. The function then tests every value in the matrix and compares it to the max. If the value is greater than the max then the max is replaced with `distance[i][j]`. Clear the `GraphDiameterList` and then add (i,j) to the graph diameter list. If `distance[i][j]` is equal to max then add (i,j) to the `GraphDiameterList`.

The pseudocode is as follows:

```

Weighted Graph Diameter{
    N = the number of vertices
    distance = new array[N][N]
    all distances start at -1

    for(i = 0 to N){
        for(j = 0 to N){
            if(i != j){
                distance[i][j] = shortest path(i, j)
            }
        }
    }

    max = Diameter[0][1]
    Add (0,1) to GraphDiameterList

    for(i = 0 to N){
        for(j = 0 to N){
            if(i != j)
            if(distance[i][j] > max){
                clear GraphDiameterList
                max = distance[i][j]
                add (i,j) to GraphDiameterList
            }
        }
    }
}

```

```

    }
    if(distance[i][j] = max){
        add (i,j) to GraphDiameterList
    }
}
return GraphDiameterList

```

3.7.1 Complexity Analysis

The time complexity of this code is $\mathcal{O}(n^2)$. You must go through the matrix twice to complete this function.

3.8 Closeness Centrality

The Closeness Centrality is defined as $C_c(i) = [\sum_{j=1}^N d(i, j)]^{-1}$. So in order to find the Closeness Centrality we first must find the sum of all the shortest paths for each vertices. Once we find this sum we simply record it as a float by dividing one by the sum and storing it in an array of Closeness values.

```

sum = 0
closeness = new array[100]
initialize closeness to 0
for i from 0 to V
    for j from 1 to V
        sum += distance[i][j]
    closeness[i] = 1/sum
    output closeness[i]
    sum = 0

```

3.8.1 Complexity Analysis

Because we are iterating over the entire matrix resulting from the Floyd-Warshall algorithm of $V \times V$ size. We arrive at a complexity of $\mathcal{O}(n^2)$.

3.9 Undirected Betweenness Centrality Distribution

This algorithms takes a vertex i from the main function. The function then looks at every shortest path possible. Starting at vertex 0 the function and going to the largest vertex. If i is the starting point (j) or the ending point (k) it is ignored. K is defined as $j + 1$ If the shortest path because in undirected graphs, the shortest distance from $j \rightarrow k ==$ the shortest distance from $k \rightarrow j$. If the shortest distance from j to k includes the vertex i , the value of the betweenness is increased by 1. The total betweenness is then returned to the main function.

```

Undirected Betweenness Centrality Distribution of Vetex i(vertex i){
    N = max vertex
    B = betweennesses = 0

```

```

for(j = 0 to N){
    if(i != j){
        for(k = j+1 to N){
            if(i != k){
                shortestpath between j and k
                if i is included, increase betweenesses by 1
            }
        }
    }
}

return B;
}

```

This function has a complexity of n . Each vertex check every vertex larger than itself to determine all of the shortest paths. The first vertex must check nearly other ever vertex, except i . This gives it a complexity of $\mathcal{O}(n)$.

3.10 Directed Betweenness Centrality Distribution

This algorithm takes a vertex i from the main function. The function then looks at every shortest path possible. Starting at vertex 0 the function and going to the largest vertex. If i is the starting point (j) or the ending point (k) it is ignored. The path is also ignored if j and k are equal. If the shortest distance from j to k includes i , then the betweenness is increased by 1. The total betweenness is then returned to the main function.

```

Directed Betweenness Centrality Distribution of Vertex i(vertex i){
    N = max vertex
    B = betweenesses = 0
    for(j = 0 to N){
        if(i != j){
            for(k = 0 to N){
                if(i != k && j != k){
                    shortest path between j and k
                    if i is included, increase betweenesses by 1
                }
            }
        }
    }
}

```

3.10.1 Complexity Analysis

This function has a complexity of n^2 . Each vertex must check every other vertex to determine all of the shortest paths. This gives it a complexity of $\mathcal{O}(n^2)$.

3.11 Undirected Unweighted Betweenness Centrality Distribution

This algorithm takes an edge e from the main function. The function then looks at every shortest path possible. Starting at vertex 0 the function and going to the largest vertex. K is defined as $j + 1$ because the shortest path in undirected graphs, the shortest distance from j to k == the shortest distance from k to j . If the shortest distance from j to k includes the edge e , the value of the betweenness is increased by 1. The total betweenness is then returned to the main function.

```
Undirected Unweighted Betweenness Centrality Distribution of Edge e(edge e){
    N = max vertex
    B = betweennesses = 0
    for(j = 0 to N){
        for(k = j+1 to N){
            shortest path between j and k
            if e is included, increase betweennesses by 1
        }
    }
    return B;
}
```

3.11.1 Complexity Analysis

This function has a complexity of n . Each vertex check every vertex larger than itself to determine all of the shortest paths. The first vertex must check nearly other ever vertex. This gives it a complexity of n .

3.12 Directed Unweighted Betweenness Centrality Distribution of Edge

This algorithm takes an edge e from the main function. The function then looks at every shortest path possible. Starting at vertex 0 the function and going to the largest vertex. If the shortest distance from j to k includes the edge e , the value of the betweenness is increased by 1. The total betweenness is then returned to the main function.

```
Directed Unweighted Betweenness Centrality Distribution of Edge e(edge e){
    N = max vertex
    B = betweennesses = 0
    for(j = 0 to N){
        for(k = 0 to N){
            if(j != k){
                shortest path between j and k
                if e is included, increase betweennesses by 1
            }
        }
    }
    return B;
}
```

```

    }
  }
}
return B;
}

```

3.12.1 Complexity Analysis

This function has a complexity of n^2 . Each vertex must check every other vertex to determine all of the shortest paths. This gives it a complexity of $\mathcal{O}(n^2)$.

3.13 Undirected Weighted Betweenness Centrality Distribution of Edge e

This algorithm takes an edge e from the main function. The function then looks at every shortest path possible. Starting at vertex 0 the function and going to the largest vertex. K is defined as $j + 1$ because the shortest path in undirected graphs, the shortest distance from $j \rightarrow k ==$ the shortest distance from $k \rightarrow j$. If the shortest distance from j to k includes the edge e , the value of the betweenness is increased by 1. The total betweenness is then returned to the main function divided by the weight of e .

```

Undirected Weighted Betweenness Centrality Distribution of Edge e(edge e){
  N = max vertex
  B = betweenness = 0
  for(j = 0 to N){
    for(k = j+1 to N){
      shortest path between j and k
      if e is included, increase betweenness by 1
    }
  }
  return B/(weight of e);
}

```

3.13.1 Complexity Analysis

This function has a complexity of n . Each vertex check every vertex larger than itself to determine all of the shortest paths. The first vertex must check nearly other ever vertex. This gives it a complexity of $\mathcal{O}(n)$.

```

Directed Weighted Betweenness Centrality Distribution of Edge e(edge e){
  N = max vertex
  B = betweenness = 0
  for(j = 0 to N){
    for(k = 0 to N){
      if(j != k){

```

```

        shortest path between j and k
        if e is included, increase betweennesses by 1
    }
}
}
return B/(weight of edge e);
}

```

This algorithm takes an edge e from the main function. The function then looks at every shortest path possible. Starting at vertex 0 the function goes to the largest vertex. If the shortest distance from j to k includes the edge e , the value of the betweenness is increased by 1. The total betweenness is then returned to the main function divided by the edge e .

3.13.2 Complexity Analysis

This function has a complexity of n^2 . Each vertex must check every other vertex to determine all of the shortest paths. This gives it a complexity of $\mathcal{O}(n^2)$.

3.14 Community Detection

To detect communities of vertices within the data set we set out to remove those edges that are most likely between those communities to make those communities more apparent. To do this we take the betweenness centrality of edges and sort them in descending order. Then we remove the edge with the highest value (or edges if there are more than one of the same value). Once this is complete we simply recalculate the matrix of shortest paths using the Floyd-Warshall algorithm and the graph diameter using the algorithm detailed above. We repeat this for a total of five times and the resulting diameters are our likely communities.

```

UWBetween[] = Unweighted Betweenness Centrality Edges
V = Original Network

```

```

For k from 0 to 4:
    DescendSort(UWBetween)
    UWBetween[0] = x
    while (UWBetween[0] == x)
        V.remove(UWBetween[i])
        UWBetween.remove(UWBetween[0])
    Floyd(V)           //Run Floyd-Warshall Algorithm on revised data set
    Diameter(V)         // Calculate and output max diameter based on new matrix from above
    Betweenness(V)      // Calculate Betweenness centrality based on
                        // new matrix giving us a new UWBetween array

```

3.14.1 Complexity Analysis

Because the Community detection algorithm calls the algorithms to find all shortest paths, the diameter and the unweighted betweenness centrality edges within it and since we are executing this algorithm a total of five times, the complexity of this algorithm is five times the sum of the complexities of these algorithms.

4 Plan of Experiments

The major purpose of our experiment is to dissect and examine networks, so we propose the following plan of experiments:

1. Extrapolate data and store in a relevant data structure — in our case, a sorted `map<int, vector<pair<int, double>>>`
 - The `int` is the key to be used, signifying the origin.
 - The `vector<pair>` holds all the adjacent edges.
2. Iterate over the entirety of the data structure to determine degree distribution.
 - For *weighted and unweighted out degree*, it is simply counting the the `vector` size with respect to each key.
 - For *weighted and unweighted in degree*, making an efficient algorithm is still difficult — not only is by default $\mathcal{O}(n)$ but there is an efficient way of finding where the edges lead to. We accomplish this by a [Binary Search Tree](#). By iterating over every vertex and storing their destination in a binary search tree, we have achieved a sufficient algorithm.
3. Detect the shortest path via the [Floyd-Warshall algorithm](#) for directed graphs.
 - Make the graph undirected by making it symmetric about the $a_{i,i}$ elements $\forall i \in \text{edges}$.
 - Test again.
4. Detect closeness centrality and betweenness.
5. Implement community detection.
6. Output results to user.

5 Team Roles

- Illya Starikov
 - Project Manager
 - Official Write-up
- Timothy Ott
 - Pseudocode Write-up
 - Algorithm Analysis
- Claire Trebing
 - Pseudocode Write-up
 - Algorithm Analysis

Programming Project II, Second Report

Illya Starikov, Claire Trebing, Timothy Ott

Due Date: May 1st, 2016

1 Abstract

As stated in the prior report, social networks have revolutionized the way we communicate. Seeing as social *networks* are a real-life representation of a graph, we would like to more closely examine them. Particularly, we would like to test

- Degree Distribution
- Shortest Path Distribution
- Graph Diameter
- Closeness Centrality
- Betweenness Centrality Distribution
- Community Detection

This report will showcase our results — more specifically, we would like to discuss, in detail, our implementation and experiments. Also, we will list any relevant, interesting results we obtain.

2 Implementation

Our implementation is as follows, from a higher level:

1. Get and parse graph input.
 - (a) Because our data was given in the form of a `csv`, we decided to just pipe that input directly to a `vector<string>`.
 - (b) We then pass said `vector<string>` to a function that parses using C++ [string functions](#).
 - (c) We pipe the parsed data to a data structure of `map<int, vector<pair<int, double>>>`
 - The `int` is the key for retrieval of the `vector`

- The `vector<pair<int, double>>` stores a vector of the edges, in pairs — where the pair `<int, double>` are proportional to the target vertex and weight.
2. Move on to calculating the out degree of the vertexes.
 - (a) Initialize a `map` of `<int, int>` for unweighted and `<int, double>` for weighted.
 - (b) For both weighted and unweighted simply use the source as the `key`.
 - (c) For unweighted, use the `size()` method of the vector class to determine the out degree¹.
 - (d) For weighted, sum the `second` property of the `pair`s in the `vector` — note that the second property of `pair` is the weight. This gives a total weight.
 3. Move to calculating the in degree of the vertexes.
 - (a) This is done almost the same way, except there is a weight `map`.
 - (b) Iterate over the entirety of our data structure², and store where the target vertex points to in all the edges in the weight `map`.
 4. Calculate shortest path via the Floyd-Warshall algorithm.
 - (a) Initialize an adjacency matrix — a `vector<vector<int>>`
 - Default all values to infinity — in our case, 999999.
 - (b) Copy over data from our `map` to said adjacency matrix.
 - If unweighted and an edge exists, default to 1.
 - (c) If requested an undirected shortest path, make the graph undirected.
 - This is done by making a mirror image of the adjacency matrix A , by setting $\forall i, j \in A, A_{i,j} = A_{j,i}$. Just copying over the diagonal.
 - (d) Run the Floyd-Warshall algorithm, with triple C-Style `for` loops.
 5. Implemented Graph Diameter
 - (a) Took the map and using the Floyd Walsh algorithm, found the shortest paths between each vertex
 - (b) Begin by assuming that the path from 0 to 1 is the longest path
 - (c) Compare each other path to the longest path. If a longer path is found, replace the longest path with that path. If a path of equal length is found, add the path to the list of paths.

¹Remember, the key return the a `vector` of pairs. The number of pairs are directly proportional the out degree.

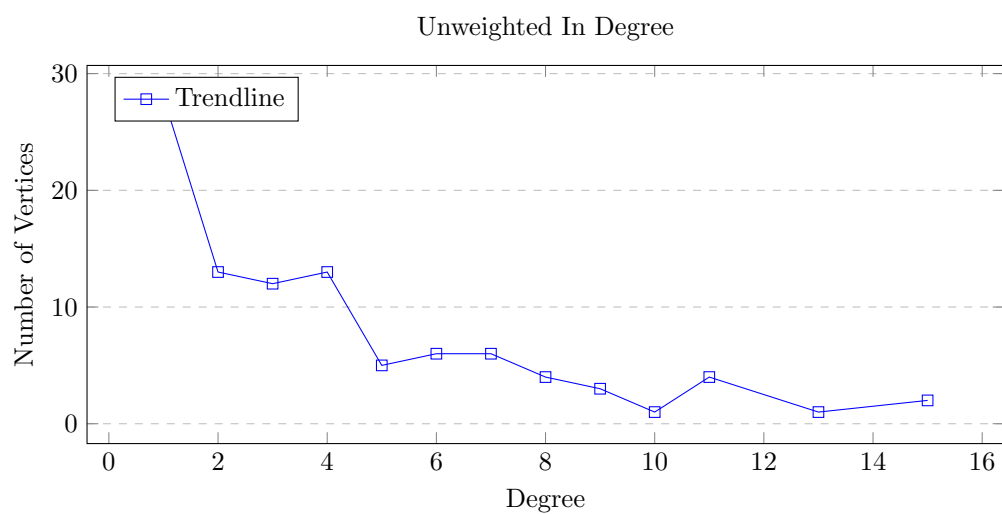
²An adjacency map of sorts, `map<int, vector<pair<int, double>>>`.

- (d) Output the list of all paths of the maximum length.
6. Next we endeavor to find the various closeness centrality of the vertices.
- (a) Initialize one vector to hold the solutions set and four separate vectors to contain the vertices that correspond to the four vertices with the highest closeness centrality values.
 - (b) Add each edge weight for each vertex together (iterating over each vertex and each edge) and pushing the result into the solution set vector.
 - (c) After this has been pushed we check the value against the recorded highest four values to see if this vertex either equals or exceeds those four values and change our recorded values/vertices accordingly.
 - (d) Finally, we output both our results for highest closeness centrality values and the data needed to construct the graph.
7. Calculate Betweenness Centrality
- Betweenness Vertex
 - (a) Take the map and use the Floyd Walsh algorithm to determine the shortest paths between each vertex
 - (b) While constructing the paths, if a vertex is used in a path, increase the betweenness value for that vertex.
 - (c) Go through the array of betweenness values and find the one with the maximum value. If two are equal betweenness, record them both.
 - (d) Output the results.
 - Betweenness Edge
 - (a) Take the map and use the Floyd Walsh algorithm to determine the shortest paths between each vertex
 - (b) While constructing the paths, if an edge is used in a path, increase the betweenness value for that edge.
 - (c) Go through the array of betweenness values and find the one with the maximum value. If two are equal betweenness, record them both.
8. Lastly we must run an algorithm to detect the communities present within the graph.
- (a) Take the results from the Unweighted Edge Betweenness algorithm and take the maximum value.
 - (b) Remove the edge that corresponds to this result from the original network.
 - (c) Re-run the Shortest Path and Graph Diameter Algorithms on the new network.
 - (d) Repeat these steps until a total of five deletions from the original network have been completed.

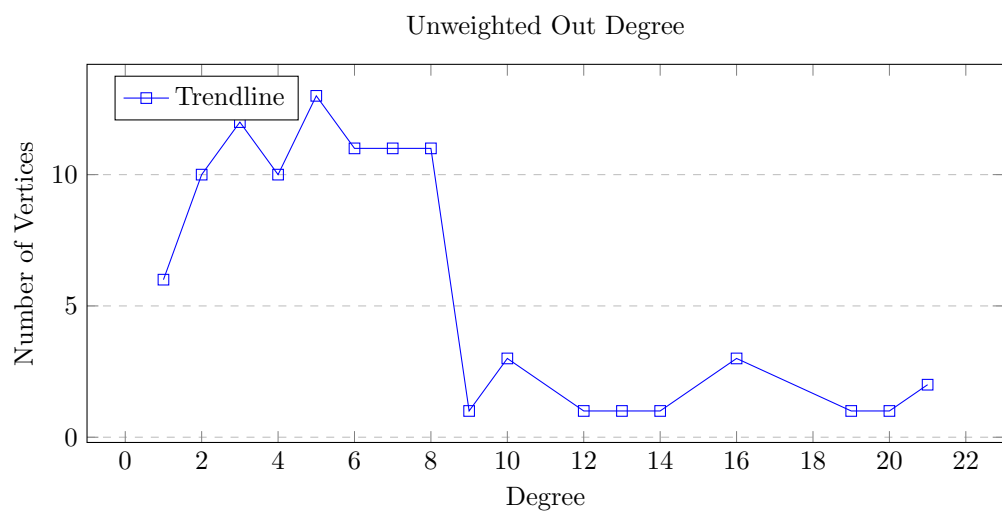
3 Experiments

3.1 Degree Distribution

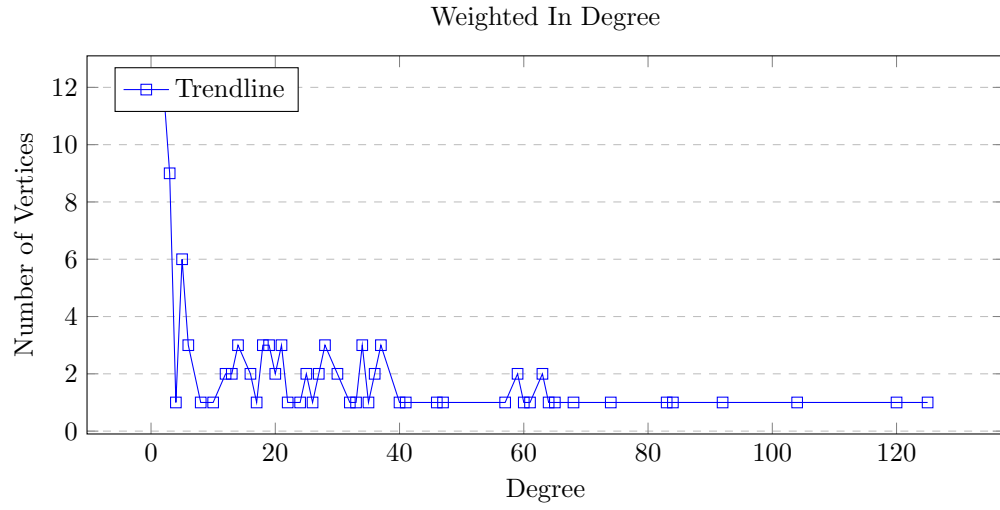
3.1.1 Unweighted In Degree



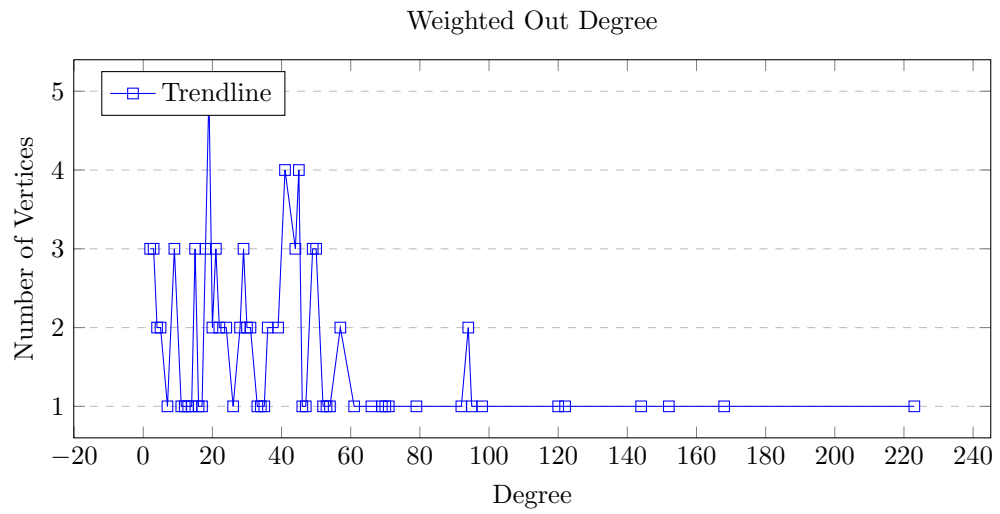
3.1.2 Unweighted Out Degree



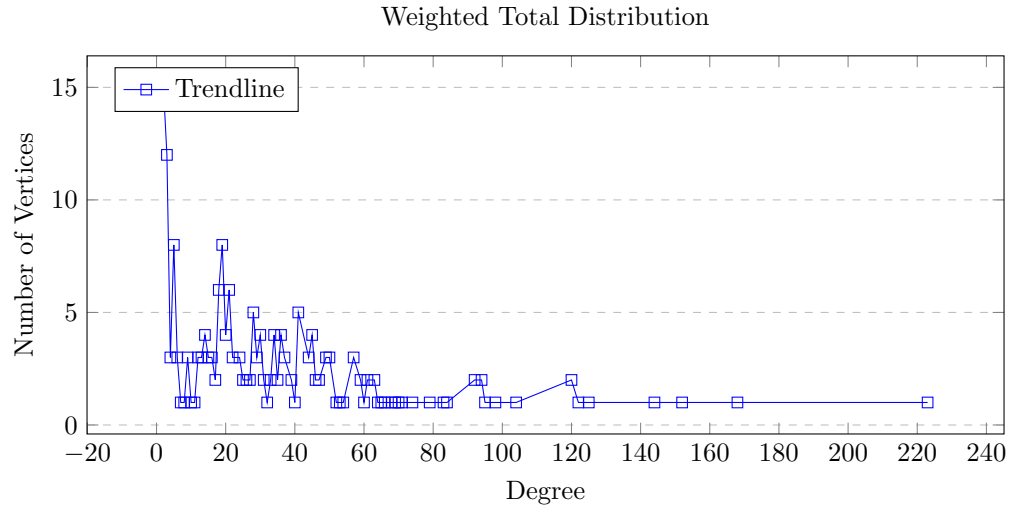
3.1.3 Weighted In Degree



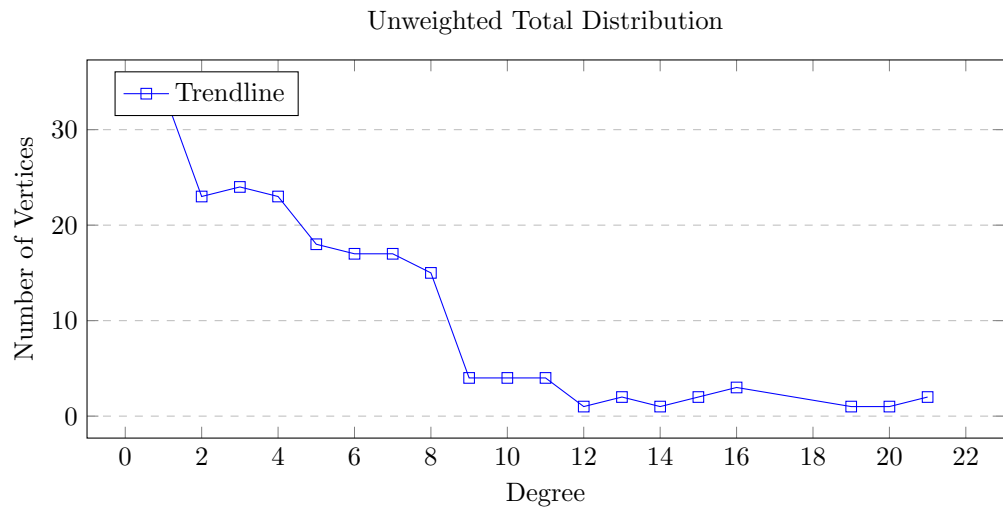
3.1.4 Weighted Out Degree



3.1.5 Weighted Total Distribution



3.1.6 Unweighted Total Distribution



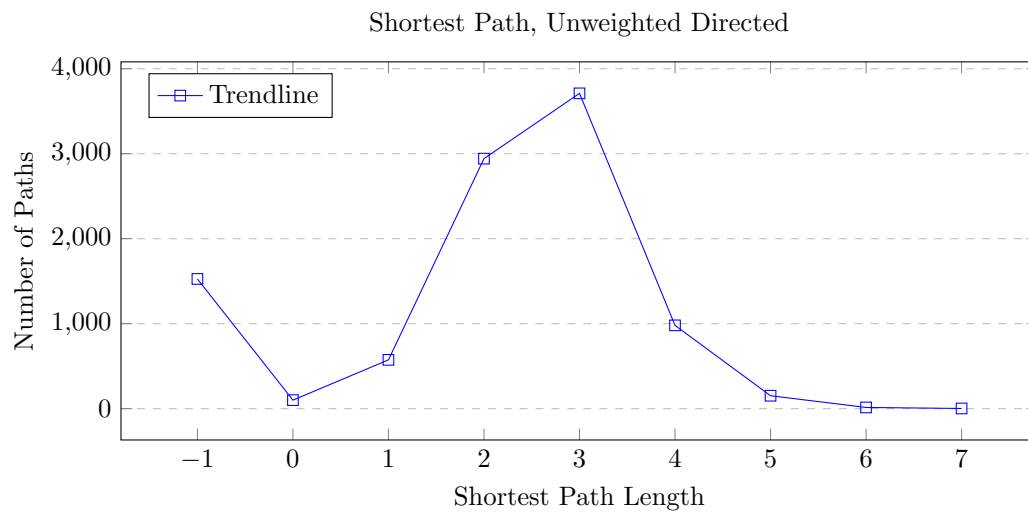
3.1.7 Highest In Degree

OUT DEGREE, UNWEIGHTED: 21	• 5 • 7
OUT DEGREE, WEIGHTED: 223	• 3
IN DEGREE, UNWEIGHTED: 15	• 70 • 78
IN DEGREE, WEIGHTED: 125	• 70
TOTAL DEGREE, UNWEIGHTED: 22	• 5 • 7
TOTAL DEGREE, WEIGHTED: 226	• 3

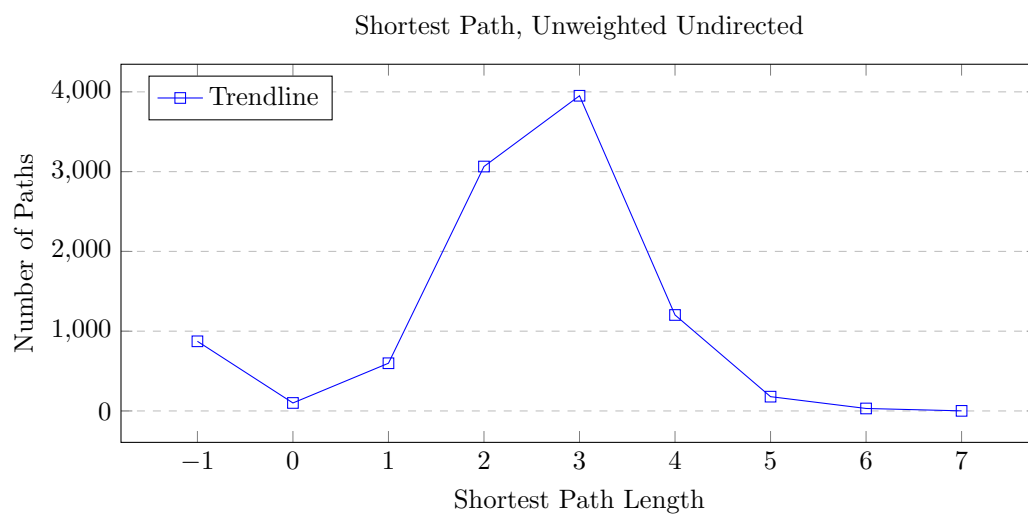
3.2 Shortest Path

Please note that -1 corresponds to a path between two vertices not existing.

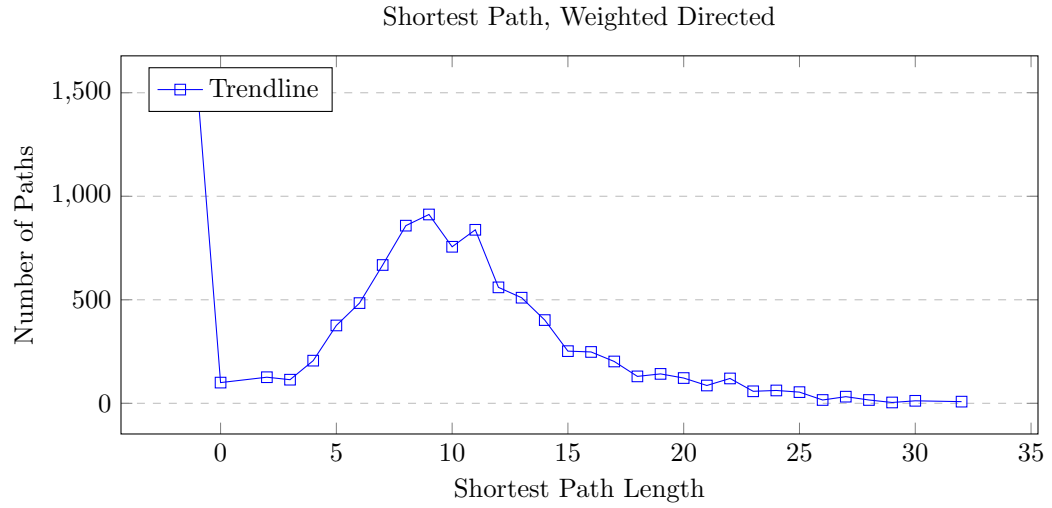
3.2.1 Shortest Path, Unweighted Directed



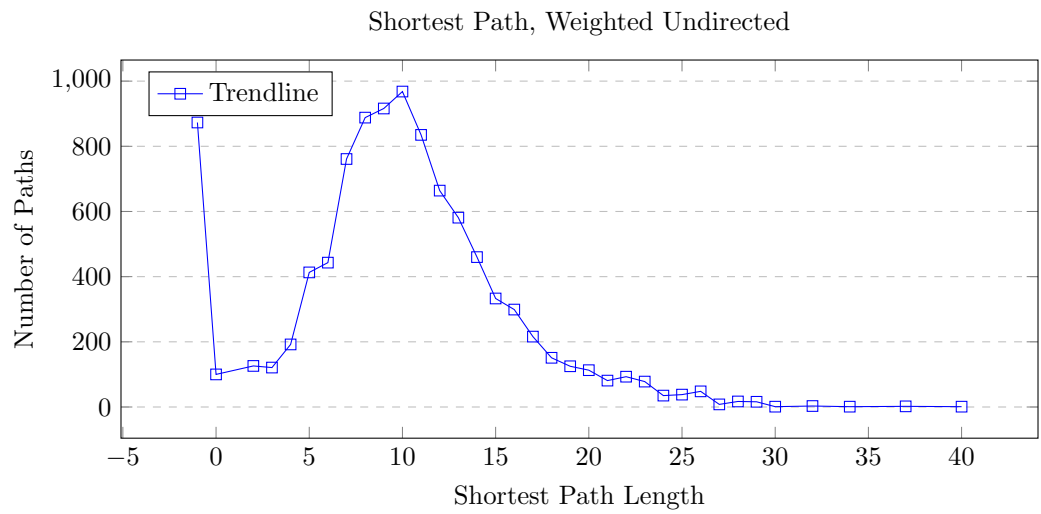
3.2.2 Shortest Path, Unweighted Undirected



3.2.3 Shortest Path, Weighted Directed

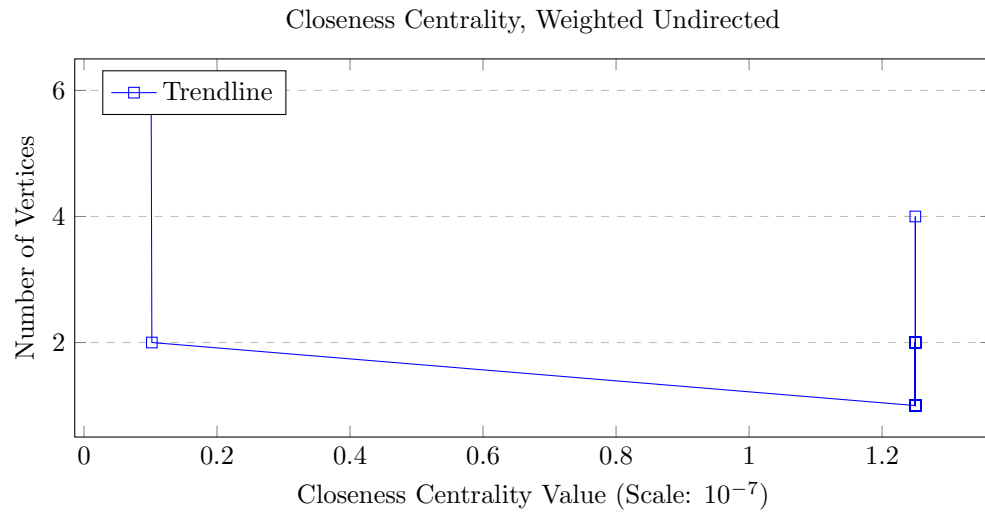


3.2.4 Shortest Path, Weighted Undirected

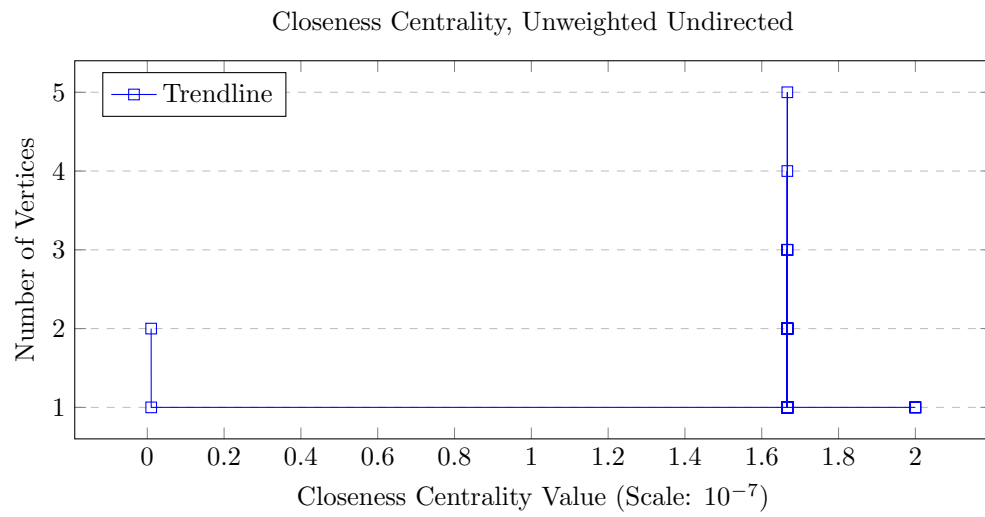


3.3 Closeness Centrality

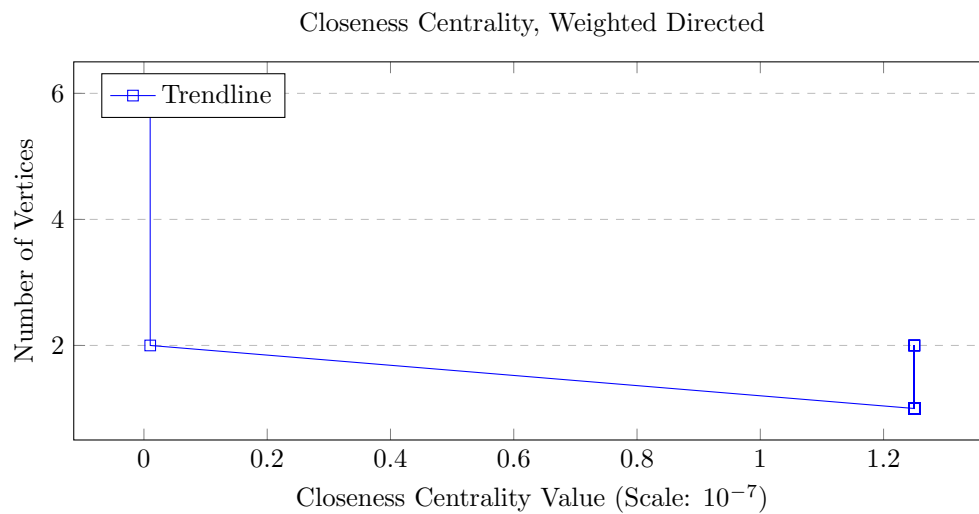
3.3.1 Closeness Centrality, Unweighted Directed



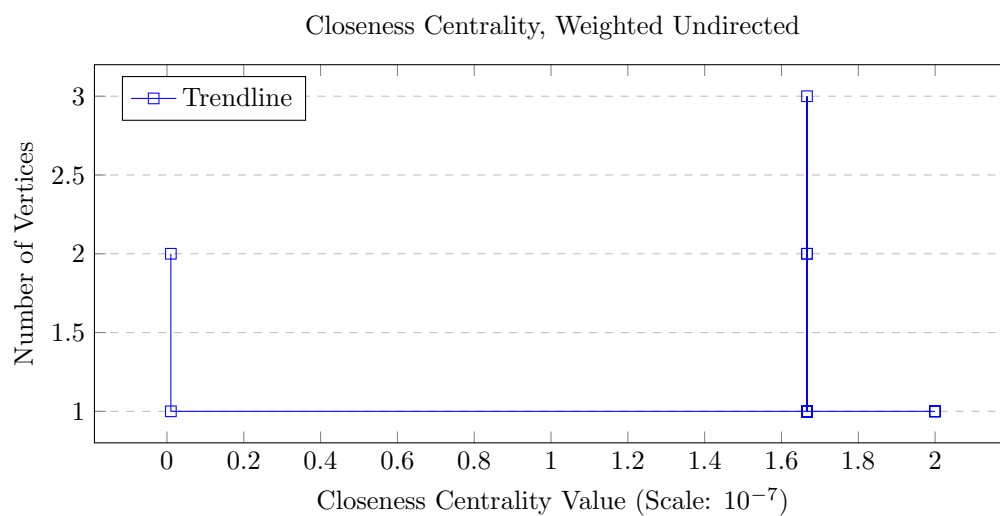
3.3.2 Closeness Centrality, Unweighted Undirected



3.3.3 Closeness Centrality, Weighted Directed



3.3.4 Closeness Centrality, Weighted Undirected

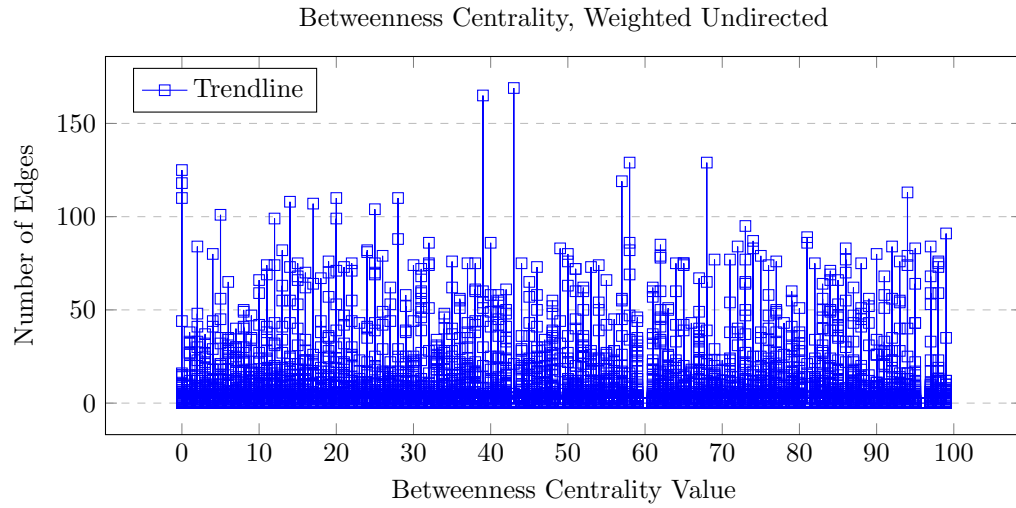


3.3.5 Highest Closeness Centrality

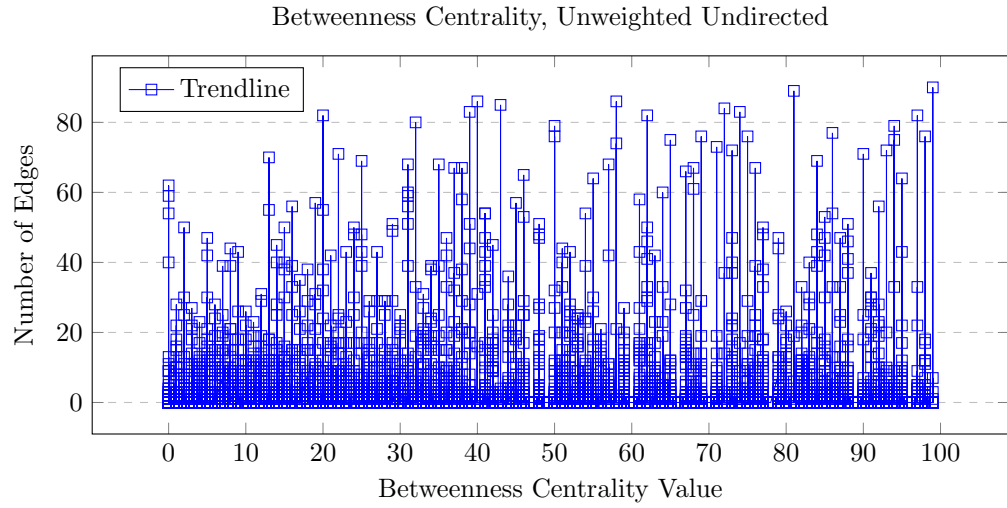
Type	Value	Vertex
Unweighted, Directed	0.000000124997e	5
	0.000000124997	7
	0.000000124997	1
	0.000000124997	3
Weighted, Directed	0.00000012498937590	5
	0.00000012498914157	64
	0.00000012498907908	52
	0.00000012498875101	11
Weighted Undirected	0.00000019996612574	47
	0.00000019996412644	0
	0.00000019996168734	70
	0.00000019995872852	89
Unweighted Undirected	0.00000019999132038	0
	0.00000019999036046	89
	0.00000019998952055	70
	0.00000019998928057	47

3.4 Betweenness Centrality

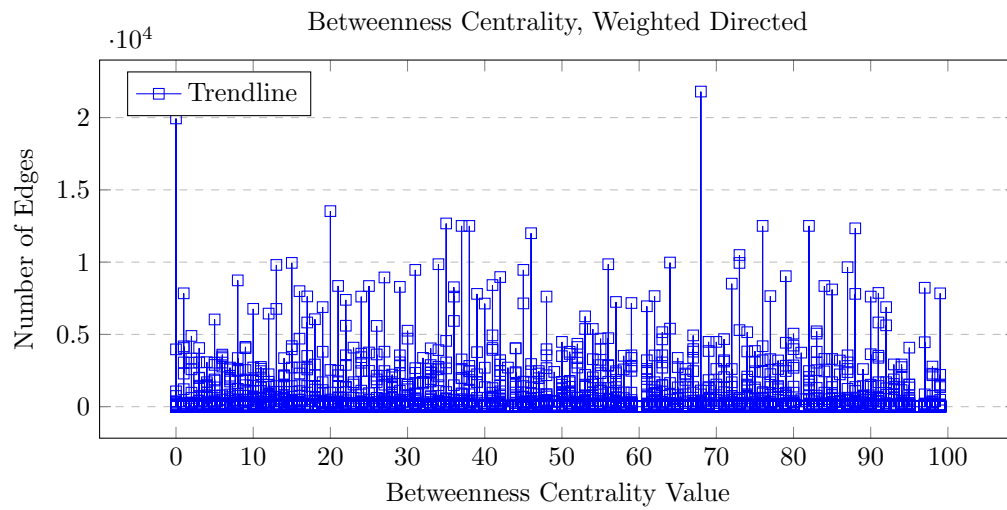
3.4.1 Betweenness Edge, Unweighted Directed



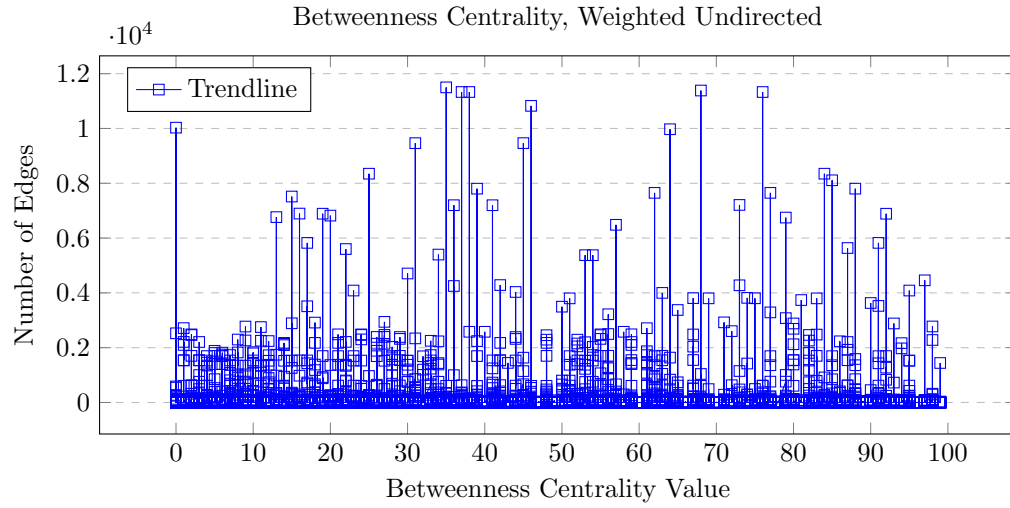
3.4.2 Betweenness Edge, Unweighted Undirected



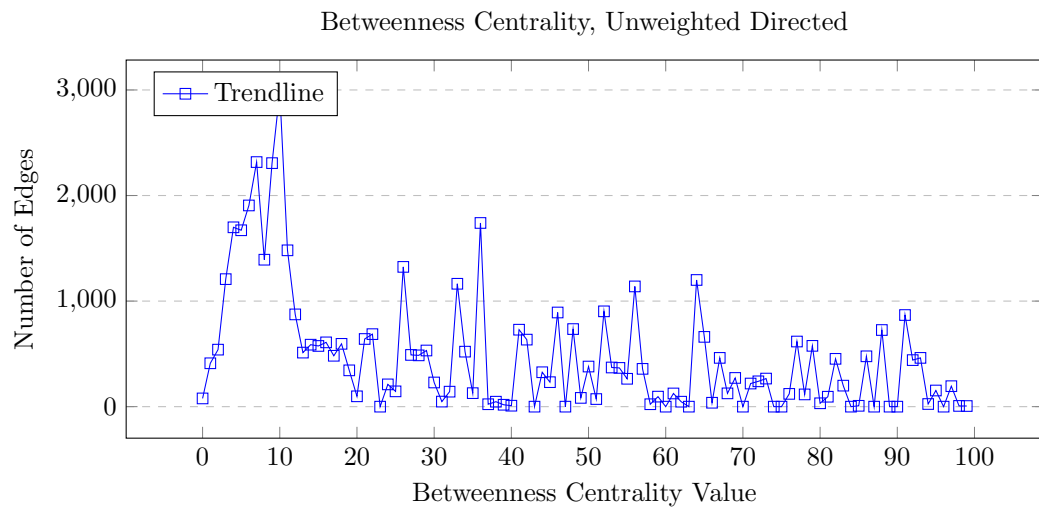
3.4.3 Betweenness Edge, Weighted Directed



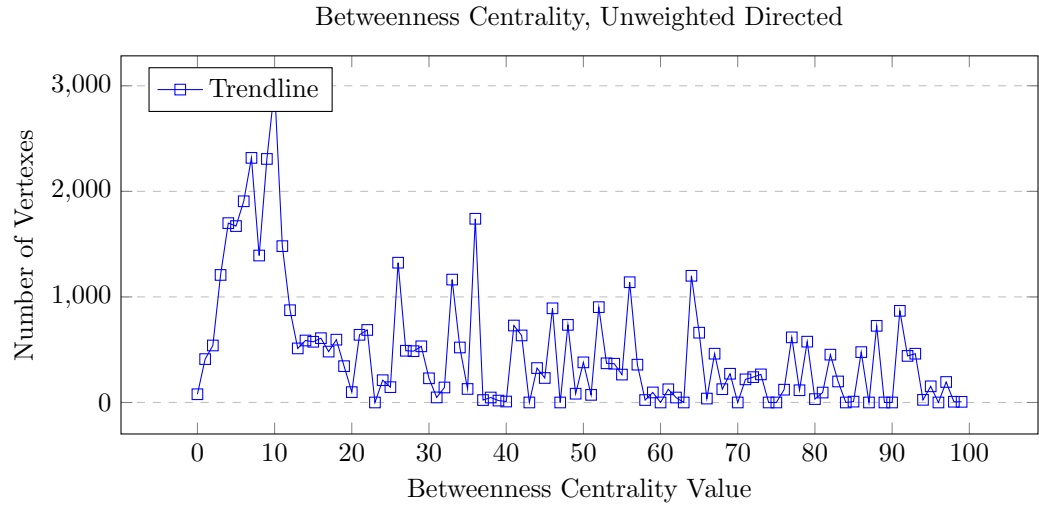
3.4.4 Betweenness Edge, Weighted Undirected



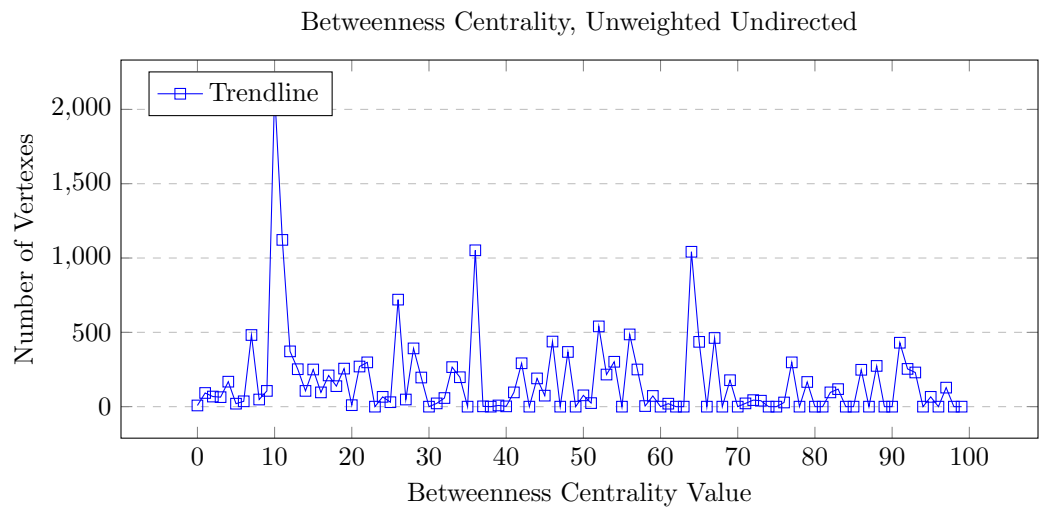
3.4.5 Betweenness Vertex, Unweighted Directed



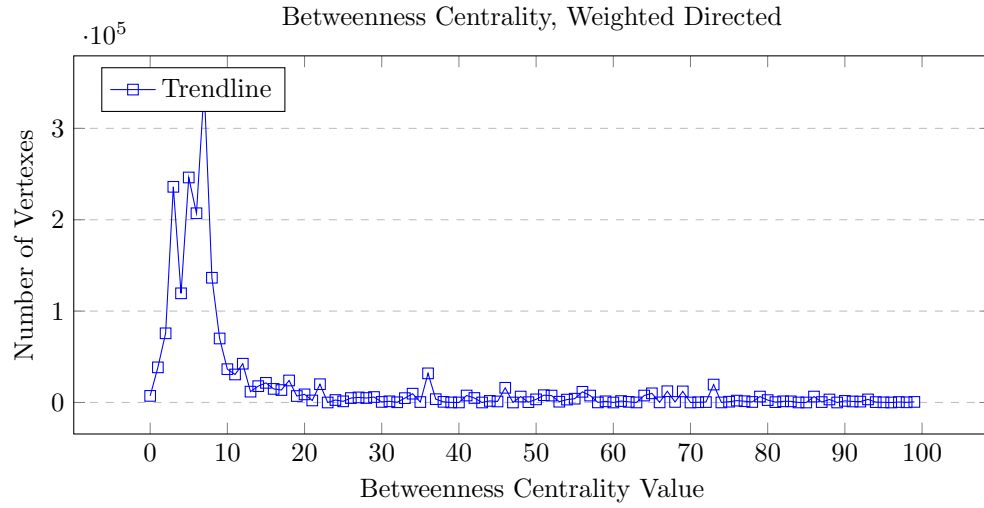
3.4.6 Betweenness Vertex, Weighted Undirected



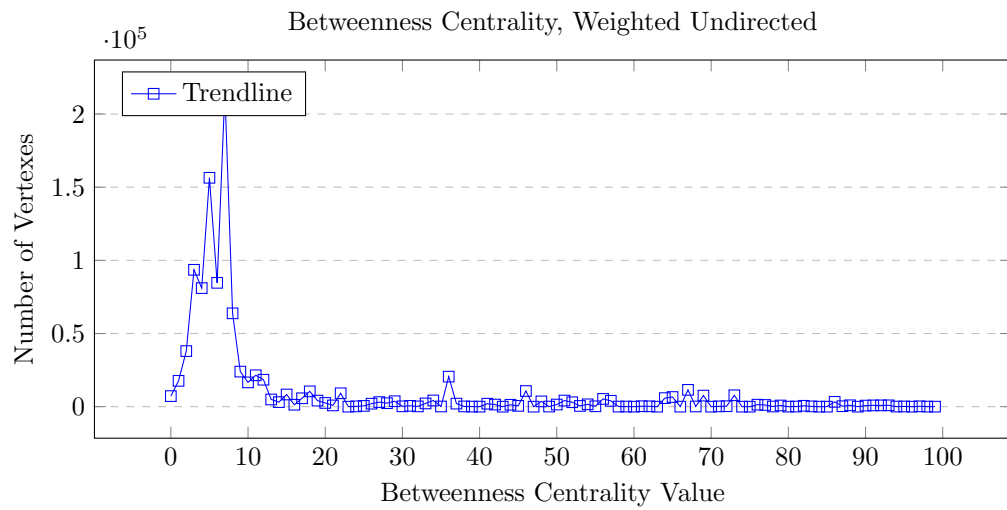
3.4.7 Betweenness Vertex, Unweighted Undirected



3.4.8 Betweenness Vertex, Weighted Directed



3.4.9 Betweenness Vertex, Weighted Undirected



4 Team Roles

- Illya Starikov
 - Project Manager

- Implementation
 - * Weight Distribution
 - * Shortest Path
- Timothy Ott
 - Report Writeup
 - Implementation
 - * Closeness Centrality
 - * Community Detection
- Claire Trebing
 - Report Writeup
 - Implementation
 - * Unweighted/Weighted Graph Diameter
 - * Betweenness Centrality Distribution

5 Interesting Results

Below are interesting results we have found in regards to the project so far.

- Our pick of data structure (essential a Binary Search Tree) made the project easily testable. Because of Binary Search Tree's $\mathcal{O}(\lg n)$ retrieval complexity, this made our project run under 100 macro-seconds every compilation.
- For degree distribution, there seemed to be roughly 10 vertexes that were very dense. Although the graph seems fairly homogeneous many vertexes had a total degree of 15 or more.
 - When observing total weighted distribution (which appears almost to be $f(x) = \frac{1}{x}$), there is a tail caused by three nodes.
- There are always more paths between an arbitrary two vertexes than there are not. Meaning from any two nodes, it is more likely that there is a path.
 - This furthers our claim that this graph is dense.
- Closeness Centrality is so miniscule it cannot be computed by a standard `double` — the accuracy was not good enough. `long double` was used instead.

6 Conclusions

The experiments detailed above seek to find meaning in what would ordinarily be only raw data, the results of which, while being open to some interpretation, describe the small subsection of a social community that is given by the data set we were provided. For instance, the distribution of degrees is a measure of how interconnected individual nodes or profiles are to the rest of the graph. Because this graph is directed, it closely resembles a social network such as Tumblr where individuals can follow another profile but that profile is not required to follow back. From our results, particularly those of the unweighted distribution, we can tell that most of our nodes are followed by under 5 profiles and in turn follow between 2 and 8 profiles. The algorithms for Shortest Path are also a test of interconnectedness, though this time measuring the degrees of separation between nodes. From our results we can see that on average the nodes of our network are no more than two or three connections away from each other. The Graph Diameter measurements are meant to give an idea of the outliers in this community or to define the size of the outward edges of this network. Centrality metrics seek to identify the most important nodes within the network, using a variety of qualifications to define what is most important. A measure of Closeness centrality is effectively the inverse of a nodes farness or distance from other nodes. The next centrality metric that we tested for was the Betweenness Centrality which attempts to determine those nodes that most often act as a bridge between two other nodes. This measurement has applications within social networks to determine which individual (or in our social network examples, profile) has the most influence on communication between other individuals. Lastly we attempted to locate and determine the smaller communities within our social network. One way to accomplish this is to locate those edges that have the highest betweenness centrality and eliminate them from the network. These edges are most likely to be used as bridges between communities so by eliminating them we are able to separate and identify those communities. Once we have eliminated those edges we simply recalculate the shortest paths and diameters of the network to see how the network has changed. As we can see, these experiments and algorithms are of great use to us to sift through a large quantity of raw data and attribute meaning to them where there originally was little to none. After all, after applying these algorithms to a simple adjacency list we are now able to infer a large amount of information about the underlying situation that is being represented by this graph.

CS3001

Skills Development



Illya Starikov (isgx2)

CS5400-SP2018 Puzzle3 Grading Rubric

Datastructures: 100.00%

Looks good.

Action & state generation: 100.00%

Looks good.

Search Algorithm: 100.00%

Looks good.

Programming Practice: 100.00%

Good documentation!

Code reliability: 100.00%

Good.

Bonus: 100.00%

Good analysis!

**Total = 3.00%(Datastructures) + 2.00%(Action & state generation) + 75.00%(Search Algorithm) +
10.00%(Programming Practice) + 10.00%(Code reliability) + 15.00%(Bonus)**

Total: 115.00%



FACULTY
OF EDUCATION
Masaryk University

A Special Presentation (Presentation #1)

Special Topics (CS3001)

Illya Starikov

A Brief Introduction

- Generally, excelled at presenting.
- These were not without a lot of effort.
- I would generally spend several hours practicing presentation. I hope to change that for this class.
- From here on out, every presentation will generally improved. (I don't know if this is allowed but better to ask for forgiveness than permission).
- Why?

A Brief Introduction

Because of the massive workload this semester, I did not get to work on this skill after the first day. I had to skip class regularly to make hours at work. However, I was able to learn some things from it.

Prior Knowledge

- I attended some improv sessions last semester.
- Besides that, I had no prior knowledge.

Goals

- To be able to give a good, improved presentation.

Goal Accomplishment

- I successfully gave a winged presentation during the first lecture. Although it made me really nervous, after I got used to it, I felt a bit better about myself.

In Closing

Thank you for having me! All question, comments, and insults can be directed towards me:

 starikov@mst.com

 [Illya Starikov](#)

 [Illya Starikov](#)

 [FreneticArray.com](#)



FACULTY
OF EDUCATION
Masaryk University

Unit Tests (Presentation #2)

Special Topics (CS3001)

Illya Starikov

A Brief Introduction

- Good code is broken into many functions.
- In a perfect world, these functions would always produce the right results.
- Unfortunately, this isn't a perfect world.
- Unit tests allow to test all these blocks to guarantee correctness.

Prior Knowledge

- I started writing unit tests last semester for my internship (in C#).
- However, they were small and minimal.

Goals

- To become fluent in several unit testing frameworks.
- To become better at penetration testing of functions.

Tangible Evidence

Python Unit Tests (AI)

```
def test_grid_size(self):
    not_long_enough = [list(range(2)), list(range(2)), list(range(2))]

    rows, columns = 5, 7
    just_right = [list(range(columns)) for i in range(rows)]

    self.assertRaises(AssertionError, MechanicalMatch.grid_size,
                      not_tall_enough)

    grid_rows, grid_columns = MechanicalMatch.grid_size(
        just_right)
    self.assertEqual(grid_rows, rows)
    self.assertEqual(grid_columns, columns)
```

Tangible Evidence

C++ Unit Tests (Numerical Modeling)

```
SECTION("Stream Operator") {
    auto polarPair = PolarPair<R, Theta>(defaultModulus,
        defaultArgument);
    std::stringstream expected, actual;

    polarPair.setPrecision(6);

    expected << std::setprecision(6) << std::fixed << "(" <<
        defaultModulus << ", " << defaultArgument << ")";
    actual << polarPair;

    REQUIRE(expected.str() == actual.str());
}
```

Tangible Evidence

C# Unit Tests (Work)

I can't shared source code from my work because of security reasons/non disclosure agreements.

Resources

- Online resources.
 - C++ Unit Tests.
 - C# Unit Tests.
 - Python Unit Tests.
- Looking at other people's source code.
- Discussing ideas with boss and coworkers.

Goal Accomplishment

- Since the beginning of the semester, I have drastically become better at writing unit tests.
- I am fluent in writing unit tests in C++, C#, and Python.
- Although I have gotten better at writing unit tests, I could progress a lot more.

In Closing

Thank you for having me! All question, comments, and insults can be directed towards me:

 starikov@mst.com

 [Illya Starikov](#)

 [Illya Starikov](#)

 [FreneticArray.com](#)



FACULTY
OF EDUCATION
Masaryk University

UML Modeling (Presentation #3)

Special Topics (CS3001)

Illya Starikov

A Brief Introduction

- It's difficult to imagine a lot of objects interacting at the same time.
- It's also difficult for someone else to imagine these objects interacting when they have never seen them in use before.
- This where Uniform Modeling Language (UML) Diagramming comes in handy.

A Brief Introduction

- Uniform Modeling Language (UML) is a modeling language that helps visual a system, typically in programming.
- UML can be broken down into two simple interactions: Entities and Relationships.

Entities The actors in the system.

Relationships How actors interact with one another. These relationships can include: dependencies (A depends on B), generalization (A is a generalization of B), aggregation (A is an aggregation of B, C, D).

A Brief Introduction

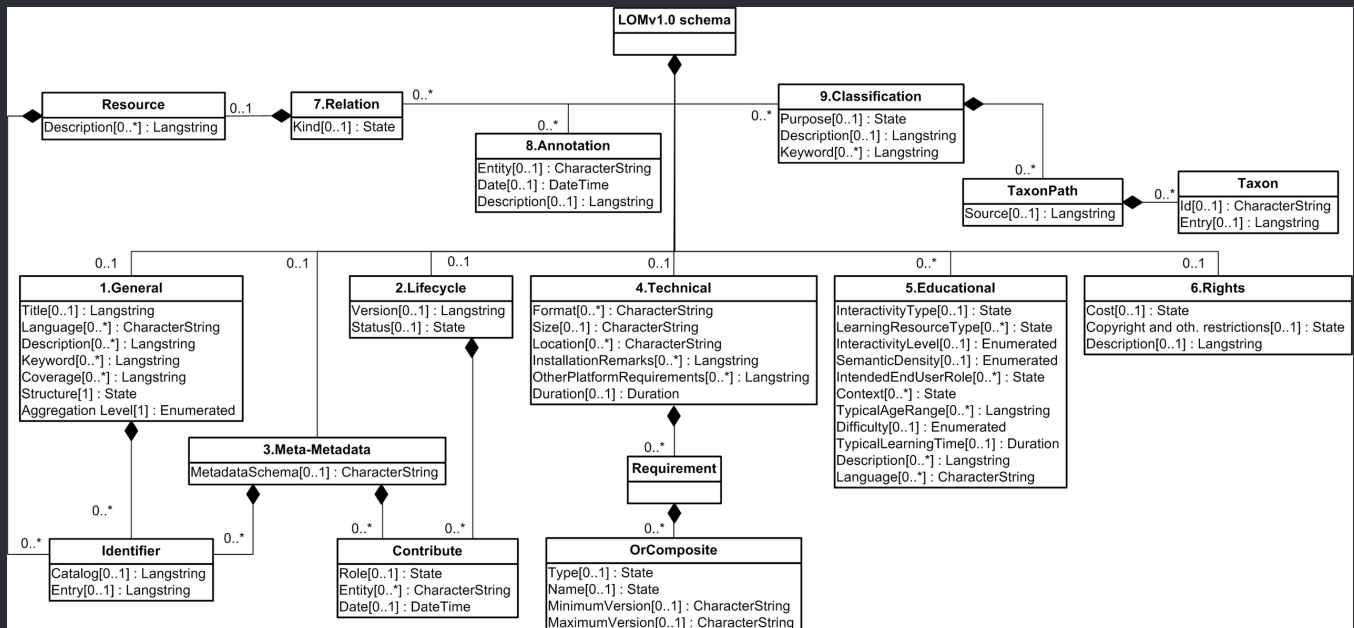


Figure: An example of a UML Diagram

Prior Knowledge

- I have had no prior experience with UML.

Goals

- To become fluent in UML Diagramming, with all associated entities and relationships.
- To become fluent in an application that is meant to do powerful UML diagramming.

StarUML

After a lot of searching, I settled on [StarUML](#). It was recommended all over the internet, and it seemed to be able to handle complicated diagrams.

StarUML

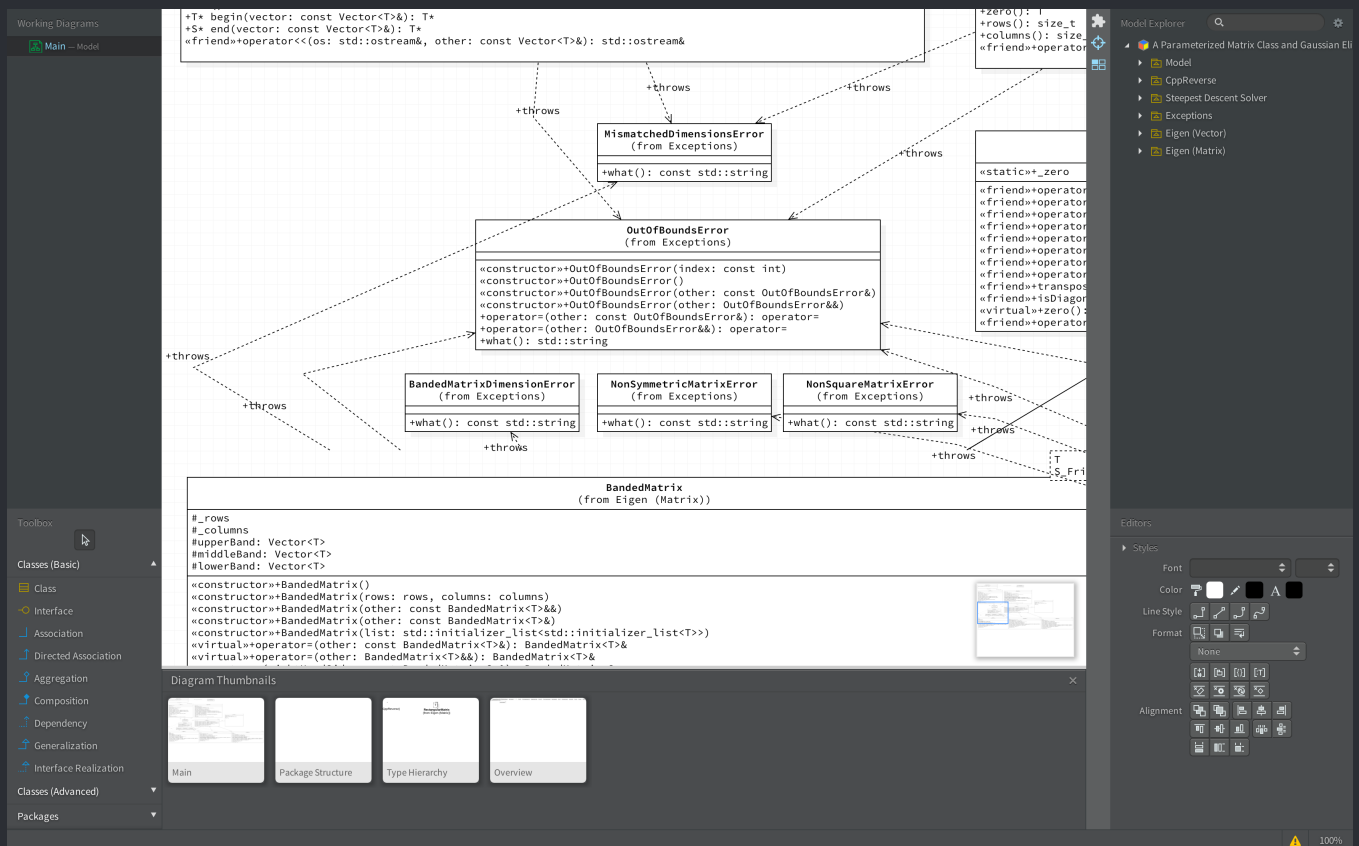


Figure: An example of a UML Diagram

Resources

- Most of the learning came from playing around the tool. However, there were other resources I found to be useful.
- Online resources.
 - The official website.
 - The extension gallery, to help automate the tedious parts.
 - A UML cheatsheet.

Goal Accomplishment

- Since the beginning of the semester, I have drastically become better at creating UML Diagrams. The evidence can be found with this presentation; homework - 2 . png was the first real UML Diagram I created, homework - 6 . png was the most recent one created.

In Closing

All question, comments, and insults can be directed towards me:

 starikov@mst.com

 [Illya Starikov](#)

 [Illya Starikov](#)

 [FreneticArray.com](#)



FACULTY
OF EDUCATION
Masaryk University

Vim (Presentation #4)

Special Topics (CS3001)

Illya Starikov

A Brief Introduction

They often say

“A Developer Is Only As Good As His Tools”

Although I don't agree with this sentiment, I always wanted to become better with tools I was constantly using. One of the tools I use most is my code editor: **Vim**.

A Brief Introduction

- Vim has been around since 1991, and is almost ubiquitous on all Unix machines. It has served mostly as a terminal text editor (see attached video).
- Also, Vim has greatly improved a lot since then, to incorporate many of the new features that IDEs and Text Editors have brought to the table.
- Along with that, a lot of projects have set out to make Vim look and feel more like a regular text editor (as apposed to a terminal editor).

A Brief Introduction

```
matrix-base.hpp matrix-base.h
8 //
9 // matrix-base.h
10 // lib
11 //
12 // Illya Starikov/Rachel Million, Section 1A, 18083141
13 // cs6201 - Object Oriented Numerical Modeling
14 //
15 //
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027

```

Prior Knowledge

- I've been using Vim since my freshmen year. However, I never worked with the advanced features of Vim, only the top level features.
- I knew that Vim had an expansive feature set, and I wanted to dive into that.

Motivation

- Upon graduation, I will likely be using Vim full time at my time at Garmin.

Goals

- To become more efficient at using Vim.
- To become familiar with the modern parts of Vim.
- To learn more of the keystrokes and shortcuts for Vim.

Resources

- A lot of the learning came from playing around with Vim. Anytime I found myself doing something repetitive, I simply tried to find a way to automate. Most of the time, there was a way.
- Book Resources (That I Actually Bought!):
 - [Modern Vim](#)
 - [Practical Vim](#)
- Online Resources:
 - [The Vim Subreddit](#), for everyday improvements or general knowledge on Vim.
 - [VimCasts](#), for when I want to get a deeper look into a subject.
 - [Vim Adventures](#), a little game to help understand Vim through a text adventure.

Goal Accomplishment

- I am much more fluent in using Vim, using shortcuts that significantly improve my workflow.
 - See attached videos for details.
- I have also learned to use plugins, to do completions.
- I have also learned to use many of the different parts of Vim I was unfamiliar with, such as:
 - Buffers
 - Registers
 - Advanced Search
 - Tags

And a lot more!

Side Note

```
presentation-4.tex buffers
36 \begin{itemize}
37   \item \href{https://www.reddit.com/r/vim/}{The Vim Subreddit}, for everyday improvements or general knowledge on Vim.
38   \item \href{http://vimcasts.org/}{VimCasts}, for when I want to get a deeper look into a subject.
39   \item \href{https://vim-adventures.com/}{Vim Adventures}, a little game to help understand Vim through a text adventure.
40 \end{itemize}
41 \end{frame}
42
43 \begin{frame}
44   \frametitle{Goal Accomplishment}
45
46   \begin{itemize}
47     \item I am much more fluent in using Vim, using shortcuts that significantly improve my workflow.
48   \end{itemize}
49   \begin{itemize}
50     \item See attached videos for details.
51   \end{itemize}
52
53   \item I have also learned to use plugins, to do completions.
54   \item I have also learned to use many of the different parts of Vim I was unfamiliar with, such as:
55   \begin{itemize}
56     \item Buffers
57     \item Registers
58     \item Advanced Search
59     \item Tags
60   \end{itemize}
61
62   And a lot more!
63 \end{itemize}
64 \end{frame}
65
66 \begin{frame}
67   \frametitle{Side Note}
68
69   \begin{figure}[H]
70     \centering
71     \includegraphics[width=0.8\linewidth]{presentation}
72     \caption{The entirety of this presentation was made in Vim.}
73     \label{fig:presentation}
74   \end{figure}
75 \end{frame}
76
77 \begin{frame}
78   \frametitle{In Closing}
79
80   All question, comments, and insults can be directed towards me:
81
82   \begin{center}
83     \begin{description}
84       \item \faComment \href{mailto:starikov@mst.edu}{starikov@mst.com}
85       \item \faLinkedin \href{https://www.linkedin.com/in/illyastarikov/}{illyastarikov/} Illya Starikov
86       \item \faGithub \href{https://github.com/IllyaStarikov/}{github.com/IllyaStarikov/} Illya Starikov
87       \item \faRss \href{https://freneticarray.com/}{freneticarray.com/} FreneticArray.com
88     \end{description}
89   \end{center}
90 \end{frame}
```

NORMAL /Users/starboy/Desktop/cs3801/presentation-4.tex tex utf-8[unix] 547 words 87% 139/159 : 71 W:2(L56)

Figure: The entirety of this presentation was made in Vim.

In Closing

All question, comments, and insults can be directed towards me:

 starikov@mst.com

 [Illya Starikov](#)

 [Illya Starikov](#)

 [FreneticArray.com](#)



FACULTY
OF EDUCATION
Masaryk University

Documentation (Presentation #5)

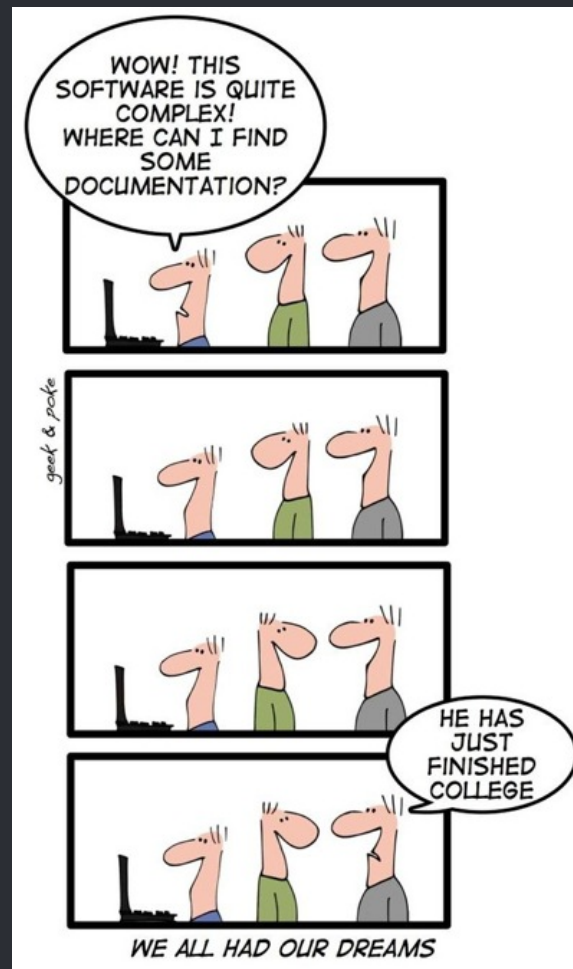
Special Topics (CS3001)

Illya Starikov

A Brief Introduction

One of the most critical aspects of computer science is documentation.

A Brief Introduction



A Brief Introduction

- Documentation was one of those things discussed once briefly in Introduction to Programming. You were expected to do it, but never really taught how to do it.
- One of my classes this semester (Object Oriented Numerical Modeling) require *a lot* of documentation.
- Throughout the semester, I wanted to write good, thorough documentation.

Prior Knowledge

- Mediocre documentation is easy to write.
 - This is the kind of documentation I wrote my freshmen year.
- Great documentation is much harder to write.
 - This is documentation I strive to write now.

Tools

- Often, it's hard to read documentation when it's directly inside code.
- This is why I've picked up Doxygen, a tool to scrap all documentation outside of a codebase.

Tools (Example)

file:///Users/starboy/Desktop/ninjas/2018-sp-a-hw6-lsgx2/doc/html/class_matrix.html

Eigen

A Linear Algebra Library

Main Page Classes Files

Search

Matrix< T > Class Template Reference

Public Member Functions | List of all members

An interface for a [Matrix](#). [More...](#)

#include <matrix.h>

Inheritance diagram for Matrix< T >:

```
graph BT; BandedMatrix["BandedMatrix< T >"] --> BaseMatrix["BaseMatrix< T >"]; DiagonalMatrix["DiagonalMatrix< T >"] --> BaseMatrix; RectangularMatrix["RectangularMatrix< T >"] --> BaseMatrix; SymmetricMatrix["SymmetricMatrix< T >"] --> BaseMatrix; BaseMatrix --> Matrix["Matrix< T >"];
```

Public Member Functions

virtual void	resize	(const size_t rows, const size_t columns)=0
virtual T &	operator()	(const int row, const int column)=0
virtual const T &	operator()	(const int row, const int column) const =0
virtual T &	zero	() const noexcept=0
virtual size_t	rows	() const noexcept=0
virtual size_t	columns	() const noexcept=0

Detailed Description

template<class T>
class Matrix< T >

An interface for a [Matrix](#).

Template Parameters

T The type of the elements the matrix stores.

The documentation for this class was generated from the following file:

- lib/matrix.h

Generated by [doxygen](#) 1.8.14

Goals

- To be able to use Doxygen efficiently.
- To write documentation that's easy to read, but also informative.

Before — Intro To Programming

```
// Description:      To get user input to dictate what menu option
                     is selected.
// Pre-Condition:    A valid character must be entered.
// Post-Condition:    A value for charMenuOption is received,
// dictating what functions are used form there.
```

After — Object Oriented Numerical Modeling

```
/**
 * @brief Read the specified input file and ensure proper
 *        data format.
 *
 * @param filename The filename where file is to be read from.
 *        Can be relative or absolute.
 *
 * @return An `std::string` with the entirety of the file
 *        contents in them (`\n` and all).
 *
 * @warning This does terminate prematurely if the file
 *        contents is not in the proper format.
 **/
```

Resources

- A great starting point was a [Fermilab introduction article](#).
- From there, I would reference:
 - [The doxygen website](#).
 - [The special doxygen command guide](#).
- I would also read articles about better documentation.
 - [HackerNoon](#).
 - [Documentation Handbook](#).
 - [Hacker News](#).

Goal Accomplishment

- So far, this semester, I have only missed two points on documentation for all of my classes.
 - Some of my graders even commented on my documentation ability (see attached).
- I have been more conscious about my technical writing.
- Overall, I saw an improvement in my documentation skills. This is reflected in my school work and my work life.

In Closing

All question, comments, and insults can be directed towards me:

 starikov@mst.com

 [Illya Starikov](#)

 [Illya Starikov](#)

 [FreneticArray.com](#)



FACULTY
OF EDUCATION
Masaryk University

Observation (Presentation #7)

Special Topics (CS3001)

Illya Starikov

Backstory

At the beginning of the semester, one of my friends got me into the television *Psych*. Spoiler lies ahead. If you don't want to be spoiled, simply watch the first episode (it's on Amazon Prime Video).

Backstory



Figure: The Psych Sitcom

Backstory

The basic premise of Psych is thus: there is an extremely observant protagonist that pretends to be psychic. His “Psychic visions” are simply conclusions he draws from details he observes. I took note of the exceptional attention to detail he paid, and wanted to emulate this in my everyday life.

Goals

- The goal of becoming more observant were difficult to describe.
- I wanted to generally go by feeling.
 - Some days I feel as if I was very observant, other days I felt as if I was not very observant.

Actions Taken

I started generally paying attention to things, such as:

- What people wear
- Their mannerism
- What would make them upset
- What would make them happy

I would even try to test my ability to pay attention by trying to memorize license plates on my way walking to class, and trying to see how many I was able to recognize on my way back from class (generally a five hour delay).

Resources

- I read one book on the subject: [What Every BODY Saying](#)
- Other than that, a lot of the resources were the people I observed.

Goal Accomplishment

- I feel as if I have become a lot more observant this semester.
- However, this is a skill that does leave when you don't use it.
- I want to continually practice this the rest of my life.

An Example

A couple of my roommates and I were out at the bar. When we went outside, I noticed a man smoking a cigarette. Using my observational ability, I notice near the filter of the cigarette it said “Marlboro”, and I noticed his drink was something with coke. Using my deducing ability, I started off a conversation with him with “Marlboro/Rum and Coke I see”. He responded, with astonishment, “Are you physic?”.

In Closing

All question, comments, and insults can be directed towards me:

 starikov@mst.edu

 [Illya Starikov](#)

 [Illya Starikov](#)

 [FreneticArray.com](#)



FACULTY
OF EDUCATION
Masaryk University

Productivity (Presentation #8)

Special Topics (CS3001)

Illya Starikov

A Brief Introduction

I knew this semester was going to be difficult. I decided I needed better time and assignment management. After some research, I decided I wanted to learn how to practice Getting Things Done (GTD).

A Brief Introduction

Getting Things Done is a time management method created by David Allen in 2001. It centers around the idea of planning actionable items in an external system. This allows the person following the program to not ponder on next actions, since they have all of them in front of them. The Getting Things Done lifestyle can best be described by David Allen in his own book, [Getting Things Done: The Art of Stress Free Productivity](#).

A Brief Introduction

As part of the Getting Things Done lifestyle, I need some sort of system to store all of my actions in. After *a lot* of research and experimenting, I landed on **OmniFocus**. It got by far the best reviews.

A Brief Introduction



Figure: OmniFocus On The Mac

Prior Knowledge

- I was always interested in productivity, reading books such as The Power of Full Engagement and The Seven Habits of Highly Effective People.
- However, I never considered creating a system for all my actions.

Goals

- I wanted to maintain a consistent system throughout the semester that would organize mostly my school work.
- I wanted to always be able to plan ahead for assignments.
- I wanted to have better time management skills.

Goals*

- I realize I'm submitting almost all of my assignments the week before they're due, but that's mostly at the fault of other classes taking priority.

Resources

- I read the entirety of David Allen's Getting Things Done book. This gave the foundation to the methodology.
- I bought and watched all of David Spark's [OmniFocus video field guide](#). This taught me how to use OmniFocus.
- I read almost all of [SimplicityBliss's guide to OmniFocus](#). This gave me an idea of how to structure my OmniFocus projects.

Goal Accomplishment

- I was able to keep a system consistently throughout the semester.
- Along the way, I learned a lot of valuable lessons about productivity.

Lessons Learned

- Try to remain a constant routine. Currently, I have four routines.
 - Morning Routine
 - Night Routine
 - Saturday Routine (typically my least productive day, do self maintenance on this day)
 - Weekly Review (entire review of my system)
- An example of my morning routine is as follows:
 1. Shower
 2. Breakfast Reddit
 3. ...
 4. Open OmniFocus

Lessons Learned

- Keep checklists for everything.
- As soon as an actionable items comes into your life, write it down immediately.
- Structure you projects accordingly.
- Break everything up as much as possible.
 - For examples, Algorithms Project → Create Outline For Project, Code The Project, Document The Project, Create The UML.

In Closing

All question, comments, and insults can be directed towards me:

 starikov@mst.edu

 [Illya Starikov](#)

 [Illya Starikov](#)

 [FreneticArray.com](#)



FACULTY
OF EDUCATION
Masaryk University

Bartending (Presentation #9)

Special Topics (CS3001)

Illya Starikov

A Brief Introduction

Disclaimer: I am 21.

A Brief Introduction

I got onto a streak of cool bartending videos earlier in the semesters, such as:

- This one.
- This one.
- This one.

Inspired by this, I wanted to become a decent bartender. First, I knew I had to get some supplies.

A Brief Introduction



Figure: Some of the Alcohol Necessary For Bartending

A Brief Introduction



Figure: Some of the Tools Necessary For Bartending

Prior Knowledge

- I did not have much prior knowledge to this.

Goals

- To become well acquainted with some of the more famous cocktails and what is in them.
- To become make these drinks with some flair.

Resources

- To learn the mechanics of bartending, a lot of it came from practice and YouTube.
 - How to pour.
 - How to flip pour.
 - How to set up a bar.
- I also picked up a book to passively read: [Liquid Intelligence](#).

Goal Accomplishment

- I have learned how to make several of the famous cocktails relatively well.
- Some of them were very easy.
 - Screw Driver
 - Fuzzy Naval
 - Sex on the Beach
- Some of them were more difficult to make.¹
 - Mojitos
 - Mai Tai
 - Martini

Goal Accomplishment

- I also learned how to do some of the cool bar tricks in the previous videos.

Lessons Learned

Aside from the cocktails I made, there were some lessons I learned.

- The real difference between shaken and stirred and how it effects a drink.
- How much ice can effect a cocktail.
- What different liquor combinations can be combined to produce different flavors.

In Closing

All question, comments, and insults can be directed towards me:

 starikov@mst.edu

 [Illya Starikov](#)

 [Illya Starikov](#)

 [FreneticArray.com](#)



FACULTY
OF EDUCATION
Masaryk University

Financial Planning (Presentation #10)

Special Topics (CS3001)

Illya Starikov

A Brief Introduction

I have always had a problem with being financially responsible. As I am about to graduate, and will have exponentially more bills, I want a way to be able to handle all of them reasonably. So, I decided I needed a budget.

A Brief Introduction

I knew I would need a tool to help budget; after some search, I settled on EmPower. I also knew I needed something to invest money into, so I settled on Acorns.

A Brief Introduction

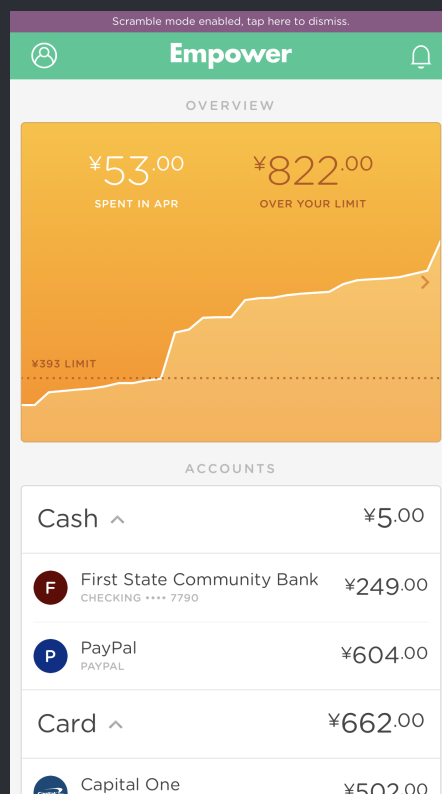


Figure: Empower (With Scramble Mode On, So All Data Is Randomize)

A Brief Introduction



Figure: Acorns

Prior Knowledge

- I've had mediocre money management skills in the past.
- I would basically ballpark expenses and such.
- Although this would suffice in college, it would not last me in the real world long.

Goals

- To keep a reliable budget for the entirety of the semester.
- To be able to update my budget when I graduate easily.

Resources

- All of the learning (besides a couple of articles online) came from experimenting.

Goal Accomplishment

- I successfully created a budget within EmPower.
 - It handles all my weekly expenses (i.e., grocery shopping), monthly expenses (i.e., utilities or internet), and other miscellaneous expenses (entertainment).
- I also was able to save a considerable amount into Acorns.
- The EmPower/Acorns system will be very useful when I graduate and move onto full time.

Goal Not Accomplishment (Yet)

- Sticking to my budget was very hard. Some months I would hit the price point, other times I would miss it by orders of magnitude.
- I would sometimes pull money out of my savings account a little prematurely.

Lessons Learned

The lesson I learned that was most valuable to me was to check my budget everyday. If I don't check it daily, I won't know how much I can or cannot spend. This could be for better or worse.

In Closing

All question, comments, and insults can be directed towards me:

 starikov@mst.edu

 [Illya Starikov](#)

 [Illya Starikov](#)

 [FreneticArray.com](#)

CS3200

Numerical Methods



Homework #9

Illya Starikov

Due Date: December 1st, 2016

1 Simpson 1/3 Rule For $1 + 2x + 3x^2$

$$\begin{aligned}\int_0^4 1 + 2x + 3x^2 dx &\approx h/6(f(a) + 4f(a+h) + f(b)) \\ &= 4/6(1 + 4(1 + 4 + 12) + (1 + 8 + 48)) \\ &= 2/3(1 + 68 + 57) \\ &= 84\end{aligned}$$

2 Simpson's 3/8 Rule For x^3

$$\begin{aligned}\int_0^3 x^3 dx &\approx 3/8h(f(a) + 3f(a+h) + 3f(a+2h) + f(b)) \\ &= 3/8(0 + 1 + 8 + 27) \\ &= 3/8 \cdot 36 \\ &= 13.5\end{aligned}$$

3 Trapezoidal, Richardson and Romberg Method of x^4

We know the equation for the trapezoid rule to be

$$I = \frac{h}{2} \left[f(x_0) + 2 \sum_{i=1}^n f(x_i) \right]$$

From this, we obtain

$$\begin{aligned} I_4 &= \frac{4}{2} (f(0) + f(4)) = 512 \\ I_2 &= \frac{2}{2} (f(0) + 2f(2) + f(4)) = 288 \\ I_1 &= \frac{1}{2} (f(0) + 2f(2) + 2f(3) + f(4)) = 226 \end{aligned}$$

For Richardson, we calculate the terms as follows:

$$\begin{aligned} I &\approx \frac{4}{3}I(h_2) - \frac{1}{3}I(h_1) \\ I_2 &= \frac{4}{3}(288) - \frac{1}{3}(512) = \frac{640}{3} = 213.\bar{3} \\ I_1 &= \frac{4}{3}(226) - \frac{1}{3}(288) = \frac{616}{3} = 205.\bar{3} \end{aligned}$$

And for Romberg,

$$\begin{aligned} I_{j,k} &= \frac{4^{k-1}I_{j+1,k-1} - I_{j,k-1}}{4^{k-1}} \\ I_1 &= \frac{4^2(\frac{616}{3}) - \frac{640}{3}}{15} = \frac{1024}{5} = 204.8 \end{aligned}$$

We observe that the table looks like

h	trapazoidal	Richardson	Romberg
4	512		
2	288	213. $\bar{3}$	
1	226	205. $\bar{3}$	204.8

4 Derivation of Richardson Extrapolation

We recall that the value of integration I is equal to approximation + error, which we can eloquently write

$$I = I(h) + \mathcal{E}(h) \quad (1)$$

Where $\mathcal{E}$ represents our error and $I(h)$ represents our approximation. Supposing we have two different steps size (h_1 and h_2), we can rewrite equation (1) in the form

$$I = I(h_1) + \mathcal{E}(h_1) = I(h_2) + \mathcal{E}(h_2) \quad (2)$$

Now, let us expand $\mathcal{E}(h)$ from equation (1) and equation (2).

$$\mathcal{E} \approx -\frac{b-a}{2}h^2 \bar{f}'' \quad (3)$$

Where b and a are the upper and lower bounds, respectively, and $\bar{f}''$ is the average value of $\frac{d}{dx}f(x)$ (where $f(x)$ is the function we are integrating). Therefore, we can a ratio of the two errors

$$\frac{\mathcal{E}(h_1)}{\mathcal{E}(h_2)} \approx -\frac{\frac{b-a}{2}h_1^2 \bar{f}''}{\frac{b-a}{2}h_2^2 \bar{f}''} \approx \frac{h_1^2}{h_2^2}$$

Which can be algebraically manipulated to

$$\mathcal{E}(h_1) \approx \mathcal{E}(h_2) \left(\frac{h_1}{h_2} \right)^2 \quad (4)$$

Now we can substitute equation (4) back into equation (2) to obtain

$$I \approx I(h_1) + \mathcal{E}(h_2) \left(\frac{h_1}{h_2} \right)^2 \approx I(h_2) + \mathcal{E}(h_2)$$

Which can be solve for

$$\mathcal{E}(h_2) \approx \frac{I(h_2) - I(h_1)}{(h_1/h_2)^2 - 1}$$

Now that we have an estimate (hence the $\approx$) of the truncation error, We once plug this back into equation (2) to get

$$I \approx I(h_2) + \frac{I(h_2) - I(h_1)}{(h_1/h_2)^2 - 1}.$$

Because we know the interval to be halved, we can safely assume $h_2 = h_1/2$. Thus, our equation reduces to

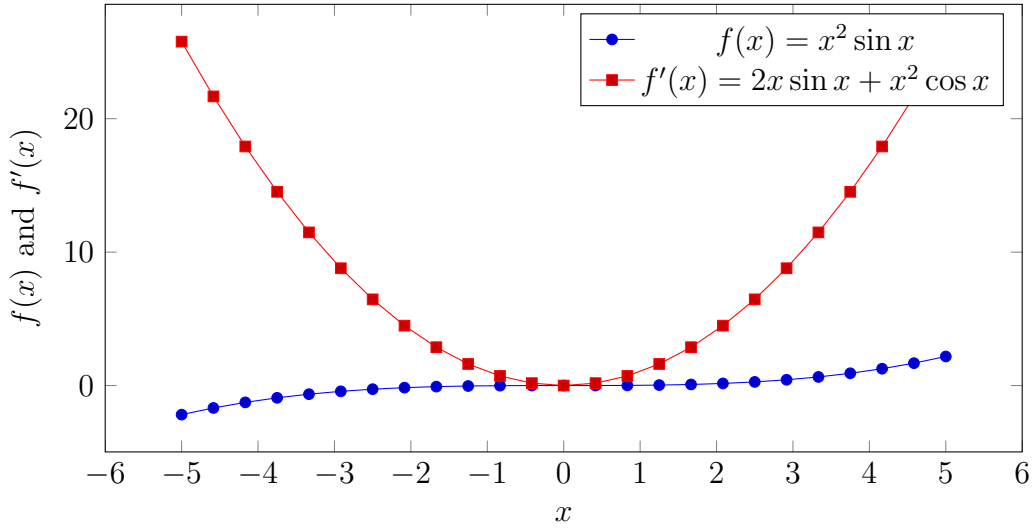
$$I \approx \frac{4}{3} I(h_2) - \frac{1}{3} I(h_1) \quad (5)$$

5 Derivative of $x^2 \sin x$

From the chain rule, we get

$$\frac{d}{dx} x^2 \sin x = 2x \sin x + x^2 \cos x$$

It can be plotted as follows



To compute the two derivatives for $h_1 = 0.2$ and $h_2 = 0.1$, we use the centered difference formulas. For h_1

$$\begin{aligned} \frac{d}{dx} &= \frac{f(x_{i+1}) - f(x_{i-1}))}{2h} \\ &= \frac{f(x+h) - f(x-h)}{2h} \\ &= \frac{f(x + 1/5) - f(x - 1/5)}{2/5} \\ &= \frac{(x + 1/5)^2 \sin(x + 1/5) - ((x - 1/5)^2 \sin(x - 1/5))}{2/5} \end{aligned}$$

Similarly, for h_2

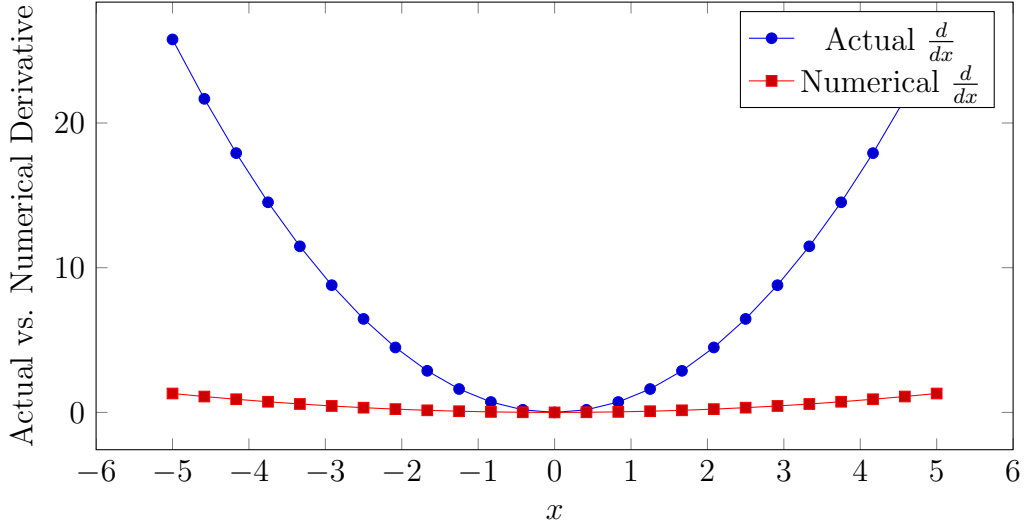
$$\frac{(x + 1/10)^2 \sin(x + 1/10) - ((x - 1/10)^2 \sin(x - 1/10))}{1/5}$$

We combine these for obtain the Richardson extrapolation formula,

$$D \approx 4/3 D(h_2) - 1/3 D(h_1) \quad (6)$$

$$\approx 4/3 \frac{(x + 1/10)^2 \sin(x + 1/10) - ((x - 1/10)^2 \sin(x - 1/10))}{1/5} \quad (7)$$

$$- 1/3 \frac{(x + 1/5)^2 \sin(x + 1/5) - ((x - 1/5)^2 \sin(x - 1/5))}{2/5} \quad (8)$$



6 Central Difference of x^4

6.1 First Order

From the power rule, we notice that

$$\frac{d}{dx} x^4 = 4x^3 \implies f'(1) = 4$$

Using centered difference formula,

$$\begin{aligned}
\frac{d}{dx} &= \frac{f(x_{i+1}) - f(x_{i-1}))}{2h} \\
&= \frac{(1 + 1/2)^4 - (1 - 1/2)^4}{(2^{1/2})} \\
&= 5
\end{aligned}$$

We notice the error to be $|\frac{4-5}{4}| = |- .25| \implies \mathbf{25\%}$

6.2 Second Order

From the power rule, we notice that

$$\frac{d^2}{dx^2} x^4 = 12x^2 \implies f'(1) = 12.$$

Using centered difference formula,

$$\begin{aligned}
\frac{d^2}{dx^2} &= \frac{f(x_{i+1}) - 2f(x_i) + f(x_{i-1}))}{h^2} \\
&= \frac{(1 + 1/2)^4 - 2(1^4) + (1 - 1/2)^4}{(1/2)^2} \\
&= 25/2
\end{aligned}$$

We notice the error to be $|\frac{12-25/2}{12}| = |- .041\bar{6}| \implies \mathbf{4.16\%}$.

Homework #10

Illya Starikov

Due Date: December 6th, 2016



CALENDAR CAMPUS MAP FIND PEOPLE A-Z INDEX

COMMUNITY FACULTY & STAFF ALUMNI & FRIENDS CURRENT STUDENTS FUTURE STUDENTS

Course Evaluation

Instructor Evaluation

Instructor: **Sabharwal, Chaman L**
Section: **COMP SCI 3200 - 1A - LEC**
Department: **Computer Science**
Course: **Intro Numerical Methods**

Your updates to this evaluation have been recorded. Thank you for your participation.

[Return to list of courses](#)

For any problems or corrections please contact the Solution Center at 573-341-HELP
or submit a [Help Request Ticket](#).

© Missouri University of Science and Technology | Rolla, MO 65409 | 573-341-4111 | 1-800-522-0938
Contact: [IT HelpDesk](#) · webmaster@mst.edu

CS3800

Operating Systems



Homework #2

Illya Starikov

Due Date: April 6th, 2017

For this assignment, the tasks were to simulate a memory manager with three page replacement algorithms

1. Clock Page Replacement
2. Least Recently Used Page Replacement
3. First In First Out Page Replacement

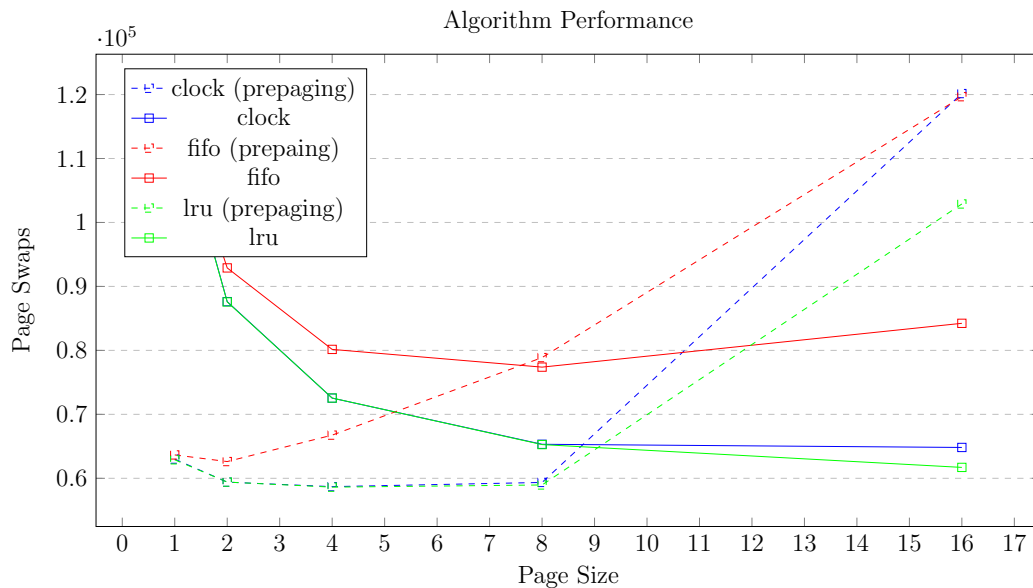


Figure 1 – Page Size vs. Page Swaps

This was accomplished by creating objects for the individual pages (eloquently named `Page`), programs (`Program`), and the memory simulator.

The individual algorithms were implemented as methods of the `MemorySimulation` class.

The results can be summarized by Figure 1. For the most part, the page swaps ranged from 60 000 to 120 000. The different page sizing affected the algorithms the same, *depending on if there was prepaging*. For prepaging, higher page size increased the page swaps. The inverse also holds, for on demand paging, the algorithms fared better for higher page size. Overall, *Least Recently Used* and *Clock Based Paging* with prepaging worked the best for page sizes of 2, 4, and 8. The worst was *Clock Based Paging* and *First In First Out* with prepaging. The median solution was *Least Recently Used* with prepaging.

In retrospect, one algorithm was no significantly more difficult to implement than another. Conceptually, the *Least Recently Used* algorithm was the most difficult, and even that one was not hard. *First In First Out* was the easiest to implement, but was the worst performance wise. *Clock Based Paging* was the hardest, but had some of the best performance.

Overall, the complexity of the algorithms wasn't the deciding factor on which one to use in day-to-day situations. With very high page swapping happening in an operating system, the deciding factor is the performance.

CS5200

Analysis Of Algorithms



Homework #1

Analysis of Algorithms

Illya Starikov

Due Date: September 1st, 2017

Question #1

- (a) 4.54 ± 0.05 billion years (https://en.wikipedia.org/wiki/Age_of_the_Earth)
- (b) 5 billion years (http://earthguide.ucsd.edu/virtualmuseum/ita/05_3.shtml)
- (c) 13.6 billion years $\pm$ 800 million years (<https://www.space.com/263-milky-age-narrowed.html>)
- (d) 13.82 billion years (http://www.slate.com/blogs/bad_astronomy/2013/03/21/age_of_the_universe_planck_results_show_universe_is_13_82_billion_years.html)
- (e) 7.79 billion years (<https://www.livescience.com/39775-how-long-can-earth-support-life.html>)
- (f) 3.25 billion years. Guaranteed existential risks, human prevention not possible.
 - Asteroid impact
 - Extraterrestrial invasion (i.e. , aliens taking over the world)
 - Geomagnetic reversal of the North and South pole

Prevention possible.

- Artificial super intelligence enslavement of the man race. Steps are unclear, besides keeping all experiments in a Faraday cage.
- Biotechnology engineering. A bio-engineered virus or disease that gets loose can cause a global pandemic. Steps to prevent are clear.
- Nuclear holocaust. Steps to prevent are clear.

Extinction prevention matters are to become a space-ferrying species, attempt some sort of peace treaty amongst all nations, and to try to prevent global pandemics. (https://en.wikipedia.org/wiki/Global_catastrophic_risk#Asteroid_impact)

- (g) 10-12 billion years. The sun will become a Red Giant, no longer being able to provide fuel. Will consume several planets along the way. (<https://www.forbes.com/sites/startswithabang/2017/01/27/how-our-solar-system-will-end-in-the-far-future/#20ea714f4e5d>)

Eventually, about 5–7 billion years down the line, we'll run out of nuclear fuel in the Sun's core, which will cause our parent star to become a Red Giant, engulfing Mercury and Venus in the process. Due to the particulars of stellar evolution, the Earth/Moon system will probably be pushed outwards, and be spared the fiery fate of our inner neighbors.

- (h) Either the universe will stop contracting, reach an influx in the gravitational pull on all cosmic bodies, and come back to a singularity; or, heat death, where entropy reaches its max, and all energy is evenly dissipated throughout the universe, freezing

everything. The first theory has 1-100 trillion years, the second has no estimate.
(<http://www.bbc.com/earth/story/20150602-how-will-the-universe-end>)

- (i) 584.6 billion years. Between 0.585%–58.5%.

Question #2

- (a) A standard, 12pt L^AT_EX document can fit roughly 39 lines of text.

$$\frac{9 \times 10^8}{39} \approx 2.31 \times 10^8 \text{ sheets}$$

- (b) 1500 sheets is approximately \$14.99, before tax. (https://www.amazon.com/Georgia-Pacific-Spectrum-Standard-Multipurpose-998606/dp/B00BB5DJU6/ref=sr_1_2?s=office-products&ie=UTF8&qid=1503934559&sr=1-2&keywords=printer+paper)

$$(2.31 \times 10^8 \text{ sheets}) \times \frac{\$14.99}{1500 \text{ sheets}} \approx \$2.31 \times 10^6$$

Assuming a standard filing cabinet can store roughly 15 reams, with 100 sheets in a ream,

$$2.31 \times 10^8 \text{ sheets} \times \frac{1 \text{ ream}}{100 \text{ sheets}} \times \frac{1 \text{ cabinet}}{15 \text{ reams}} = 154\,000$$

- (c) The monks expected the project to take *15,000 years* by hand, and *3 months* by modern technology. Assuming an average human lifespan of 79 years, approximately 50 years could be used writing.

$$\frac{15\,000 \text{ years}}{50 \text{ years}} = 300 \text{ people}$$

- (d) The time allocated to the computer was *3 months*. To print all 3 billion, it would have to print approximately 1120.07 names per second.
- (e) I enjoyed the story, and the first thing I did was a mental check to see if the numbers added up. Naturally, they did not, but that did not detract from the story.

Question #3

```
1 def alternating_difference(numbers):  
2     if not numbers:  
3         return 0  
4  
5     return numbers[0] - alternating_difference(numbers[1:])
```

Question #4

```
1 def f53q(n):
2     if n < 8:
3         return ValueError('Value_less_than_8')
4     elif n % 5 == 0:
5         return (0, n // 5)
6     elif n % 5 == 1:
7         return (2, (n - 5) // 5)
8     elif n % 5 == 2:
9         return (4, (n - 12) // 5)
10    elif n % 5 == 3:
11        return (1, n // 5)
12    else:
13        return (3, (n - 9) // 5)
```

Question #5

$$\begin{aligned}f(1) &= 91 \\f(-6) &= 91 \\f(200) &= 190 \\f(27) &= 91\end{aligned}$$

A more appropriate, non-recursive function would be as follows:

```
1 def f(n):
2     if n < 101:
3         return 91
4     else:
5         return n - 10
```

This will always produce the same output. Let us write $f(n)$ as a piecewise-defined function.

$$f(x) = \begin{cases} n - 10 & 100 < n < \infty \\ f(f(n + 11)) & -\infty < n \leq 100 \end{cases}$$

We see that, for values less than 100, n will grow until it hits the first case. For values $90 \dots 200$, we see that n will oscillate until it reaches 101, upon which one final $f(x)$ will be applied.

A more simple explanation would be to insert $n - 10$ into $f(f(n + 11))$ to see that $f(x) = f(f(n + 1))$ until it reaches 101, 1 past the bounds, and gets $n - 10$ applied to it one more time.

Question #6

The function is copy and pasted from implementation.

```
1 def A(x, y):  
2     if x == 0:  
3         return y + 1  
4     elif y == 0:  
5         return A(x - 1, 1)  
6     else:  
7         return A(x - 1, A(x, y - 1))
```

The maximum value of my system (Late-2013 MacBook Pro) is roughly $A(3, 5)$. For $A(2, 2)$, there should be 42 438 function calls.

Question #7

```
1 def gcd2(a, b):  
2     if a == 0:  
3         return (b, 0, 1)  
4     else:  
5         g, s, t = gcd2(b % a, a)  
6         return [g, t - (b // a) * s, s]
```

Question #8

```
1 # defined as list_ to not stomp out the typename _list_
2 def super_reverse(list_):
3     if len(list_) < 2:
4         return list_
5     else:
6         last_element = list_[-1]
7         first_element = list_[0]
8
9         # if is a list or a tuple, sort that as well
10        if isinstance(first_element, list):
11            first_element = super_reverse(first_element)
12        if isinstance(last_element, list):
13            last_element = super_reverse(last_element)
14
15        return [last_element] + super_reverse(list_[1:-1]) + [
            first_element]
```

Question #9

```

1 def anagram(string_):
2     if string_ == "":
3         return [""]
4
5     result = []
6
7     for partial_anagram in anagram(string_[1:]):
8         for position in range(len(partial_anagram) + 1) :
9             right_side = partial_anagram[position:]
10            left_side = partial_anagram[:position]
11            original_element = string_[0]
12
13            result += [left_side + original_element + right_side]
14
15     return result

```

Table 1: Results (units in milliseconds)

String Size	One Recursive Call	n Recursive Calls	Absolute Difference	Percent Difference
1	0.006	0.004	-0.002	-56.250%
2	0.009	0.009	-0.000	-2.703%
3	0.015	0.026	0.011	42.202%
4	0.036	0.104	0.068	65.367%
5	0.139	0.528	0.389	73.668%
6	0.672	3.124	2.452	78.486%
7	5.312	25.050	19.738	78.794%
8	39.671	204.850	165.179	80.634%
9	332.021	1936.445	1604.424	82.854%
10	3471.336	21813.828	18342.492	84.087%
11	43223.737	267635.574	224411.837	83.850%

Homework #2

Analysis of Algorithms

Illya Starikov

Due Date: September 11th, 2017

Question #1

We define $Q = \{q | q + n = n + k\}$, and $n \in \mathbb{Z}^+$. Given that $n + 0 = n = 0 + n$

$$\begin{aligned} q + (k + 1) &= (q + k) + 1 \\ &= (k + q) + 1 \\ &= k + (1 + k) \\ &= (k + 1) + m \\ \therefore q &\in Q \\ \therefore n + q &= q \end{aligned}$$

Question #2

Theorem 1. $\forall i \in \mathbb{Z}^+$,

$$\sum_{i=1}^n \frac{i^2}{(2i-1)(2i+1)} = \frac{n(n+1)}{2(2n+1)} \quad (1)$$

Proof. We will use the principle of recursion to solve this problem.

Step #1 Our problem states for an arbitrary integer i , summed to an arbitrary integer n , we have the summation (we denote by $f(x)$) and explicit forms ($g(x)$) described by Equation 1. We take the stopping value to be 0.

Step #2 Checking two values is trivial; take 1 and 2.

$$\begin{aligned} \sum_{i=1}^1 \frac{i^2}{(2i-1)(2i+1)} &= \frac{1(1+1)}{2(2*1+1)} = \frac{1}{3} \\ \sum_{i=1}^2 \frac{i^2}{(2i-1)(2i+1)} &= \frac{2*(2+1)}{2(2*2+1)} = \frac{3}{5} \end{aligned}$$

To check the stopping value, we check the 0th value. We clearly see that $f(x) = g(x) = 0$.

Step #3 If n in $\mathbb{Z}^+$ triggers a recursive call, then $n > 0$. The only value used in the call is $n-1$, which is in $\mathbb{Z}$ and greater than or equal to 0, because it is an integer and $n-1 \geq 0$ since $n > 0$.

Step #4 We use the integer n as the counter. When recursion is called the function is called with the value $n-1$. The counter strictly decreases and the recursion halts.

Step #5 To prove recursion stops, we take the following:

$$\begin{aligned} f(n) &= \sum_{i=1}^n \frac{i^2}{(2i-1)(2i+1)} && \text{definition} \\ &= f(n-1) + \frac{n^2}{(2n-1)(2n+1)} && \text{recursive structure} \\ &= g(n-1) + \frac{n^2}{(2n-1)(2n+1)} && \text{by our assumption} \\ &= \frac{(n-1)((n-1)+1)}{2(2(n-1)+1)} + \frac{n^2}{(2n-1)(2n+1)} && \text{replacing } g(n) \text{ with definition} \\ &= \frac{n(n+1)}{2(2n+1)} && \text{algebraic simplification} \\ &= g(n) \end{aligned}$$

Because $f(n) = g(n)$, the property is inherited recursively.

Step #6 Since Steps 1–5 have been verified, it follows from the Principle of Recursion that P holds for all values in $\mathbb{Z}^+$, i.e., $f(n) = g(n), \forall n \in \mathbb{Z}, n \geq 0$

□

Table 1: The results from the explicit form, the series form, and the difference for the first 40 values.

Sum Value	Explicit Values	Difference
0.333 333 333 333	0.333 333 333 333	0.000 000 000 000 000
0.600 000 000 000	0.600 000 000 000	0.000 000 000 000 000
0.857 142 857 143	0.857 142 857 143	0.000 000 000 000 000
1.111 111 111 111	1.111 111 111 111	0.000 000 000 000 000
1.363 636 363 636	1.363 636 363 636	0.000 000 000 000 000
1.615 384 615 385	1.615 384 615 385	0.000 000 000 000 000
1.866 666 666 667	1.866 666 666 667	0.000 000 000 000 000
2.117 647 058 824	2.117 647 058 824	0.000 000 000 000 000
2.368 421 052 632	2.368 421 052 632	0.000 000 000 000 000
2.619 047 619 048	2.619 047 619 048	0.000 000 000 000 000
2.869 565 217 391	2.869 565 217 391	0.000 000 000 000 000
3.120 000 000 000	3.120 000 000 000	0.000 000 000 000 001
3.370 370 370 370	3.370 370 370 370	0.000 000 000 000 001
3.620 689 655 172	3.620 689 655 172	0.000 000 000 000 001
3.870 967 741 935	3.870 967 741 935	0.000 000 000 000 001
4.121 212 121 212	4.121 212 121 212	0.000 000 000 000 001
4.371 428 571 429	4.371 428 571 429	0.000 000 000 000 001
4.621 621 621 622	4.621 621 621 622	0.000 000 000 000 002
4.871 794 871 795	4.871 794 871 795	0.000 000 000 000 001
5.121 951 219 512	5.121 951 219 512	0.000 000 000 000 002
5.372 093 023 256	5.372 093 023 256	0.000 000 000 000 002
5.622 222 222 222	5.622 222 222 222	0.000 000 000 000 002
5.872 340 425 532	5.872 340 425 532	0.000 000 000 000 003
6.122 448 979 592	6.122 448 979 592	0.000 000 000 000 003
6.372 549 019 608	6.372 549 019 608	0.000 000 000 000 003
6.622 641 509 434	6.622 641 509 434	0.000 000 000 000 004
6.872 727 272 727	6.872 727 272 727	0.000 000 000 000 004
7.122 807 017 544	7.122 807 017 544	0.000 000 000 000 004
7.372 881 355 932	7.372 881 355 932	0.000 000 000 000 004
7.622 950 819 672	7.622 950 819 672	0.000 000 000 000 004
7.873 015 873 016	7.873 015 873 016	0.000 000 000 000 003
8.123 076 923 077	8.123 076 923 077	0.000 000 000 000 004
8.373 134 328 358	8.373 134 328 358	0.000 000 000 000 004
8.623 188 405 797	8.623 188 405 797	0.000 000 000 000 004
8.873 239 436 620	8.873 239 436 620	0.000 000 000 000 004
9.123 287 671 233	9.123 287 671 233	0.000 000 000 000 004
9.373 333 333 333	9.373 333 333 333	0.000 000 000 000 004
9.623 376 623 377	9.623 376 623 377	0.000 000 000 000 004
9.873 417 721 519	9.873 417 721 519	0.000 000 000 000 004

Question #3

Theorem 2. For an arbitrary $n \in \mathbb{Z}$, such that $n \geq 0$,

$$2^n \leq \text{fib}(n) \leq 2^{\frac{n}{2}}$$

Proof. *Step #1* Prove by induction that for $n > 0$

$$2^n \leq \text{fib}(n) \leq 2^{\frac{n}{2}}$$

Step #2 Check the base case and two other values,

$$2^0 \leq \text{fib}(0) \leq 2^0, 2^1 \leq \text{fib}(1) \leq 2^{\frac{1}{2}}, 2^2 \leq \text{fib}(2) \leq 2^1$$

Step #3 To prove the recursive case, we take the following.

$$\begin{aligned} 2^n &\leq \text{fib}(n) \leq 2^{\frac{n}{2}} \\ 2^{n-1} &\leq \text{fib}(n) + \text{fib}(n-1) \leq 2^{\frac{n-1}{2}} \\ \frac{2^n}{2} &\leq \text{fib}(n) + \text{fib}(n-1) \leq \frac{2^{\frac{n}{2}}}{\sqrt{2}} \end{aligned}$$

Because we know $\frac{2^n}{2} \leq 2^n$ and $\frac{2^{\frac{n}{2}}}{\sqrt{2}} \leq 2^{\frac{n}{2}}$, we know that $f(n)$ must be bounded by those function.

Step #4 Since the hypotheses of Simple Induction are true, the conclusion follows, namely, for all natural numbers, $f(n) \bmod 133 = 0$.

□

Question #4

Upon inspection, it's quite apparent that the series is the sum of all natural numbers, with every other number being negative. More formally,

$$\sum_{q=1}^n (-1)^{q-1} q^2 = (-1)^{n-1} \frac{n * (n + 1)}{2}$$

The results can be summarized below.

Table 2: The results from the explicit form, the series form, and the difference for the first 40 values.

Sum Value	Explicit Values	Sum of Numbers
1.0000000000	1.0000000000	1.0000000000000000
-3.0000000000	-3.0000000000	3.0000000000000000
6.0000000000	6.0000000000	6.0000000000000000
-10.0000000000	-10.0000000000	10.0000000000000000
15.0000000000	15.0000000000	15.0000000000000000
-21.0000000000	-21.0000000000	21.0000000000000000
28.0000000000	28.0000000000	28.0000000000000000
-36.0000000000	-36.0000000000	36.0000000000000000
45.0000000000	45.0000000000	45.0000000000000000
-55.0000000000	-55.0000000000	55.0000000000000000
66.0000000000	66.0000000000	66.0000000000000000
-78.0000000000	-78.0000000000	78.0000000000000000
91.0000000000	91.0000000000	91.0000000000000000
-105.0000000000	-105.0000000000	105.0000000000000000
120.0000000000	120.0000000000	120.0000000000000000
-136.0000000000	-136.0000000000	136.0000000000000000
153.0000000000	153.0000000000	153.0000000000000000
-171.0000000000	-171.0000000000	171.0000000000000000
190.0000000000	190.0000000000	190.0000000000000000
-210.0000000000	-210.0000000000	210.0000000000000000
231.0000000000	231.0000000000	231.0000000000000000
-253.0000000000	-253.0000000000	253.0000000000000000
276.0000000000	276.0000000000	276.0000000000000000
-300.0000000000	-300.0000000000	300.0000000000000000
325.0000000000	325.0000000000	325.0000000000000000
-351.0000000000	-351.0000000000	351.0000000000000000
378.0000000000	378.0000000000	378.0000000000000000
-406.0000000000	-406.0000000000	406.0000000000000000
435.0000000000	435.0000000000	435.0000000000000000
-465.0000000000	-465.0000000000	465.0000000000000000
496.0000000000	496.0000000000	496.0000000000000000
-528.0000000000	-528.0000000000	528.0000000000000000
561.0000000000	561.0000000000	561.0000000000000000
-595.0000000000	-595.0000000000	595.0000000000000000
630.0000000000	630.0000000000	630.0000000000000000
-666.0000000000	-666.0000000000	666.0000000000000000
703.0000000000	703.0000000000	703.0000000000000000
-741.0000000000	-741.0000000000	741.0000000000000000
780.0000000000	780.0000000000	780.0000000000000000

Question #5

Question #6

```
1 def depth(n):
2     if n < 2:
3         return 1
4     if n % 2 == 1:
5         return 1 + depth(3 * n + 1)
6     else:
7         return 1 + depth(n // 2)
8
9
10 def main():
11     for i in range(101):
12         print("%3.0f_&_5.0f\\\\" % (i, depth(i)))
13
14
15 if __name__ == "__main__":
16     main()
```

Iteration	Depth				
0	1	34	14	68	15
1	1	35	14	69	15
2	2	36	22	70	15
3	8	37	22	71	103
4	3	38	22	72	23
5	6	39	35	73	116
6	9	40	9	74	23
7	17	41	110	75	15
8	4	42	9	76	23
9	20	43	30	77	23
10	7	44	17	78	36
11	15	45	17	79	36
12	10	46	17	80	10
13	10	47	105	81	23
14	18	48	12	82	111
15	18	49	25	83	111
16	5	50	25	84	10
17	13	51	25	85	10
18	21	52	12	86	31
19	21	53	12	87	31
20	8	54	113	88	18
21	8	55	113	89	31
22	16	56	20	90	18
23	16	57	33	91	93
24	11	58	20	92	18
25	24	59	33	93	18
26	11	60	20	94	106
27	112	61	20	95	106
28	19	62	108	96	13
29	19	63	108	97	119
30	19	64	7	98	26
31	107	65	28	99	26
32	6	66	28	100	26
33	27	67	28		

Question #7

Theorem 3. For an arbitrary $n \in \mathbb{Z}$, such that $n \geq 0$,

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

Proof. *Step #1* Prove by induction that for $n = 0$

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

We choose the summation form as $f(x)$ and the explicit form $g(x)$.

Step #2 Check the base case and two other values,

$$\begin{aligned} \sum_{i=1}^1 n^2 &= 1 = \frac{1(1+1)(2 \cdot 1 + 1)}{6} \\ \sum_{i=1}^2 n^2 &= 5 = \frac{1(1+1)(2 \cdot 1 + 1)}{6} \\ \sum_{i=1}^3 n^2 &= 14 = \frac{2 \cdot (2+1)(2 \cdot 2 + 1)}{6} \end{aligned}$$

Step #3 $\forall n > 0$, prove that $P(n)$ is true if $P(n-1)$ is true,

$$\begin{aligned} f(x) &= \sum_{i=1}^n i^2 && \text{by definition} \\ &= f(x-1) + n^2 && \text{by recursion} \\ &= g(x-1) + n^2 && \text{by hypothesis} \\ &= \frac{(n-1)(n-1+1)(2(n-1)+1)}{6} + n^2 \\ &= \frac{n(n+1)(2n+1)}{6} && \text{algebraic simplification} \end{aligned}$$

Step #4 Since the hypotheses of Simple Induction are true, the conclusion follows, namely, for all natural numbers, $f(n) = g(n)$.

□

Question #8

Theorem 4. For an arbitrary $n \in \mathbb{Z}$, such that $n \geq 0$,

$$11^{n+2} + 12^{2n+1} \mod 133 = 0$$

Proof. *Step #1* Prove by induction that for $n > 0$

$$11^{n+2} + 12^{2n+1} \mod 133 = 0$$

Step #2 Check the base case and two other values,

$$\begin{aligned} 11^3 + 12^3 \mod 133 &= 0 \\ 11^4 + 12^5 \mod 133 &= 0 \end{aligned}$$

Step #3 $\forall n > 0$, prove that $P(n)$ is true if $P(n-1)$ is true,

$$\begin{aligned} f(x) &= 11^{n+2} + 12^{2n+1} \\ &= 11^{n+1} + 12^{2n-1} + 11^{n+2} + 12^{2n+1} \end{aligned}$$

Because we assume $11^{n+2} + 12^{2n+1}$ to be divisible, we can rewrite it to be $133a_1$, signifying it is evenly divisible by 133.

$$\begin{aligned} f(x) &= 11 * 11^n + 12^{-1} * 144^n + 133a_1 \\ &= a_2 11^n + a_3 144^n + 133a_1 \end{aligned}$$

We know the superposition of all of these numbers to be divisible by 133.

Step #4 Since the hypotheses of Simple Induction are true, the conclusion follows, namely, for all natural numbers, $f(n) \mod 133 = 0$.

□

Question #9

Let $F(0)$ denote the root of the tree T . If we start at $F(0)$, there must be an adjacent vertex to it, namely, v_1 . There are infinitely many vertexes by going through the tree (this must be the case, or the tree T would be finite).

We can repeat this process, for at any vertex v_n , there exists a vertex v_{n+1} . Because the tree is infinite, there are infinitely many v_{n+1} .

By induction, for any n , there exists a path of length L in T .

Question #10

```
1 class Binary_Search_Tree:
2     def __init__(self, data):
3         if data is None:
4             self.data = [0, [], []]
5         else:
6             self.data = data
7
8     def add_node(self, node):
9         if node.data[0] < self.data[0]:
10             self.__add_node_left(node)
11         else:
12             self.__add_node_right(node)
13
14     def __add_node_left(self, node):
15         if self.data[1] == []:
16             self.data[1] = node
17         else:
18             self.data[1].add_node(node)
19
20     def __add_node_right(self, node):
21         if self.data[2] == []:
22             self.data[2] = node
23         else:
24             self.data[2].add_node(node)
25
26     def leaf_count(self):
27         if self.data[1] == []:
28             if self.data[2] == []:
29                 return 1
30             else:
31                 return self.data[2].leaf_count()
32         elif self.data[2] == []:
33             return self.data[1].leaf_count()
34         else:
35             return self.data[1].leaf_count() + self.data[2].
36 leaf_count()
37
38     def internal_count(self):
39         if self.data[1] == []:
40             if self.data[2] == []:
41                 return 0
42             elif self.data[2] == []:
43                 return 1 + self.data[1].internal_count()
44             else:
45                 return 1 + self.data[1].internal_count() + self.data[2].
46 internal_count()
47
48 def main():
49     T = Binary_Search_Tree([
```

```

49         42,
50         Binary_Search_Tree([29, [], []]),
51         Binary_Search_Tree([51, [], []])
52     ])
53
54     print("Leaf:␣{0},␣Internal:␣{1}".format(T.internal_count(), T.
leaf_count()))
55
56     T.add_node(Binary_Search_Tree([25, [], []]))
57     T.add_node(Binary_Search_Tree([22, [], []]))
58     T.add_node(Binary_Search_Tree([45, [], []]))
59     T.add_node(Binary_Search_Tree([8, [], []]))
60     T.add_node(Binary_Search_Tree([1000, [], []]))
61     T.add_node(Binary_Search_Tree([3, [], []]))
62     T.add_node(Binary_Search_Tree([1, [], []]))
63
64     print("Leaf:␣{0},␣Internal:␣{1}".format(T.internal_count(), T.
leaf_count()))
65
66
67 if __name__ == "__main__":
68     main()

```

From the results, we can very much see that Internal Nodes = Leaf Nodes − 1.

Proof

Proof. Take n to be the number of nodes, $I(T)$ the number of internal nodes and $L(T)$ to be the number of leaf nodes. We know the internal nodes to be the sum of previous internal nodes (i.e., $I(t) = I(t_1) + I(t_2) + 1$).

From this, we get

$$I(T) = I(t_1) + I(t_2) + 1 \quad (2)$$

$$I(t_1) = L(t_1) - 1 \quad (3)$$

$$I(t_2) = L(t_2) - 1 \quad (4)$$

$$= L(T) - 1 \quad (5)$$

□

Homework #3

Analysis of Algorithms

Illya Starikov

Due Date: September 19th, 2017

Question #1

```
1 def fn(n):  
2     if n > 100:  
3         return n - 10  
4     return ((n - 101) % (c - 10)) + 91
```

Question #2

The following function tests random values up to in $\{-1\,000\,000, \dots, 1\,000\,000\}$. `is_gcd_fast` is a faster alternative because $\gcd \in \mathcal{O}\left(\log_{\frac{a}{\gcd(a,b)}}(a \times b)\right)$ while $\gcd_{\text{fast}} \in \mathcal{O}(c)$.

```
1 from fractions import gcd
2 import random
3
4
5 def is_gcd(gcd_value, a, b):
6     return gcd_value == gcd(a, b)
7
8
9 def is_gcd_fast(gcd_value, a, b):
10     return a % gcd_value == b % gcd_value == 0
11
12 for i in range(1000):
13     a = random.randint(-1000000, 1000000)
14     b = random.randint(-1000000, 1000000)
15     g = gcd(a, b) if random.choice([True, False]) else random.
        randint(-1000000, 1000000)
16
17     print(is_gcd_fast(g, a, b) == is_gcd(g, a, b))
```

Question #3

All Sums

```
1 # Warning, for all lists (with the exclusion of [1], [0] and [])
  this function does not stop
2 def generate_all_sums(l):
3     for element in l:
4         if element == 0:
5             return [0] + generate_all_sums(element[1:])
6         else:
7             return [element] + [generate_all_sums([element*element])]
8
9     return l
```

$Sum(T) \notin \{\text{all odds}\}$

Take Sum to be the function that maps all elements to all of their multiples.

Theorem 1. *There is not set T such that $Sum(T) = \{\text{all odd integers}\}$.*

We prove so by proof of contradiction.

Proof. Suppose not. That is, suppose that is a set T such that $Sum(T) = \{\text{all odd integers}\}$. Set T , at a minimum, must have a single element (for the empty set $\emptyset$ cannot produce any integers). We take an arbitrary element from this set T and name it q . There are two forms for q ($\exists m \in \mathbb{Z}$),

$$q = \begin{cases} 2m & \Leftrightarrow q \text{ is even} \\ 2m + 1 & \Leftrightarrow q \text{ is odd} \end{cases}$$

If q is even, there is already a contradiction. If q is odd, then we have the following contradiction. Supposing we sum q twice, we have the following form:

$$q = a(2m + 1) + b(2m + 1)$$

Because a and b are arbitrary, we take them to be 1.

$$\begin{aligned} q &= (2m + 1) + (2m + 1) \\ &= 4m + 2 \\ &= 2(2m + 1) \end{aligned}$$

Because we know the sum of integers and multiplication of integers to be integers, $2(2m + 1) \equiv 2n, \exists n \in \mathbb{Z}$, which has the form of an even integer.

This has lead us to a contradiction. Therefore, our hypothesis is false, concluding there is not set T such that $Sum(T) = \{\text{all odd integers}\}$.

□

Fast Algorithm For S

To get d from an arbitrary set S , we simply take the greatest common divisor from all the sets.

```
1 from functools import reduce
2
3 def gcd(numbers):
4     def gcd_single(a, b):
5         while b:
6             a, b = b, a % b
7         return a
8
9     return reduce(gcd_single, numbers)
```

$d \in \{540051690381, 5404079462298, 3485942644184\}$

547.

Question #4

Theorem 2. $\forall i \in \mathbb{Z}^+,$

$$\sum_{i=1}^n (-1)^{i-1} i^3 = -\frac{1}{8}(-1)^n (4n^3 + 6n^2 - 1) - \frac{1}{8}$$

Proof. *Step #1* We wish to prove that for all natural numbers $f(n) = g(n)$, where f and g are as defined as follows:

$$f(x) = \sum_{i=1}^n (-1)^{i-1} i^3$$
$$g(x) = -\frac{1}{8}(-1)^n (4n^3 + 6n^2 - 1) - \frac{1}{8}$$

Let D be the set of natural numbers (i.e., $D = \mathbb{Z}^+$). D includes the stopping value 1.

Step #2 Checking two values is trivial; take 2 and 3.

$$\sum_{i=1}^2 (-1)^{i-1} i^3 = -\frac{1}{8}(-1)^2 (4 * 2^3 + 6 * 2^2 - 1) - \frac{1}{8} = -7$$
$$\sum_{i=1}^3 (-1)^{i-1} i^3 = -\frac{1}{8}(-1)^3 (4 * 3^3 + 6 * 3^2 - 1) - \frac{1}{8} = 20$$

To check the stopping value, we check the first value. We clearly see that $f(x) = g(x) = 1$.

Step #3 If n in $\mathbb{Z}^+$ triggers a recursive call, then $n > 0$. The only value used in the call is $n - 1$, which is in $\mathbb{Z}$ and greater than or equal to 0, because it is an integer and $n1 \geq 0$ since $n > 0$.

Step #4 We use the integer n as the counter. When recursion is called the function is called with the value $n - 1$. The counter strictly decreases and the recursion halts.

Step #5 To prove recursion stops, we take the following:

$$f(x) = f(n-1) + f(n) \quad (1)$$

$$= f(n-1) + (-1)^{n-1} n^3 \quad (2)$$

$$= -\frac{1}{8}(-1)^{(n-1)} (4(n-1)^3 + 6(n-1)^2 - 1) - \frac{1}{8} + (-1)^{n-1} n^3 \quad (3)$$

$$= -\frac{1}{8}(-1)^n (4n^3 + 6n^2 - 1) - \frac{1}{8} \quad (4)$$

$$= g(x) \quad (5)$$

Because $f(n) = g(n)$, the property is inherited recursively.

Step #6 Since Steps 1–5 have been verified, it follows from the Principle of Recursion that P holds for all values in $\mathbb{Z}^+$, i.e., $f(n) = g(n), \forall n \in \mathbb{Z}^+$

□

Table 1: The results from the explicit form, the series form, and the difference for the first 40 values.

Sum Value	Explicit Values	Difference
1	1	0
-7	-7	0
20	20	0
-44	-44	0
81	81	0
-135	-135	0
208	208	0
-304	-304	0
425	425	0
-575	-575	0
756	756	0
-972	-972	0
1225	1225	0
-1519	-1519	0
1856	1856	0
-2240	-2240	0
2673	2673	0
-3159	-3159	0
3700	3700	0
-4300	-4300	0
4961	4961	0
-5687	-5687	0
6480	6480	0
-7344	-7344	0
8281	8281	0
-9295	-9295	0
10388	10388	0
-11564	-11564	0
12825	12825	0
-14175	-14175	0
15616	15616	0
-17152	-17152	0
18785	18785	0
-20519	-20519	0
22356	22356	0
-24300	-24300	0
26353	26353	0
-28519	-28519	0
30800	30800	0

```

1  def sum_solution(n):
2      if n <= 1:
3          return 1
4      else:
5          return (-1)**(n - 1) * n**3 + sum_solution(n - 1)
6
7
8  def explicit_solution(n):
9      return -int(
10         (1.0 / 8.0) *
11         ((-1)**n) *
12         (4 * n**3 + 6 * n**2 - 1) +
13         1.0 / 8.0)
14
15
16 def main():
17     for n in range(1, 40):
18         print(sum_solution(n), explicit_solution(n), sum_solution(n)
19             - explicit_solution(n))
20
21 if __name__ == "__main__":
22     main()

```

Homework #4

Analysis of Algorithms

Illya Starikov

Due Date: September 26th, 2017

Question #1

```
1 def sum_of_two_elements(S, x):
2     S = sorted(S)
3     i, j = 0, len(S) - 1
4     while i <= j:
5         if S[i] + S[j] == x:
6             return True
7         elif S[i] + S[j] < x:
8             i += 1
9         else:
10            j -= 1
11
12     return False
```

Table 1: Output for `sum_of_two_elements`

List	x	$n \lg n$ solution	n^2 solution
[219, 526, 87, 829, 59, 384, 849, 65, 463, 934]	449	True	True
[393, 983, 584, 160, 421, 652, 41, 719, 686, 181]	1338	True	True
[491, 154, 345, 508, 208, 50, 11, 183, 723, 994]	931	True	True
[798, 810, 263, 786, 177, 314, 211, 708, 300, 286]	1061	True	True
[262, 494, 533, 38, 105, 937, 383, 625, 733, 428]	645	True	True
[730, 371, 356, 591, 126, 416, 732, 84, 505, 165]	717	True	True
[469, 821, 559, 492, 479, 91, 621, 55, 550, 507]	605	True	True
[822, 407, 527, 858, 498, 360, 551, 532, 503, 274]	1054	True	True
[278, 265, 997, 628, 563, 536, 783, 817, 725, 124]	1380	True	True
[455, 745, 422, 274, 335, 781, 909, 867, 669, 681]	943	True	True
[209, 836, 717, 858, 446, 773, 507, 693, 907, 59]	268	True	True
[18, 639, 711, 738, 583, 69, 714, 503, 597, 280]	1452	True	True
[690, 867, 901, 558, 367, 927, 439, 590, 651, 447]	1459	True	True
[156, 129, 61, 363, 948, 347, 874, 914, 775, 73]	503	True	True
[781, 985, 385, 523, 753, 804, 740, 7, 155, 441]	596	True	True
[897, 799, 83, 402, 144, 820, 621, 22, 640, 660]	1420	True	True
[478, 364, 216, 907, 638, 576, 835, 487, 571, 883]	1051	True	True
[193, 116, 483, 17, 363, 276, 14, 534, 145, 636]	162	True	True
[626, 446, 185, 716, 514, 225, 953, 826, 758, 809]	1072	True	True
[268, 778, 934, 880, 347, 306, 90, 767, 626, 230]	1712	True	True
[170, 591, 380, 744, 868, 242, 736, 756, 45, 798]	624	False	False
[85, 354, 59, 185, 916, 42, 567, 532, 106, 285]	704	False	False
[938, 218, 584, 440, 574, 748, 450, 931, 955, 588]	909	False	False
[930, 337, 292, 491, 668, 486, 630, 320, 91, 797]	17	False	False
[277, 754, 116, 486, 75, 868, 788, 346, 326, 188]	847	False	False
[951, 37, 80, 72, 515, 45, 925, 533, 626, 767]	346	False	False
[529, 286, 483, 105, 283, 224, 461, 245, 447, 861]	482	False	False
[563, 577, 795, 911, 692, 462, 755, 311, 898, 268]	706	False	False
[798, 617, 27, 746, 149, 763, 546, 752, 692, 279]	816	False	False
[334, 339, 184, 870, 776, 716, 375, 752, 414, 453]	522	False	False
[683, 969, 687, 274, 870, 566, 139, 664, 699, 325]	17	False	False
[641, 411, 4, 735, 742, 205, 264, 548, 331, 617]	604	False	False
[876, 939, 653, 487, 148, 433, 807, 238, 848, 556]	872	False	False
[385, 21, 172, 598, 463, 721, 187, 670, 328, 917]	510	False	False
[3, 246, 181, 435, 937, 974, 817, 109, 841, 383]	518	False	False
[889, 542, 73, 455, 946, 307, 189, 988, 440, 349]	58	False	False
[383, 691, 490, 938, 233, 139, 818, 231, 825, 720]	170	False	False
[772, 543, 127, 538, 273, 508, 924, 793, 495, 150]	229	False	False
[311, 843, 307, 863, 305, 602, 696, 351, 910, 776]	42	False	False

Question #2

1. As follows, the inversions (i, j) are:

- a) (1, 5)
- b) (2, 5)
- c) (3, 4)
- d) (3, 5)
- e) (4, 5)

$(A[i], A[j])$ are:

- a) (2, 1)
- b) (3, 5)
- c) (8, 6)
- d) (8, 1)
- e) (6, 1)

2. For a reverse sorted array, there would $\binom{n}{2} = \frac{n(n-1)}{2}$ total inversions.

3. Divide the array recursively into half and count number of inversions in sub-arrays (an $\lg n$ algorithm). To count all inversions, it takes n steps. $n - \text{operations} \times \lg n - \text{recursive steps}$. Combined, $\Theta(n \lg n)$ algorithm.

Question #3

We recall that the geometric series, and its derivatives, state that:

$$\sum_{k=0}^{\infty} x^k = \frac{1}{1-x} \quad \text{and} \quad \sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2} \quad (1)$$

Next, we know that our sum is as follows:

$$\sum_{k=0}^{\infty} \frac{(k-1)}{2^k}$$

Breaking these up we have the following:

$$\sum_{k=0}^{\infty} k \left(\frac{1}{2}\right)^k - \sum_{k=0}^{\infty} \left(\frac{1}{2}\right)^k \quad (2)$$

Replacing Equation 2 with Equation 1, we get the following:

$$\begin{aligned} \sum_{k=0}^{\infty} k \left(\frac{1}{2}\right)^k - \sum_{k=0}^{\infty} \left(\frac{1}{2}\right)^k &= \frac{1/2}{(1-1/2)^2} - \frac{1}{1-1/2} \\ &= 2 - 2 \\ &= 0 \end{aligned}$$

Question #4

Because k^3 is monotonically increasing, we have the following bounds upper bound:

$$\begin{aligned}\sum_{k=1}^n k^3 &\leq \int_1^{n+1} x^3 dx \\ &= \left. \frac{x^4}{4} \right|_1^{n+1} \\ &= \frac{(n+1)^4 - 1}{4}\end{aligned}$$

For the lower bound,

$$\begin{aligned}\sum_{k=1}^n k^3 &\geq \int_0^n x^3 dx \\ &= \left. \frac{x^4}{4} \right|_0^n \\ &= \frac{(n)^4}{4}\end{aligned}$$

So, the bounds are as follows:

$$\frac{(n)^4}{4} \leq \sum_{k=1}^n k^3 \leq \frac{(n+1)^4 - 1}{4}$$

Question #5

We use integral approximations for all of the following.

Problem #5.1

$$\begin{aligned} \int_0^n x^r dx &\leq \sum_{k=1}^n k^r \leq \int_1^{n+1} x^r dx \\ \frac{n^{r+1}}{r+1} &\leq \sum_{k=1}^n k^r \leq \frac{(n+1)^{r+1} - 1}{r+1} \end{aligned}$$

Therefore, the bound is $\Theta(n^{r+1})$.

Problem #5.2

For the following, we take s for 1,

$$\begin{aligned} \int_0^n \lg^1 x \, dx &\leq \sum_{k=1}^n \lg^1 k \leq \int_1^{n+1} \lg^1 x \, dx \\ n (\lg(n) - 1) &\leq \sum_{k=1}^n \lg^1 k \leq (n+1) (\lg(n+1) - 1) + 1 \end{aligned}$$

Therefore, the bound is $\Theta(n \lg^1 n)$.

For the following, we take s for 2,

$$\begin{aligned} \int_0^n \lg^2 x \, dx &\leq \sum_{k=1}^n \lg^2 k \leq \int_1^{n+1} \lg^2 x \, dx \\ n ((\lg(n) - 2) \lg(n) + 2) &\leq \sum_{k=1}^n \lg^2 k \leq (n+1) ((\lg(n+1) - 2) \lg(n+1) + 2) - 2 \end{aligned}$$

Therefore, the bound is $\Theta(n \lg^2 n)$.

For the following, we take s for 3,

$$\int_0^n \lg^3 x \, dx \leq \sum_{k=1}^n \lg^3 k \leq \int_1^{n+1} \lg^3 x \, dx$$

From this, we get,

$$\begin{aligned} &n (\lg(n) ((\lg(n) - 3) \lg(n) + 6) - 6) \\ &\leq \sum_{k=1}^n \lg^3 k \\ &\leq (n+1) (\lg(n+1) ((\lg(n+1) - 3) \lg(n+1) + 6) - 6) + 6 \end{aligned}$$

Therefore, the bound is $\Theta(n \lg^3 n)$.

In general, the integral requires s integration by parts, multiplying $\lg n$ s times. So, for $\sum_{k=1}^n \lg^s k$, the bound would be $\Theta(n \lg^s n)$.

Problem #5.3

In general, we know that the lower-bound integral we have will have the form

$$\int_0^n x^r \lg^s x \, dx$$

And likewise for the upper bound. Using the same process as before (omitted due to tedious mathematics), we get the tight bound $\Theta(n^{r+1} \lg^s n)$.

Homework #5

Analysis of Algorithms

Illya Starikov

Due Date: October 4th, 2017

Question #1

Theorem 1. *If $L \geq 2$, then every binary tree with L leaves contains a subtree having between $\frac{L}{3}$ and $\frac{2L}{3}$ leaves, inclusive.*

Proof. Suppose not. That is, suppose that there exists a binary tree with more than 1 leaves with all subtrees containing leaves in the range $\frac{L}{3} > n > \frac{2L}{3}$. The root contains L leaves, and all bottommost leaves contain 1 leaves.

This implies that two subtrees with size less than $\frac{L}{3}$ (because our hypothesis states all leaves must in the range $\frac{L}{3} > n > \frac{2L}{3}$) combined to make a tree of size greater than $\frac{2L}{3}$. This is a contradiction

$$\frac{L}{3} + \frac{L}{3} \not> \frac{2L}{3}$$

Because we have reached a contradiction, our hypothesis does not hold. Therefore, binary tree with L leaves contains a subtree having between $\frac{L}{3}$ and $\frac{2L}{3}$ leaves, inclusive. $\square$

Question #2

Theorem 2. For every binary tree T , let us associate a “weight” $w(q) = 2^{-\text{depth}(q)}$. For all leaves $q \in T$,

$$\sum_q w(q) \leq 1$$

Proof. We prove so by induction.

(Base Case) For the root,

$$\sum_{x \in T} w(x) = 2^{-0} = 1$$

(Inductive Hypothesis) Suppose for any tree of n nodes, $\sum_{x \in T} w(x) \leq 1$.

(Inductive Step) Suppose the binary tree T_0 to have $n - 1$ nodes. We must show that $\sum_{x \in T_0} w(x) \leq 1$. Suppose we take the left subtree T_L or the right subtree T_R . By the inductive hypothesis, the sum of the weights are less than or equal to 1; however, since the depth of a node in T_0 is one greater than the depth of T_L or T_R , the respective weights of every leaf in T_0 is halved. Thus,

$$\sum_{x \in T_L} w(x) \leq 1/2 \quad \sum_{x \in T_R} w(x) \leq 1/2$$

Because both of these are bounded by $1/2$, their sum forms T_0 , which is bounded by 1. By the principle of mathematical induction, $\sum_q w(q) \leq 1$ for any tree. $\square$

Question #3

Theorem 3. *Any planar graph can be colored with six or fewer colors.*

Proof. We prove so by induction. We take n to be an arbitrary number of nodes in an arbitrary graph G .

(Base Case) For n vertices, where $1 \leq n \leq 6$, we can color the graph G with 6 or less colors (for every vertices maps to a single, unique color).

(Inductive Hypothesis) Suppose for $n - 1$ vertices, where $n > 1$, the graph G can be colored with 6 or less colors.

(Inductive Step) We now prove that G can be colored with n vertices can be colored with 6 or fewer colors. Recall all connected, simple planar graph contains a maximum of 5 degrees. Suppose that the any vertex $v \in G$ has this max degree 5.

We remove this vertex, name it v_0 , with all incident edges. We now have less than n vertices, and by our inductive hypothesis, this graph now can be colored with 6 or less colors. Adding this vertex back, we see that there are 5 incident vertexes, hence 5 colors. We use the 6th color for v_0 .

Hence, the inductive step holds. By the principle of mathematical induction, any planar graph can be colored with six or fewer colors. $\square$

Question #4

Theorem 4. *Any point x in a graph is a cut-point iff there exists two vertices in the graph, a and b , such that every path between a and b has to pass through x .*

Proof. Assume G to be a connected graph, and $\exists$ vertices $a, b \in G$, where every path $a \longleftrightarrow b$ passes through x . Also assume there exists no path $a \longleftrightarrow b \in G - \{x\}$. Therefore, $G - \{x\}$ is disconnected, for a and b are in two different components of $G - \{x\}$. This proves that x is a cut-point. $\square$

Question #5

Euler Path A path that uses every edge of a graph exactly once.

Hamiltonian Cycle A path through a graph that starts and ends at the same vertex and includes every other vertex exactly once.

Yes, all **cyclic graphs** meet both these conditions.

Theorem 1. *Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the non-negative numbers by the recurrence*

$$T(n) = aT(n/b) + f(n) \tag{1}$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:

1. *If $f(n) = \mathcal{O}(n^{\epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\epsilon})$.*
2. *If $f(n) = \Theta(n^{\epsilon})$, then $T(n) = \Theta(n^{\epsilon})$.*
3. *If $f(n) = \Omega(n^{\epsilon})$ for some constant $\epsilon > 0$, and if*

$$af(n/b) < cf(n)$$

for some constant $c < 1$ and sufficiently large n , then $T(n) = \Theta(n^{\epsilon})$.

Question #1

Problem #1.1

From Equation 1, we have the following:

$$a = 2 \quad b = 4 \quad f(n) = 1$$

From this,

$$\begin{aligned} f(n) &= \mathcal{O}(n^4) \\ 1 &= \mathcal{O}(n^4) \\ &= \mathcal{O}(n^4) \\ &= \mathcal{O}(n^4) \\ &= \mathcal{O}(n^4) \end{aligned}$$

Therefore, the tight asymptotic bound for the recurrence is

$$\begin{aligned} T(n) &\in \Theta(n^4) \\ &\in \Theta(n^4) \\ &\in \Theta(n^4) \\ &\in \Theta(n^4) \end{aligned}$$

Therefore, the bound is $\Theta(n^4)$.

Problem #1.2

From Equation 1, we have the following:

$$a = 2 \quad b = 4 \quad f(n) = \sqrt{n}$$

From this,

$$\sqrt{n} = \Theta(n^{1/2})$$

Therefore, the tight asymptotic bound for the recurrence is

$$T(n) \in \Theta(n^{1/2})$$

Problem #1.3

From Equation 1, we have the following:

$$a = 2 \quad b = 4 \quad f(n) = n$$

From this,

$$n = \Omega(n \log m_1)$$

Therefore, the tight asymptotic bound for the recurrence is

$$T(n) \in \Theta(n \log m_1)$$

Problem #1.4

From Equation 1, we have the following:

$$a = 2 \quad b = 4 \quad f(n) = n^2$$

From this,

$$n^2 = \Omega(n \log n)$$

Therefore, the tight asymptotic bound for the recurrence is

$$T(n) \in \Theta(n \log n)$$

Question #2

Recall Strassen's algorithm belongs to the complexity class $\Theta(n^{\lg 7})$. By using the Case 1 of the Main Theorem, we must find a that solves the inequality $\log_4 a < \lg 7$. The integer that solves the equation $\log_4 a = \lg 7$ is 49; therefore,

$$a = 48$$

Question #3

Because the process is identical to Problem , the work is omitted; only answers are provided.

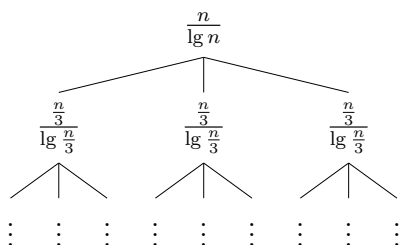
1. Main Theorem (*Case #3*), $T(n) \in \Theta(n \log n)$
2. Main Theorem (*Case #3*), $T(n) \in \Theta(n \log n)$
3. Main Theorem (*Case #2*), $T(n) \in \Theta(n \log n)$
4. Main Theorem (*Case #3*), $T(n) \in \Theta(n \log n)$
5. Main Theorem (*Case #1*), $T(n) \in \Theta(n \log n)$
6. Main Theorem (*Case #2*), $T(n) \in \Theta(n \log n)$
7. Because $(n - 2) \in \mathcal{O}(n \log n)$, we can write an equivalent recurrence relation,

$$T(n) = T(n/2) + n^2$$

Because $\log_2 1$ is undefined, we cannot use the Main Theorem; but we do not need it. We have n^2 work to do n times. This implies that $T(n) \in \Theta(n^3)$.

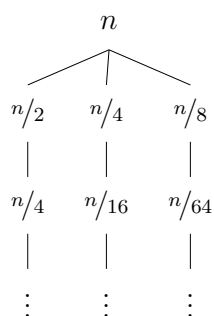
Question #4

1. Main Theorem (*Case #1*), $T(n) \in \Theta(n \lg n)$
2. Using a recursion tree,



We see that the $T(n) \in \Theta(n \lg n)$.

3. Main Theorem (*Case #3*), $T(n) \in \Theta(n \lg n)$.
4. We augment the equation to be $3T(n/3) + n/2$, where we can use Main Theorem (*Case #2*), $T(n) \in \Theta(n \lg n)$.
5. With similar reasoning as in Step #2, we use a recursion tree to get the results $T(n) \in \Theta(n \lg n)$
6. We solve so via the substitution method. We use a recursion tree to get our initial guess.



Guessing $T(n) \in \Theta(n \lg n)$, we get the following:

$$\begin{aligned}
 T(n) &= T(n/2) + T(n/4) + T(n/8) + n \\
 &= c^{n/2} + c^{n/4} + c^{n/8} + n \\
 &= \frac{7}{8}n + n \\
 &\leq cn
 \end{aligned}$$

We see this inequality holds for $c \geq 8$. Therefore, $T(n) \in \Theta(n \lg n)$.

7. Using a recursion tree,

$$\begin{array}{c} \frac{1}{n} \\ | \\ \frac{1}{n-1} \\ | \\ \frac{1}{n-2} \\ | \\ \vdots \end{array}$$

We see that $T(n) \in \Theta(n \log n)$.

8. Using a recursion tree,

$$\begin{array}{c} \lg n \\ | \\ \lg(n-1) \\ | \\ \lg(n-2) \\ | \\ \vdots \end{array}$$

We see that $T(n) \in \Theta(n \log n)$.

9. Using a recursion tree,

$$\begin{array}{c} \frac{1}{\lg n} \\ | \\ \frac{1}{\lg(n-2)} \\ | \\ \frac{1}{\lg(n-4)} \\ | \\ \vdots \end{array}$$

We see that $T(n) \in \Theta(n \log n)$.

10. We guess

$$T(n) \in \Theta(n \log n)$$

$$\begin{aligned}
T(n) &= \sqrt{n} T(\sqrt{n}) + n \\
&= \sqrt{n} T(c\sqrt{n} \lg \lg \sqrt{n}) + n \\
&= c n \lg \lg n - c n + n \\
&\leq c n \lg \lg n
\end{aligned}$$

We see this inequality for $c \in \mathbb{R}^+$. Therefore $T(n) \in \Theta(n \lg \lg n)$.

Question #5

Options c and e .

Question #6

1. We choose a root among the vertices, call it v_0 . If we choose the k th smallest element, the smaller subtree will have $i - 1$ vertices and the larger will have $n - i$ vertices. Summing over all possibilities, we get the following form for b_n :

$$\begin{aligned} b_n &= \sum_{k=1}^n b_{k-1} b_{n-k} \\ &= \sum_{k=0}^{n-1} b_k b_{n-k-1} \end{aligned}$$

- 2.

$$\begin{aligned} B(x) &= \sum_{n=0}^{\infty} b_n x^n \\ &= 1 + \sum_{n=1}^{\infty} b_n x^n \\ &= 1 + \sum_{n=1}^{\infty} \sum_{k=0}^{n-1} b_k b_{n-k-1} x^{n-k-1} \\ &= 1 + x \sum_{n=1}^{\infty} \sum_{k=0}^{n-1} b_k x^k b_{n-k-1} x^{n-k-1} \\ &= 1 + x \sum_{n=0}^{\infty} \sum_{k=0}^n b_k x^k b_{n-k} x^{n-k} \\ &= 1 + x B(x)^2 \end{aligned}$$

3. We use the Taylor expansion of $\sqrt{1-4x}$, we have the following

$$\begin{aligned} B(x) &= \frac{1}{2x} \left(1 - \sum_{n=0}^{\infty} \frac{1}{1-2n} \binom{2n}{n} x^n \right) \\ &= -\frac{1}{2x} \sum_{n=1}^{\infty} \sum_{n=1}^{\infty} \frac{1}{1-2n} \binom{2n}{n} x^n \\ &= \frac{1}{2} \sum_{n=1}^{\infty} \frac{1}{2n-1} \binom{2n}{n} x^{n-1} \\ &= \frac{1}{2} \sum_{n=0}^{\infty} \frac{1}{2n+1} \binom{2n+2}{n+1} x^n \end{aligned}$$

Therefore $b_n = \frac{1}{n+1} \binom{2n}{n}$.

4.

$$\begin{aligned}
b_n &= \frac{1}{n+1} \frac{(2n)!}{n!n!} \\
&\approx \frac{1}{n+1} \frac{\sqrt{4\pi n} (2n/e)^{2n}}{2\pi n (n/e)^{2n}} \\
&= \frac{1}{n+1} \frac{4^n}{\sqrt{\pi n}} \\
&= \left(\frac{1}{n} + \left(\frac{1}{n+1} - \frac{1}{n} \right) \right) \frac{4^n}{\sqrt{\pi n}} \\
&= \left(\frac{1}{n} - \frac{1}{n^2+n} \right) \frac{4^n}{\sqrt{\pi n}} \\
&= \frac{1}{n} \left(1 - \frac{1}{n+1} \right) \frac{4^n}{\sqrt{\pi n}} \\
&= \frac{4^n}{\sqrt{\pi n}^{3/2}} (1 + O(1/n))
\end{aligned}$$

Homework #7

Analysis of Algorithms

Illya Starikov

Due Date: October 20th 2017

```
1 class Node:
2     value = -1
3     left_child, right_child = None, None
4
5     def __init__(self, value):
6         self.value = value
7         self.left_child, self.right_child = None, None
8
9     @property
10    def number_of_children(self):
11        if self.left_child is not None and self.right_child is not
None:
12            return 2
13        elif self.left_child is not None or self.right_child is not
None:
14            return 1
15        else:
16            return 0
17
18    @property
19    def valid_children(self):
20        number_of_children = self.number_of_children
21
22        if number_of_children == 0:
23            return None
24        elif number_of_children == 1:
25            if self.left_child is not None:
26                return self.left_child
27            else:
28                return self.right_child
29        else:
30            return (self.left_child, self.right_child)
31
32    def __str__(self):
33        return str(self.value)
34
```

```

35
36 class BinarySearchTree:
37     root = -1
38
39     def __init__(self):
40         self.root = None
41
42     def insert(self, value):
43         if self.root is None:
44             self.root = Node(value)
45         else:
46             self.__insert_node(self.root, value)
47
48     def exists(self, value):
49         return False if self.__find_node(self.root, value) is None
50     else True
51
52     def delete(self, value):
53         self.root = self.__delete_node(self.root, value)
54
55     def print_tree(self):
56         if not self.root:
57             return
58
59         current_level = [self.root]
60
61         while current_level:
62
63             print('└─'.join(str(node) for node in current_level))
64
65             next_level = list()
66             for n in current_level:
67                 if n.left_child:
68                     next_level.append(n.left_child)
69                 if n.right_child:
70                     next_level.append(n.right_child)
71             current_level = next_level
72
73     # MARK: Private Methods
74     def __insert_node(self, current_node, value):
75         # for the values <= `value`, we put on the left side of the
76         tree
77         if value <= current_node.value:
78             if current_node.left_child is None:
79                 current_node.left_child = Node(value)
80             else:
81                 self.__insert_node(current_node.left_child, value)
82
83         # for values > `value`, we put on the right side of the tree
84         else:
85             if current_node.right_child is None:
86                 current_node.right_child = Node(value)

```

```

85         else:
86             self.__insert_node(current_node.right_child, value)
87
88     def __find_node(self, current_node, value):
89         if current_node is None:
90             return None
91         elif value < current_node.value:
92             return self.__find_node(current_node.left_child, value)
93         elif value > current_node.value:
94             return self.__find_node(current_node.right_child, value)
95         else:
96             return current_node
97
98     def __delete_node(self, current_node, key):
99         if not current_node:
100             return current_node
101
102         if current_node.value > key:
103             current_node.left_child = self.__delete_node(
current_node.left_child, key)
104         elif current_node.value < key:
105             current_node.right_child = self.__delete_node(
current_node.right_child, key)
106         else:
107             if not current_node.left_child:
108                 right_child = current_node.right_child
109                 del current_node
110                 return right_child
111
112             elif not current_node.right_child:
113                 left_child = current_node.left_child
114                 del current_node
115                 return left_child
116
117             else:
118                 successor = current_node.right_child
119                 while successor.left_child:
120                     successor = successor.left_child
121
122                 current_node.value = successor.value
123                 current_node.right_child = self.__delete_node(
current_node.right_child, successor.value)
124
125         return current_node

```

Question #1

With the following elements,

42	2555	4830	7516
104	2718	4883	7604
321	2849	4961	7632
578	2954	5016	7706
600	2974	5119	7754
734	3063	5136	7989
929	3159	5246	8106
1004	3262	5253	8126
1120	3397	5502	8135
1128	3405	5520	8204
1149	3476	5550	8223
1213	3694	5644	8338
1323	3739	5675	8359
1347	3902	6082	8565
1512	4310	6306	8611
1522	4466	6425	8740
1730	4475	6562	8890
1831	4494	6570	8961
1853	4584	6653	9300
1886	4644	6712	9304
1899	4699	6940	9509
2040	4750	6966	9614
2082	4769	7164	9762
2164	4783	7324	9971
2253	4786	7499	9985

Problem #1.1

Determining if elements in the binary search tree are there. All these values should be true.

1	Element	3739	Found:	True
2	Element	5550	Found:	True
3	Element	600	Found:	True
4	Element	3262	Found:	True
5	Element	8611	Found:	True
6	Element	4750	Found:	True
7	Element	5246	Found:	True
8	Element	6712	Found:	True
9	Element	1323	Found:	True
10	Element	1831	Found:	True
11	Element	5520	Found:	True
12	Element	3063	Found:	True

13	Element	9300	Found: True
14	Element	1004	Found: True
15	Element	7164	Found: True
16	Element	4466	Found: True
17	Element	8890	Found: True
18	Element	8338	Found: True
19	Element	1853	Found: True
20	Element	8223	Found: True
21	Element	3159	Found: True
22	Element	4644	Found: True
23	Element	7706	Found: True
24	Element	8740	Found: True
25	Element	1120	Found: True
26	Element	2040	Found: True
27	Element	6653	Found: True
28	Element	578	Found: True
29	Element	9762	Found: True
30	Element	8565	Found: True
31	Element	7604	Found: True
32	Element	9985	Found: True
33	Element	4961	Found: True
34	Element	929	Found: True
35	Element	2082	Found: True
36	Element	5253	Found: True
37	Element	3405	Found: True
38	Element	7516	Found: True
39	Element	6306	Found: True
40	Element	8106	Found: True
41	Element	1347	Found: True
42	Element	4310	Found: True
43	Element	4584	Found: True
44	Element	2954	Found: True
45	Element	9614	Found: True
46	Element	2253	Found: True
47	Element	1522	Found: True
48	Element	8135	Found: True
49	Element	4475	Found: True
50	Element	1512	Found: True
51	Element	9304	Found: True
52	Element	6966	Found: True
53	Element	321	Found: True
54	Element	2974	Found: True
55	Element	6940	Found: True
56	Element	7754	Found: True
57	Element	1213	Found: True
58	Element	1149	Found: True
59	Element	8961	Found: True
60	Element	8359	Found: True
61	Element	4883	Found: True
62	Element	7499	Found: True
63	Element	4786	Found: True
64	Element	8126	Found: True

65	Element	42	Found: True
66	Element	4699	Found: True
67	Element	3902	Found: True
68	Element	1730	Found: True
69	Element	4494	Found: True
70	Element	5502	Found: True
71	Element	104	Found: True
72	Element	4830	Found: True
73	Element	6082	Found: True
74	Element	1899	Found: True
75	Element	5136	Found: True
76	Element	5644	Found: True
77	Element	2849	Found: True
78	Element	5675	Found: True
79	Element	8204	Found: True
80	Element	7632	Found: True
81	Element	1886	Found: True
82	Element	5119	Found: True
83	Element	5016	Found: True
84	Element	6562	Found: True
85	Element	3397	Found: True
86	Element	9971	Found: True
87	Element	3694	Found: True
88	Element	4783	Found: True
89	Element	2718	Found: True
90	Element	734	Found: True
91	Element	7989	Found: True
92	Element	7324	Found: True
93	Element	1128	Found: True
94	Element	6425	Found: True
95	Element	4769	Found: True
96	Element	2555	Found: True
97	Element	3476	Found: True
98	Element	9509	Found: True
99	Element	2164	Found: True
100	Element	6570	Found: True

Problem #1.2

Determining if elements **not** in binary search tree are found. These values should be false.

1	Element	9170	Found:	False
2	Element	1059	Found:	False
3	Element	6807	Found:	False
4	Element	7990	Found:	False
5	Element	9177	Found:	False
6	Element	9933	Found:	False
7	Element	9290	Found:	False
8	Element	5479	Found:	False
9	Element	611	Found:	False
10	Element	5458	Found:	False
11	Element	1563	Found:	False
12	Element	5870	Found:	False
13	Element	3134	Found:	False
14	Element	178	Found:	False
15	Element	3890	Found:	False
16	Element	7225	Found:	False
17	Element	9820	Found:	False
18	Element	2292	Found:	False
19	Element	5313	Found:	False
20	Element	4489	Found:	False
21	Element	3835	Found:	False
22	Element	2543	Found:	False
23	Element	5679	Found:	False
24	Element	46	Found:	False
25	Element	1812	Found:	False
26	Element	4628	Found:	False
27	Element	7310	Found:	False
28	Element	6839	Found:	False
29	Element	5411	Found:	False
30	Element	2929	Found:	False
31	Element	2351	Found:	False
32	Element	9107	Found:	False
33	Element	7498	Found:	False
34	Element	5810	Found:	False
35	Element	6692	Found:	False
36	Element	3986	Found:	False
37	Element	2941	Found:	False
38	Element	9506	Found:	False
39	Element	3485	Found:	False
40	Element	1113	Found:	False
41	Element	4268	Found:	False
42	Element	9848	Found:	False
43	Element	1525	Found:	False
44	Element	7304	Found:	False
45	Element	1792	Found:	False
46	Element	2707	Found:	False
47	Element	4111	Found:	False
48	Element	7696	Found:	False

49	Element	4897	Found: False
50	Element	120	Found: False
51	Element	9588	Found: False
52	Element	819	Found: False
53	Element	5893	Found: False
54	Element	9990	Found: False
55	Element	6444	Found: False
56	Element	2619	Found: False
57	Element	3091	Found: False
58	Element	8431	Found: False
59	Element	7965	Found: False
60	Element	4863	Found: False
61	Element	9778	Found: False
62	Element	9601	Found: False
63	Element	8084	Found: False
64	Element	7459	Found: False
65	Element	7299	Found: False
66	Element	1	Found: False
67	Element	6819	Found: False
68	Element	9085	Found: False
69	Element	5574	Found: False
70	Element	8748	Found: False
71	Element	8790	Found: False
72	Element	6042	Found: False
73	Element	5852	Found: False
74	Element	5341	Found: False
75	Element	7671	Found: False
76	Element	8456	Found: False
77	Element	1818	Found: False
78	Element	6918	Found: False
79	Element	9723	Found: False
80	Element	3980	Found: False
81	Element	6496	Found: False
82	Element	5121	Found: False
83	Element	2553	Found: False
84	Element	7316	Found: False
85	Element	1461	Found: False
86	Element	3896	Found: False
87	Element	9382	Found: False
88	Element	8610	Found: False
89	Element	5181	Found: False
90	Element	3305	Found: False
91	Element	549	Found: False
92	Element	7306	Found: False
93	Element	4612	Found: False
94	Element	9525	Found: False
95	Element	4847	Found: False
96	Element	6420	Found: False
97	Element	2643	Found: False
98	Element	4538	Found: False
99	Element	5934	Found: False
100	Element	7969	Found: False

Problem #1.3

Deleting entire tree. This output is ≈ 2000 , so please refer to output for entire tree.

```
1 Current Tree Before Deleting Element 4769
2 3739
3 2974 7632
4 1149 3405 4783 9509
5 734 1899 3063 3694 4475 6562 8890 9985
6 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
7 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
   7604 8565 8961 9614
8 321 929 1213 1853 2040 4466 4644 4769 4830 5520 6082 6653 7516 8126
   8611 9300 9762
9 578 1323 1831 1886 2718 4699 4786 4883 5502 5550 6306 6570 7164 7989
   8223
10 600 1730 2253 4961 5253 6425 6940 7706 8106 8135 8338
11 2164 2555 5136 6712 6966 7754 8204 8359
12 2082 5119 5246
13
14 Current Tree After Deleting Element 4769
15 3739
16 2974 7632
17 1149 3405 4783 9509
18 734 1899 3063 3694 4475 6562 8890 9985
19 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
20 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
   7604 8565 8961 9614
21 321 929 1213 1853 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
   9300 9762
22 578 1323 1831 1886 2718 4699 4786 4883 5502 5550 6306 6570 7164 7989
   8223
23 600 1730 2253 4961 5253 6425 6940 7706 8106 8135 8338
24 2164 2555 5136 6712 6966 7754 8204 8359
25 2082 5119 5246
26 -----
27 Current Tree Before Deleting Element 1853
28 3739
29 2974 7632
30 1149 3405 4783 9509
31 734 1899 3063 3694 4475 6562 8890 9985
32 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
33 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
   7604 8565 8961 9614
34 321 929 1213 1853 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
   9300 9762
35 578 1323 1831 1886 2718 4699 4786 4883 5502 5550 6306 6570 7164 7989
   8223
36 600 1730 2253 4961 5253 6425 6940 7706 8106 8135 8338
37 2164 2555 5136 6712 6966 7754 8204 8359
38 2082 5119 5246
39
```

```

40 Current Tree After Deleting Element 1853
41 3739
42 2974 7632
43 1149 3405 4783 9509
44 734 1899 3063 3694 4475 6562 8890 9985
45 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
46 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
47 321 929 1213 1886 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
48 578 1323 1831 2718 4699 4786 4883 5502 5550 6306 6570 7164 7989 8223
49 600 1730 2253 4961 5253 6425 6940 7706 8106 8135 8338
50 2164 2555 5136 6712 6966 7754 8204 8359
51 2082 5119 5246
52 -----

53 Current Tree Before Deleting Element 4699
54 3739
55 2974 7632
56 1149 3405 4783 9509
57 734 1899 3063 3694 4475 6562 8890 9985
58 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
59 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
60 321 929 1213 1886 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
61 578 1323 1831 2718 4699 4786 4883 5502 5550 6306 6570 7164 7989 8223
62 600 1730 2253 4961 5253 6425 6940 7706 8106 8135 8338
63 2164 2555 5136 6712 6966 7754 8204 8359
64 2082 5119 5246
65
66 Current Tree After Deleting Element 4699
67 3739
68 2974 7632
69 1149 3405 4783 9509
70 734 1899 3063 3694 4475 6562 8890 9985
71 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
72 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
73 321 929 1213 1886 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
74 578 1323 1831 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223
75 600 1730 2253 4961 5253 6425 6940 7706 8106 8135 8338
76 2164 2555 5136 6712 6966 7754 8204 8359
77 2082 5119 5246
78 -----

79 Current Tree Before Deleting Element 2082
80 3739
81 2974 7632
82 1149 3405 4783 9509
83 734 1899 3063 3694 4475 6562 8890 9985

```

```

84 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
85 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
86 321 929 1213 1886 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
87 578 1323 1831 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223
88 600 1730 2253 4961 5253 6425 6940 7706 8106 8135 8338
89 2164 2555 5136 6712 6966 7754 8204 8359
90 2082 5119 5246

```

```

91
92 Current Tree After Deleting Element 2082
93 3739

```

```

94 2974 7632
95 1149 3405 4783 9509
96 734 1899 3063 3694 4475 6562 8890 9985
97 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
98 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
99 321 929 1213 1886 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
100 578 1323 1831 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223
101 600 1730 2253 4961 5253 6425 6940 7706 8106 8135 8338
102 2164 2555 5136 6712 6966 7754 8204 8359
103 5119 5246

```

```

104 -----

```

```

105 Current Tree Before Deleting Element 4961
106 3739
107 2974 7632
108 1149 3405 4783 9509
109 734 1899 3063 3694 4475 6562 8890 9985
110 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
111 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
112 321 929 1213 1886 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
113 578 1323 1831 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223
114 600 1730 2253 4961 5253 6425 6940 7706 8106 8135 8338
115 2164 2555 5136 6712 6966 7754 8204 8359
116 5119 5246

```

```

117
118 Current Tree After Deleting Element 4961
119 3739
120 2974 7632
121 1149 3405 4783 9509
122 734 1899 3063 3694 4475 6562 8890 9985
123 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
124 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
125 321 929 1213 1886 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
126 578 1323 1831 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223

```

127 600 1730 2253 5253 6425 6940 7706 8106 8135 8338
128 2164 2555 5136 6712 6966 7754 8204 8359
129 5119 5246
130 -----

131 Current Tree Before Deleting Element 1512
132 3739
133 2974 7632
134 1149 3405 4783 9509
135 734 1899 3063 3694 4475 6562 8890 9985
136 42 1120 1512 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
137 104 1004 1128 1347 1522 2849 3159 3397 4310 4494 4750 5016 5675 7324
7604 8565 8961 9614
138 321 929 1213 1886 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
9300 9762
139 578 1323 1831 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223
140 600 1730 2253 5253 6425 6940 7706 8106 8135 8338
141 2164 2555 5136 6712 6966 7754 8204 8359
142 5119 5246
143

144 Current Tree After Deleting Element 1512
145 3739
146 2974 7632
147 1149 3405 4783 9509
148 734 1899 3063 3694 4475 6562 8890 9985
149 42 1120 1522 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
150 104 1004 1128 1347 1886 2849 3159 3397 4310 4494 4750 5016 5675 7324
7604 8565 8961 9614
151 321 929 1213 1831 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
9300 9762
152 578 1323 1730 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223
153 600 2253 5253 6425 6940 7706 8106 8135 8338
154 2164 2555 5136 6712 6966 7754 8204 8359
155 5119 5246
156 -----

157 Current Tree Before Deleting Element 1730
158 3739
159 2974 7632
160 1149 3405 4783 9509
161 734 1899 3063 3694 4475 6562 8890 9985
162 42 1120 1522 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
163 104 1004 1128 1347 1886 2849 3159 3397 4310 4494 4750 5016 5675 7324
7604 8565 8961 9614
164 321 929 1213 1831 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
9300 9762
165 578 1323 1730 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223
166 600 2253 5253 6425 6940 7706 8106 8135 8338
167 2164 2555 5136 6712 6966 7754 8204 8359
168 5119 5246
169
170 Current Tree After Deleting Element 1730

```

171 3739
172 2974 7632
173 1149 3405 4783 9509
174 734 1899 3063 3694 4475 6562 8890 9985
175 42 1120 1522 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
176 104 1004 1128 1347 1886 2849 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
177 321 929 1213 1831 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
178 578 1323 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223
179 600 2253 5253 6425 6940 7706 8106 8135 8338
180 2164 2555 5136 6712 6966 7754 8204 8359
181 5119 5246
182 -----

183 Current Tree Before Deleting Element 2849
184 3739
185 2974 7632
186 1149 3405 4783 9509
187 734 1899 3063 3694 4475 6562 8890 9985
188 42 1120 1522 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
189 104 1004 1128 1347 1886 2849 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
190 321 929 1213 1831 2040 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
191 578 1323 2718 4786 4883 5502 5550 6306 6570 7164 7989 8223
192 600 2253 5253 6425 6940 7706 8106 8135 8338
193 2164 2555 5136 6712 6966 7754 8204 8359
194 5119 5246
195
196 Current Tree After Deleting Element 2849
197 3739
198 2974 7632
199 1149 3405 4783 9509
200 734 1899 3063 3694 4475 6562 8890 9985
201 42 1120 1522 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
202 104 1004 1128 1347 1886 2040 3159 3397 4310 4494 4750 5016 5675 7324
    7604 8565 8961 9614
203 321 929 1213 1831 2718 4466 4644 4830 5520 6082 6653 7516 8126 8611
    9300 9762
204 578 1323 2253 4786 4883 5502 5550 6306 6570 7164 7989 8223
205 600 2164 2555 5253 6425 6940 7706 8106 8135 8338
206 5136 6712 6966 7754 8204 8359
207 5119 5246
208 -----

209 Current Tree Before Deleting Element 5520
210 3739
211 2974 7632
212 1149 3405 4783 9509
213 734 1899 3063 3694 4475 6562 8890 9985
214 42 1120 1522 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971

```

```

215 104 1004 1128 1347 1886 2040 3159 3397 4310 4494 4750 5016 5675 7324
      7604 8565 8961 9614
216 321 929 1213 1831 2718 4466 4644 4830 5520 6082 6653 7516 8126 8611
      9300 9762
217 578 1323 2253 4786 4883 5502 5550 6306 6570 7164 7989 8223
218 600 2164 2555 5253 6425 6940 7706 8106 8135 8338
219 5136 6712 6966 7754 8204 8359
220 5119 5246
221
222 Current Tree After Deleting Element 5520
223 3739
224 2974 7632
225 1149 3405 4783 9509
226 734 1899 3063 3694 4475 6562 8890 9985
227 42 1120 1522 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
228 104 1004 1128 1347 1886 2040 3159 3397 4310 4494 4750 5016 5675 7324
      7604 8565 8961 9614
229 321 929 1213 1831 2718 4466 4644 4830 5550 6082 6653 7516 8126 8611
      9300 9762
230 578 1323 2253 4786 4883 5502 6306 6570 7164 7989 8223
231 600 2164 2555 5253 6425 6940 7706 8106 8135 8338
232 5136 6712 6966 7754 8204 8359
233 5119 5246
234 -----

235 Current Tree Before Deleting Element 8890
236 3739
237 2974 7632
238 1149 3405 4783 9509
239 734 1899 3063 3694 4475 6562 8890 9985
240 42 1120 1522 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
241 104 1004 1128 1347 1886 2040 3159 3397 4310 4494 4750 5016 5675 7324
      7604 8565 8961 9614
242 321 929 1213 1831 2718 4466 4644 4830 5550 6082 6653 7516 8126 8611
      9300 9762
243 578 1323 2253 4786 4883 5502 6306 6570 7164 7989 8223
244 600 2164 2555 5253 6425 6940 7706 8106 8135 8338
245 5136 6712 6966 7754 8204 8359
246 5119 5246
247
248 Current Tree After Deleting Element 8890
249 3739
250 2974 7632
251 1149 3405 4783 9509
252 734 1899 3063 3694 4475 6562 8961 9985
253 42 1120 1522 2954 3262 3476 3902 4584 5644 7499 8740 9304 9971
254 104 1004 1128 1347 1886 2040 3159 3397 4310 4494 4750 5016 5675 7324
      7604 8565 9300 9614
255 321 929 1213 1831 2718 4466 4644 4830 5550 6082 6653 7516 8126 8611
      9762
256 578 1323 2253 4786 4883 5502 6306 6570 7164 7989 8223
257 600 2164 2555 5253 6425 6940 7706 8106 8135 8338

```

258	5136	6712	6966	7754	8204	8359
259	5119	5246				

Question #2

```
1 class Graph:
2     adjacency_list = {}
3     is_directed = False
4
5     def __init__(self, is_directed=False):
6         self.adjacency_list = {}
7         self.is_directed = is_directed
8
9     def add_edge(self, start, destination):
10        self.adjacency_list[start].add(destination)
11
12        if self.is_directed:
13            self.adjacency_list[destination].add(start)
14
15    def add_node(self, node):
16        if node not in self.adjacency_list:
17            self.adjacency_list[node] = set()
18
19    def print_graph(self):
20        max_row, max_column = self.max_dimensions
21        list_version = [[0 for i in range(max_column)] for j in
22range(max_row)]
23
24        for source, destination_set in self.adjacency_list.items():
25            for destination in destination_set:
26                list_version[source][destination] = 1
27
28        print(matrix(list_version))
29
30    @property
31    def max_dimensions(self):
32        max_element = max(self.adjacency_list.keys())
33
34        for _, value in self.adjacency_list.items():
35            value = list(value)
36
37            if value:
38                if max(value) > max_element:
39                    max_element = max(value)
40
41        return max_element + 1, max_element + 1
42
43    def depth_first_search(self, start_node, preserve_order=True):
44        if preserve_order:
45            colors = {}
46            for vertex in self.adjacency_list:
47                colors[vertex] = "white"
48
49        return self.__depth_first_search_preserve_order(
50start_node, colors, None)
```



```

49         else:
50             return self.__depth_first_search(start_node, None)
51
52     def breadth_first_search(self, start_node, preserve_order=True):
53         if preserve_order:
54             return self.__breadth_first_search_preserve_order(
55 start_node)
56         else:
57             return self.__breadth_first_search(start_node)
58
59     def __depth_first_search(self, start_node, visited_nodes):
60         if visited_nodes is None:
61             visited_nodes = set()
62
63         visited_nodes.add(start_node)
64         for unvisited_node in self.adjacency_list[start_node] -
65 visited_nodes:
66             self.__depth_first_search(unvisited_node, visited_nodes)
67
68         return visited_nodes
69
70     def __depth_first_search_preserve_order(self, start_node, colors
71 , visited_nodes):
72         if visited_nodes is None:
73             visited_nodes = []
74
75         if colors[start_node] is "white":
76             colors[start_node] = "grey"
77             visited_nodes += [start_node]
78
79         for unvisited_node in [x for x in self.adjacency_list[
80 start_node] if x not in visited_nodes]:
81             if colors[unvisited_node] is "white":
82                 self.__depth_first_search_preserve_order(
83 unvisited_node, colors, visited_nodes)
84
85         colors[start_node] = "black"
86
87         return visited_nodes
88
89     def __breadth_first_search(self, start):
90         visited, queue = set(), [start]
91
92         while queue:
93             vertex = queue.pop(0)
94
95             if vertex not in visited:
96                 visited.add(vertex)
97                 queue.extend(self.adjacency_list[vertex] - visited)
98
99         return visited

```

```

96     def __breadth_first_search_preserve_order(self, start):
97         colors = {}
98
99         for vertex in self.adjacency_list:
100             colors[vertex] = "white"
101
102         queue = [start]
103         visited_nodes = [start]
104
105         while queue:
106             node = queue.pop(0)
107
108             for unvisited_node in list(self.adjacency_list[node]):
109                 if colors[unvisited_node] is "white":
110                     colors[unvisited_node] = "grey"
111                     queue.insert(0, unvisited_node)
112                     visited_nodes += [unvisited_node]
113
114             colors[node] = "black"
115
116         return visited_nodes

```

We test examples with the following adjacency list.

$$\begin{bmatrix}
 0 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 1 \\
 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \\
 0 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\
 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\
 1 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 \\
 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0
 \end{bmatrix}$$

Problem #2.1

For Depth-First Search, we get the following ordering (with the root taken at random).

[illegible]

Problem #2.2

For Breadth-First Search, we get the following ordering (with the root taken at random).

3 → 4 → 6 → 0 → 8 → 5 → 7 → 9 → 1 → 2
0 → 9 → 3 → 4 → 1 → 2 → 8 → 5 → 6 → 7
8 → 0 → 3 → 5 → 6 → 7 → 9 → 1 → 4 → 2
5 → 4 → 2 → 6 → 0 → 8 → 3 → 7 → 9 → 1
0 → 9 → 3 → 4 → 1 → 2 → 8 → 5 → 6 → 7
6 → 0 → 8 → 3 → 5 → 7 → 9 → 1 → 4 → 2
3 → 4 → 6 → 0 → 8 → 5 → 7 → 9 → 1 → 2
5 → 4 → 2 → 6 → 0 → 8 → 3 → 7 → 9 → 1
7 → 8 → 0 → 3 → 5 → 6 → 9 → 1 → 4 → 2
9 → 1 → 4 → 6 → 0 → 8 → 3 → 5 → 7 → 2
3 → 4 → 6 → 0 → 8 → 5 → 7 → 9 → 1 → 2
8 → 0 → 3 → 5 → 6 → 7 → 9 → 1 → 4 → 2
5 → 4 → 2 → 6 → 0 → 8 → 3 → 7 → 9 → 1
0 → 9 → 3 → 4 → 1 → 2 → 8 → 5 → 6 → 7
2 → 8 → 0 → 3 → 5 → 6 → 7 → 9 → 1 → 4

Question #3

Our graph is exactly the same as the previous version; however, this one has the boolean property (`is_directed` set to false). We test examples with the following adjacency list.

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Problem #3.1

For Depth-First Search, we get the following ordering (with the root taken at random).

3 → 9 → 0 → 8 → 1 → 4 → 2 → 5 → 7 → 6
9 → 0 → 8 → 1 → 3 → 4 → 2 → 5 → 7 → 6
2 → 0 → 8 → 1 → 3 → 9 → 4 → 5 → 6 → 7
4 → 9 → 0 → 8 → 1 → 3 → 6 → 7 → 2 → 5
9 → 0 → 8 → 1 → 3 → 4 → 2 → 5 → 7 → 6
0 → 8 → 1 → 3 → 9 → 4 → 2 → 5 → 7 → 6
4 → 9 → 0 → 8 → 1 → 3 → 6 → 7 → 2 → 5
2 → 0 → 8 → 1 → 3 → 9 → 4 → 5 → 6 → 7
4 → 9 → 0 → 8 → 1 → 3 → 6 → 7 → 2 → 5
1 → 8 → 0 → 9 → 3 → 4 → 2 → 5 → 7 → 6
0 → 8 → 1 → 3 → 9 → 4 → 2 → 5 → 7 → 6
7 → 8 → 0 → 9 → 3 → 1 → 4 → 2 → 5 → 6
7 → 8 → 0 → 9 → 3 → 1 → 4 → 2 → 5 → 6
5 → 0 → 8 → 1 → 3 → 9 → 4 → 2 → 7 → 6
4 → 9 → 0 → 8 → 1 → 3 → 6 → 7 → 2 → 5

Problem #3.2

For Breadth-First Search, we get the following ordering (with the root taken at random).

3 → 9 → 1 → 4 → 6 → 7 → 8 → 2 → 0 → 5
9 → 0 → 3 → 4 → 2 → 5 → 1 → 8 → 7 → 6
2 → 0 → 8 → 4 → 5 → 7 → 1 → 3 → 9 → 6
4 → 9 → 2 → 3 → 5 → 1 → 8 → 7 → 0 → 6
9 → 0 → 3 → 4 → 2 → 5 → 1 → 8 → 7 → 6
0 → 8 → 9 → 2 → 5 → 4 → 3 → 1 → 7 → 6
4 → 9 → 2 → 3 → 5 → 1 → 8 → 7 → 0 → 6
2 → 0 → 8 → 4 → 5 → 7 → 1 → 3 → 9 → 6
4 → 9 → 2 → 3 → 5 → 1 → 8 → 7 → 0 → 6
1 → 8 → 3 → 4 → 7 → 2 → 0 → 5 → 9 → 6
0 → 8 → 9 → 2 → 5 → 4 → 3 → 1 → 7 → 6
7 → 8 → 1 → 2 → 3 → 9 → 4 → 6 → 5 → 0
7 → 8 → 1 → 2 → 3 → 9 → 4 → 6 → 5 → 0
5 → 0 → 2 → 4 → 9 → 3 → 1 → 8 → 7 → 6
4 → 9 → 2 → 3 → 5 → 1 → 8 → 7 → 0 → 6

Question #4

```
1 class ParentheticalOrder(Graph):
2     def print_parenthetical_order(self, start_node):
3         colors = {}
4         for vertex in self.adjacency_list:
5             colors[vertex] = "white"
6
7         for node in sorted(self.adjacency_list):
8             if colors[node] is "white":
9                 self.__print_parenthetical_order(node, colors)
10
11     def __print_parenthetical_order(self, node, colors):
12         colors[node] = "grey"
13         print("{}␣".format(node), end='')
14         for neighbor in sorted(self.adjacency_list[node]):
15             if colors[neighbor] is "white":
16                 self.__print_parenthetical_order(neighbor, colors)
17         colors[node] = 'black'
18         print("␣{}>".format(node), end='')
```

We get the following string:

(u (v (y (x x) y) v) u) (w (z z) w)

Question #5

Question #6

```

1 class WrestleMania(Graph):
2     def determine_valid_rivalry(self):
3         rivalry, not_visited = {}, list(self.adjacency_list.keys())
4
5         for vertex in self.adjacency_list:
6             rivalry[vertex] = "none"
7
8         while "none" in rivalry.values():
9             current_depth, start = 0, not_visited[-1]
10            queue = [start]
11
12            while queue:
13                current_depth += 1
14                node = queue.pop(0)
15
16                for unvisited_node in list(self.adjacency_list[node
17            ]):
18                if rivalry[unvisited_node] is "none":
19                    if current_depth % 2 == 0:
20                        rivalry[unvisited_node] = "good_guy"
21                    else:
22                        rivalry[unvisited_node] = "bad_guy"
23
24                not_visited.remove(unvisited_node)
25                queue.insert(0, unvisited_node)
26
27            for wrestler, adjacency_wrestlers in self.adjacency_list.
28            items():
29                for adjacent_wrestler in adjacency_wrestlers:
30                    if rivalry[wrestler] == rivalry[adjacent_wrestler]:
31                        return False
32
33            return True

```

For the following sample inputs

$$X_1 = \begin{bmatrix} & x & y & z \\ x & 1 & 0 & 1 \\ y & 0 & 0 & 1 \\ z & 1 & 1 & 0 \end{bmatrix} \quad X_2 = \begin{bmatrix} & w & x & y & z \\ w & 0 & 1 & 0 & 1 \\ x & 1 & 0 & 1 & 0 \\ y & 0 & 1 & 0 & 1 \\ z & 1 & 0 & 1 & 0 \end{bmatrix} \quad X_3 = \begin{bmatrix} & u & v & w & x & y & z \\ u & 0 & 1 & 0 & 0 & 0 & 0 \\ v & 1 & 0 & 0 & 0 & 0 & 0 \\ w & 0 & 0 & 0 & 1 & 0 & 0 \\ x & 0 & 0 & 1 & 0 & 0 & 0 \\ y & 0 & 0 & 0 & 0 & 0 & 1 \\ z & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

```

1 No configuration could be found.
2 =====
3 Wrestler y Is A Baby Face
4 Wrestler x Is A Heel

```

5 Wrestler z Is A Heel
6 Wrestler w Is A Baby Face
7 =====
8 Wrestler u Is A Heel
9 Wrestler w Is A Heel
10 Wrestler v Is A Baby Face
11 Wrestler y Is A Heel
12 Wrestler x Is A Baby Face
13 Wrestler z Is A Baby Face

Question #7

Problem #7.1

For this problem we have three cases.

No Children If the root has no children it has no edges so it cannot disconnect the graph.

One Child The only edge it breaks is paths to itself so it cannot disconnect graph.

≥ 2 *Children* Because there are no cross edges between the children, and because the children nodes are in the subtrees of the root, no path exists between them. Deleting the root will delete the link between the two or more children; thus, disconnecting the graph and making the root G_π an articulation point.

Problem #7.2

Suppose there is a vertex $v \in G_\pi$, where v is a non-root. Furthermore, suppose v has a child s such that there is no back edge from or any descendent of s to ancestor of v . Thus, the removal of v will lead the disconnection of the sub-tree rooted at v from the graph G_π .

Therefore, the absence of an edge between the descendants of s and ancestors of v resulted in the disconnection of the graph after removal of non-root vertex v . This implies v is an articulation point.

Problem #7.3

Because v is discovered before all of its descendants, the only edges that could affect the minimum are ancestors of v . Thus, we can do an augmented depth-first search to determine all of the low values.

```
1 class ArticulateGraph(Graph):
2     time = 0
3     times, colors, lows = {}, {}, {}
4
5     def find_minimum(self, start_node, visited_nodes=None):
6         if visited_nodes is None:
7             self.times, self.colors, self.lows = {}, {}, {}
8             self.time = 0
9
10            for vertex in self.adjacency_list:
11                self.colors[vertex] = "white"
12
13            visited_nodes = []
14
15            for unvisited_node in [x for x in self.adjacency_list[
16start_node] if x not in visited_nodes]:
17                if self.colors[unvisited_node] is "white":
18                    self.colors[start_node] = "grey"
19                    self.time += 1
20                    visited_nodes += [start_node]
21
22                    self.times[start_node] = self.time
23                    self.lows[start_node] = self.times[start_node]
24
25                    for adjacent_node in self.adjacency_list[start_node
26]:
27                        self.find_minimum(adjacent_node, visited_nodes)
28
29                        if self.colors[adjacent_node] is "white":
30                            if self.time[start_node] < self.lows[self.
31adjacency_list]:
32                                self.lows[start_node] = self.times[
33adjacent_node]
34
35                    self.colors[start_node] = "black"
36                    self.time += 1
37
38            return self.lows
```

With the following graph,

$$\begin{bmatrix} 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

We get the following:

$$8 \rightarrow 3$$

$$2 \rightarrow 2$$

$$3 \rightarrow 1$$

$$4 \rightarrow 4$$

$$7 \rightarrow 9$$

Problem #7.4

After applying the algorithm mentioned above for all $v \in V$, we check to see if $v.low = v.d$. If it is, no descendants of v has a back edge to a proper ancestor of v , implying v is not a articulation point.

Problem #7.5

If there is a cycle from $u \rightarrow v \rightarrow w$, where u and v have unique partitions, then removing any edge of w does not disconnect the graph. Therefore, the edges from w to u and v cannot be bridges.

Problem #7.6

A simple circuit contains an edge (u, v) if and only if

- Both of its endpoints are articulation points.
- One of its endpoints is an articulation point and the other is a vertex of degree 1.

Using our previous algorithm above, we can compute this by running `find_minimum(self, v)`, and deciding if one of the two criteria are met.

Problem #7.7

Because we have already stated that any two edges of any bi-connected components lie on a common cycle, we know all edges are lying inside a component of a graph will not be bridges (via Part E). Therefore, all bi-connected components of a graph partition the non-bridges of said graph.

Problem #7.8

Locate all bridge edges using the algorithm described in Part F. Remove each bridge, the connected components are all biconnected components.

Homework #8

Analysis of Algorithms

Illya Starikov

Due Date: November 27th, 2017

Contents

Listings	1
Question #1	2
Question #2	3
Question #3	4
Question #4	5
Question #5	6
Question #6	7
Question #7	8
Question #8	9
Question #9	13

Listings

Question #1

Let the samples space $S = \{X_{\text{free}} \implies \text{prisoner } X \text{ is going free, } Y_{\text{free}} \implies \text{prisoner } Y \text{ is going free, } Z_{\text{free}} \implies \text{prisoner } Z \text{ is going free, } \}$. Furthermore, let y be event that Y is to be executed. Using Bayes Theorem, we get the following:

$$\begin{aligned}\Pr(X_{\text{free}}|y) &= \frac{\Pr(y|X_{\text{free}}) \times \Pr(X_{\text{free}})}{\Pr(y \cap X_{\text{free}}) + \Pr(y \cap Y_{\text{free}}) + \Pr(y \cap Z_{\text{free}})} \\ &= \frac{\frac{1}{3} \times \frac{1}{2}}{\frac{1}{3} \times \frac{1}{2} + 0 + \frac{1}{3} \times 1} \\ &= \frac{1}{3}\end{aligned}$$

We see that the prisoner X still has a probability of $\frac{1}{3}$.

Question #2

$$\text{minimum} = 2^h \quad \text{maximum} = 2^{h+1} - 1$$

Question #3

A simple way to achieve an $\mathcal{O}(nm_1)$ algorithm.

1. Sort all of the arrays in ascending order.
2. Create a minimum heap with all of the minimum elements in the k individual lists. Remove these elements from the respective arrays.
3. Remove the minimum element (name it $d_{\min}$) from the heap mentioned in Step 2, and replace it with the minimum from its respective list. Add $d_{\min}$ to the solution.
4. While the heap is not empty, repeat Step 3.

Question #4

Recall [Stirling's Approximation](#) for factorials:

$$n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \quad (1)$$

We skip the $\frac{1}{2}$ and $\frac{1}{n}$ case; for if we can't prove $\frac{1}{2^n} \times n! \in \Omega(n \log n)$, then neither are $\frac{1}{2}$ and $\frac{1}{n}$.

Theorem 1.

$$\forall n \in \mathbb{R}^+, \frac{1}{2^n} \times n! \notin \Omega(n \log n)$$

Proof. Suppose not. That is, suppose $\exists c \in \mathbb{R}, \frac{1}{2^n} \times n! \leq c n \log n$. Therefore, $\frac{1}{2^n} \times n!$ must be in the same growth class as $n \log n$. By definition, the limit of the two functions must be convergent to some number M .

Because we cannot directly take the limit, we use Equation 1. From this we get the result

$$\lim_{n \rightarrow \infty} \frac{\frac{1}{2^n} \times n!}{c n \log n} = \frac{\frac{1}{2^n} \times \sqrt{2\pi n} \left(\frac{n}{e}\right)^n}{c n \log n} = \infty \quad \forall c \in \mathbb{R}^+$$

This has led us to a contradiction. □

Question #5

Proof. Suppose we have n integers of length L (we can make this assumption because we can always pad the leading digits with 0s).

Base Case Solving with $L = 1$ is trivial. Because it assumed our sorting algorithm is correct, this will simply sort the digits.

Inductive Hypothesis Suppose that it is true for $n - 1$; we will show it to be true for n .

When comparing the n th digit, call it d_1 and d_2 from the two respective lists, it appears something like such:

...	...	d_1	...	...
...	...	d_2	...	...

From this, there are three cases:

$d_1 < d_2$ Then d_1 is placed before d_2 .

$d_1 > d_2$ Then d_1 is placed after d_1 .

$d_1 = d_2$ Then the numbers go unchanged. **Because our sorting algorithm is stable**, the order is preserved from the lower digits. This is shown below.

...	...	42	64	...
...	...	42	16	...

□

Question #6

We can accomplish $n + \lg n - 2$ runtime as such:

1. Split all elements into pairwise elements ($S \Rightarrow (s_1, s_2), (s_3, s_4), \dots, (s_{n-1}, s_n)$).
2. Compare all of the elements with respect to the smallest element in the pair. This will get the smallest. This requires n iterations.
3. The second smallest value must have been an element that lost to the original minimum, so we cache all of the lost contenders. We then run the previous step on this to get the second minimum. This requires at most $\lg n - 1$ iterations.

We first the following sample input $n = 10$:

342	133	40	365	796	716	354	38	256	614
-----	-----	----	-----	-----	-----	-----	----	-----	-----

We get the following output:

$$\begin{aligned}\text{Number of Comparison} &\in \Omega(n \lg n) \\ &\leq 10 + \lg 10 - 2 \\ &\leq 12 \\ &= 7\end{aligned}$$

Question #7

Theorem 2. *The Set-Partition Problem is NP-Complete.*

Proof. It is trivial to prove that The Set-Partition problem is NP. Given two candidate solutions A and $\bar{A}$, verify that following things:

$$\sum_{x \in A} x - \sum_{x \in \bar{A}} x = 0 \quad \text{and} \quad A \cup \bar{A} = S$$

Recall The Subset Sum problem as follows:

Given a set of natural numbers S and a natural number t , is there a subset of S that sums to t

We will prove that The Set-Partition problem reduces down to The Subset Sum problem.

Suppose we have $p, t \in \mathbb{R}$ and a set P , where t is a particular number and p is $p = \sum_{x \in P} x$. We give input to Set-Partition $P^* = P \cup s - 2t$. From this, we have two cases.

1. If there exists $t \in P$, then remaining numbers in P sum to $s - t$. This satisfies the Set-Partition problem.
2. There exists a partition of X^* into two sets Q, Q^* such that the $\sum_{x \in Q} = \sum_{x \in Q^*} = s - t$. This implies $s - 2t \in Q^* \cup Q$. Removing this number, one set sums to t , and we can use the previous case.

Because we have proved The Set-Partition Problem is NP and it reduces to an NP-Complete problem, we conclude that the Set-Partition problem is NP Complete. $\square$

Question #8

Problem #8.1

There exists an algorithm. Because there are only two denominations, we can enumerate all possible solutions. By producing pairs of all values in amount of x and the amount of y , we get a solution that's $\mathcal{O}(nm_1) = \mathcal{O}(nm_1)$.

Problem #8.2

There exists an algorithm. And it works like such:

1. Sort the coins in ascending order.
2. Pop off the largest coin, give it to Bonnie.
3. Keep taking smallest coins and assign them to Clyde. If finished, and still have coins to assign, go to Step 2.
4. If at any there is not enough coins to match Bonnie when assigning to Clyde in Step 3, there exists no such configuration.

Problem #8.3

There exists no polynomial algorithm.

Proof. Note to Reader: We shall call this the Bonnie-Clyde Check problem, and the solution set of checks S_{Bonnie} for Bonnie and S_{Clyde} for Clyde.

It is trivial to prove that this Bonnie-Clyde check problem is NP. Given a solution S_{Bonnie} and S_{Clyde} , we must verify

$$\sum_{x \in S_{\text{Bonnie}}} x\text{'s worth} = \sum_{x \in S_{\text{Clyde}}} x\text{'s worth} \quad |S_{\text{Bonnie}}| + |S_{\text{Clyde}}| = n$$

where $|A|$ is the cardinality of A ¹. This is clearly a polynomial algorithm.

We will now show that the Bonnie-Clyde Check problem reduces down to the Partition Problem.

Assign every checks value as it's representation in the input set S . Feed S into The Partition problem. All output from The Partition problem is also valid output for the Bonnie Clyde problem.

Because we have proved that the Bonnie-Clyde problem is NP and reduces to an NP Complete problem, we conclude that the Bonnie-Clyde problem is NP complete. $\square$

¹Cardinality meaning the amount of elements in the set

Problem #8.4

The problem is also NP Complete.

Proof. Note to Reader: We shall call this the Bonnie-Clyde 100 Check problem, and the solution set of checks S_{Bonnie} for Bonnie and S_{Clyde} for Clyde.

If the minimum check value is greater than 100\$, the proof is identical to the previous.

If the minimum check is less than 100\$, first we must prove it is in NP. Given a solution S_{Bonnie} and S_{Clyde} , we must verify

$$\left| \sum_{x \in S_{\text{Bonnie}}} x\text{'s worth} - \sum_{x \in S_{\text{Clyde}}} x\text{'s worth} \right| = 100 \quad |S_{\text{Bonnie}}| + |S_{\text{Clyde}}| = n$$

where $|A|$ is the cardinality of A ². This is clearly a polynomial algorithm.

We will now show that the Bonnie-Clyde 100 Check problem reduces to the partition problem.

If there is a valid solution via Partition problem, we know there to be a solution. If not, we append a 1 to the distribution of checks, and rerun Set-Partition. Does this until either:

- We reach a valid solution.
- We reach 100.

Either way, this problem reduces to the Partition Problem.

Because we have proved that the Bonnie-Clyde problem is NP and reduces to an NP Complete problem, we conclude that the Bonnie-Clyde problem is NP complete. $\square$

²Cardinality meaning the amount of elements in the set

Question #9

A greedy algorithm is as follows:

1. Pick two arbitrary vertices u and v .
2. Add both u and v to the vertex cover.
3. Delete all incident edges to u and v .
4. If the adjacency matrix still has edges, go to Step 1.

For the adjacency matrix

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

We get the minimum vertex cover:

$$\{1, 2, 3, 4, 5\}$$

CS5400

Artificial Intelligence



Exercise #1

Introduction To Artificial Intelligence

Illya Starikov

Due Date: January 22nd, 2018

We defined a rational agent as follows:

For each possible percept sequence, a rational agent should select an action that is expected to maximize its performance measure, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

We also have the following assumptions:

1. The performance measure awards one point for each clean square at each time step, over a “lifetime” of 1000 time steps.
2. The “geography” of the environment is known a priori (Figure 2.2) but the dirt distribution and the initial location of the agent are not. Clean squares stay clean and sucking cleans the current square. The Left and Right actions move the agent left and right except when this would take the agent outside the environment, in which case the agent remains where it is.
3. The only available actions are Left, Right, and Suck.
4. The agent correctly perceives its location and whether that location contains dirt.

Problem Statement. *Show that the simple vacuum-cleaner agent function described in Figure 2.3 is indeed rational under the assumptions listed on Page 38.*

We couple our assumptions with the fact the agent moves to the adjoining square if there is no dirt, and “Suck”s if there is dirt. Also, we are under the assumption that the environment cannot become dirtier from some external agent within the 1000 time steps.

Under these assumptions, with our performance measure (see Item 1), we can conclude the agent is rational. Because the performance measure cannot decrease, it would always be beneficial for the agent to move to a new square to clean (since there is always a

possibility of increasing the performance measure). By design, our agent always moves to a new square, consequently, always having the possibility of increasing the performance measure.

Problem Statement. *Describe a rational agent function for the case in which each movement costs one point. Does the corresponding agent program require internal state?*

Yes. Because movement now costs one point, it is no longer in the agent's best interest to constantly move. However, because our agent is purely reflexive and we are unsure of the initial conditions, the agent will receive a negative performance measure; quite contrary to our definition of a rational agent.

For our agent to achieve maximum performance measure, the optimal situation would be to keep all clean squares in some sort of memory, and when the entirety of the vacuum world is clean, constantly perform a *NoOp*; a reflexive agent cannot do this.

Problem Statement. *Discuss possible agent designs for the cases in which clean squares can become dirty and the geography of the environment is unknown. Does it make sense for the agent to learn from its experience in these cases? If so, what should it learn? If not, why not?*

Learning would be optimal in this situation, for the following reasons:

Boundaries An agent with no clue about the world boundaries is doomed to constantly bump in the walls. This would waste precious time cycles on simply finding the boundaries.

Obstacles Just as one might bump in the boundaries, an agent might bump into obstacles.

Common Accumulation Some areas might become dirtier faster than others (i.e., a doorway). If an agent could determine where these areas are, it can check those areas more often than areas with less dirt.

In essence, learning some sort of a map would greatly improve the performance measure of the agent.

After learning some sort of map, we use the following agent design:

1. Start traversing to the most common dirty areas on the map in order of closeness.
2. With every step, if the current step is dirty, clean it; if not, *NoOp*.
3. If all common areas are **not** traversed, proceed to Step 1.
4. Proceed to the closest edge of the world boundary.
5. Proceed around the edges of the map.
6. If **not** reached original position, repeat Step 5.

7. Move one unit closer to the inside of the world. Treat the previous movement as the new world boundaries.
8. If **not** reached center of world, repeat Step 5.
9. Proceed to Step 1.

Exercise #2

Introduction To Artificial Intelligence

Illya Starikov

Due Date: January 24th, 2018

For these problems, we split the river into two sections into two section: Left and Right. Left is assumed to be the initial state.

1 Problem Formulation

Problem Statement. *Precisely formulate the problem.*

1. A method of describing a state would be with the tuple: $(C_L, M_L, B_{L/R})$, where:

C_L = The number of Cannibals on the Left

M_L = The number of Missionaries on the Left

$B_{L/R}$ = The location (Left/Right) of the boat

However, we restrict these numbers to as follows:

We do not care about the number of Cannibals/Missionaries on the right, because we can calculate them as follows:

$$C_R = 3 - C_L \quad M_R = 3 - M_L$$

Although these values are not strictly within the state description, from here forth (for readability purposes) we act as if it is, mentally replacing C_R and M_R with these values. The final description is still $(C_L, M_L, B_{L/R})$, and these are merely computed properties of the state.

Note, however, because we want to restrict our state space to only valid states, we place the following restrictions:

$$M_L \geq C_L \quad \text{and} \quad M_R \geq C_R$$

Furthermore,

$$B_{L/R} = \text{Left} \iff M_L + B_L = 6 \quad B_{L/R} = \text{Right} \iff M_R + B_R = 6$$

2. The initial state is as follows:

$$(C_L, M_L, B_{L/R}) = (3, 3, L)$$

3. Our ACTIONS are defined as follow. We use synonyms $L \leftrightarrow$ Left and $R \leftrightarrow$ Right.

Basically, we check to see if we can send one of both **Cannibal** and **Missionary**. If we can, add them to the solution set. Next, we check to see if we can send either 1 **Cannibal** and 1 **Missionary**. After, we check to see if we can send 2 **Cannibal** and 2 **Missionary**. Because there are rules we have to account for (i.e., there must be more **Missionary** than **Cannibal** *unless there's 0 Missionary*).

```

1: function  $\neg$ (direction)
2:   if direction = Left then
3:     return Right
4:   else
5:     return Left
6:   end if
7: end function
8:
9:
10: function ACTIONS( $s$ )
11:   PossibleActions  $\leftarrow \{\}$  ▷ A set of sets of moves
12:
13:   for all direction  $d \in \{\text{Left}, \text{Right}\}$  do
14:     if  $a.M_d > 0$  and  $a.C_d > 0$  then
15:       NewAction  $\leftarrow \{\text{send } \text{Cannibal } d \rightarrow \neg d\}$ 
16:       NewAction  $\leftarrow \{\text{send } \text{Missionary } d \rightarrow \neg d\}$  ▷ Note this has the form
17:        $\{\dots, \dots\}$ 
18:       PossibleActions  $\leftarrow \text{PossibleActions} \cup \{\text{NewAction}\}$  ▷ Note this has the
19:       form  $\{\dots, \{\dots, \dots\}, \dots\}$ 
20:     end if
21:
22:     for  $i \leftarrow 1$  to 2 do
23:       if  $a.M_d - i \geq 0$  then ▷ Is there even enough missionaries to remove?
24:         if  $(a.M_d - i \geq a.C_d \text{ or } a.M_d - i = 0)$  and  $a.M_{\neg d} + i \geq a.C_{\neg d}$  then
25:           NewAction  $\leftarrow \{\text{send } i \text{ Missionary } d \rightarrow \neg d\}$ 1
26:           PossibleActions  $\leftarrow \text{PossibleActions} \cup \{\text{NewAction}\}$ 
27:         end if
28:       end if

```

¹Note by saying “send i **Missionary**”, we are just saying “send a missionary” i times. For $i = 2$, we have NewAction $\leftarrow \{\text{send a missionary}, \text{send a missionary}\}$.

```

28:
29:         if  $a.C_d - i \geq 0$  then           ▷ Is there even enough cannibals to remove?
30:             if  $a.C_{\neg d} + i \leq a.M_{\neg d}$  or  $a.M_{\neg d} = 0$  then
31:                 NewAction  $\leftarrow \{\text{send } i \text{ Cannibal } d \rightarrow \neg d\}$ 2
32:                 PossibleActions  $\leftarrow$  PossibleActions  $\cup \{\text{NewAction}\}$ 
33:             end if
34:         end if
35:     end for
36: end for
37:
38:     return PossibleActions
39: end function

```

4. We define RESULTS(s, a) as follows.

```

1: function RESULTS( $s, a$ )
2:      $a' \leftarrow a$ 
3:      $a'.B_{L/R} = \neg a'.B_{L/R}$ 
4:
5:     for all action  $\in a$  do           ▷ We defined an action as a set of steps
6:         if action = send Cannibal Left  $\rightarrow$  Right then
7:              $a'.C_L = a'.C_L - 1$ 
8:         end if
9:
10:        if action = send Cannibal Right  $\rightarrow$  Left then
11:             $a'.C_L = a'.C_L + 1$ 
12:        end if
13:
14:        if action = send Missionary Left  $\rightarrow$  Right then
15:             $a'.M_L = a'.M_L - 1$ 
16:        end if
17:
18:        if action = send Missionary Right  $\rightarrow$  Left then
19:             $a'.M_L = a'.M_L + 1$ 
20:        end if
21:    end for
22:
23:    return  $a'$ 
24: end function

```

5. We define ISGOAL(s) as follows.

```

1: function ISGOAL( $s$ )

```

²See previous footnote.

```
2:   return  $s = (0, 0, \text{Right})$ 
3: end function
```

6. We define $C(s, a, s')$

```
1: function  $C(s, a, s')$ 
2:   return  $|a|$ 3
3: end function
```

2 State Space

Figure 1 provides the complete space for the problem, with **M** representing Missionaries and **C** representing cannibals.

3 Possible Solution

Figure 2 outlines a path for a solution, with **M** representing Missionaries and **C** representing cannibals.

³We define the $|x|$ operator to return the length of x .

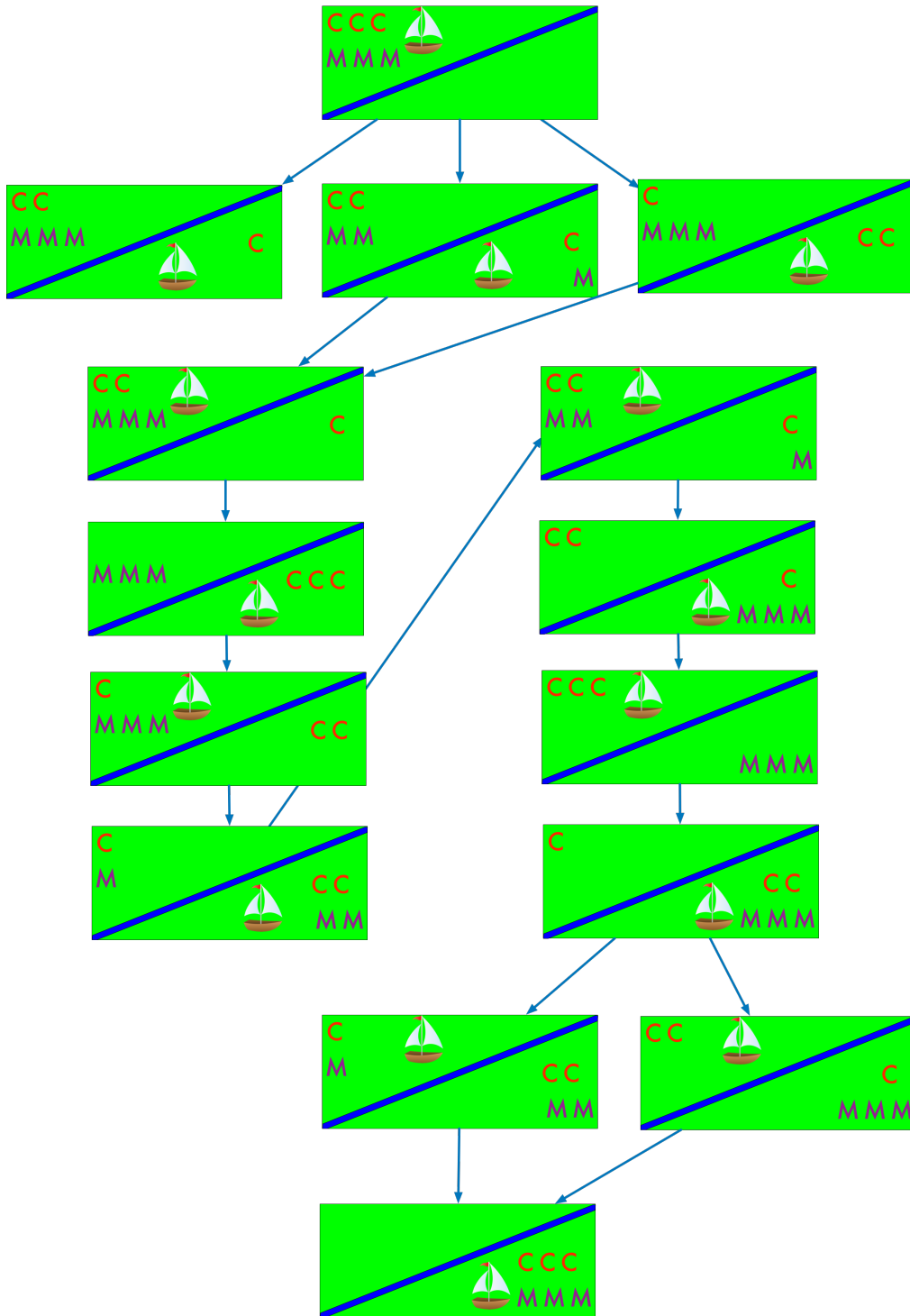


Figure 1: The Complete State Space

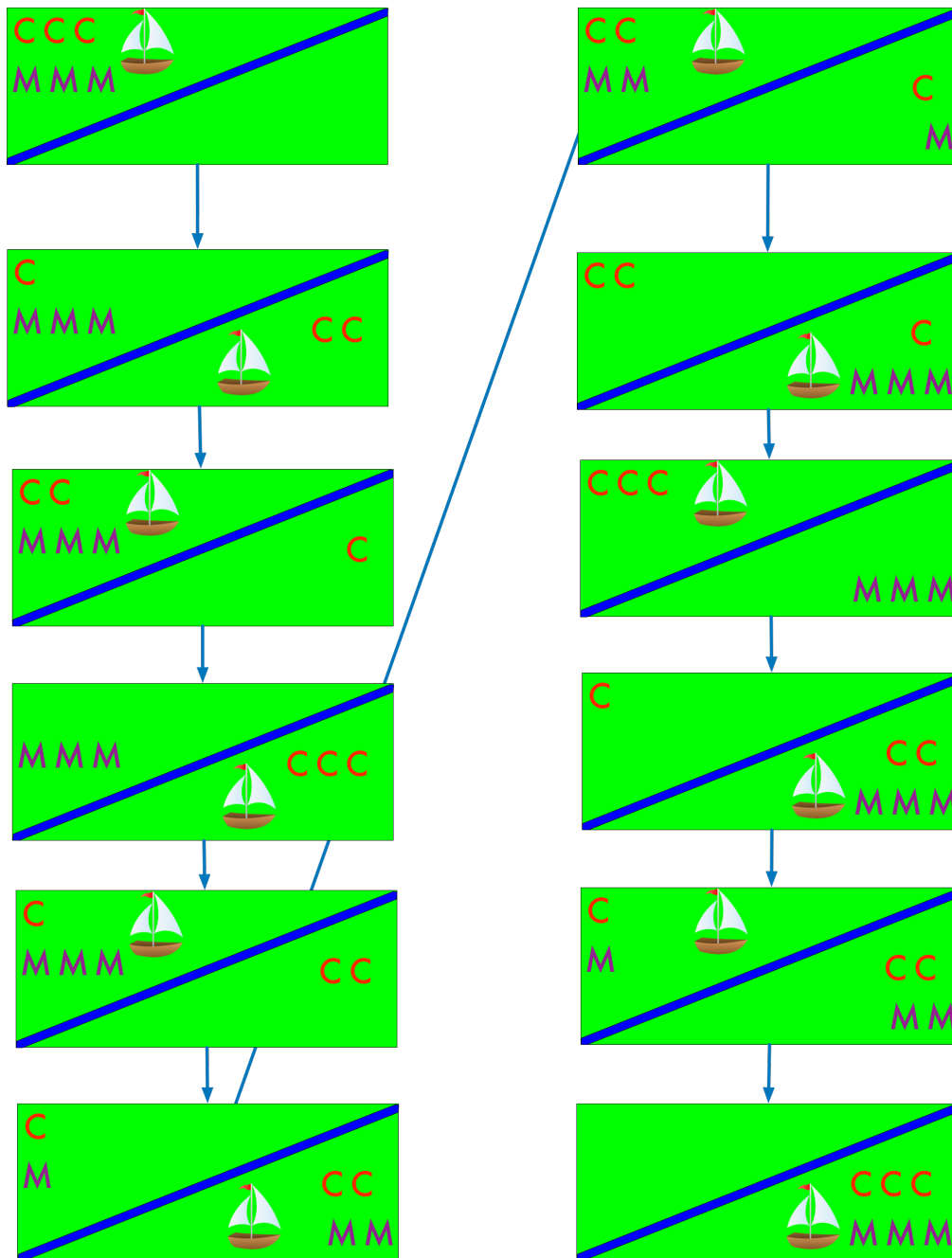


Figure 2: Path To A Solution

Exercise #3

Introduction To Artificial Intelligence

Illya Starikov

Due Date: February 5th, 2018

1 Negative Path Optimality

Problem Statement. *Suppose that actions can have arbitrarily large negative costs; explain why this possibility would force any optimal algorithm to explore the entire state space.*

Even if for any path we have an arbitrary path $P_{\max}$ with largest path cost $N_{\max}$,

$$N_{\max} = \sum_{\text{vertex } v \in P_{\max}} \text{PATH-COST}(v)$$

There might be a singular path P after it,

$$N = \sum_{\text{vertex } v \in P} \text{PATH-COST}(v)$$

Such that

$$N_{\max} + N = \text{Minimum Path Cost}$$

Therefore, any algorithm that is to find the optimal path cost must search the entire state space.

Also, there is the possibility of a negative cycle in a graph, which can only be discovered by exploring the entire state space.

2 Bounds on Negative Path Cost

Problem Statement. *Does it help if we insist that step costs must be greater than or equal to some negative constant c ? Consider both trees and graphs.*

For trees, it can be helpful in limiting the search space, because there can only be so much gained from negative paths.

For graphs, there is the aforementioned issue of negative cycles, where the lowest path cost could be $-\infty$ (i.e., keep repeating the negative cycle infinitely).

3 Loops In Negative Path Cost

Problem Statement. *Suppose that a set of actions forms a loop in the state space such that executing the set in some order results in no net change to the state. If all of these actions have negative cost, what does this imply about the optimal behavior for an agent in such an environment?*

This implies that the optimal behavior in this environment is to loop around the states in the negative cycle.

Exercise #4

Introduction To Artificial Intelligence

Illya Starikov

Due Date: February 12th, 2018

Problem Statement. *On page 90, we mentioned iterative lengthening search, an iterative analog of uniform cost search. The idea is to use increasing limits on path cost. If a node is generated whose path cost exceeds the current limit, it is immediately discarded. For each new iteration, the limit is set to the lowest path cost of any node discarded in the previous iteration.*

1 Iterative Lengthening Optimality

Problem Statement. *Show that this algorithm is optimal for general path costs.*

Because the nodes are discovered in order of path cost, naturally the node with the lowest path cost will be discovered first.

2 Iterative Lengthening Performance

Problem Statement. *Consider a uniform tree with branching factor b , solution depth d , and unit step costs. How many iterations will iterative lengthening require?*

The runtime will be $\mathcal{O}(b^d)$.

3 Iterative Lengthening Performance II

Problem Statement. *Now consider step costs drawn from the continuous range $[\epsilon, 1]$, where $0 < \epsilon < 1$. How many iterations are required in the worst case?*

The runtime will be $\mathcal{O}\left(\frac{d}{\epsilon}\right)$.

Exercise #5

Introduction To Artificial Intelligence

Illya Starikov

Due Date: February 19th, 2018

1 A^* vs. DFS

Problem Statement. *True/False: Depth-first search always expands at least as many nodes as A search with an admissible heuristic.*

False. DFS has the best case of $\Theta(d)$ (for the case where there is no backtracking to find a solution). A^* can make no such guarantee, because it ranges by heuristic.

2 $h(n) = 0$ For 8-Puzzle

Problem Statement. *True/False: $h(n) = 0$ is an admissible heuristic for the 8-puzzle.*

True. For positive path costs, $h(n) = 0$ will always be an admissible heuristic.

$$\forall \epsilon \in \mathbb{R}^+, h(n) = 0 \leq C^*(n) = \epsilon$$

3 A^* In Robotics

Problem Statement. *True/False: A is of no use in robotics because percepts, states, and actions are continuous.*

True. A^* operates on discrete states. However, there is always the possibility to create discrete states from continuous states.

4 Breadth-First Search Completeness

Problem Statement. *True/False: Breadth-first search is complete even if zero step costs are allowed.*

True. Per our definition of completeness:

A search algorithm is complete iff it will find a goal when one exists.

5 Rook Admissibility

Problem Statement. *True/False: Assume that a rook can move on a chessboard any number of squares in a straight line, vertically or horizontally, but cannot jump over other pieces. Manhattan distance is an admissible heuristic for the problem of moving the rook from square A to square B in the smallest number of moves.*

False. Using the chess depicted in Figure 1, the Manhattan Distance from A1 to A8 would be 8; however, a rook could get there in one move.

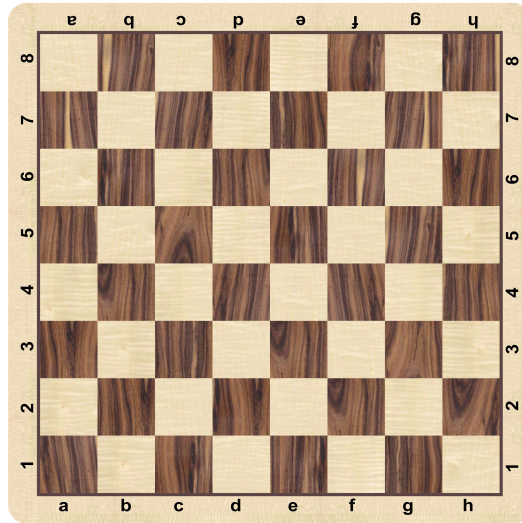


Figure 1: A Standard Chessboard

Exercise #6

Introduction To Artificial Intelligence

Illya Starikov

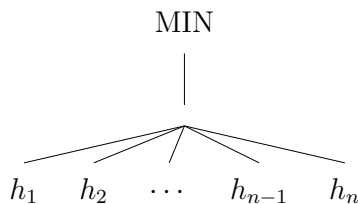
Due Date: March 13th, 2018

1 Suboptimal Min vs Optimal Min

Problem Statement. *Prove the following assertion: For every game tree, the utility obtained by MAX using minimax decisions against a suboptimal MIN will be never be lower than the utility obtained playing against an optimal MIN. Can you come up with a game tree in which MAX can do still better using a suboptimal strategy against a suboptimal MIN?*

We will do so by induction.

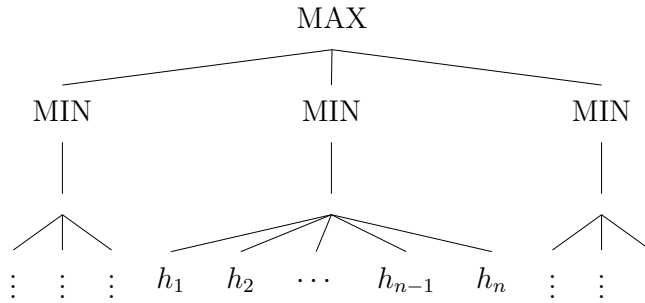
Basis Case The base case is a *MIN* node with all terminals. Suppose we have the following leaf such that for any node h_x , they are ordered in ascending order based on the utility value $h_1 \leq h_2 \leq \dots \leq h_{n-1} \leq h_n$:



An optimal *MIN* would always pick h_1 , while a suboptimal would pick from $\{h_1, h_2, \dots, h_{n-1}, h_n\}$. From this, we can conclude

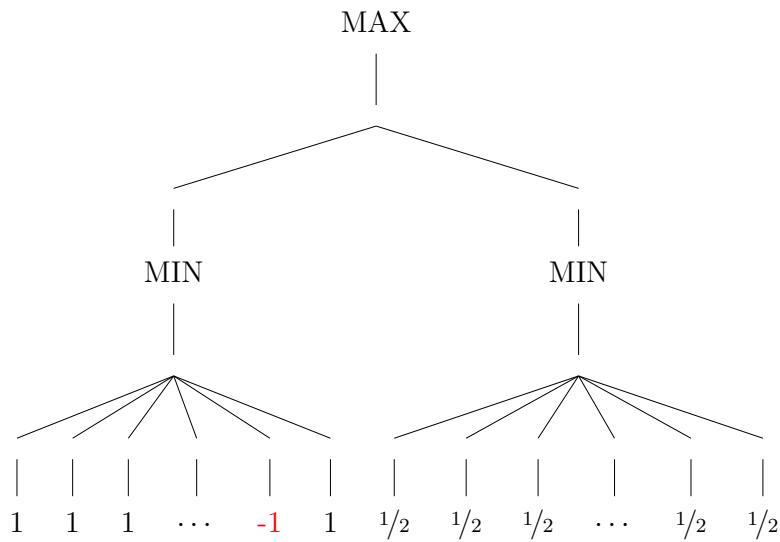
$$\text{Optimal } MIN\text{'s Utility} \leq \text{Suboptimal } MIN \text{ Utility} \quad (1)$$

Inductive Case The recursive case is for *MIN* with parents and children.



However, because the game tree can be reduced to a *MIN* node with all terminals (such as the base case), our inductive step holds.

For a game tree of a suboptimal *MAX* vs a *MIN*, we can produce the following chess game:



An optimal *MAX* would pick the rightmost *MIN*, which would result in only a draw; however, a suboptimal *MIN* would only pick the leftmost *MIN*, resulting in in almost all wins, with only one possibility of losing.

Exercise #7

Introduction To Artificial Intelligence

Illya Starikov

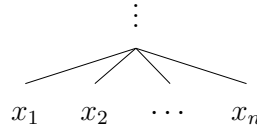
Due Date: March 9th, 2018

1 x To $ax + b$ Leaf Node Transformation

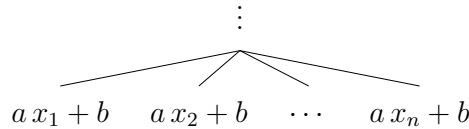
Problem Statement. *Prove that with a positive linear transformation of leaf values (i.e., transforming a value x to $ax + b$ where $a > 0$), the choice of move remains unchanged in a game tree, even when there are chance nodes.*

Proof. We will prove the following via proof of mathematical induction.

Base Case We will take the base case with a leaf nodes, so the transformation would be as follows:



would transform into:



Recalling the definition of MINIMAX, we get the following:

$$\text{MINIMAX}(s) = \begin{cases} \text{MAX'S UTILITY}(s) & \leftrightarrow \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \leftrightarrow \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{ACTIONS}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \leftrightarrow \text{PLAYER}(s) = \text{MIN} \end{cases} \quad (1)$$

Recalling that ACTIONS returns a set of values (let us name them $x_1, x_2, \dots, x_n$) our $\min()$ and $\max()$ values can be rewritten as:

$$\begin{aligned} \min(x_1, x_2, \dots, x_n) &= x_p \\ \max(x_1, x_2, \dots, x_n) &= x_p \end{aligned}$$

where x_p is the chosen node. For the probability, we get

$$p(x_1) + p(x_2) + \cdots + p(x_n)$$

Applying the transformation, we get the following:

$$\begin{aligned} \min(a x_1 + b, a x_2 + b, \dots, a x_n + b) &= a x_p + b \\ \max(a x_1 + b, a x_2 + b, \dots, a x_n + b) &= a x_p + b \\ p_1(a x_1 + b) + p_2(a x_2 + b) + \cdots + p_n(a x_n + b) & \end{aligned}$$

From this, we factor out a and b :

$$\begin{aligned} a \min(x_1, x_2, \dots, x_n) + b &= a x_p + b \\ a \max(x_1, x_2, \dots, x_n) + b &= a x_p + b \\ a(p_1 x_1 + p_2 x_2 + \cdots + p_n x_n) + b & \end{aligned}$$

Upon which we can clearly see a ($\forall a \in \mathbb{R}^+$) and b have no choice on which node gets chosen.

Inductive Step As we scale our problem up for MINIMAX (Equation 1), we see that if the decision of the terminal node is unchanged, the decision at the root will be changed. The same terminal value will be returned; and as we go up the game tree, each ply acts as a terminal.

□

Exercise #8

Introduction To Artificial Intelligence

Illya Starikov

Due Date: April 22nd, 2018

1 State-Space Search Algorithms

Problem Statement. *Give the name of the algorithm that results from each of the following special cases.*

1. Local beam search with $k = 1$. **Steepest-Ascent Hill-Climbing.**
2. Local beam search with one initial state and no limit on the number of states retained. **Breadth First Tree Search.**
3. Simulated annealing with $T = 0$ at all times (and omitting the termination test). **Steepest-Ascent Hill-Climbing.**
4. Simulated annealing with $T = \infty$ at all times. **Random Walk.**
5. Genetic algorithm with population size $N = 1$. **Random Walk.**

Bonus #1

Illya Starikov

Due Date: Sunday, February 18th, 2018

We know the definition of branching factor b^* to be defined as

$$N = b^* + (b^*)^2 + \dots + (b^*)^d$$

With N being the total number of nodes being generated, and d being the solution depth. From this, we can get an estimate for the effective branching factor as follows:

$$b^* = N^{\frac{1}{d}}$$

Note this is not a perfect equation, and has the possibility for high error tolerances; for our purposes, it should be okay. The two heuristics for this assignment are as follows:

$$\begin{aligned} h_1 &= \text{Score Difference} \\ &= \frac{\text{Quota} - \text{Current Score}}{\text{Max Swaps} - \text{Current Swaps}} \\ h_2 &= \text{Homogeneous Board} \\ &= \text{Standard Deviation of the} \\ &= \text{Number of Devices Types on the Board} \end{aligned}$$

Results can be found in Table 1. Essentially, the branching factors are similar, but the Homogeneous Board heuristic has a tendency to explore more of the board.

Table 1: The computations for the effective branching factor.

Input	Heuristic	Nodes Generated (N)	Solution Depth (d)	Branching Factor (b^*)
Puzzle #1	Score Difference	4	1	4
	Homogeneous Board	4	1	4
Puzzle #2	Score Difference	99	11	1.52
	Homogeneous Board	144	10	1.64
Puzzle #3	Score Difference	94	12	1.46
	Homogeneous Board	890	25	1.31

CS5402

Data Mining



Homework #1

CS5402 — Intro To Data Mining

Illya Starikov

Due Date: July 6th, 2018

1 1-Rule Method

Problem Statement. Use the 1-rule (1R) method to find the best single attribute to determine restaurant. In order to demonstrate that you actually know how this method works (and aren't just guessing at which attribute is best), you must fill in ALL of the blank values in the table below; otherwise, you will not receive any credit for this problem.

Attribute	Attribute Value	# Rows	Most Frequent Value	Errors	Total Errors
Meal Preference	Hamburger	3	McDonalds (3)	0	2
	Fish	2	Burger King (2)	0	
	Chicken	4	Wendy's (2)	2	
Gender	M	5	Burger King/McDonald's (2/2)	3	5
	F	4	McDonald's (2)	2	
Drink Preference	Pepsi	3	Burger King (2)	1	3
	Coke	6	McDonald's (4)	2	

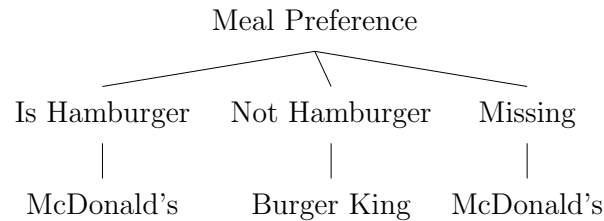
From this, the following rules would be generated:

Meal Preference = *Hamburger* $\implies$ McDonald's
Meal Preference = *Fish* $\implies$ Burger King
Meal Preference = *Chicken* $\implies$ Wendy's

2 Decision Stump

Problem Statement. Create the dataset given in problem 1. as an arff or csv file, and run *DecisionStump* on it in Weka. List the classification rules that are produced by Weka (you can just include a screenshot of your Weka output) AND hand-draw the decision tree that corresponds to those rules.

From Weka's Decision Stump, we get the following output:
 For the figure, we get the following decision tree:



3 Statistical Modeling

Problem Statement. *Statistical modeling can be used to compute the probability of occurrence of an attribute value. Based on the data given in the table below, if we have a new instance where $ageGroup = youngAdult$, $gender = M$, and $bookPreference = nonFiction$, what is the likelihood that $musicPreference = country$? Just set up the equation to compute this with the appropriate values; you don't have to actually calculate the final answer.*

$$\text{Likelihood of Country} = \frac{2}{3} \times \frac{1}{3} \times \frac{1}{3} \times \frac{3}{8} = \frac{1}{36} \approx 0.02\bar{7}$$

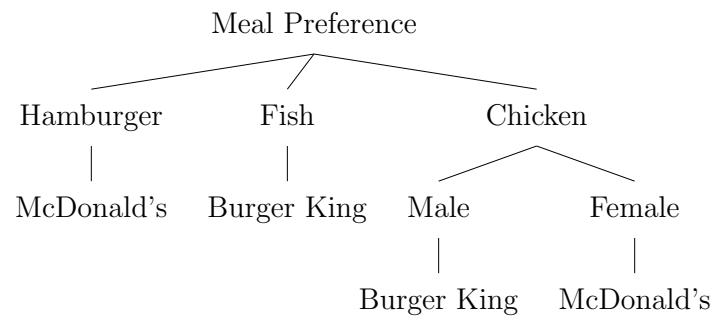
4 Id3

Problem Statement. *Create the dataset given in problem 1. as an arff or csv file, and run Id3 on it in Weka. Include a screenshot of the decision tree output that is produced by Weka AND hand-draw the decision tree. Note: You may have to install the package containing Id3.*

From Weka's Id3, we get the following output:



For the figure, we get the following decision tree:



5 Entropy

Problem Statement. *If we want to make a decision tree for determining restaurant, we must decide which of the three non-decision attributes (mealPreference, gender, or drinkPreference) to use as the root of the tree.*

- *Set up the equation to compute what in lecture we called `entropyBeforeSplit` for restaurant. You do not have to actually solve (i.e., calculate the terms in) the*

equation, just set up the equation with the appropriate values.

$$\text{Entropy Before Split} = -\frac{3}{10} \log_2 \left(\frac{3}{10}\right) - \frac{5}{10} \log_2 \left(\frac{5}{10}\right) - \frac{2}{10} \log_2 \left(\frac{2}{10}\right) \approx 1.485$$

- Set up the equation to compute entropy for mealPreference when its value is chicken. That is, a tree with mealPreference at the root would have three branches (one for hamburger, one for chicken, and one for fish), requiring us to compute entropyHamburger, entropyChicken, and entropyFish; here we only want you to set up the equation to compute entropyChicken. You do not have to actually solve (i.e., calculate the terms in) the equation, just set it up using the appropriate values.

$$\text{Entropy Chicken} = -\frac{1}{4} \log_2 \left(\frac{1}{4}\right) - \frac{1}{4} \log_2 \left(\frac{1}{4}\right) - \frac{2}{4} \log_2 \left(\frac{2}{4}\right) \approx 1.040$$

- Suppose that instead of considering mealPreference to be the root of this decision tree, we had instead considered drinkPreference. Set up the equation to compute information gain for drinkPreference given the variables specified below.

$$\text{Information Gain} = X - \left(\frac{7}{10} C + \frac{3}{10} P\right)$$

6 isSticky

Problem Statement. Create the dataset shown below as an arff or csv file and run the Prism algorithm on it in Weka specifying isSticky as the decision attribute. List the classification rules that are produced (you can just include a screenshot of your Weka output). Then work out the Prism algorithm by hand to show what classification rules you would get; who knows, they might be different!

6.1 Is Sticky = No

consistency	packaging	chocolate	roundShaped	isSticky
soft	individuallyWrapped	yes	no	no
soft	box	yes	no	no
hard	sack	no	yes	no
hard	box	yes	yes	no
hard	individuallyWrapped	no	no	yes
hard	box	no	yes	yes
soft	individuallyWrapped	no	no	yes

Which leads us to the following T and P values:

6.2 Is Sticky = No

Label	T	P	T/P
soft	3	2	0.66
hard	4	2	0.50
individually wrapped	3	1	0.33
box	3	2	0.66
sack	1	1	1
chocolate	3	3	1
not chocolate	4	1	0.25
round	3	2	0.66
not round	4	2	0.5

This leads to the following rule:

$$\text{chocolate} = \text{yes} \implies \text{is sticky} = \text{no}$$

Reducing Our dataset to:

6.3 Is Sticky = No

consistency	packaging	chocolate	roundShaped	isSticky
hard	sack	no	yes	no
hard	individuallyWrapped	no	no	yes
hard	box	no	yes	yes
soft	individuallyWrapped	no	no	yes

Which leads us to the following T and P values:

6.4 Is Sticky = No

consistency	packaging	chocolate	roundShaped	isSticky
hard	sack	no	yes	no
hard	individuallyWrapped	no	no	yes
hard	box	no	yes	yes
soft	individuallyWrapped	no	no	yes

Which leads us to the following T and P values:

Label	T	P	T/P
soft	1	0	0
hard	3	1	0.33
individually wrapped	2	0	0
box	1	0	0
sack	1	1	1
chocolate	0	0	
not chocolate	4	1	0.25
round	2	1	0.5
not round	2	0	0

Which leads us to the following rule:

$$\text{packaging} = \text{sack} \implies \text{is sticky} = \text{no}$$

6.5 Is Sticky = Yes

hard	individuallyWrapped	no	no	yes
hard	box	no	yes	yes
soft	individuallyWrapped	no	no	yes

Which leads us to the following T and P values:

6.6 Is Sticky = No

Label	T	P	T/P
soft	1	1	1
hard	2	2	1
individually wrapped	2	2	1
box	1	1	1
sack	0	0	–
chocolate	0	0	–
not chocolate	3	3	1
round	1	1	1
not round	2	2	1

Which leads us to the following rule:

$$\text{chocolate} = \text{no} \implies \text{is sticky} = \text{yes}$$

6.7 Final Ruleset

From this, the following rules would be generated:

chocolate = yes $\implies$ is sticky = no
packaging = sack $\implies$ is sticky = no
chocolate = no $\implies$ is sticky = yes

6.8 Weka Output

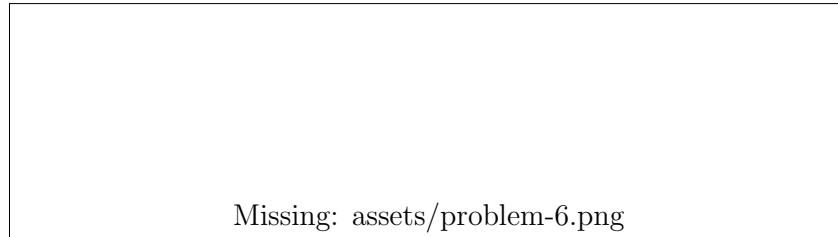


Figure 1: Weka output for problem 6

Homework #2

CS5402 — Intro To Data Mining

Illya Starikov

Due Date: July 10th, 2018

1 Association Rules

Problem Statement. *Compute the coverage (i.e., support) of each item set listed below.*

drinkPref = pepsi 2

married = no, pet = dog 3

sportPref = football, musicPref = classical, drinkPref = tea 0

Problem Statement. *Write down every association rule that could be generated from the item set listed below, regardless of whether or not there are actually any instances of that rule in our given dataset.*

married = no $\implies$ **pet = dog**
pet = dog $\implies$ **married = no**
- $\implies$ **married = no and pet = dog**

Problem Statement. *Compute the accuracy (i.e., confidence) of each rule listed below. Express accuracy as a fraction (e.g., 2/3, 2/2, etc.), NOT as a decimal number (e.g., 0.67, 1.0, etc.).*

With the case *If pet = dog then income = middle*, we get as follows:

$$\text{Accuracy} = \frac{1}{4}$$

With the case *If married = no and sportPref = football, then pet = dog and musicPref = rock*, we get as follows:

$$\text{Accuracy} = \frac{2}{3}$$

With the case *If* $\neg$, then $\text{drinkPref} = \text{coke}$ and $\text{married} = \text{yes}$, we get as follows:

$$\text{Accuracy} = 0$$

Problem Statement. In each rule listed below, specify whether any condition(s) in the antecedent (i.e., the “if” part) can be dropped.

If married = no and sportPref = football then pet = dog and drinkPref = coke No.

If married = yes and pet = cat then musicPref = country Yes, married = yes can be dropped.

2 RICO

Problem Statement. Using the Rule Induction from Coverings algorithm discussed in class, find each of the partitions specified below.

$$\begin{aligned} \{a\}^* &= \{\{x_1, x_5\}, \{x_2, x_3, x_4\}\} \\ \{c\}^* &= \{\{x_1\}, \{x_2, x_3\}, \{x_4\}, \{x_5\}\} \\ \{d\}^* &= \{\{x_1, x_2\}, \{x_3, x_4\}, \{x_5\}\} \\ \{d, e\}^* &= \{\{x_1\}, \{x_2\}, \{x_3, x_4\}, \{x_5\}\} \end{aligned}$$

Problem Statement. Suppose that we are looking for a covering for decision attribute f which has partition $\{f\}^* = \{\{x_1, x_2\}, \{x_3, x_4, x_5\}\}$.

- No, the block $\{x_1, x_5\}$ is not a subset of any blocks in $\{f\}^*$.
- No, the block $\{x_2, x_3\}$ is not a subset of any blocks in $\{f\}^*$.
- Yes, all blocks are subsets of all blocks in $\{f\}^*$.
- No, because $\{d, e\}^*$ is not minimal.

Problem Statement. Find a covering for f . If you found one in part b that worked, just use one of those. Using your covering, give a set of rules for decision attribute f .

$$\begin{aligned} d = 0 &\implies f = \text{True} \\ d = 1 &\implies f = \text{False} \\ d = 2 &\implies f = \text{False} \end{aligned}$$

3 KD-Tree

Problem Statement. Consider the dataset given below where the decision attribute is the one labeled decision. Build a kd-tree where $k = 3$. No partial credit will be given unless you show your work!

Sorting the tuples by x , we get as follows:

$$[(1, 7, 9), (2, 5, 8), (3, 6, 7), (4, 4, 0), (5, 3, 4), (6, 1, 6), (7, 2, 2)]$$

Seeing a our median = 4, we partition as follows:

$$[(1, 7, 9), (2, 5, 8), (3, 6, 7)] \quad [(4, 4, 0), (5, 3, 4), (6, 1, 6), (7, 2, 2)]$$

Sorting these tuples by y , we get as follows:

$$[(2, 5, 8), (3, 6, 7), (1, 7, 9)] \quad [(6, 1, 6), (7, 2, 2), (5, 3, 4), (4, 4, 0)]$$

With our medians as follows, we get as follows:

$$\begin{array}{ccccc} \text{median} = 6 & & \text{median} = 2.5 & & \\ [(2, 5, 8)] & [(3, 6, 7), (1, 7, 9)] & [(6, 1, 6), (7, 2, 2)] & [(5, 3, 4), (4, 4, 0)] & \end{array}$$

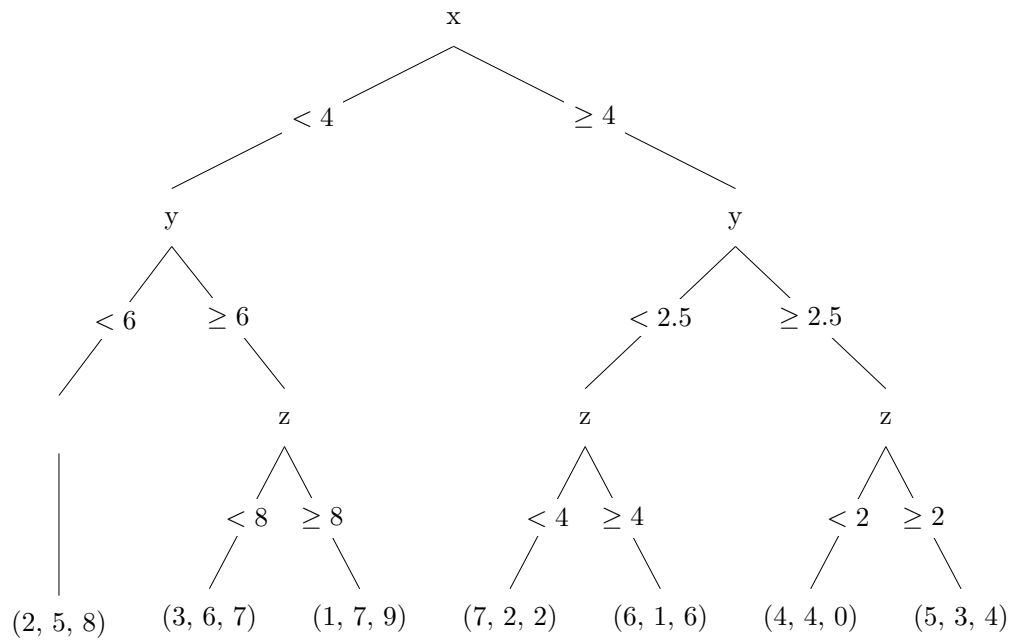
Sorting these tuples by z , we get as follows:

$$[(2, 5, 8)] \quad [(3, 6, 7), (1, 7, 9)] \quad [(7, 2, 2), (6, 1, 6)] \quad [(4, 4, 0), (5, 3, 4)]$$

With our medians as follows, we get as follows:

$$\begin{array}{ccccccccc} \text{median} = 8 & \text{median} = 8 & & \text{median} = 4 & & \text{median} = 2 & & & \\ [(2, 5, 8)] & [(3, 6, 7)] & [(1, 7, 9)] & [(7, 2, 2)] & [(6, 1, 6)] & [(4, 4, 0)] & [(5, 3, 4)] & & \end{array}$$

To produce the following KD-Tree:



4 Winnow

Instance	Calculation	Predicted	Actual	Resultant Weights
x_1	$2 \times 1 + 2 \times 0 + 2 \times 1 = 4 > 2$	1	0	$w_a = 1, w_b = 2, w_c = 1$
x_2	$1 \times 0 + 2 \times 1 + 1 \times 0 = 2 \leq 2$	0	1	$w_a = 1, w_b = 4, w_c = 1$
x_3	$2 \times 1 + 4 \times 1 + 2 \times 0 = 6 > 2$	1	1	$w_a = 1, w_b = 4, w_c = 1$
x_4	$1 \times 1 + 4 \times 0 + 1 \times 1 = 2 \leq 2$	0	0	$w_a = 1, w_b = 4, w_c = 1$

Homework #3

CS5402 — Intro To Data Mining

Illya Starikov

Due Date: July 17th, 2018

1 C4.5

Split	Entropy Less Than	Entropy Greater Than	Entropy Before Split	Entropy After Split	Information Gain
8	0	1.55459	1.579	1.435	0.14400
10	1	1.57262	1.579	1.48453	0.09447
15	0.9183	1.52193	1.579	1.38263	0.19637
20	0.81128	1.39215	1.579	1.21342	0.36558
24	0.97095	1.40564	1.579	1.23845	0.34055
28	0.9183	0.98523	1.579	0.95434	0.62466
28	1.37878	1	1.579	1.20396	0.37504
28	1.5	0.97095	1.579	1.29652	0.28248
29	1.53049	1	1.579	1.36726	0.21174
30	1.52193	0.9183	1.579	1.38263	0.19637
30	1.49492	0	1.579	1.26493	0.31407
31	1.55459	0	1.579	1.435	0.14400
33	1.57662	0	1.579	1.57662	0.00238

2 Grouping

Grouping	Entropy	Entropy Before Split	Entropy After Split	Information Gain
Coke-Pepsi	1.5219	1.565	1.2652	0.2998
Mountain Dew	1.5850	1.565		
Coke-Mountain Dew	1.4591	1.565	1.0943	0.4706
Pepsi	0	1.565		
Pepsi-Mountain Dew	1.3710	1.565	0.9623	0.6028
Coke	0.9183	1.565		

No, they should not be grouped; the respective information gains for the groupings are less than the information gain without grouping.

3 J48

For the deepest subtree, `credit_history`, we get the following error for the individual leaves:

$$U_{90\%}(0, 1) + U_{90\%}(0, 1) + 7 \times U_{90\%}(1, 7) + 0 + 2 \times U_{90\%}(0, 2) \approx \\ 0.95 + 0.95 + 7 \times 0.5207 + 0 + 2 \times 0.77639 \approx 3.19709$$

If the subtree would just be replaced by a leaf **good (9/2)**, predicted error would be

$$9 \times U_{90\%}(2, 9) \approx 9 \times 0.47009 \approx 4.23081$$

Therefore, the new tree would be

```
property_magnitude = car
|   personal_status = male div/sep: bad (1.0)
|   personal_status = female div/dep/mar: good (7.0)
|   personal_status = male single: good (9.0/2.0)
|   personal_status = male mar/wid: good (1.0)
|   personal_status = female single: good (0.0)
```

For this subtree, we get the following error for individual leaves:

$$U_{90\%}(0, 1) + 7 \times U_{90\%}(0, 7) + 9 \times U_{90\%}(2, 9) + U_{90\%}(0, 1) + 0 \approx \\ 0.95 + 7 \times 0.348169 \times 0.470090 \approx 7.61793$$

Again, if the subtree would be replaced by a leaf **good (17/2)**, predicted error would be

$$17 \times U_{90\%}(2, 17) \approx 17 \times 0.70420 \approx 5.0286$$

Therefore, the new tree would be

```
property_magnitude = car: good (17/2)
```

4 Missing Value

4.1 Outdoor

4.1.1 Hair = Short

$$\text{entropy} = -\frac{1 + 2/7}{2 + 2/7} \log_2 \frac{1 + 2/7}{2 + 2/7} - \frac{1}{2 + 2/7} \log_2 \frac{1}{2 + 2/7} = 0.9887$$

4.1.2 Hair = Long

$$\text{entropy} = \frac{0}{\vdots} \dots = 0$$

4.2 Indoor

4.2.1 Hair = Short

$$\text{entropy} = -\frac{1}{1 + 2/7} \log_2 \frac{1}{1 + 2/7} - \frac{2/7}{1 + 2/7} \log_2 \frac{2/7}{2 + 2/7} = 0.7642$$

4.2.2 Hair = Long

$$\text{entropy} = -\frac{1}{1} \log_2 \frac{1}{1} = 0$$

4.3 Both

4.3.1 Hair = Short

$$\text{entropy} = -\frac{1 + 3/7}{1 + 3/7} \log_2 \frac{1 + 3/7}{2 + 3/7} = 0 = 0$$

4.3.2 Hair = Long

$$\text{entropy} = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$$

Homework #4

CS5402 — Intro To Data Mining

Illya Starikov

Due Date: July 20th, 2018

1 Support Vector Machine

1. Our equations would be as follows:

$$\begin{aligned}\alpha_1 \varphi(s_1) \cdot \varphi(s_1) + \alpha_2 \varphi(s_2) \cdot \varphi(s_1) &= 1 \\ \alpha_1 \varphi(s_1) \cdot \varphi(s_2) + \alpha_2 \varphi(s_2) \cdot \varphi(s_2) &= -1\end{aligned}$$

2. Assuming φ to be the identity operator, we get as follows:

$$\begin{aligned}6\alpha_1 + 11\alpha_2 &= 1 \\ 11\alpha_1 + 21\alpha_2 &= -1\end{aligned}$$

From this, we get as follows:

$$\alpha_1 = 32/5 \quad \alpha_2 = -17/5$$

3. Our discriminating hyperplane would be as follows:

$$w' = \langle -4/5, -2/5, 3 \rangle$$

Therefore, the plane equation would be as follows:

$$-4/5 x - 2/5 y + 3 = 0$$

2 K2 Bayesian Network

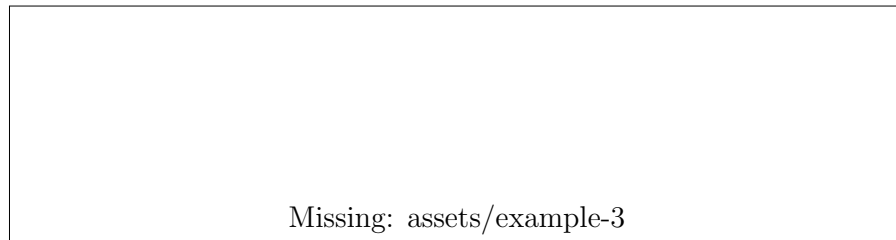
For our α values, we get the following:

$$\begin{aligned}\alpha_{411} &= 1 & (\# \text{ cases with } x_1 = 0, x_2 = N, x_4 = H) \\ \alpha_{412} &= 2 & (\# \text{ cases with } x_1 = 0, x_2 = N, x_4 = L) \\ \alpha_{413} &= 1 & (\# \text{ cases with } x_1 = 0, x_2 = N, x_4 = M) \\ \alpha_{421} &= 0 & (\# \text{ cases with } x_1 = 0, x_2 = Y, x_4 = H) \\ \alpha_{422} &= 3 & (\# \text{ cases with } x_1 = 0, x_2 = Y, x_4 = L) \\ \alpha_{423} &= 1 & (\# \text{ cases with } x_1 = 0, x_2 = Y, x_4 = M) \\ \alpha_{431} &= 1 & (\# \text{ cases with } x_1 = 1, x_2 = N, x_4 = H) \\ \alpha_{432} &= 3 & (\# \text{ cases with } x_1 = 1, x_2 = N, x_4 = L) \\ \alpha_{433} &= 1 & (\# \text{ cases with } x_1 = 1, x_2 = N, x_4 = M) \\ \alpha_{441} &= 0 & (\# \text{ cases with } x_1 = 1, x_2 = Y, x_4 = H) \\ \alpha_{442} &= 2 & (\# \text{ cases with } x_1 = 1, x_2 = Y, x_4 = L) \\ \alpha_{443} &= 1 & (\# \text{ cases with } x_1 = 1, x_2 = Y, x_4 = M)\end{aligned}$$

For our N values, we get the following:

$$\begin{aligned}N_{41} &= \alpha_{411} + \alpha_{412} + \alpha_{413} = 1 + 2 + 1 = 4 \\ N_{42} &= \alpha_{421} + \alpha_{422} + \alpha_{423} = 0 + 3 + 1 = 4 \\ N_{43} &= \alpha_{431} + \alpha_{432} + \alpha_{433} = 1 + 3 + 1 = 5 \\ N_{44} &= \alpha_{441} + \alpha_{442} + \alpha_{443} = 0 + 2 + 1 = 3\end{aligned}$$

3 Weka Output



For the Naive vs Regular Bayesian Network, the results are similar. With the same confusion matrix, they both correct and correctly classify the same number of instances (and the same instances).

4 K2

From the graph, we can first determine that x_5 is not dependent on x_1, x_2, x_3 or x_4 . However, x_3 and x_4 depend on x_2 (from the arrows being directed from x_2 to x_3 and x_4). Likewise, x_2 depends on x_1 .

Homework #5

CS5402 — Intro To Data Mining

Illya Starikov

Due Date: July 24th, 2018

1 DBSCAN

a) The table would look as follows:

Point	x	y	Density	Designation
A1	1	4	3	Core Point
A2	5	2	3	Core Point
A3	4	3	3	Core Point
A4	5	6	2	Border Point
A5	2	5	2	Border Point
A6	5	4	4	Core Point
A7	1	2	2	Border Point
A8	3	1	1	Noise

b) The points in the two clusters would be:

- {A1, A5, A7}
- {A2, A3, A4, A6}

The clusters are formed by first identifying the main points; this is done via calculating the ϵ , or the density of the points (i.e., how many points are around those points). After the core points have been identified, the border points are found via observing which points are close (within an ϵ value) to the core points. Finally, the remaining points are simply noise.

2 Bagging Vs Boosting

2.1 Iris

Looking at the confusion matrix of using bagging:


```

a  b  c  <-- classified as
50  0  0 |  a = Iris-setosa
  0  0 50 |  b = Iris-versicolor
  0  0 50 |  c = Iris-virginica

```

And comparing to the confusion matrix of boosting:

```

a  b  c  <-- classified as
50  0  0 |  a = Iris-setosa
  0 45  5 |  b = Iris-versicolor
  0  1 49 |  c = Iris-virginica

```

We see that boosting is far more accurate; the kappa statistic agrees, where bagging had a kappa statistic of 0.5 versus the boosting kappa statistic of 0.94.

2.2 Iris

Looking at the confusion matrix of bagging:

```

445  13 |  a = benign
   7 234 |  b = malignant

```

And comparing to the confusion matrix of the boosting:

```

a  b  <-- classified as
445 13 |  a = benign
 17 224 |  b = malignant

```

We see there is very little difference in accuracy. The kappa statistic agrees; where bagging had a kappa statistic of 0.937 versus the boosting kappa statistic of 0.9046.

3 KMeans In R

assets/problem_3.pdf

4 DBSCAN In R

assets/problem_4.pdf

5 Frames For Days

6 She's So Mean

7 Animals

Homework #6

CS5402 — Intro To Data Mining

Illya Starikov

Due Date: July 27th, 2018

1 Support Vectors

The output is as follows:

```
Setting default kernel parameters
[[1]]
  X1 X2
2  2  1
3  4  2

[1] 1
[1] -1
```

2 Bayes Network

The network looks as follows:

The output looks as follows:

```
$location
location
Good  Bad
0.6  0.4
```

```
$quality
quality
Good Normal  Bad
0.3    0.5   0.2
```

```

$cost
, , location = Good

      quality
cost   Good Normal Bad
  High  0.8      0.6 0.1
  Low   0.2      0.4 0.9

, , location = Bad

      quality
cost   Good Normal  Bad
  High  0.6      0.6 0.05
  Low   0.4      0.4 0.95

$number_of_people
, , location = Good

      cost
number_of_people High Low
      High  0.6 0.8
      Low   0.4 0.2

, , location = Bad

      cost
number_of_people High Low
      High  0.1 0.6
      Low   0.9 0.4

prediction Good Normal
  Good      0      0
  Normal    1      1
  Bad       0      0
[1] Normal Normal
Levels: Good Normal Bad

```

3 DBScan

```
Clustering.DbscanResult(  
  [1, 5, 7, 12, 13, 14, 15, 16, 17, 22, 23, 24, 25, 29, 31],  
  [1, 1, 1, 1, 2, 1, 3, 1, 1, 1, 1, 4, 5, 6, 7, 8, 9, 1, 1, 1,  
    1, 10, 11, 12, 13, 1, 1, 1, 14, 1, 15, 1],  
  [18, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

Gadfly could not produce a graph.

4 Decision Tree

```
Feature 3, Threshold 118.0  
L-> Feature 3, Threshold 96.0  
  L-> Feature 3, Threshold 65.5  
    L-> Feature 1, Threshold 1.5  
      L-> 33.9 : 1/1  
      R-> Feature 3, Threshold 57.0  
        L-> 30.4 : 1/1  
        R-> 24.4 : 1/1  
    R-> Feature 3, Threshold 92.0  
      L-> Feature 1, Threshold 1.5  
        L-> 27.3 : 1/2  
        R-> 26.0 : 1/1  
      R-> 22.8 : 2/2  
  R-> Feature 1, Threshold 3.0  
    L-> Feature 3, Threshold 107.0  
      L-> Feature 2, Threshold 5.0  
        L-> 21.5 : 1/1  
        R-> 18.1 : 1/1  
      R-> Feature 4, Threshold 4.5  
        L-> 21.4 : 2/2  
        R-> 30.4 : 1/1  
    R-> 21.0 : 2/2  
R-> Feature 3, Threshold 192.5  
  L-> Feature 2, Threshold 7.0  
    L-> Feature 4, Threshold 4.5  
      L-> 17.8 : 1/2  
      R-> 19.7 : 1/1  
    R-> Feature 1, Threshold 2.5  
      L-> Feature 3, Threshold 162.5  
        L-> 15.2 : 1/2  
        R-> 19.2 : 1/2  
      R-> 16.4 : 1/3
```

```

R-> Feature 3, Threshold 237.5
  L-> Feature 3, Threshold 222.5
    L-> 10.4 : 2/2
    R-> 14.7 : 1/1
  R-> Feature 3, Threshold 254.5
    L-> 14.3 : 1/2
    R-> Feature 3, Threshold 299.5
      L-> 15.8 : 1/1
      R-> 15.0 : 1/1

```

5 KMeans

Iters	objv	objv-change	affected
0	1.000000e+02		
1	4.880000e+01	-5.120000e+01	0
2	4.880000e+01	0.000000e+00	0

K-means converged with 2 iterations (objv = 48.800000000000001)

Assignments: [1 1 1 1 2 1 3]

Counts: [5 1 1]

MATH2100

Foundations Of Mathematics



Homework #1

Illya Starikov

Thursday, January 26th, 2017

Theorem 1. *There does not exist a level set with more than one bottom level.*

Proof. Suppose that there exists two, different levels P and Q in the level set M (where M may contain two — P and Q — or more levels). Per our definition, P and Q are bottom levels of the level set M if there exists a level x in M where x is not above P and Q . This implies P cannot be above Q and Q cannot be above P , for a level x must not be above P and Q . However, this violates Axiom 3 (which states that P is above Q or Q is above P). This has lead us to a contradiction. $\therefore$ There does not exists a level set with more than one bottom level. $\square$

PHIL3225

Business Ethics



Essay #2

Illya Starikov

Due Date: May 10th, 2017

Utilitarianism is the doctrine that actions are right if they benefit the majority. Examples of this are prevalent everyday, from voting systems to workplace ethics to criminal justice; it would intuitively makes sense that utilitarianism ethics is a great model for ethics. This is not always the case. Utilitarianism is a terrible model for ethics because it can give way to arbitrariness (even from the definition), can produce poor results, and is simply not pragmatic.

The first problem of utilitarianism is the definition of "benefit". It's easy to give a textbook definition, such as "an advantage or profit gained from something"¹, coming to the realization that what some people believe to be a benefit can take to be harm. If one is to believe that everybody could benefit from religion, one would be met by wide criticism. If one was to believe that a non-GMO diet would be beneficial, one would be wrong. One could make the case that having cellular devices to keep in contact with loved ones all the time is beneficial, but one could make the case that addiction to constant connectivity is not a good habit to have. These examples are quite simple, but they are *prevelant* in our everyday life. The task of defining what is beneficial is borderline impossible.

The next impossible task is coming to a universal definition of how to distribute the benefits. An oligarchy approach would be to give the people with the most benefit more benefit. Without getting too political, this has some problems in itself. This system is proven to produce incredible inequality. There is a hypothetical trickle down effect, where the top would redistribute the benefit to people with less — we know this to not always be the case.

A different approach could be maximizing equality — everyone gets to be on a more level playing field. This would typically mean providing benefit to

the lower class before to the higher class. This would attempt to put everyone at a more level playing field.

The last option is to simply distribute equally. If there are p people and b benefit, everyone gets $\frac{b}{p}$ however it shares some common qualities to communism, a not-particularly favored form of government. This appears to be the most fair system, but its relation to a particular form of government will always make someone not like it.

An alternative solution would a democratic vote to find the superior system. The voting system holds its own problems, but its the most reasonable solution.

Supposing these impossible problems are side-stepped, there is a subsequent problem: although we have established something is "truly" beneficial, the outcome may be less than desirable. Some examples of this might include:

- Agreeing on never lying being beneficial, yet telling the truth when it is harmful (i.e. imagine the "Does this dress make me look fat" example scaled to the masses).
- The trolley problem. Supposing "living" is purely beneficial, one would be inclined to agree that pulling the lever or pushing the larger person in front of the track is more beneficial than not.
- If overall well being is considered most beneficial, euthanasia might have to have serious consideration.

These problems are real, and do become a concern when seriously considering utilitarianism.

Last, and although utilitarianism may work well for smaller groups of people, the model completely falls apart for an fractions of the population. The problem of this model is when bringing humanity as a whole into the mix; which, unfortunately, nulls most of the cases. Large groups of people are, in a large part, selfish, unsystematic, arbitrary and short-sighted. Not only is decisions making often misguided and irrational (i.e. voting to leave the European Union before knowing what it is²), it is not always based on logic or reasoning. The idealized model may work fine, but would never get mass approval.

There are several solutions to this, one of which is to have a representative for your views. This has intrinsic problem of your representative not always having your interest at heart, but its a more feasible solution. If the

representative always has the *majority* in mind, then it is already a better system than having a democracy of the people.

Another alternative would be to have an idealistic “perfect” person make a decision for the majority. Assuming this person always has the majority’s interest at heart, then this system could hold. However, it closely resembles a dictatorship, a seemingly ostracized form of government.

As stated in the thesis, utilitarianism makes a good heuristic for decision making, it does not live past that (that it, being a heuristic). The definition of utilitarianism is much too vague, and gives way to too much ambiguity when actually using it. Two of the ambiguities is defining what is benefit, and how to distribute it. If this problem was side-stepped, it is not always clear whether one would like the benefit. Lastly, the problem does not scale properly. Larger than small groups of people create massive problems. There are two solutions to this: a republic and a dictatorship.

Notes

¹<http://www.dictionary.com/browse/benefit>

²<http://fortune.com/2016/06/24/brexit-google-trends/>

Essay #1

Illya Starikov

Due Date: February 10th, 2017

As of the time of this writing, there are currently approximately 30,000¹ deaths in the United States. To put said number in perspective, one is more likely to be in a fatal vehicular accident than be a victim of homicide², overdose from heroine³, or be injured from an intentional fire⁴. Even worse, those three statistics combined cause more deaths, per year, than a vehicle does. These deaths aren't at the fault of mechanical error, animal intervention, or even dangerous weather conditions: they are caused by humans. Approximately 94% of traffic accidents are *primarily caused by humans*⁵. This begs the question, how much safer can an algorithmic, non-self aware autonomous vehicle be, compared to a human?

Because self driving cars are in their infancy (with Carnegie Mellon having the first record of self-driving technology dating back to 1984⁶), it would seem to be almost unfair to compare such a computer to a human driver; or so it would seem. Although humans had a century head start, car projects have *already* proven to be sufficiently better drivers. Google's well known self-driving car project (and their PR department) can attest to such a claim:

“We just got rear-ended again yesterday while stopped at a stop-light in Mountain View. That's two incidents just in the last week where a driver rear-ended us while we were completely stopped at a light! So that brings the tally to 13 minor fender-benders in more than 1.8 million miles of autonomous and manual driving — and still, not once was the self-driving car the cause of the accident.” Jacquelyn Miller, Google Spokeswoman⁷

Google is not the only company who is interested; Tesla's autopilot software is expected to be fully autonomous (and commercially available)

come 2017. Along with Google and Tesla, BMW, Mercedes Benz, and Ford have made public claims to be working on self driving car capabilities. There is no dispute; self driving cars will be radically safer in comparison to human drivers, and they will be available soon. Assuming the standard S-curve technological adoption, price to significantly drop, and luddite-esque apposition, the adoption should be mainstream within fifty years at most. Now, what about a hundred years?

Within a hundred years, self driving cars should be illegal; this is not a moral dilemma, this is a “saving over 3000 lives every day”⁸ solution. Every time a person gets in a car and chooses to drive and causes an accident, that person chose to cause the accidents. It would come to no surprise that people from the 22nd century that would look at the 21st century driving as “barbaric”. When strictly looking at the statistics and the logical arguments, driving in the 22nd century would not only be morally impermissible, it would be illegal.

Notes

¹<https://www-fars.nhtsa.dot.gov/Main/index.aspx>

²<https://www-fars.nhtsa.dot.gov/Main/index.aspx>

³<https://www.cdc.gov/drugoverdose/data/heroin.html>

⁴<http://www.nfpa.org/news-and-research/fire-statistics-and-reports/fire-statistics/fire-causes/arson-and-juvenile-firesetting/intentional-fires>

⁵<https://crashstats.nhtsa.dot.gov/Api/Public/ViewPublication/812115>

⁶https://engineering.cmu.edu/alumni/magazine/2015_spring/self-driving_cars.html

⁷<http://bigthink.com/ideafeed/googles-self-driving-car-is-ridiculously-safe>

⁸<http://asirt.org/initiatives/informing-road-users/road-safety-facts/road-crash-statistics>

Essay #2

Illya Starikov

Due Date: February 23rd, 2017

Utilitarianism is the doctrine that actions are right if they benefit the majority. Examples of this are prevalent everyday, from voting systems to workplace ethics to criminal justice; it would intuitively make sense that utilitarianism ethics is a great model for ethics. This is not the case. Utilitarianism is a terrible model for ethics because it can give way to arbitrariness, produce poor results, and is simply not pragmatic.

The first problem of utilitarianism is the definition of "benefit". It's easy to give a textbook definition, such as "an advantage or profit gained from something"¹, coming to the realization that what some people believe to be a benefit can take to be harm. If one is to believe that everybody could benefit from religion, one would be met by wide criticism. If one was to believe that a non-GMO diet would be beneficial, one would be wrong. One could make the case that having cellular devices to keep in contact with loved ones all the time is beneficial, but one could make the case that addiction to constant connectivity is not a good habit to have. These examples are quite simple examples, but they are *prevelant* in our everyday life.

Supposing this problem is side-stepped, there is a subsequent problem: although we have established something is "truly" beneficial, the outcome may be less than desirable. Some examples of this might include:

- Agreeing on never lying being beneficial, yet telling the truth when it is harmful (i.e. imagine the "Does this dress make me look fat" example scaled to the masses).
- The trolley problem. Supposing "living" is purely beneficial, one would be inclined to agree that pulling the lever or pushing the larger person in front of the track is more beneficial than not.

- If overall well being is considered most beneficial, euthanasia might have to have serious consideration.

These problems are real, and do become a concern when seriously considering utilitarianism.

Last, and although utilitarianism may work well for smaller groups of people, the model completely falls apart for an fractions of the population. The problem of this model is when bringing humanity as a whole into the mix; which, unfortunately, nulls most of the cases. Large groups of people are, in a large part, selfish, unsystematic, arbitrary and short-sighted. Not only is decisions making often misguided and irrational (i.e. voting to leave the European Union before knowing what it is²), it is not always based on logic or reasoning. The idealized model may work fine, but would never get mass approval.

As stated in the thesis, utilitarianism makes a good heuristic for decision making, it does not live past that (that it, being a heuristic). A working definition of being truly "beneficial" is impossible to put into words, being purely "beneficial" might not produce fruitful results, and masses of people would never abide utilitarianism. It makes for a great thought experiment, but the pragmatism simply isn't there.

Notes

¹<http://www.dictionary.com/browse/benefit>

²<http://fortune.com/2016/06/24/brexit-google-trends/>

Essay #3

Illya Starikov

Due Date: March 20th, 2017

Very often when making a decision, one might hear advice such as the following:

- “Go with your heart”
- “Go with your gut”
- “When you need to make a hard decision, flip a coin. Why? Because when that coin is in the air you suddenly know what your hoping for.”

This advice is awful; simply because it forces you to rely one thing: emotion. Emotions should never be considered in decision making. As result, they should not be used in ethical decision making; simply because emotions are subjective, emotions are inconsistent, and there are much better systems and frameworks.

The first problem about including emotions in ethics is incredible subjectivity that goes into it. People just simply feel a certain way, yet they can never explain it. This is a biproduct of so much influence from surroundings (whether it be from people, media, or simply culture). Whether one believes in nature vs. nurture, one cannot deny that certain influences effect the way a person feels.

A famous example of this is the Trolley problem; or, rather, a modified version of it.

Suppose there is a runaway train heading towards an unenviable collisions with five people (who are tied to the tracks). However, there is a switch you, the observed, can pull to save the five people at the cost of one person.

When asked this question, roughly 90% of participants ¹ stated they would pull the lever, holding a utilitarianism viewpoint. However, when posed with a twist:

The one person you must kill to save five is your child, parent, or sibling.

The number shot down to 33%, nearly two-thirds less. The best guess is humans are wired to be emotionally selfish, from an evolutionary standpoint. It made sense in more primitive, hunter-gathered times; but this has an impact on us today.

Applying this to business ethics, humans will worry about self-interest first, and ethical implications will be an afterthought. Take the case of the ²Ford Pinto, ³Enron, or, a more modern scandal, ⁴Samsung's battery recall. Emotion is rarely based on logic or reasoning, which ties to the second reason emotions should never be included in ethical decisions: systematic reasoning.

Emotions are rarely consistent; one might feel a particular way about a subject today, but an event might change their view on the subject later. Some great examples of this are religion, patriotism, and political beliefs. It is quite common to hear of "Born again Christians" or disavowed Christians, those who would die for their country become less steadfast, or people transitions from conservatism to a more liberal viewpoint. Those long-held beliefs usually take years to change; the trouble lies when it is a shorter timespan.

The real danger is emotional inconsistency is not always on a yearly timescale, but often daily or hourly. A simple "bad day" is enough to make one feel less empathy. One friend's remark might sway your feelings one way or another. Simple entanglement of other feelings would be enough to change one's mind about something; all of this is dangerous.

The solutions to all these problems is simple, do not allow emotions into ethical decisions. There are much better systems to use for ethical decision making that are well structured, unambitious, and not subjective.

Emotions are should never be allowed to influence a decisions in regards to ethics. Emotions are incredibly subjective. They do not always reflect what one is thinking at that point in time. The inconsistency of emotions make it difficult to systematize them. In short: they are *much* better alternatives to using emotions in decisions making.

Notes

¹<http://healthland.time.com/2011/12/05/would-you-kill-one-person-to-save-five-new-research-on-a-classic-debate/>

²<https://philosophia.uncg.edu/phi361-metivier/module-2-why-does-business-need-ethics/case-the-ford-pinto/>

³<http://www.investopedia.com/updates/enron-scandal-summary/>

⁴<http://fortune.com/2016/12/28/biggest-corporate-scandals-2016/>

Essay #4

Illya Starikov

Due Date: April 21st, 2017

In summary, the Ultimatum game is thus:

Suppose you are approached by host of a game, name him Joel, and asks if you would like to play. Reluctantly, you agree. Joel hands you \$100 in \$1 bills and says that you are to make an offer to another player, who has just joined both of you. You keep what you don't offer if and only if the person accepts your offer; otherwise, you both go home empty-handed.

The question is:

- What would you offer, from a *rational* point of view? Why?
- What would you offer, from a *moral* point of view? Why?
- If different, why different?

From a *rational* point of view, the obvious answers would typically be in fractions/wholes of the money: give away 50\$, 25\$, or \$0. However, from a personal standpoint, I would take a much more radical standpoint: give away \$100 with the condition that the other person knows **that is all the money to give**.

By giving the full \$100, it may appear I have side-stepped the argument. No negotiation is made, the game is effectively irrelevant to me — I have instantaneously gained and lost \$100, as if I never had it from the start. However, this is not taking into consideration the other participant.

To the other participant, I appear giving and caring (whether those are self-reflective traits is up to you), even though **I have done nothing**. By

acting as a middle man in this transaction, I have swooned the third party. Now, this is not a random act of kindness or “returning the karma”. I could pull \$100 out of my bank and hand it to the person next to me as I am writing this paper, and it would never have the same effect as it would in this situation.

In regards to what the offer would be from a *moral* standpoint, I would offer a fair \$50 split. Surely there are more implications from taking advantage of such a way, like:

- Utilitarianism would say maximizing the benefit between the people would be the two participants gaining an equal split (unless we were in radically different social classes).
- Justice and fairness ethics would say split the money evenly.
- The general deceitfulness of the situation (on my part) might raise some ethical concerns.

There are only two options for why the numbers are radically different: *I'm immoral* or *pure morality is not a great framework for this situation* — personally, I believe it a mixture of both. Selfishly, I try to maximize my own benefit for the situation (as most people would). Also, this is a test designed to show the distinct lines of morality and ethics (such as a trolley vs transplant problem).

PHILOS 3235: Test #1

Due on Wednesday, April 5th, 2017

Dittmer 15:00

Illya Starikov

Monday, April 3rd, 2017

Problem I

Characterize the Trolley vs. Transplant Problem. Explain how some people do not face the Trolley vs. Transplant Problem. What do you take to be the most important implication from this problem?

The Trolley vs. Transplant Problem is the juxtaposition of two problems, the Trolley problem:

Suppose you are standing on train tracks. You notice on one side of the tracks, there is an oncoming trolley. On the other side, there is a fork in the tracks. On one side, there is a person tied to the tracks; on the other, five people are tied. The tracks are oriented towards the five people; however, there is a lever to divert the tracks. Do you save five at the expense of one?

And the Transplant problem:

Suppose you are an outstanding surgeon working in the top medical facility in your area. During a shift, you have five subsequent patients needing transplants within the hour (with their life in jeopardy). In a stroke of luck, a healthy patient walks in for a checkup that has all the organs necessary. Do you save five at the expense of one?

The last sentence in both cases are identical because the problems are identical; however, the vast majority say “yes” to Trolley *while saying no “no” to Transplant*. This is the Trolley vs. Transplant.

Some people do not face this problem because they break the problem into its fundamentals: save five at the expense of one. First principles thinking helps to avoid falling in this ambiguity trap.

The most important implication from this is showcasing the enigmatic decision-making process a person might go through. When given a straightforward prompt:

Save five at the expense of one.

The answer is always utilitarian: “yes”. Muddled with a semi-plausible scenario; the answer is a resounding “no”. This is humans inability to think from first principles.

Problem II

Define utilitarianism. Characterize two problems with/objections to utilitarianism. Attempt a reply to just one of the two problems/objections.

Utilitarianism can be defined as follows:

The doctrine that actions are right if they benefit the majority.

It may seem a suitable framework for ethics; however, once one starts dissecting it, very visible holes are evident. Two of the biggest holes are:

1. Defining what benefit entails.
2. Defining how to distribute the benefit.

The Oxford-style definition of benefit “An advantage or profit gained from something”¹ may work in most cases, but it leaves room open for ambiguity. One’s benefit can be defined as other’s hindrance.

Even if a valid definition of “benefit”, with all subsequent consequences calculated, was created, one might not like the outcome. In a purely utilitarian framework, the work of computer scientist can be done remote while engineers would be obliged to work in third world countries. That much accountability can be quite taxing.

The second problem is how to distribute the benefit. Oligarchy utilitarianism would be to give the people with the most benefit more benefit. There is also the apposing side; where the people with the lowest amount of benefit would get the benefit.

A different approach could be maximizing equality — whereas the benefit is distributed evenly amongst all participants. This seems reasonable; however it shares some common qualities to communism, a seemly ostracized form of government.

To counter this point, one could simply assemble all that would be involved, and hold a democratic vote to see which system they would favor — whether it be communism or an oligarchy. The voting system holds its own problems (i.e., Brexit), but its the most reasonable solution.

¹<https://en.oxforddictionaries.com/definition/benefit>

Problem III

Characterize the Prisoner's Dilemma. Why is it a dilemma? Discuss some application of the dilemma.

The Prisoner's Dilemma can be characterized as so:

Suppose Bonnie and Clyde are detained by the police for a burglary without enough evidence to convict either of them. Knowing they need a confession, the police place Bonnie and Clyde into separate rooms and propose the following to both of them:

You can incriminate your partner, and you will have *zero years* in prison; however, your partner will have *three years*. This also works in reverse; if your partner incriminates you, you will have *three years* while your partner will have *zero years*. If both of you incriminate each other, both of you will have *two years* each. If neither of you incriminate each other you both will have *one year* each.

The point where this becomes a dilemma apposed to problem is as follows: collectively, it benefits to cooperate (i.e., not to incriminate the partner), as described in Table 1. Looking at it from one person's perspective, however, it always benefits to defect (i.e., incriminate your partner). If your partner defects, you will get two years; if not, there will be zero years. The barrier of not knowing your partner's response turns this into a dilemma.

Table 1: Collective benefit from prisoner's dilemma.

	👤 (Cooperate)	👤 (Defect)
👤 (Cooperate)	2 Years Total	3 Years Total
👤 (Defect)	3 Years Total	4 Years Total

One application of this is politics; specifically, military. Suppose two apposing nations both heavily invest in their military (i.e., nuclear arsenals, army, weapons), out of fear of the other nation doing the same. That's hundreds of billions² that could have gone to:

- Education
- Economy
- Infrastructure

and so forth. Other applications could include media (i.e., two apposing companies advertising at the cost of their own), business (i.e., companies lowering margins at the expense of profits), and stock market (i.e., overvaluing at the expense of future shares).

²In 2017 currency.

PHILOS 3235: Test 2

Due on Wednesday, May 10th, 2017

Dittmer 15:00

Illya Starikov

Monday, April 3rd, 2017

Problem I

Utilitarianism states that actions are right just in case they produce the most over-all (available) goodness. Discuss some ways in which according to Kantian ethics, utilitarianism is false.

Utilitarianism and Kantian ethics seem to have a large overlap. After all, utilitarianism is the doctrine that actions are right if they benefit the majority and Kantian Ethics reflect much of this thinking. However, when analyzing the two frameworks of ethics, there are areas where they differ. The biggest area of difference is in the first Categorical Imperative.

The first Categorical Imperative states that

Act only according to that maxim by which you can at the same time will that it should become a universal law.

This may seem like a perfect utilitarian definition; until one realizes that “universal” is not the same as “maximum benefit”. To show how this juxtaposition is true, let us take two examples.

One of the most widely known examples of a wrong in Kantian Ethics is suicide. If everyone committed suicide (making it universal), the human-centric ethics would fall apart. However, we *know* there to be situation where self-sacrifice is the correct in a utilitarian framework. Consider the transplant problem.

Suppose you are an outstanding surgeon working in the top medical facility in your area. During a shift, you have five subsequent patients needing transplants within the hour (with their life in jeopardy). In a stroke of luck, a healthy patient walks in for a checkup that has all the organs necessary. Do you save five at the expense of one?

The one patient is **obligated** to sacrifice himself to save five other patients. This is not the case in a Kantian framework.

Take another example: a situation where you are forced to lie. Imagining back to Nazi Germany in WWII, no sane human being would admit to harboring a fugitive Jew. In a utilitarian framework, not getting the innocent person killed would definitely be considered the maximum benefit. However, in pure Kantian Ethics, lying is very much breaking the first Categorical Imperative.

Admittedly, there is a lot of overlap in Utilitarian and Kantian; most of which is at the intersection of maximum benefit and universal benefit. However, from the point of maximum (whether it be at the 51% mark or on) to the point of universal (which has to lie at the 100% mark), there is a tangible difference.

Problem II

Utilitarianism states that actions are right just in case they produce the most over-all (available) goodness. Discuss some ways in which according to Kantian ethics, utilitarianism is false.

Upon making decisions, the goal should always be to set create a policy that everyone can agree to. This goal cannot scale, of course, but it should give a general heuristic for how to gauge the conversation. For every situation, there can be a *hypothetical* common ground that everyone can agree to.

This may seem impossible, seeing how everyday situations do not reflect this thinking. However, when considering everyday situations, we are often blindsided by a different way of thinking: the popular vote. When trying to agree on something, we fight for the maximum benefit instead of the most benefit for the group. To achieve a common policy, one must take a more utilitarian point of view.

Take a common, everyday example of a deciding where and what to eat. This monumental decision seemingly never is able to get settled. Instead of the group all blurting what they would like, set up a list of food, and have everyone vote on what they *would be willing* to eat. While the popular vote would leave people unhappy, this system allows for everyone to be satisfied.

Of course, scaling this is very difficult. Having a common ground for the entirety of the United States on foreign policy is borderline impossible because of sheer scale. But on a smaller scale, the framework is feasible.

The main limitation of this framework is the “the weakest link” principle. Almost always, it always appears to be a Pareto principle, where 80% of the people can agree on most things while 20% always disagree. Being at the mercy of the 20% or less often makes it very difficult to come to a common policy.

In summary, it is technically feasible to come to a common policy. By taking a more utilitarian approach, and not being selfish and trying to meet your own goals, a common ground can be established quite easily. However, the difficulty lies in the small subset that prohibit progress by disagreeing with the established policy.

Problem III

Visit `moralmachine.mit.edu`. Take “the test”. Characterize two of the (thirteen) scenarios presented to you, and what you chose in those scenarios. Then explain why you made those choices.

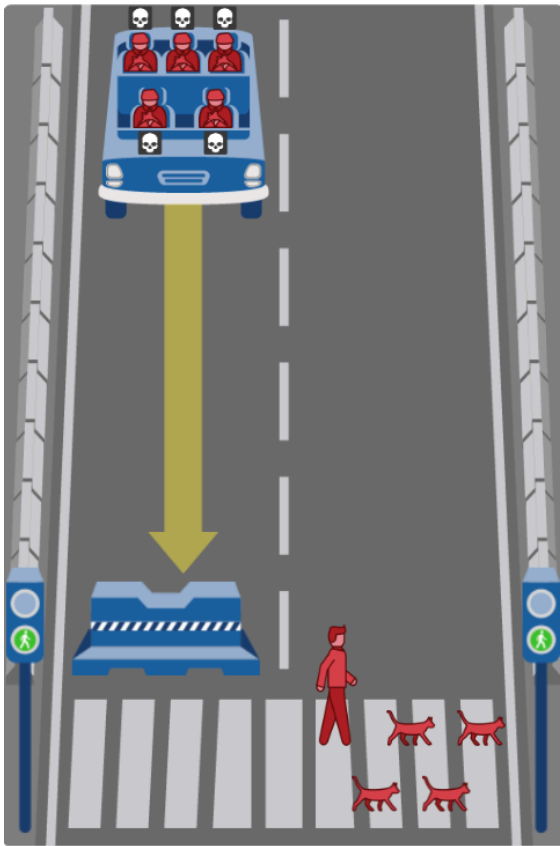
When considering the moral issues of the self driving car, it should always be made clear: *The car has no “mind” of its own.* Many people fall into the trap of empathizing with the self driving car, not realizing it’s a two ton metal machine with algorithmic decision making. When considering the trolley-esque problem of selfing driving cars deciding what/who to impact, it is not the car deciding — the programmer is deciding. So it always better to empathize with the programmer, instead of the car. Unfortunately, this is not always the case. For the arguments presented here, the approach will be completely from the perspective of the programmer.

For the first scenario, described in Figure ??, I believe the scenario that should be chosen is the straight into the concrete barrier. This may seem counterintuitive, favoring five human lives over 1 human life and 4 dead animals. However, remember the empathy is with the programmer. The best heuristic is often the simplest one; in this case, the simpler heuristic is going straight into a barricade.

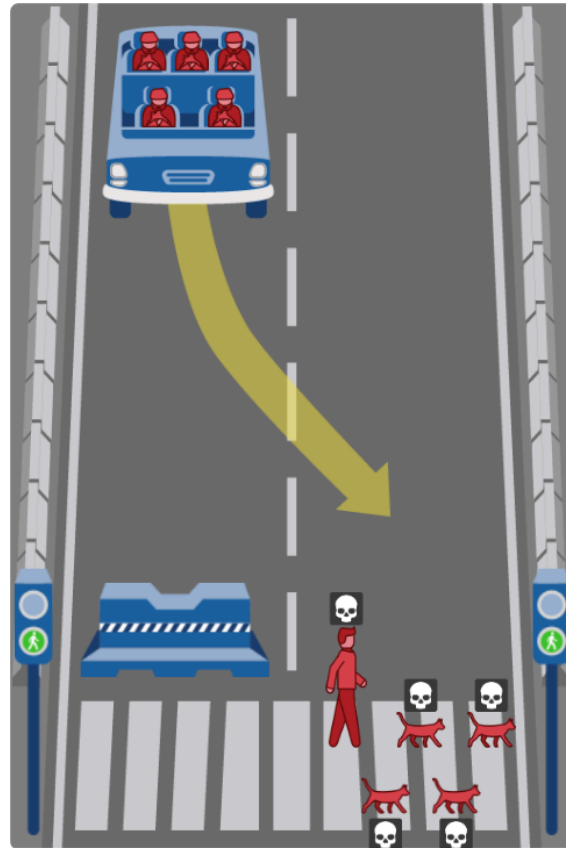
To further expand, this heuristic *should not* always be used. This heuristic should be used in situations where there are a *comparable* number of passengers and pedestrians.

For the second scenario, described in Figure ??, I believe the scenario that should be chosen is to swerve — contradicting my first decision! This is simply because this is no longer a pedestrian vs. passenger situation. It is a pedestrian vs. pedestrian situation. This alternative situation calls for a different heuristic, because it a completely different situation.

From the perspective of the programmer, both of these situations are a good trade off between simplicity and favorable for the passengers/pedestrians — and this is something the consumer should want; even though it is not always what the consumer *does* want.

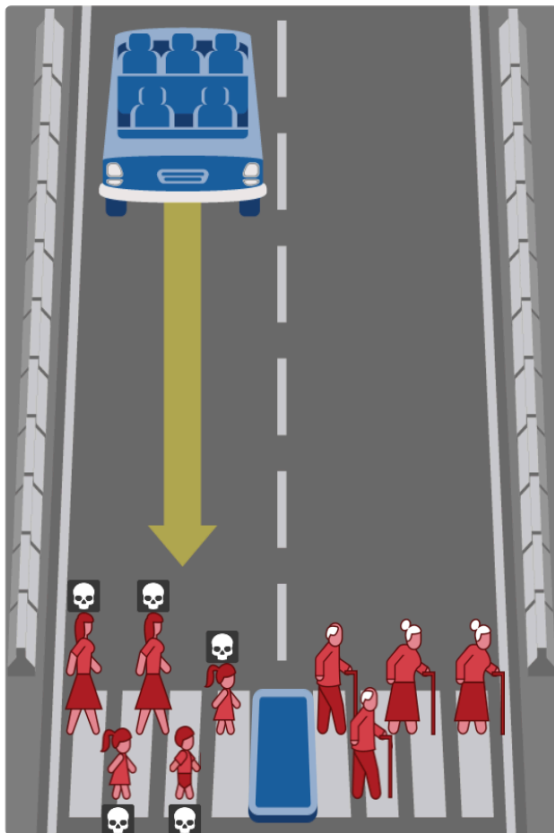


(a) In this case, the self-driving car with sudden brake failure will continue ahead and crash into a concrete barrier. *This will result in 5 dead homeless people.*

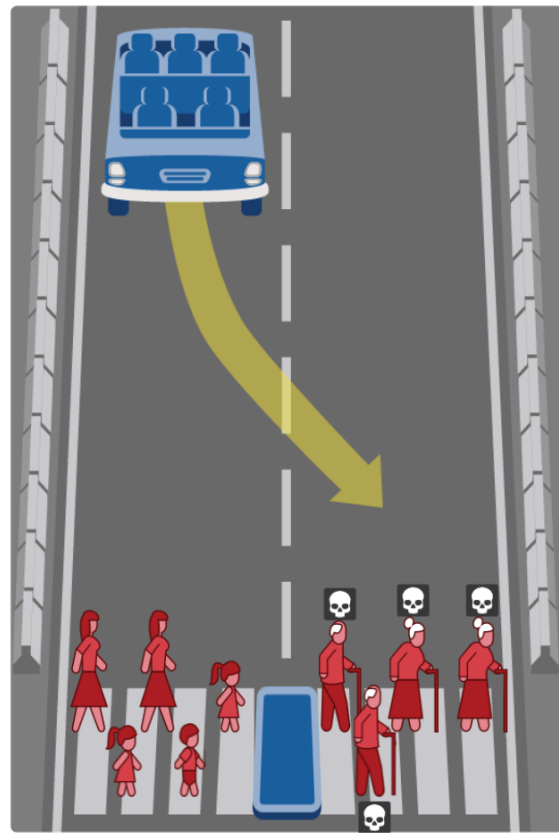


(b) In this case, the self-driving car with sudden brake failure will swerve and drive through a pedestrian crossing in the other lane. *This will result in 1 dead man, 4 dead cats.*

Figure 1: First scenario.



(a) In this case, the self-driving car with sudden brake failure will continue ahead and drive through a pedestrian crossing ahead. *This will result in 2 women, 2 girls, 1 boy dead.*



(b) In this case, the self-driving car with sudden brake failure will swerve and drive through a pedestrian crossing in the other lane. *This will result in 2 elderly men, 2 elderly women.*

Figure 2: Second scenarios

PSYC1101

Psychology 101

S&TTM

Illya Starikov

Professor Amanda Burch

General Psychology

May 5, 2026

Sleep States and Memory Processes In Humans: Procedural Versus Declarative Memory Systems

“Sleep on it”. This single cliché has permeated society for quite a long time. Although when applied strictly to the context it is usually said (typically when faced with a difficult decision), it does not have merit. Sleep does not help one make a decision no more than a physics textbook helps make you a theoretical physicist; however, just as the book can be a *tool* to make one a physicist, sleep can be a *tool* used in the decision making process. We will try to unravel this in this paper.

The relationship for finding correlation between sleep and learning/memory had been, for a long time, something like this:

“There was substantial evidence from animal studies to support the idea that post-training REM sleep was important for memory consolidation. The human studies, however, provided mixed results. The human studies cited as showing no relationship between sleep and memory utilized simple memorization tasks.”

These findings would later come to be proven false. The main reason for the discrepancies came from a naive understanding of how memory worked:

“These included a very short registration phase taking less than 1 s to occur followed by a period (minutes to hours) after training, during which learned material existed as relatively labile short-term memory that was vulnerable to various agents such as

strong drugs or electroconvulsive therapy (ECT). As the minutes turned to hours and memory processing continued, the learned material was considered less vulnerable to disruption. After 24 h (or even less), memory formation was considered complete and, thus, no longer vulnerable to disruption. At this point a stable, permanent long-term memory was considered to have been formed. This process has been called consolidation and most workers still use the concept of a permanent “memory trace” existing in the brain”

This poses a problem when testing two different types of learning: “declarative” and “procedural” knowledge. Declarative knowledge “refers to memories accessible to conscious recollection”, while procedural “are memories of how to do some skill or how to solve a problem”. This is the discrepancy: psychological studies have always assumed that memory was a single, colossal process; however, we know this not to be the case.

There have been four types of studies performed to study the relationship between sleep and memory: “One approach has been to train subjects on some task and then to record the changes in sleep parameters”, “deprive the subject of total sleep or of some specific phase or aspect of sleep following task acquisition in order to induce possible memory impairments”, and finally “A third approach has been to have the subject learn the task either just before bed or during the night at some particular point”. These four types have been carried out in regards to declarative material, and the results were, as aforementioned, strongly positive. It had been clearly shown that “after learning a PA [paired associates] task showed superior memory to participants that stayed awake for an equal amount of time”. The research did not stop there — further research showed that total and even *selective* sleep deprivation was harmful; so much so that you

could not tell the selectively sleep deprived and totally sleep deprived apart.

Switching to procedural, patients that underwent study for procedural knowledge were “to engage in cognitive activity that was more complex than simple recall or recognition of previously memorized material. These studies demonstrated a relationship between REM sleep and memory.” Again, the studies were overwhelming positive:

“Mandai. . . found an increase in REM sleep duration and number of REM episodes in subjects who underwent 90min training in translation of Morse code. They also reported a high correlation between retention, number of REMs and REM density.”

And this was not limited to just partial sleep deprivation, but also to total and selective sleep deprivation as well. Those who experienced any sleep loss with 48-72 hours also were susceptible to memory loss related to procedural tasks. At this point, there is no denying the relationship between sleep and memory recall; as stated, “The level of recall was positively correlated with the average duration of the NREM-REM sleep cycles”.

This all comes back to the “Sleep on it” statement made previously — although you cannot use sleep a sole deducer for your choices; sleep can make you more effectively a decision maker by allowing you to more quickly recall declarative and procedural knowledge that might be relevant to the question.

I do not think this article is that controversial; humans needs sleep to recharge, reboot, and restore our energy. The article also did not fall into the trap of a “catchall” for sleep, stating that a person needs x-hours of sleep to full function. I also agree with the statements being made; it is quite intuitive that a person will need sleep to recall declarative and procedural knowledge.

Steven Pinker: The Language of Cognition

Illya Starikov
Professor Amanda Burch
General Psychology
May 5, 2026

Illya Starikov

Professor Amanda Burch

General Psychology

May 5, 2026

Steven Pinker: The Language of Cognition

Steven Pinker is one of the most influential psychologists of our time. Aside from being named Time's 100 most influential people in 2004 and winning Foreign Policy's 100 top public intellectuals in 2004 and 2008, he has won awards from the National Academy of Sciences (which he was elected to in 2016), the Royal Institution, American Psychological Association, the American Humanist Association, and the Cognitive Neuroscience Society. Although his work is primarily in linguistics and its relation to psychology, he has also does work work in cognitive science and visual cognition. From his research he has published ten books; some which include *The Language Instinct*, *How the Mind Works*, *The Blank Slate*, *The Stuff of Thought* and, more recently, *The Better Angels of Our Nature*. Before diving into his formal research, it might be more helpful to look at his earlier years and their influence on his later life.

Steven Pinker was born in Montreal, Quebec, in 1954 to a Jewish family. Although his parents were devoted Jewish, he proudly renounced the faith, a sentiment that is found to this day:

““The Bible is a manual for rape, genocide, and the destruction of families...Religion has given us stonings, witch burnings, crusades, Inquisitions, jihads, fatwas, suicide bombers...and mothers who drown their children in the river,” he said.”

This rejection of religion ultimately lead him down a life of science; specifically, psychology.

After graduating with a bachelor's degree from McGill University in 1976, he later finished his doctorate only three years later from Harvard. Before finally settling as the Johnstone Family Professor in the Department of Psychology at Harvard, where he currently works, he bounced around between Stanford, MIT, University of California, Santa Barbara and Harvard. The attendance of these prestigious colleges was not without merit, of course; Steven Pinker had accomplished quite a bit at this time.

Pinker's most notable work is not just the work of the mind, but of his enthusiasm for language. This fascination came to him in graduate school, which he decided to pursue in his research. Roughly during this time, he had published two books. His first book, *Language Learnability and Language Development*, was regarded as "A fiercely reasoned, bently written landmark of psychological science.". As hinted by the name, the book sheds light on the problem of language learning; in particular, language acquisition in children. In the book, he gives a brief summary on the matter:

"The core assumption of the theory is that children are innately equipped with algorithms designed to acquire the grammatical rules and lexical entries of a human language. The algorithms are triggered at first by the meanings of the words in the input sentences and knowledge of what their referents are doing, gleaned from the context. Their outputs, the first grammatical rules, are used to help analyze subsequent inputs and to trigger other learning algorithms, which come in sets tailored to the major components of language. Empirically, children do seem to show the rapid but piecemeal acquisition of adultlike rules predicted by the theory, with occasional systematic errors that betray partly acquired rules." (Pinker, 2)

As such, he has spent many years of his later years still studying the lexical structure of language. However, he had another passion, which he touched on in his second book, *Visual Cognition: Computational Models of Cognition and Perception*.

Visual Cognition addresses some of the more difficult question of human visualization, such as:

“How do we recognize objects? How do we reason about objects when they are absent and only in memory? How do we conceptualize the three dimensions of space? Do different people do these things in different ways? And where are these abilities located in the brain?”

Throughout the remainder of the book, Pinker goes about answer these questions through past studies, modeling of cognitive processes, and new experimental techniques.

Although his main claim to fame was progressive studies in these two fields, visual cognition and language, he has done additional studies regarding violence, written additional books (regarding writing, violence, humanitarian efforts, and the human mind), advocated for science based thinking, participated in public debates, and occasionally done interviews with notable online celebrities.

In general, I do agree with Steven Pinker. His work for language, in my limited exposure to it, has always been intuitive and downright spot on. In one of his Ted Talks, *What our language habits reveal*, a particular statement stands out:

I think the key idea is that language is a way of negotiating relationships, and human relationships fall into a number of types. There’s an influential taxonomy by the anthropologist Alan Fiske, in which relationships can be categorized, more or less,

into communality, which works on the principle “what’s mine is thine, what’s thine is mine,” the kind of mindset that operates within a family, for example; dominance, whose principle is “don’t mess with me;” reciprocity, “you scratch my back, I’ll scratch yours;” and sexuality, in the immortal words of Cole Porter, “Let’s do it.”

This gives a more modest expression of language — instead of an overly-romanticized art, language can be a negotiation, among many things. And Pinker continues through the talk discussing these strictly-scientific and psychological definitions of language and their effects on our daily lives.

Although I do agree with his work, I do not particularly see his work around me; even if it is ever-present. Language does give a direct method of communication that I may not particularly give attention to, but it does affect my daily life.

Works Cited

- Pinker, Steven A. "About." Steven Pinker. Harvard, n.d. Web. 28 June 2016.
- Pinker, Steven A. "Language Learnability and Language Development." Steven Pinker. Steven Pinker, 17 Aug. 2015. Web. 28 June 2016.
- Pinker, Steven A. "Visual Cognition: Computational Models of Cognition and Perception." Steven Pinker. Harvard, 08 Aug. 2015. Web. 28 June 2016.
- Pinker, Steven. Language Learnability and Language Development. Cambridge, MA: Harvard UP, 1984. Print.
- Pinker, Steven. "Transcript of "What Our Language Habits Reveal"" Steven Pinker: What Our Language Habits Reveal. TED, Sept. 2007. Web. 28 June 2016.

Illya Starikov

Professor Amanda Burch

General Psychology

May 5, 2026

Quantum Cognition: A New Theoretical Approach to Psychology

For decades, judgments and human cognition has been preceded on two different concepts — heuristic and rational. The heuristic approach “posits that, to make judgments and decisions, people tend to employ simple heuristics (e.g., representativeness, anchoring-and-adjustment, take-the-best)”, while the rational approach “posits that people can derive inferences from the Bayes rule and decisions from the expected utility rule in a rational manner”. These two methods have been proven an intuitive model; however, a third approach has emerged: quantum cognition. This framework for cognition assumes that humans abide basic axioms from probability theory and quantum physics. However, these axioms can be proven to be less intuitive than their former counterparts (heuristic and rational).

The theory is quite younger than quantum physics (quantum cognition being 20 years while quantum physics being over 100 years old), but that time discrepancy has given researches the ability to draw “inspiration from both the mathematical structure of quantum theory and its associated dynamic principles.”

As for how quantum physics relates to cognition, less us first look at an example of the disparity between the logic of quantum physics and the logic of boolean algebra. The statement A and B (more formally, $A \cap B$ denotes A and B , and $P(A \cap B)$ denotes *the probability* of A and B) in classical boolean algebra has the commutative property, meaning one can write $P(A \cap B)$ or $P(B \cap A)$ and there would be no difference. This is not the case in quantum physics, nor is it

in cognition. In quantum physics, the ordering does matter; as it does for human cognition.

Occasionally, the $\cup$ can be replaced with *before* (i.e. *A before B*).

To further clarify this, let us take two examples. First, suppose a student (name him Illya) turns in two papers, paper *A* and paper *B*. In paper *A*, he does an excellent job, citing the paper, pulling examples from the paper, etc. The second paper, Illya does well, but not nearly to the level of excellence as the first. In this case, the order matters: if his first paper (paper *A*) was extraordinary, there will be that bias for the second paper (paper *B*) to be done. In this instance, the probability of — $P(\dots)$ — doing well is not commutative; $P(A \cup B) \neq P(B \cup A)$, the probability of doing well on paper *A* and paper *B* are not quite the same as doing well on paper *B* and *A*. However, this is not always the case; for example we can contemplate whether to fail Illya on the second paper while we contemplate what is for dinner, as we often might do. This provides a simple introduction to the principle of complementarity, which “leads directly to the best-known principle of quantum theory — the uncertainty principle.”

“The [uncertainty] principle holds that when we are certain about a quantum particle’s position, we are necessarily uncertain about its momentum, and vice versa.” This is important in the context of incompatible events (events that can’t be considered at the same time, such as Illya’s papers). When one has two events dependent on the order, they can’t think of them, therefore they can’t think of them at the same time; just like the uncertainty of a particle’s position and momentum.

Now that we have this background knowledge, we shall consider another quantum physics approach to understanding the brain itself: vector spaces. A vector space, in mathematics, is a space of vectors; and a vector is an object with direction and magnitude. This might seem abstract, but “it is analogous to the distributed input across nodes in a connectionist neural

network”. With this vector space, and a projector (something that maps a space to a subspace), this can “provide algorithm-level predictions for this more complex neural implementation.”

However, these are all just new applications of quantum theory to cognition — we can “highlight the expressive power of quantum models by pointing out that they have already been applied to a broad range of cognitive phenomena, including perception, memory, conceptual combinations, attitudes, probability judgments, causal reasoning, decision making, and strategic games”. This is still a maturing field, as we still do not quite fully understand all of quantum physics, but applications are being found and used everyday.

In my personal opinion, I believe the article is controversial. If psychology does a paradigm shift where quantum physics is the framework to understand human cognition, this particular field might not be quite as fruitful. Can you imagine a third-year psychology major walking into class and seeing $-\frac{\hbar^2}{2m} \frac{d^2\psi}{dx^2} + V\psi = E\psi$ (time-independent Schrödinger’s equation) for the first time? Quantum cognition is based on quantum mechanics, and quantum mechanics is based on heavy math (differential equations, complex linear algebra, multi-dimensional calculus) and every kind of physics imaginable (Newtonian mechanics, electromagnetism, theoretical, etc.). Even taking the most primitive example of physics in this article, the uncertainty principle is understood through analogy, not physics. The moving parts of this problem are: the principle itself ($\sigma_x\sigma_p \geq \frac{\hbar}{2}$), momentum ($\vec{p} = m\vec{v}$), and position (usually denoted by the vector $\vec{r}$). These are hard, mathematical and physics problems to understand. So either this framework of cognition will be left to the few that are also studying higher-level physics or be left out of the curriculum outright.

I do agree with the article. It shows the uncertainty and complexity that the human mind possesses, and how it can be described through such complex means. Although I am skeptical if quantum physics is truly the best framework for understanding the human mind, I think it holds as

the biggest contender at this point in time.